

weft

A Modern Literate Programming Tool

Cuauhtemoc Pacheco
Derived from nuweb 1.64 by Preston Briggs¹
with contributions by Keith Harwood, Simon Wright,
Marc W. Mengel, John D. Ramsdell, and Gregor Goldbach

September 14, 2026

¹Original nuweb work supported by ARPA, through ONR grant N00014-91-J-1989.

Contents

1	Introduction	1
1.1	weft	1
1.2	Writing weft	3
1.2.1	Source File Extension: <code>.weft</code>	3
1.2.2	The Major Commands	4
1.2.3	The Minor Commands	7
1.3	Sectioning commands	8
1.4	Running weft	9
1.5	Markdown Output	10
1.6	Section Markers and Language Auto-Detection	11
1.6.1	Overriding the Language with <code>@l</code>	12
1.6.2	Mid-Line Inline Expansion	12
1.6.3	The <code>@L</code> Command: Defining New Languages	13
1.6.4	The Language Table	13
1.6.5	Syntax Highlighting in Markdown Output	14
1.6.6	Output Format of Section Markers	14
1.6.7	Debugging Tangled Code	14
1.7	Restrictions	18
1.8	Project Introspection	19
1.8.1	Map (<code>-m</code>)	19
1.8.2	Extract (<code>-e</code>)	19
1.8.3	Reverse Map (<code>-R</code>)	19
1.9	Acknowledgements	20
2	The Overall Structure	21
2.1	Files	21
2.1.1	The Main Files	23
2.1.2	Support Files	25
2.2	The Main Routine	25
2.2.1	Command-Line Arguments	26
2.2.2	Help Message	30
2.2.3	Short Flags	34
2.2.4	File Names	37
2.3	Pass One	43
2.3.1	Accumulating Definitions	47
2.3.2	Block Comments	49
2.3.3	Fixing the Cross References	53
2.3.4	Dealing with fragment parameters	53
2.4	Writing the Latex File	56
2.4.1	Formatting Definitions	63
2.4.2	Generating the Indices	80
2.5	Writing the Markdown File	86

2.5.1	Formatting Definitions in Markdown	90
2.5.2	Generating the Indices in Markdown	99
2.5.3	Identifier Cross-References in Markdown	104
2.6	Writing the HTML File	106
2.6.1	HTML escaping helpers	106
2.6.2	The driver	107
2.6.3	HTML scrap definitions	109
2.6.4	HTML indices	111
2.7	Map and Extract Output	113
2.7.1	JSON Map Output	113
2.7.2	Fragment Extraction	116
2.7.3	Callers (Upward Call Chain)	118
2.7.4	Lint (Static Analysis)	119
2.7.5	Reverse Map Output	122
2.7.6	Error Annotator	126
2.8	Writing the Output Files	128
3	The Support Routines	133
3.1	Source Files	133
3.1.1	Global Declarations	133
3.1.2	Local Declarations	133
3.1.3	Reading a File	134
3.1.4	Opening a File	137
3.2	Scraps	138
3.2.1	Collecting Page Numbers	160
3.3	Names	162
3.3.1	Language Auto-Detection	168
3.3.2	User-Defined Languages	175
3.3.3	Weave Format Declaration	177
3.3.4	Collecting File and Fragment Names	178
3.3.5	Comment Styles	180
3.4	Searching for Index Entries	188
3.4.1	Retrieving scrap uses	190
3.4.2	Building the Automata	192
3.4.3	Searching the Scraps	195
3.5	Labels	200
3.6	Memory Management	203
3.6.1	Allocating Memory	203
3.6.2	Freeing Memory	204
4	Man page	205
5	Indices	210
5.1	Files	210
5.2	Fragments	210
5.3	Identifiers	215

Chapter 1

Introduction

In 1984, Knuth introduced the idea of *literate programming* and described a pair of tools to support the practise [3]. His approach was to combine Pascal code with T_EX documentation to produce a new language, WEB, that offered programmers a superior approach to programming. He wrote several programs in WEB, including `weave` and `tangle`, the programs used to support literate programming. The idea was that a programmer wrote one document, the web file, that combined documentation (written in T_EX [4]) with code (written in Pascal).

Running `tangle` on the web file would produce a complete Pascal program, ready for compilation by an ordinary Pascal compiler. The primary function of `tangle` is to allow the programmer to present elements of the program in any desired order, regardless of the restrictions imposed by the programming language. Thus, the programmer is free to present his program in a top-down fashion, bottom-up fashion, or whatever seems best in terms of promoting understanding and maintenance.

Running `weave` on the web file would produce a T_EX file, ready to be processed by T_EX. The resulting document included a variety of automatically generated indices and cross-references that made it much easier to navigate the code. Additionally, all of the code sections were automatically prettyprinted, resulting in a quite impressive document.

Knuth also wrote the programs for T_EX and METAFONT entirely in WEB, eventually publishing them in book form [5, 6]. These are probably the largest programs ever published in a readable form.

Inspired by Knuth's example, many people have experimented with WEB. Some people have even built web-like tools for their own favorite combinations of programming language and typesetting language. For example, CWEB, Knuth's current system of choice, works with a combination of C (or C++) and T_EX [8]. Another system, FunnelWeb, is independent of any programming language and only mildly dependent on T_EX [10]. Inspired by the versatility of FunnelWeb and by the daunting size of its documentation, Preston Briggs decided to write his own, very simple, tool for literate programming.¹

`weft` is a fork of `nuweb`. The original `nuweb` code, written by Preston Briggs and maintained over the years by several contributors, provided an excellent foundation: a simple, language-independent literate programming tool. `weft` builds on that foundation with a modernized codebase, a modular source structure organized by concept, the removal of the unmaintained HTML output path, and the addition of a Markdown weave output format. The goal is to continue evolving the tool while preserving the simplicity and philosophy that made `nuweb` valuable in the first place.

1.1 `weft`

The version number is defined once, as a single fragment, and threaded everywhere it is needed: the binary's `--version`, the default value of the `@v` directive (via `WEFT_VERSION`), and the generated documentation. Cutting a release is therefore a one-line edit to this scrap.

¹There is another system similar to `nuweb`, written by Norman Ramsey, called *noweb* [9]. It perhaps suffers from being overly Unix-dependent and requiring several programs to use. On the other hand, its command syntax is very nice. In any case, `nuweb` certainly owes its name and a number of features to his inspiration.

`<version 2> ≡`

`2.0.1`

□

`2 · used in 23b`

weft works with any programming language and produces documentation in L^AT_EX [7] or Markdown format. L^AT_EX was the original choice because it supports a multi-level sectioning scheme and has facilities for drawing figures. Markdown support was added to allow literate programs to produce lightweight documentation that renders well on platforms like GitHub, GitLab, and any Markdown viewer, without requiring a T_EX installation.

Support for arbitrary programming languages was a design goal because programmers work in many languages (and sometimes combinations of several languages), *e.g.*, C, Fortran, C++, yacc, lex, Scheme, assembly, Postscript, and so forth. The need to support arbitrary programming languages has many consequences:

No prettyprinting Both WEB and CWEB are able to prettyprint the code sections of their documents because they understand the language well enough to parse it. Since weft aims to support *any* language, this feature must be abandoned. However, weft does allow particular individual formulas or fragments of L^AT_EX code to be formatted and still be parts of output files. Also, keywords in scraps can be surrounded by @_ to have them be bold in the output.

No index of identifiers Because WEB knows about Pascal, it is able to construct an index of all the identifiers occurring in the code sections (filtering out keywords and the standard type identifiers). Unfortunately, this isn't as easy for a language-independent tool. weft doesn't know what an identifier looks like in each language and certainly doesn't know all the keywords. (On the other hand, see the end of Section 1.2.3)

Of course, there must be some compensation for these losses or the whole idea would be a waste. Here are the advantages:

Simplicity The majority of the commands in WEB are concerned with control of the automatic prettyprinting. Since weft doesn't prettyprint, many commands are eliminated. A further set of commands is subsumed by L^AT_EX and may also be eliminated. As a result, the set of commands is reduced to only four members (explained in the next section). This simplicity is also reflected in the size of this tool, which is quite a bit smaller than the tools used with other approaches.

No prettyprinting Everyone disagrees about how their code should look, so automatic formatting annoys many people. One approach is to provide ways to control the formatting. weft's approach is simpler—it performs no automatic formatting and therefore allows the programmer complete control of code layout. weft does allow individual scraps to be presented in either verbatim, math, or paragraph mode in the T_EX output.

Control weft also offers the programmer complete control of the layout of his output files (the files generated during tangling). Of course, this is essential for languages that are sensitive to layout; but it is also important in many practical situations, *e.g.*, debugging.

Speed Since weft doesn't do too much, the weft tool runs quickly. By default it performs only tangling; weaving is activated with `-w`. When weaving is requested, both operations execute in a single pass.

Page numbers Inspired by the example of noweb, weft refers to all scraps by page number to simplify navigation. If there are multiple scraps on a page (say, page 17), they are distinguished by lower-case letters (*e.g.*, 17a, 17b, and so forth).

Multiple file output The programmer may specify more than one output file in a single weft file. This is required when constructing programs in a combination of languages (say, Fortran and C). It's also an advantage when constructing very large programs that would require a lot of compile time.

This last point is very important. By allowing the creation of multiple output files, the need for monolithic programs is avoided. This supports the creation of very large programs by groups of people.

A further reduction in compilation time is achieved by first writing each output file to a temporary location, then comparing the temporary file with the old version of the file. If there is no difference, the temporary file can be deleted. If the files differ, the old version is deleted and the temporary file renamed. This approach works well in combination with `make` (or similar tools), since `make` will avoid recompiling untouched output files.

1.2 Writing weft

The bulk of a weft file will be ordinary L^AT_EX (or Markdown, when using the `-w md` flag). In fact, any L^AT_EX file can serve as input to weft and will be simply copied through, unchanged, to the documentation file—unless a weft command is discovered. Similarly, any Markdown file can be processed with `-w md` and will be copied through to the `.md` output. All weft commands begin with an “at-sign” (`@`). Therefore, a file without at-signs will be copied unchanged. weft commands are used to specify *output files*, define *fragments*, and delimit *scraps*. These are the basic features of interest to the weft tool—all else is simply text to be copied to the documentation file.

1.2.1 Source File Extension: `.weft`

weft source files use the `.weft` extension. This single, format-agnostic extension replaces the earlier `.w` and `.wmd` conventions. The prose surrounding scraps may be written in L^AT_EX or Markdown; the at-sign commands (`@o`, `@d`, `@<`, etc.) are the same in both cases.

The weave output format is specified in one of two ways:

In the source file The `@W` directive declares the intended format: `@W tex` for L^AT_EX, `@W md` for Markdown. This should appear in the preamble, before any `@o` or `@d` directives.

On the command line `-w md` or `-w tex` overrides the `@W` directive. Using `-w` alone (without a format token) reads the format from `@W`.

If no extension is given on the command line, weft appends `.weft` automatically.

Default Behavior: Tangle Only

By default, `weft myfile.weft` performs only tangling (producing the output files). No documentation is generated unless `-w` is given:

<code>weft programa.weft</code>	tangle only (default)
<code>weft -w programa.weft</code>	weave per <code>@W</code> (error if missing)
<code>weft -w md programa.weft</code>	weave Markdown (override)
<code>weft -w tex programa.weft</code>	weave LaTeX (override)

The `-t` flag suppresses weaving even if `-w` is given. The `-o` flag suppresses tangling.

Cosmetic Fences

When writing `.weft` files with Markdown prose, it is often convenient to surround scrap definitions with Markdown fenced code blocks (triple backticks) so that the source file renders nicely in Markdown previewers such as GitHub or VS Code. For example:

Some prose in Markdown.

```
'''c
@d My fragment
@{code here
```

```
@}
'''
```

More prose.

weft silently strips these fence lines (lines that consist of three or more backticks, optionally followed by a language identifier) during Markdown weaving. They do not appear in the generated .md file and do not affect tangling. This is purely a cosmetic convenience for making the source file readable in any Markdown viewer.

Note that fences are only stripped in Markdown mode (`-w md`). In L^AT_EX mode, backtick lines are treated as ordinary text.

1.2.2 The Major Commands

Files and fragments are defined with the following commands:

@o *file-name flags scrap* Output a file. The file name is terminated by whitespace.

@d *fragment-name scrap* Define a fragment. The fragment name is terminated by a return or the beginning of a scrap.

@q *fragment-name scrap* Define a fragment. The fragment name is terminated by a return or the beginning of a scrap. This a quoted fragment.

A specific file may be specified several times, with each definition being written out, one after the other, in the order they appear. The definitions of fragments may be similarly specified piecemeal.

Fragment Names with Embedded Parameters

Fragment names can carry *parameters*—placeholder values embedded directly in the name using the `@'value@'` syntax. Inside the fragment body, `@1`, `@2`, ... `@9` refer to the first, second, ... ninth parameter in order of appearance.

For example, a reusable validation fragment:

```
@d Validate field @'name@' type @'string@'
@{if (!data.@1) throw new Error("Missing @1");
if (typeof data.@1 !== '@2')
  throw new TypeError("@1 must be @2");
@}
```

Each invocation supplies its own arguments:

```
@<Validate field @'email@' type @'string@'@>
@<Validate field @'age@' type @'number@'@>
```

The first invocation expands to:

```
if (!data.email) throw new Error("Missing email");
if (typeof data.email !== 'string')
  throw new TypeError("email must be string");
```

Note three key points:

- The values in the *definition* (`@'name@'`, `@'string@'`) serve as default arguments and as documentation—they make the fragment name readable.
- At each *invocation*, different values can be substituted (`@'email@'` replaces `@1`, `@'string@'` replaces `@2`).
- If a fragment name is abbreviated with ..., any missing arguments fall back to the defaults given in the definition.

The difference between a quoted fragment (`@q`) and an ordinary one (`@d`) is that inside a quoted fragment fragments are not expanded on output. Rather, they are formatted as uses of fragments so that the output file can itself be `weft` source. This allows you to create files containing fragments which can undergo further processing before the fragments are expanded, while also describing and documenting them in one place.

You can have both quoted and unquoted fragments with the same name. They are written out in order as usual, with those introduced by `@q` being quoted and those with `@d` expanded as normal.

In quoted fragments, the `@f` filename is quoted as well, so that when it is expanded it refers to the finally produced file, not any of the intermediate ones.

Scraps

Scraps have specific begin markers and end markers to allow precise control over the contents and layout. Note that any amount of whitespace (including carriage returns) may appear between a name and the beginning of a scrap. Scraps may also appear in the running text where they are formatted (almost) identically to their use in definitions, but don't appear in any code output.

`@{anything@}` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in verbatim mode. Using the `-l` option will cause the program to typeset the scrap with the help of L^AT_EX's `listings` package.

`@[anything@ ]` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in paragraph mode, allowing sections of T_EX documents to be scraps, but still be prettyprinted in the document.

`@(anything@)` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in math mode. This allows this scrap to contain a formula which will be typeset nicely.

Inside a scrap, several special sequences are recognized.

Fragment invocation.

`@<fragment-name@>` Expands the named fragment inline. It is an error to specify recursive invocations. If the fragment name contains embedded parameters, the invocation supplies arguments at the same positions (see above).

When the reference appears *mid-line*—that is, after at least one visible character on the same line—section markers, `#line` directives, and fragment-name comments are automatically suppressed. The fragment content is spliced seamlessly into the surrounding text. At start-of-line positions (including indented lines), markers are emitted normally. See Section 1.6 for details.

Argument substitution.

`@1, @2, ..., @9` Inside a fragment body, substitutes the *n*th argument. If the argument was not supplied, the empty string is used.

At each argument position in an invocation, four forms are accepted:

`@'string@'` A literal string, copied verbatim (only `@` is interpreted, producing a single `@`). This is the most common form.

`@<fragment-name@>` The fragment is expanded and its output becomes the argument value.

`@{text@}` The text is expanded as an anonymous fragment (inheriting the caller's arguments) and the result is passed as the argument. Useful for combining text and arguments into a single value.

`@1, @2, ..., @9` Forwards an argument from the calling fragment to the called one.

Other in-scrap commands.

@xlabel@x Marks a place inside a scrap and associates it to the label (which can be any text not containing a @). Expands to the reference number of the scrap followed by a numeric value. Outside scraps it expands to the same value. It is used so that text outside scraps can refer to particular places within scraps.

@f Inside a scrap this is replaced by the name of the current output file.

@t Inside a scrap this is replaced by the title of the fragment as it is at the point it is used, with all parameters replaced by actual arguments.

@# At the beginning of a line in a scrap this will suppress the normal indentation for that line. Use this, for example, when you have a `#ifdef` inside a nested scrap. Writing `@##ifdef` will cause it to be lined up on the left rather than indented with the rest of its code.

@s Inside a scrap suppresses indentation for the following fragment expansion.

Note that fragment names may be abbreviated, either during invocation or definition. For example, it would be very tedious to have to type, repeatedly, the fragment name

```
@d Check for terminating at-sequence and return name if found
```

Therefore, `weft` provides a mechanism (borrowed from Knuth) of indicating abbreviated names.

```
@d Check for terminating...
```

Basically, the programmer need only type enough characters to identify the fragment name uniquely, followed by three periods. An abbreviation may even occur before the full version; `weft` simply preserves the longest version of a fragment name. Note also that blanks and tabs are insignificant within a fragment name; each string of them is replaced by a single blank.

Sometimes, for instance during program testing, it is convenient to comment out a few lines of code. In C or Fortran placing `/* ... */` around the relevant code is not a robust solution, as the code itself may contain comments. `weft` provides the command

```
@%
```

only to be used inside scraps. It behaves exactly the same as `%` in the normal $\text{\LaTeX}$ text body.

When scraps are written to a program file or a documentation file, tabs are expanded into spaces by default. Currently, `weft` assumes tab stops are set every eight characters. Furthermore, when a fragment is expanded in a scrap, the body of the fragment is indented to match the indentation of the fragment invocation. Therefore, care must be taken with languages (*e.g.*, Fortran) that are sensitive to indentation. These default behaviors may be changed for each output file (see below).

Flags

When defining an output file, the programmer has the option of using flags to control output of a particular file. The flags are intended to make life a little easier for programmers using certain languages. They introduce little language dependences; however, they do so only for a particular file. Thus it is still easy to mix languages within a single document. There are five “per-file” flags:

- d Forces the creation of `#line` directives in the output file. These are useful with C (and sometimes C++ and Fortran) on many Unix systems since they cause the compiler’s error messages to refer to the web file rather than to the output file. Similarly, they allow source debugging in terms of the web file.
- i Suppresses the indentation of fragments. That is, when a fragment is expanded within a scrap, it will *not* be indented to match the indentation of the fragment invocation. This flag would seem most useful for Fortran programmers.

- t Suppresses expansion of tabs in the output file. This feature seems important when generating `make` files.
 - cx Puts comments in generated code documenting the fragment that generated that code. The *x* selects the comment style for the language in use: `c` for C (`/* */`), `+` for C++ (`//`), and `p` for shell/Perl (`#`). If the global `-x` cross-reference flag is set the comment includes the page reference for the first scrap that generated the code.
 - s Suppresses section markers and `#line` directives in the tangled output for this file. The language is still detected (from the extension or a preceding `@l`), so the Markdown weaver can emit syntax-highlighting classes, but the tangled code is emitted without any tangle annotations. This is useful for bootstrap source files or generated output that should remain clean.
- These flags are still available for explicit control, but in most cases the automatic language detection described in Section 1.6 is more convenient.

1.2.3 The Minor Commands

There are some very low-level utility commands that may appear anywhere in the web file.

- `@@` Causes a single “at sign” to be copied into the output.
- `@_` Causes the text between it and the next `@_` to be made bold (for keywords, etc.)
- `@i file-name` Includes a file. Includes may be nested, though there is currently a limit of 10 levels. The file name should be complete (no extension will be appended) and should be terminated by a carriage return. Normally the current directory is searched for the file to be included, but this can be varied using the `-I` flag on the command line. Each such flag adds one directory to the search path and they are searched in the order given.
- `@rx` Changes the escape character ‘`@`’ to ‘*x*’. This must appear before any scrap definitions.
- `@v` Always replaced by the string established by the `-V` flag, or a default string if the flag isn’t given. This is intended to mark versions of the generated files.
- `@l tag` Sets the language for the next `@o` or `@d` directive. The tag is looked up in the language table (Section 1.6) to determine the comment style for section markers and the `class` attribute for Markdown syntax highlighting. Placed on the line before the directive it applies to.
- `@L name style [+d]` Defines a custom language mapping for the section marker system (see Section 1.6). This must appear in the document preamble, before any `@o` or `@d` directives that depend on it.
- `@W format` Declares the weave output format for this source file. Valid formats are `md` (Markdown) and `tex` (L^AT_EX). Used by `-w` when no explicit format is given on the command line. A CLI format (`-w md` or `-w tex`) overrides this directive. Should appear in the document preamble.

In the text of the document, that is outside scraps, you may include scrap-like material.

- `@{Anything@}` The included material, the *Anything*, is typeset as if it appeared inside a scrap. This is useful for referring to fragments in the text and for describing the literate programming process itself.
- `@<Fragment name@>` The fragment named is expanded in place in the text. The expansion is presented verbatim, it is not interpreted for typesetting, so any special environment must be set up before and after this is used.
- `@c` Outside of a scrap this is used to define a block comment using all the text until the next command. Inside a scrap the last block comment is expanded using the current indentation and breaking lines between words if 60 characters are exceeded.

Finally, there are three commands used to create indices to the fragment names, file definitions, and user-specified identifiers.

`@f` Create an index of file names.

`@m` Create an index of fragment names.

`@u` Create an index of user-specified identifiers.

These are usually placed in their own section in the $\text{\LaTeX}$ document; for example, see Chapter 5.

Identifiers must be explicitly specified for inclusion in the `@u` index. By convention, each identifier is marked at the point of its definition; all references to each identifier (inside scraps) will be discovered automatically. To “mark” an identifier for inclusion in the index, it must be mentioned at the end of a scrap. For example,

```
@d a scrap @{
  Suppose the variables FOO and BAR are being declared
  inside this scrap.
@| FOO BAR @}
```

Alphabetic identifiers have been used in this example, but any string of characters (not including whitespace or `@` characters) will do. Therefore, it’s possible to add index entries for things like `<=<` if desired. An identifier may be declared in more than one scrap.

In the generated index, each identifier appears with a list of all the scraps using and defining it, where the defining scraps are distinguished by underlining. Note that the identifier doesn’t actually have to appear in the defining scrap; it just has to be in the list of definitions at the end of a scrap.

1.3 Sectioning commands

For larger documents the indexes and usage lists get rather unwieldy and problems arise in naming things so that different things in different parts of the document don’t get confused. `weft` has a sectioning command which keeps the fragment names and user identifiers separate. Thus, you can give a fragment in one section the same name as a fragment in another and the two won’t be confused or connected in any way. Nor will user identifiers defined in one section be referenced in another. Except for the fact that scraps in successive sections can go into the same output file, this is the same as if the sections came from separate input files.

However, occasionally you may really want fragments from one section to be used in another. More often, you will want to identify a user identifier in one section with the same identifier in another (as, for example, a header file defined in one section is included in code in another). Extra commands allow a fragment defined in one section to be accessible from all other sections. Similarly, you can have scraps which define user identifiers and export them so that they can be used in other sections.

`@s` Start a new section.

`@S` Close the current section and don’t start another.

`@d+ fragment-name scrap` Define a fragment which is accessible in all sections, a global fragment.

`@D+` Likewise

`@q+` Likewise

`@Q+` Likewise

`@m+` Create an index of all such fragments.

`@u+` Create an index of globally accessible user identifiers.

There are two kinds of section, the base section which is where normally everything goes, and local sections which are introduced with the `@s` command. A local section comprises everything from the command which starts it to the one which ends it. A `@s` command will start a new local section. A `@S` command closes the current local section, but doesn’t open another, so what follows goes into the base section. Note that

fragments defined in the base section aren't global; they are accessible only in the base section, but they are accessible regardless of any local sections between their definition and their use.

Within a scrap:

`@<+fragment-name>` Expand the globally accessible fragment with that name, rather than any local fragment.

`@+` Like `@|` except that the identifiers defined are exported to the global realm and are not directly referenced in any scrap in any section (not even the one where they are defined).

`@-` Like `@|` except that the identifiers are imported to the local realm. The cross-references show where the global variables are defined and defines the same names as locally accessible. Uses of the names within the section will point to this scrap.

Note that the `+` signs above are part of the commands. They are not part of the fragment names. If you want a fragment whose name begins with a plus sign, leave a space between the command and the name.

1.4 Running weft

weft is invoked using the following command:

```
weft flags file-name...
```

One or more files may be processed at a time. If a file name has no extension, `.weft` will be appended. While a file name may specify a file in another directory, the resulting documentation file will always be created in the current directory. For example,

```
weft /foo/bar/quux
```

will take as input the file `/foo/bar/quux.weft` and will produce the tangled output files only. To also generate documentation, use `-w` (see below).

By default, weft performs only tangling—it generates the output files defined by `@o` directives and nothing else. Documentation (weaving) is produced only when the `-w` flag is given. When `-w` is present, both tangling and weaving execute in a single pass. Three flags control what weft produces:

- `-t` Suppress weaving (useful to override `-w` in scripts).
- `-o` Suppress generation of the output files (weave only).
- `-c` Avoid testing output files for change before updating them.

Thus, the command

```
weft -to /foo/bar/quux
```

would simply scan the input and produce no output at all.

There are several additional command-line flags:

- `-v` For “verbose”, causes weft to write information about its progress to `stderr`.
- `-n` Forces scraps to be numbered sequentially from 1 (instead of using page numbers). This form is perhaps more desirable for small webs.
- `-s` Doesn't print list of scraps making up each file following each scrap.
- `-d` Print “dangling” identifiers – user identifiers which are never referenced, in indices, etc.
- `-p path` Prepend *path* to the filenames for all the output files.
- `-w [format]` Activate weaving (documentation generation). Without `-w`, no documentation file is produced. The optional *format* argument selects the output format:

- `-w md` — force Markdown output (`.md` file).
- `-w tex` — force L^AT_EX output (`.tex` file).
- `-w` alone — read the format from the `@W` directive in the source file. If `@W` is absent, `weft` reports an error.

A format given on the command line overrides any `@W` directive. When Markdown output is selected, sequential numbering (`-n`) is forced automatically, since Markdown has no concept of page numbers. The Markdown output uses HTML anchors (`<a id="...">`) for cross-references and `<pre><code>` blocks for scraps, allowing clickable navigation between definitions and references. Cosmetic fence lines (triple backticks) in the source are silently stripped.

- `-x` Include cross-reference numbers in scrap comments.
- `-h options` Provide options for the `hyperref` package.
- `-r` Turn on hyperlinks. You must include the `—usepackage—` options in the text.
- `-l` Use the `listings` package for formatting scraps. Use this if you want to have a pretty-printer for your scraps. In order to e.g. have pretty Perl scraps, include the following L^AT_EX commands in your document:

```
\usepackage{listings}
...
\lstset{extendedchars=true,keepspaces=true,language=perl}
```

 See the `listings` documentation for a list of formatting options. Be sure to include a `\usepackage{listings}` in your document.
- `-V string` Provide *string* as the replacement for the `@v` operation.
- `-m` Emit a JSON map of the project to stdout and exit. The map describes every output file and every fragment, their definition locations, usage relationships, and transitive dependencies. Neither weaving nor tangling is performed. See Section 2.7 for details.
- `-e name` Extract a single fragment by name and emit its code and dependency information as Markdown to stdout. The match uses the same prefix logic as `@<name...@>`. Neither weaving nor tangling is performed. See Section 2.7 for details.
- `-R file[:line]` Read section markers from a tangled output file and emit JSON mapping to stdout. If a line number is given, emit the single innermost scrap covering that line. Without a line number, emit all scrap regions in the file. No `.weft` source files are needed—the reverse map operates entirely on the tangled output. See Section 2.7 for details.

1.5 Markdown Output

When invoked with `-w md`, `weft` generates a Markdown file (`.md`) instead of a L^AT_EX file (`.tex`). This allows literate programs to be read on any platform that renders Markdown, including GitHub, GitLab, and local editors like VS Code.

The Markdown output preserves all the essential features of the L^AT_EX output—scrap definitions, cross-references, and indices—but uses a different set of formatting conventions:

Scrap headers File definitions appear as `**"filename"** **N** =` and fragment definitions as `**<Name* N>** =`, where *N* is the sequential scrap number.

Scrap bodies Code is wrapped in `<pre><code>` HTML blocks (not fenced code blocks). This allows fragment references within scraps to be rendered as clickable HTML links with italic and bold formatting, which would not be possible inside Markdown fenced blocks.

HTML escaping Characters `<`, `>`, and `&` inside scraps are escaped as `<`, `>`, and `&` respectively.

Cross-references Each scrap has an HTML anchor (`<a id="weftN">`) and cross-reference links use `<a href="#weftN">`. Footer notes (“defined by”, “referenced in”) appear as Markdown blockquotes.

Indices The `@f`, `@m`, and `@u` commands produce Markdown lists with links. In the user identifier index, defining scraps appear in bold.

Sequential numbering The `-n` flag is forced automatically, since Markdown has no page numbers. Scraps are numbered 1, 2, 3, ... without letter suffixes.

Prose Text outside scraps passes through unmodified. The author writes Markdown prose directly in the `.weft` source file; weft does not interpret or transform it.

Syntax highlighting When the language of a scrap can be determined—from a preceding `@l` tag, the `@L` command, or the file extension of an `@o` name—weft emits `<code class="language-X">` on the opening tag. This allows client-side highlighters (highlight.js, Prism) and platforms like GitHub to apply syntax colouring automatically.

Tangling (output file generation) works identically regardless of the weave format. When `-w` is given, a single invocation produces both the documentation and the tangled output files in one pass.

1.6 Section Markers and Language Auto-Detection

When debugging tangled code, it is useful to correlate lines in the generated file back to the original web source. weft can auto-detect the programming language and generate *section markers*—comments that delimit the opening and closing of each scrap in the tangled output, including the source file name and line number. For C and C++, it also generates `#line` directives automatically.

The key idea is simple:

file extension → language → comment style → section markers in tangled output

For example, `@o server.js` has the extension `.js`. weft looks up `js` in its language table, finds that JavaScript uses `//` comments, and wraps each scrap in the tangled `server.js` with `//` markers:

```
// {1: web.weft:12}
console.log("hello");
// {1: }
```

Similarly, `@o main.c` matches `c`, which uses `/* */` comments and also enables `#line` directives:

```
#line 5 "core.weft"
/* {1: core.weft:5} */
#include <stdio.h>
/* {1: } */
```

And `@o config.py` matches `py`, using `#` comments:

```
# {3: settings.weft:40}
DATABASE_URL = "postgres://localhost/app"
# {3: }
```

This happens automatically—no flags are needed. The extension alone is enough.

1.6.1 Overriding the Language with @l

Sometimes the file extension is not enough, or you want a different behaviour. The `@l` command (lowercase) placed on the line before an `@o` or `@d` directive overrides the extension:

```
@l sql
@o queries.txt
@{SELECT * FROM users;
@}
```

Here the file is `queries.txt`, whose extension `.txt` has no comment style. But `@l sql` tells weft to use `--` comments, so the tangled output gets:

```
-- {1: db.weft:5}
SELECT * FROM users;
-- {:1}
```

`@l` also works on `@d` (fragment) definitions. This is useful for fragments that will be included in files of a specific language:

```
@l python
@d Validate email
@{import re
def validate(email):
    return bool(re.match(r'^[^\@]+\@[^\@]+$', email))
@}
```

The language tag is stored on the fragment and used both for section markers in the tangled output and for syntax highlighting in the Markdown weave output.

`@l` is format-agnostic: it works identically in $\text{\LaTeX}$ and Markdown source files. Unlike the earlier approach of encoding the language in Markdown fences (`'''python`) or $\text{\LaTeX}$ markers (`%%python`), `@l` uses the same at-sign syntax as every other weft command.

Note that cosmetic fences (`'''`) may still be used around `@o/@d` definitions for readability in Markdown previewers—they are silently stripped during Markdown weaving (see Section 1.2.1). But fences no longer set the language; use `@l` for that.

Priority rules. When multiple sources of language information are available, weft resolves them in this order:

1. **Explicit per-file flags** (`-d`, `-s`, `-cx`, etc.) on the `@o` line always win for tangle behaviour.
2. **@l tag** immediately before the directive—overrides the extension.
3. **File extension** of the `@o` name—the default when no `@l` is present.

1.6.2 Mid-Line Inline Expansion

When a fragment reference `@<name>` appears in the middle of a line—after at least one visible character—weft automatically suppresses all tangle annotations for that expansion: section markers, `#line` directives, and fragment-name comments. The fragment content is spliced directly into the surrounding text, producing clean inline output.

This makes small constant fragments practical:

```
@d port @{4000@}
@d host @{"localhost"@}

@l c
```

```
@o config.c -s
@{const char *host = @<host@>;
int port = @<port@>;
@}
```

Tangling produces:

```
const char *host = "localhost";
int port = 4000;
```

If the same reference appears at the *start* of a line (even if indented), it is treated as a block expansion and markers are emitted normally. No special flags or directives are needed—the position on the line is the only thing that determines the behaviour.

This works with any language and any fragment size. When a multi-line fragment is expanded mid-line, its content is still spliced without markers—the rule is based on the *reference site*, not the fragment body.

1.6.3 The @L Command: Defining New Languages

The built-in table covers over 250 entries, but it cannot anticipate every language or every project's preferences. The @L command defines new language mappings—or overrides existing ones—directly in the document preamble.

@L *name style* [+d]

- *name* is the tag that will be recognised in @l commands and file extensions (@o *file.name*).
- *style* is the comment syntax: //, #, --, /*, or <!--.
- +d (optional) enables #line directive generation.

Examples:

```
@L wgs1 //          % WebGPU shading language: // comments
@L cedar //         % Cedar policy language
@L c //            % override built-in: use // instead of /* */
```

How @L interacts with the rest of the system. User-defined entries are searched *before* the built-in table, so @L c // effectively changes the comment style for all subsequent @o *main.c* directives (and for any @l c tag) from /* */ to //. Up to 32 user-defined languages may be registered.

@L should appear in the document preamble, before any @o or @d that depends on it. It is processed during pass 1 and silently skipped during weaving.

1.6.4 The Language Table

weft includes a built-in table of over 250 entries (languages, file extensions, and common aliases), organised by comment style. Each entry maps a tag to a comment style and optionally enables #line directives. The tag is matched against @l tags and file extensions alike—there is no distinction between them.

Comment style	Tags (representative)
<code>/* */ + #line</code>	c, h, cpp, cc, cxx, hpp, hxx, objc, objcpp, m, mm, ino, pde, cu, cuh, cilk
<code>/* */</code>	css, scss, sass, less, stylus, cl (OpenCL)
<code>//</code>	js, ts, jsx, tsx, mjs, cjs, mts, svelte, vue, java, scala, kotlin, kt, groovy, gradle, clj, cs, fs, fsx, vb, go, rust, rs, zig, swift, dart, odin, mojo, carbon, php, graphql, prisma, glsl, hlsl, wgsl, metal, vert, frag, proto, thrift, fbs, capnp, sol, reason, res, ml, sml, fut, apex, bicep, jsonnet, qml, p4
<code>#</code>	python, py, pyx, pyi, sh, bash, zsh, fish, ksh, nu, perl, pl, ruby, rb, r, tcl, awk, sed, coffee, makefile, mk, cmake, just, meson, bazel, bzl, julia, jl, nim, crystal, cr, elixir, ex, yaml, yml, toml, dockerfile, terraform, tf, hcl, nix, conf, ini, env, editorconfig, gitignore, powershell, ps1, xdc, sdc
<code>--</code>	lua, luau, sql, psql, plsql, plpgsql, mysql, sqlite, hive, haskell, hs, elm, purescript, purs, ada, adb, ads, vhdl, vhd, lean, lean4, agda, idris, idr, erlang
<code><!-- --></code>	html, htm, xml, svg, xhtml, xsl, xsd, dtd, rss, xaml, csproj, jsp, aspx, cshtml, erb, ejs, hbs, njk, twig, blade, pug, liquid, mustache, md, mdx, rst, tex, latex
(none)	json, csv, tsv, txt, diff, patch

For the full list, see the `lang_table` array in Section 3.3.1. If a tag is not found in either the user-defined (`@L`) or built-in table, weft defaults to `//` comments with no `#line` directives. Tags mapped to style 0 (e.g. `json`, `txt`) produce no markers at all.

1.6.5 Syntax Highlighting in Markdown Output

When weaving to Markdown (`-w md`), weft emits the language tag on the `<code>` element:

```
<pre><code class="language-python">def hello():
    print("hello")
</code></pre>
```

This allows client-side highlighters (highlight.js, Prism) and platforms like GitHub to apply syntax colouring automatically. The language tag comes from the same chain described above: `@L` tag or file extension.

For `@d` fragments, the tag comes from a preceding `@L` command (there is no file extension to infer from). If no language is known, weft emits a plain `<pre><code>` without a class.

1.6.6 Output Format of Section Markers

Each scrap in the tangled file is wrapped with an opening marker at the top and a closing marker at the bottom, using the comment syntax of the detected language. The opening marker includes the scrap number and the source location (file name and line number). When fragments are nested, each expansion gets its own pair of markers, producing a clear hierarchy in the output.

1.6.7 Debugging Tangled Code

When you work with literate programming, the code you execute is *tangled output*—generated files assembled from scraps scattered across `.w` files. An error at `server.js:47` does not correspond to line 47 of any file you wrote; it is line 47 of a file that weft assembled from fragments defined in `server.w`, `routes.w`, and `database.w`. Without markers, debugging means mentally reconstructing which scrap produced which line. With markers, the tangled file tells you directly.

Reading the markers. Each marker encodes three pieces of information:

Part	Meaning
{ or {:	Opening or closing delimiter
Number	Scrap ID—unique across the entire document
file:line	Source .w file and line number (opening only)

Examples by language. JavaScript (// comments):

```
// {3: routes.w:42}
app.get("/api/users", async (req, res) => {
  // {7: database.w:18}
  const users = await db.query("SELECT * FROM users");
  // {7: }
  res.json(users);
});
// {3: }
```

Python (# comments):

```
# {5: models.w:31}
class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email
# {5: }
```

C (/* */ comments with #line directives):

```
#line 12 "memory.w"
/* {2: memory.w:12} */
void *safe_malloc(size_t size) {
    void *ptr = malloc(size);
    if (!ptr) {
        /* {4: errors.w:8} */
        fprintf(stderr, "out of memory\n");
        exit(1);
        /* {4: } */
    }
    return ptr;
}
/* {2: } */
```

Lua (-- comments):

```
-- {9: config.w:55}
local defaults = {
    port = 8080,
    host = "localhost",
}
-- {9: }
```

HTML (<!-- --> comments):

```
<!-- {6: layout.w:20} -->
<nav class="sidebar">
  <!-- {11: navigation.w:7} -->
```

```

    <ul>
      <li><a href="/">Home</a></li>
    </ul>
    <!-- {11} -->
  </nav>
  <!-- {6} -->

```

Debugging workflow. The following four-step procedure traces any error in tangled output back to the literate source.

Step 1: Locate the error in tangled output. You receive an error such as:

```

TypeError: Cannot read property 'id' of undefined
    at Object.<anonymous> (server.js:47)

```

Open `server.js` and go to line 47.

Step 2: Read the nearest opening marker. Scan *upward* from the error line until you find an opening marker:

```
// {3: routes.w:42}
```

This tells you: scrap 3 generated this code, defined in `routes.w` starting at line 42.

Step 3: Go to the literate source. Open `routes.w` at line 42. You will find the scrap with the bug, surrounded by prose that explains its intent:

```

We fetch the user by ID from the database. If the user
doesn't exist, we should return a 404, but currently we
assume the user always exists --- this is the bug.

```

```

@d Handle user fetch
@{const user = await getUser(req.params.id);
  res.json(user.id);
@}

```

The prose makes the bug obvious: there is no null check. Fix it in the `.w` file, re-tangle, and the error is gone.

Step 4: Nested markers = nested fragments. When fragments invoke other fragments, markers nest:

```

// {3: routes.w:42}
app.get("/api/users/:id", async (req, res) => {
  // {7: database.w:18}
  const user = await db.query(
    "SELECT * FROM users WHERE id=$1",
    [req.params.id]);
  // {7}
  if (!user) return res.status(404).json({error: "not found"});
  res.json(user);
});
// {3}

```

If the bug is inside the `{7: ... {7}}` block, go to `database.w:18`, not `routes.w:42`. The *innermost enclosing markers* point to the most specific source location. Always look at the tightest pair around the error line.

Interpreting nested hierarchies. Complex tangled output can have several nesting levels. Read them like parentheses—each opening matches its corresponding close:

```
/* {1: main.w:10} */          outermost: main file structure
#include <stdio.h>
/* {2: io.w:25} */           middle: I/O subsystem
void print_report() {
    /* {5: format.w:8} */     innermost: formatting detail
    printf("Report v%d\n", VERSION);
    /* {5} */
}
/* {2} */
int main() {
    /* {3: init.w:14} */
    init_subsystems();
    /* {3} */
    print_report();
    return 0;
}
/* {1} */
```

Scrap numbers are *document-global*—scrap 1 is the first `@{...@}` in the entire woven document, scrap 5 is the fifth. They are stable as long as you do not reorder scraps.

C/C++: #line directives. For C-family languages, weft also generates **#line** directives *above* the opening marker. These make the C compiler itself report errors against the `.w` source file:

```
#line 12 "memory.w"
/* {2: memory.w:12} */
void *safe_malloc(size_t size) { ... }
/* {2} */
```

Now gcc will say:

```
memory.w:14: error: implicit declaration of function 'malloc'
```

instead of:

```
memory.c:47: error: implicit declaration of function 'malloc'
```

The **#line** directive and the section marker are complementary: **#line** makes the compiler report errors against the `.w` file directly; the `{N: file:line}` markers are human-readable when browsing the tangled output.

Common debugging scenarios. Runtime error with stack trace.

```
Error: ENOENT: no such file or directory, open '/data/config.json'
at readConfig (server.js:123)
```

Open `server.js:123`, find the nearest marker `// {12: config.w:35}`, open `config.w:35`—the scrap that reads the config—and fix the path or add an existence check in the `.w` file.

Compiler error in C. With **#line** directives active, the compiler already reports:

```
parser.w:84: error: 'token_type' undeclared
```

Go directly to `parser.w:84`. The prose around the scrap explains what `token_type` should be and where it is defined.

Wrong output, no error. The program runs but produces wrong results. Open the tangled file and search for the relevant code. The markers tell you which `.w` file and scrap to examine:

```
# {15: transform.w:67}
result = [x * 2 for x in data]    # should be x * 3?
# {:15}
```

Go to `transform.w:67`. The prose says “triple each value” but the code says `* 2`. Bug found.

Identifying which fragment owns a line. Sometimes you know the line of code but not which `@d` definition produced it (multiple fragments with the same name accumulate). The scrap number is unique:

```
// {14: handlers.w:89}
validateInput(data);
// {:14}
// {22: handlers.w:112}
validatePermissions(user);
// {:22}
```

Scraps 14 and 22 are two different definitions (or continuations) of the same fragment. The line numbers point to each one independently.

Tips.

- *Never edit tangled files.* The markers remind you these are generated. Always trace back to the `.w` source and edit there.
- *Scrap numbers are stable* as long as you do not add or remove `@{...@}` blocks before them. If numbering shifts after edits, re-tangle and the markers update automatically.
- *Use `grep` on markers* to find all scraps from a specific file:

```
grep -n '{[0-9]*: database.w:' server.js
```
- *Closing markers as verification.* If you see `{:7}` but no `{7:` above it in your viewport, scroll up—the scrap is larger than your screen. If you see `{7:` but no `{:7}`, the scrap extends below.
- *Markers in test output.* If your test framework shows code snippets on failure, the markers point straight to the literate source.
- *Disable markers selectively.* Use explicit flags on `@o` (e.g. `@o file -t`) with no comment flags to suppress markers for a specific output file when comments would interfere—for example, binary formats, data files, or files where comments are not valid syntax.

1.7 Restrictions

Because `weft` is intended to be a simple tool, a few restrictions have been established. Over time, some of these may be eliminated; others seem fundamental.

- The handling of errors is not completely ideal. In some cases, `weft` simply warns of a problem and continues; in other cases it halts immediately. This behavior should be regularized.
- `weft` warns about references to fragments that haven’t been defined, but doesn’t halt. The name of the fragment is included in the output file surrounded by `<>` signs. This seems most convenient for development, but may change in the future.
- File names and index entries should not contain any `@` signs.
- Fragment names may be (almost) any well-formed `TeX` string. It makes sense to change fonts or use math mode; however, care should be taken to ensure matching braces, brackets, and dollar signs.
- Anything is allowed in the body of a scrap; however, very long scraps (horizontally or vertically) may not typeset well.

- Temporary files (created for comparison to the eventual output files) are placed in the current directory. Since they may be renamed to an output file name, all the output files should be on the same file system as the current directory. Alternatively, you can use the `-p` flag to specify where the files go.
- Because page numbers cannot be determined until the document has been typeset, `weft` must be rerun after `LATEX` to obtain a clean version of the document (very similar to the way it is sometimes necessary to rerun `LATEX` to obtain an up-to-date table of contents after significant edits). `weft` will warn (in most cases) when this needs to be done; in the remaining cases, `LATEX` will warn that labels may have changed. This restriction does not apply to Markdown output (`-w md`), which uses sequential numbering and requires no additional processing passes.

Very long scraps may be allowed to break across a page if declared with `@O` or `@D` (instead of `@o` and `@d`). This doesn't work very well as a default, since far too many short scraps will be broken across pages; however, as a user-controlled option, it seems very useful.

1.8 Project Introspection

The `-m` and `-e` flags support machine-readable introspection of a `weft` project. They run after pass 1 has built the name trees and do *not* invoke the weave or tangle phases.

1.8.1 Map (`-m`)

The command

```
weft -m project.weft
```

writes a JSON object to stdout with two arrays, `"files"` and `"fragments"`. For each entry the map records the name, language tag, definition scraps (with source file and line), and dependency relationships. File entries list the fragments they use; fragment entries list the scraps that invoke them and the fragments they reference in turn.

This gives an AI assistant or any external tool a complete dependency graph of the project in a single invocation, without having to parse the `weft` source files.

1.8.2 Extract (`-e`)

The command

```
weft -e "fragment name" project.weft
```

finds the named fragment (using prefix matching), then emits its scrap code and dependency information as Markdown to stdout. Referenced fragments are followed transitively, so the output contains the full closure of the requested fragment.

This allows an AI assistant to retrieve exactly the code it needs to understand or modify a specific fragment, together with all its dependencies, in one step.

1.8.3 Reverse Map (`-R`)

The `-m` and `-e` flags require the `.weft` source files. The reverse map provides the opposite direction: given a tangled output file, it parses the section markers already embedded in that file and reports which `.weft` source and line produced each region. No source files are needed and pass 1 is never invoked.

Single-line query.

```
weft -R server.js:47
```

emits a JSON object identifying the innermost scrap covering line 47:

```
{"scrap": 3, "file": "routes.weft", "line": 42}
```

If the line is outside any scrap region, an empty object `{}` is returned.

Full-file mapping.

```
weft -R server.js
```

emits a JSON object listing every scrap region in the file:

```
{ "source": "server.js", "regions": [
  { "start": 2, "end": 15, "scrap": 3,
    "file": "routes.weft", "line": 42 },
  { "start": 5, "end": 8, "scrap": 7,
    "file": "database.weft", "line": 18 }
]}
```

Regions may overlap when fragments are nested.

This enables CI/CD pipelines to translate compiler errors back to the literate source automatically, and lets AI agents resolve tangled-output locations without loading the `.weft` files at all.

1.9 Acknowledgements

Several people have contributed their time, ideas, and debugging skills. In particular, the contributions of the following people are gratefully acknowledged: Osman Buyukisik, Manuel Carriba, Adrian Clarke, Tim Harvey, Michael Lewis, Walter Ravenek, Rob Shillingsburg, Kayvan Sylvan, Dominique de Waleffe, and Scott Warren. Of course, most of these people would never have heard of nuweb (the original tool) without the efforts of George Greenwade.

Since maintenance has been taken over by Marc Mengel, Simon Wright and Keith Harwood online contributions have been made by:

- Walter Brown <wb@fnal.gov>
- Nicky van Foreest <n.d.vanforeest@math.utwente.nl>
- Javier Goizueta <jgoizueta@jazzfree.com>
- Alan Karp <karp@hp.com>

Chapter 2

The Overall Structure

Processing a web requires three major steps:

1. Read the source, accumulating file names, fragment names, scraps, and lists of cross-references.
2. Reread the source, copying out to the documentation file, with protection and cross-reference information for all the scraps.
3. Traverse the list of file names. For each file name:
 - (a) Dump all the defining scraps into a temporary file.
 - (b) If the file already exists and is unchanged, delete the temporary file; otherwise, rename the temporary file.

2.1 Files

I have divided the program into several files for quicker recompilation during development.

```
<global.h 21a> ≡  
<Include files 21b>  
<Type declarations 23a, ...>  
<Limits 23b>  
<Global variable declarations 27, ...>  
<Function prototypes 42a, ...>  
<Operating System Dependencies 26>  
□  
21a
```

We'll need at least five of the standard system include files. We also require `_POSIX_C_SOURCE` at level 200809L so that POSIX functions like `strdup` are visible when compiling under strict C11 mode (`-std=c11`).

```
<Include files 21b> ≡  
  
#define _POSIX_C_SOURCE 200809L  
  
#include <stdlib.h>  
#include <stdio.h>  
#include <string.h>  
#include <ctype.h>  
#include <signal.h>  
#include <locale.h>
```

□

21b · used in 21a · defines `exit` 25f, 30, 36e, 37b, 39b, 47ab, 68b, 116ab, 121, 127a, 128b, 131ab, 132, 133b, 139ab, 140c, 147a, 148b, 152e, 170, 173a, 176b, 177a, 178cd, 179d, 180, 185a, 190b, 200, `fclose` 56a, 76b, 84d, 104, 121, 123, 126b, 127ac, 128a, 133a, 144, 156b, `FILE` 31, 55b, 56a, 63c, 67d, 69a, 70c, 71ab, 76b, 77, 78b, 79c, 82b, 84bcd, 88c, 91ab, 95ab, 97a, 98b, 100a, 101a, 102c, 103c, 104, 109ac, 110b, 111ab, 114, 121, 123, 126b, 127c, 128a, 129d, 130a, 135acd, 136c, 143b, 144, 147c, 155a, 156b, 192ab, 193bc, 195bc, 200, `fopen` 56a, 84d, 104, 121, 123, 127ac, 128a, 131b, 133b, 156b, `fprintf` 34, 37ab, 39b, 42c, 44d, 47ab, 50c, 54b, 56a, 58a, 61, 62, 64a, 65b, 67ac, 68b, 73a, 75abc, 77, 78c, 82c, 84d, 87c, 89a, 90be, 94bcd, 95b, 97b, 100b, 102a, 103a, 104, 107c, 111ab, 114, 116ab, 118ab, 121, 126b, 127a, 128b, 131ab, 132, 133b, 135c, 137d, 139ab, 140c, 143b, 144, 147a, 148b, 150abc, 152de, 154c, 155a, 160b, 170, 172, 173ab, 174a, 176b, 177a, 178cd, 179d, 180, 185a, 190b, 192d, 193e, 195de, 196ab, `fputs` 50e, 52c, 54b, 56b, 57, 58b, 59a, 60b, 61, 62, 63ac, 64abc, 65ab, 66aef, 67abcd, 69a, 70c, 71ab, 73b, 74a, 75c, 76ab, 77, 78acd, 79a, 80cd, 81a, 82ac, 87ac, 88ac, 89a, 90bde, 91ab, 92a, 93a, 94abd, 95ab, 96, 97bcd, 98c, 99abc, 100b, 101b, 102abd, 103ac, 105ab, 106a, 107abc, 108bcd, 109abc, 110ab, 111ab, 113a, 114, 115c, 121, 124, 127c, 135c, 144, 148c, 152d, 153, 192cd, 193ade, `getc` 128a, 130b, 131ab, 132, 133ab, `getenv` 37a, `isgraph` 132, 173a, 179d, `islower` 161c, `isspace` 66a, 89b, 108a, 140c, 173ab, 176b, 178d, `isupper` never used, `malloc` 79c, 98b, 185a, 198b, `putc` 50e, 51a, 59bc, 61, 69a, 71b, 72c, 75c, 76b, 77, 78cd, 80cd, 81a, 82c, 85ab, 86a, 91b, 92ab, 93a, 94d, 95b, 97b, 98c, 99a, 103c, 105c, 106a, 109c, 135c, 148c, 150e, 151bc, 152e, 153, 154ab, `remove` 127bc, 128ab, `setlocale` 37a, `size_t` 76b, 119b, 124, 197be, 198b, `stderr` 34, 37ab, 39b, 42c, 44d, 47ab, 56a, 61, 67ac, 68b, 73a, 75c, 77, 84d, 90be, 94d, 95b, 104, 116ab, 118ab, 121, 126b, 127a, 128b, 131ab, 132, 133b, 137d, 139ab, 140c, 147a, 148b, 152e, 160b, 170, 172, 173ab, 174a, 176b, 177a, 178cd, 179d, 180, 185a, 190b, 195e, 196ab, `strlen` 50e, 117a, 121, 126a, 156c, 159bc, 186d, 194b, `toupper` 161c

I like to use `TRUE` and `FALSE` in my code. I'd use an `enum` here, except that some systems seem to provide definitions of `TRUE` and `FALSE` by default. The following code seems to work on all the local systems.

< Type declarations 23a > ≡

```
#ifndef FALSE
#define FALSE 0
#endif
#ifndef TRUE
#define TRUE 1
#endif

#if defined(__GNUC__) && __GNUC__ >= 7
#define FALLTHROUGH __attribute__((fallthrough))
#elif defined(__clang__)
#define FALLTHROUGH __attribute__((fallthrough))
#else
#define FALLTHROUGH ((void)0)
#endif
```

□

23a · defined at 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e · used in 21a · defines `FALLTHROUGH` 44d, 59c, 73b, 86a, 106a, 130b, 160a, `FALSE` 27, 28, 34, 35ab, 36abcd, 39b, 59bc, 65a, 66d, 67d, 71ab, 76b, 80d, 81c, 82c, 85ab, 86a, 87b, 89c, 91a, 95b, 99a, 100b, 102a, 103a, 106a, 115a, 128b, 131a, 133b, 152e, 156b, 161a, 163ab, 170, 173b, 174b, 191b, 192d, 193e, 194ab, `TRUE` 27, 28, 30, 34, 36b, 39b, 54b, 59c, 64b, 65b, 67d, 73a, 76b, 80d, 81c, 82c, 85ab, 86a, 87b, 89ab, 91a, 95b, 99a, 100b, 102a, 103a, 106a, 107c, 114, 115c, 131a, 135d, 137d, 152e, 161a, 163ab, 170, 173b, 191b, 192d, 193e, 194ab

The `FALLTHROUGH` macro marks intentional switch case fall-throughs as a statement rather than a comment. GCC 7+ and Clang recognise `__attribute__((fallthrough))` and suppress `-Wimplicit-fallthrough` warnings; other compilers see a harmless `((void)0)`. The prose surrounding each switch already explains why the fall-through is intended, so no inline comment is needed.

Here we define the maximum length for names (of fragments etc), and the version string reported by `--version`. The version is a hard-coded literal — the single source of truth for weft's own version — so that *any* build from a fresh checkout carries the correct version, whether it is compiled with `make bootstrap`, re-tangled with plain `make`, or released with `make dist`. Cutting a release is therefore a one-line edit here. (The `-V` flag and the `@v` directive remain a separate, user-facing feature for stamping a version into *other* documents woven by weft; they no longer drive weft's own version.)

< Limits 23b > ≡

```
#ifndef MAX_NAME_LEN
#define MAX_NAME_LEN 1024
#endif

#define WEFT_VERSION "< version 2 >"
```

□

23b · used in 21a · defines `MAX_NAME_LEN` 140c, 143b, 147a, 173a, 176b, 179d, 181, 190b, 195a, `WEFT_VERSION` 28, 30, 31

2.1.1 The Main Files

The code is divided into four main files (introduced here) and five support files (introduced in the next section). The file `main.c` will contain the driver for the whole program (see Section 2.2).

< main.c 23c > ≡

```
#include "global.h"
```

□

23c · defined at 23c, 25f

The first pass over the source file is contained in `pass1.c`. It handles collection of all the file names, fragments names, and scraps (see Section 2.3).

```
<pass1.c 24a> ≡  
#include "global.h"  
□  
24a · defined at 24a, 42bc
```

The `.tex` file is created during a second pass over the source file. The file `latex.c` contains the code controlling the construction of the `.tex` file (see Section 2.4).

```
<latex.c 24b> ≡  
#include "global.h"  
static int scraps = 1;  
□  
24b · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · uses scraps 134b
```

The Markdown output is created by the code in `markdown.c` (see Section 2.5).

```
<markdown.c 24c> ≡  
#include "global.h"  
static int scraps = 1;  
□  
24c · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · uses scraps 134b
```

The HTML output is created by the code in `html.c` (see Section 2.6).

```
<html.c 24d> ≡  
#include "global.h"  
static int scraps = 1;  
□  
24d · defined at 24d, 103c, 104, 109ac, 110b · uses scraps 134b
```

The code controlling the creation of the output files is in `output.c` (see Section 2.8).

```
<output.c 24e> ≡  
#include "global.h"  
□  
24e · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a
```

2.1.2 Support Files

The support files contain a variety of support routines used to define and manipulate the major data abstractions. The file `input.c` holds all the routines used for referring to source files (see Section 3.1).

```
< input.c 25a > ≡  
#include "global.h"
```

□

25a · defined at 25a, 129d, 130abd, 133b

Creation and lookup of scraps is handled by routines in `scraps.c` (see Section 3.2).

```
< scraps.c 25b > ≡  
#include "global.h"
```

□

25b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b

The handling of file names and fragment names is detailed in `names.c` (see Section 3.3).

```
< names.c 25c > ≡  
#include "global.h"
```

□

25c · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a

Memory allocation and deallocation is handled by routines in `arena.c` (see Section 3.6).

```
< arena.c 25d > ≡  
#include "global.h"
```

□

25d · defined at 25d, 197cde, 199

Finally, for best portability, I seem to need a file containing (useless!) definitions of all the global variables.

```
< global.c 25e > ≡  
#include "global.h"  
< Operating System Dependencies 26 >  
< Global variable definitions 28, ... >
```

□

25e · defined at 25e, 196d

2.2 The Main Routine

The main routine is quite simple in structure. It wades through the optional command-line arguments, then handles any files listed on the command line.

```
< main.c 25f > + ≡  
  
#include <stdlib.h>  
int main(int argc, char **argv)  
{  
    int arg = 1;  
    < Interpret command-line arguments 29e, ... >  
    < Set locale information 37a >
```

```

    < Handle reverse map mode 36e >
    initialise_delimit_scrap_array();
    < Process the remaining arguments (file names) 37b >
    exit(0);
}
□

```

25f · defined at 23c, 25f · defines main 200 · uses exit 21b

We only have two major operating system dependencies; the separators for file names, and how to set environment variables. For now we assume the latter can be accomplished via `putenv()` in `stdlib.h`.

< Operating System Dependencies 26 > ≡

```

#if defined(VMS)
#define PATH_SEP(c) (c=='|' || c==':')
#define PATH_SEP_CHAR ""
#define DEFAULT_PATH ""
#elif defined(MSDOS)
#define PATH_SEP(c) (c=='\\')
#define PATH_SEP_CHAR "\\\"
#define DEFAULT_PATH "."
#else
#define PATH_SEP(c) (c=='/')
#define PATH_SEP_CHAR "/"
#define DEFAULT_PATH "."
#endif
□

```

26 · used in 21a, 25e

2.2.1 Command-Line Arguments

There are numerous possible command-line arguments:

- t Suppresses generation of the `.tex` file.
- o Suppresses generation of the output files.
- d list dangling identifier references in indexes.
- c Forces output files to overwrite old files of the same name without comparing for equality first.
- v The verbose flag. Forces output of progress reports.
- n Forces sequential numbering of scraps (instead of page numbers).
- s Doesn't print list of scraps making up file at end of each scrap.
- x Include cross-reference numbers in scrap comments.
- p *path* Prepend *path* to the filenames for all the output files.
- mkdirs Create output directories as needed (a recursive `mkdir -p`) before writing each tangled file — both the `-p` prefix and any subdirectory embedded in an `@o` name. Without it, the directories must already exist (the strict default, which catches mistyped paths); a missing directory is then a hard error.
- h *options* Provide options for the hyperref package.
- r Turn on hyperlinks. You must include the `—usepackage—` options in the text.

- 1 Deprecated no-op. Code scraps are always formatted with the `listings` package now, so this flag has no effect and is kept only for backward compatibility. Remember to `\usepackage{listings}` in your document preamble.

There are two ways to get hyper-links into your documentation. With one, the `-r` flag simply arranges for fragment cross-references to be hyperlinks. You have to provide all the rest of the `hyperref` material yourself. The `-h` flag does more work, but requires you provide the `hyperref` options on the command line. You would use the first if you know you will use the same tools all the time. You would use the second if you have to distribute the weft file and use the build system to provide the `hyperref` options.

The `-h` option is used as follows. If you want hyperlinks in your document give a string to this option which is a valid option for the `hyperref` package. For example, "`dvips,colorlinks=true`" sets it to use the `dvips` driver and mark the links in colour. For more information about these options, see the `hyperref` documentation.

Then in your document you put the command `\WEFTuseHyperlinks` immediately after your `usepackage` commands. If you do both of these you will get hyperlinks. If you miss out either then nothing interesting happens.

Global flags are declared for each of the arguments.

< Global variable declarations 27 > ≡

```
extern int tex_flag; /* if FALSE, don't emit the documentation file */
extern int output_flag; /* if FALSE, don't emit the output files */
extern int compare_flag; /* if FALSE, overwrite without comparison */
extern int verbose_flag; /* if TRUE, write progress information */
extern int number_flag; /* if TRUE, use a sequential numbering scheme */
extern int scrap_flag; /* if FALSE, don't print list of scraps */
extern int dangling_flag; /* if FALSE, don't print dangling identifiers */
extern int xref_flag; /* If TRUE, print cross-references in scrap comments */
extern int prepend_flag; /* If TRUE, prepend a path to the output file names */
extern char * dirpath; /* The prepended directory path */
extern char * path_sep; /* How to join path to filename */
extern int listings_flag; /* if TRUE, use listings package for scrap formatting */
extern int version_info_flag; /* If TRUE, set up version string */
extern char * version_string; /* What to print for @v */
extern int hyperref_flag; /* Are we preparing for hyperref
                           package. */
extern int hyperopt_flag; /* Are we preparing for hyperref options */
extern char * hyperoptions; /* The options to pass to the
                             hyperref package */
extern int includepath_flag; /* Do we have an include path? */
extern struct incl{char * name; struct incl * next;} * include_list;
/* The list of include paths */
extern int markdown_flag; /* if TRUE, generate Markdown instead of LaTeX */
extern int weave_flag; /* if TRUE, weave was requested via -w */
extern int weave_format; /* 0=none, 1=tex, 2=md, 3=html */
extern int weave_format_from_cli; /* if TRUE, CLI override of @W */
extern int map_flag; /* if TRUE, emit JSON map to stdout */
extern int extract_flag; /* if TRUE, next arg is extract name */
extern char * extract_name; /* fragment name to extract */
extern int reverse_map_flag; /* if TRUE, reverse map mode */
extern char * reverse_map_arg; /* file[:line] argument for -R */
extern int bodies_flag; /* if TRUE, include scrap bodies in -m JSON */
extern int prose_flag; /* if TRUE, include prose context in -m JSON */
extern int diff_flag; /* if TRUE, show diff instead of writing files */
extern int callers_flag; /* if TRUE, show callers with -e */
extern int lint_flag; /* if TRUE, run static analysis */
extern int errors_flag; /* if TRUE, annotate compiler errors from stdin */
extern int mkdirs_flag; /* if TRUE, create output directories as needed */
```

□

27 · defined at 27, 29ac, 45c, 50a, 66b, 129b, 137e, 158b, 197a · used in 21a · defines `bodies_flag` 28, 30, 111b, `callers_flag` 28, 30, 39b, `compare_flag` 28, 34, 127b, `dangling_flag` 28, 34, 82c, 100b, 110b, `diff_flag` 28, 30, 127b, `errors_flag` 28, 30, 36e, `extract_flag` 28, 34, 36c, `extract_name` 28, 36c, 39b, `hyperoptions` 28, 36b, 56b, 58a, `hyperopt_flag` 28, 34, 36b, `hyperref_flag` 28, 34, 36b, 56b, `incl` 28, 35b, 39a, 131b, `includepath_flag` 28, 34, 35b, `lint_flag` 28, 30, 39b, `map_flag` 28, 34, 39b, `markdown_flag` 28, 34, 39b, `makedirs_flag` 28, 30, 126b, 128b, `number_flag` 28, 34, 39b, 156b, `output_flag` 28, 34, 39b, `prose_flag` 28, 30, 42d, 43b, 111b, `reverse_map_arg` 28, 36de, `reverse_map_flag` 28, 34, 36d, `scrap_flag` 28, 34, 62, 90a, 108b, `tex_flag` 28, 34, 39b, 42c, 152e, `verbose_flag` 28, 34, 42c, 56a, 84d, 104, 126b, `version_info_flag` 28, 34, 36a, `version_string` 28, 36a, 60b, 74a, 87a, 106a, `weave_flag` 28, 34, 39b, `weave_format` 28, 34, 39b, 172, `weave_format_from_cli` 28, 34, 39b, 172, `xref_flag` 28, 34, 153, 154b · uses `FALSE` 23a, `next` 133c, `prose` 134a, `reverse` 183a, `scraps` 134b, `TRUE` 23a

The flags are all initialized for correct default behavior.

⟨ *Global variable definitions* 28 ⟩ ≡

```
int tex_flag = TRUE;
int output_flag = TRUE;
int compare_flag = TRUE;
int verbose_flag = FALSE;
int number_flag = FALSE;
int scrap_flag = TRUE;
int dangling_flag = FALSE;
int xref_flag = FALSE;
int prepend_flag = FALSE;
char * dirpath = DEFAULT_PATH; /* Default directory path */
char * path_sep = PATH_SEP_CHAR;
int listings_flag = FALSE;
int version_info_flag = FALSE;
char default_version_string[] = WEFT_VERSION;
char * version_string = default_version_string;
int hyperref_flag = FALSE;
int hyperopt_flag = FALSE;
char * hyperoptions = "";
int includepath_flag = FALSE; /* Do we have an include path? */
struct incl * include_list = NULL;
/* The list of include paths */
int markdown_flag = FALSE;
int weave_flag = FALSE;
int weave_format = 0;
int weave_format_from_cli = FALSE;
int map_flag = FALSE;
int extract_flag = FALSE;
char *extract_name = NULL;
int reverse_map_flag = FALSE;
char *reverse_map_arg = NULL;
int bodies_flag = FALSE;
int prose_flag = FALSE;
int diff_flag = FALSE;
int callers_flag = FALSE;
int lint_flag = FALSE;
int errors_flag = FALSE;
int mkdirs_flag = FALSE;
```

□

28 · defined at 28, 29bd, 46a, 50b, 66c, 129c, 137f, 158c · used in 25e · uses `bodies_flag` 27, `callers_flag` 27, `compare_flag` 27, `dangling_flag` 27, `diff_flag` 27, `errors_flag` 27, `extract_flag` 27, `extract_name` 27, `FALSE` 23a, `hyperoptions` 27, `hyperopt_flag` 27, `hyperref_flag` 27, `incl` 27, `includepath_flag` 27, `lint_flag` 27, `map_flag` 27, `markdown_flag` 27, `makedirs_flag` 27, `number_flag` 27, `output_flag` 27, `prose_flag` 27, `reverse_map_arg` 27, `reverse_map_flag` 27, `scrap_flag` 27, `tex_flag` 27, `TRUE` 23a, `verbose_flag` 27, `version_info_flag` 27, `version_string` 27,

[weave_flag 27](#), [weave_format 27](#), [weave_format_from_cli 27](#), [WEFT_VERSION 23b](#), [xref_flag 27](#)

A global variable `nw_char` will be used for the weft meta-character, which by default will be `@`.

⟨ Global variable declarations 29a ⟩ + ≡

```
extern int nw_char;
```

□

29a · defined at [27](#), [29ac](#), [45c](#), [50a](#), [66b](#), [129b](#), [137e](#), [158b](#), [197a](#) · used in [21a](#) · defines `nw_char` [29b](#), [31](#), [39b](#), [42d](#), [44d](#), [47ab](#), [48c](#), [49c](#), [50d](#), [51c](#), [53](#), [54ab](#), [59bc](#), [62](#), [68a](#), [70c](#), [71b](#), [72a](#), [73b](#), [74d](#), [75a](#), [78c](#), [82c](#), [85a](#), [86a](#), [91b](#), [93a](#), [94b](#), [105c](#), [106a](#), [130b](#), [131a](#), [133b](#), [139ab](#), [140ac](#), [142ab](#), [143b](#), [147a](#), [148c](#), [151ac](#), [152de](#), [154c](#), [155ac](#), [160b](#), [170](#), [172](#), [173a](#), [176b](#), [177ab](#), [178d](#), [179d](#), [180](#), [181](#), [190ab](#), [192d](#), [193e](#), [194a](#), [195a](#)

⟨ Global variable definitions 29b ⟩ + ≡

```
int nw_char='@';
```

□

29b · defined at [28](#), [29bd](#), [46a](#), [50b](#), [66c](#), [129c](#), [137f](#), [158c](#) · used in [25e](#) · defines `nw_char` [29a](#), [31](#), [39b](#), [42d](#), [44d](#), [47ab](#), [48c](#), [49c](#), [50d](#), [51c](#), [53](#), [54ab](#), [59bc](#), [62](#), [68a](#), [70c](#), [71b](#), [72a](#), [73b](#), [74d](#), [75a](#), [78c](#), [82c](#), [85a](#), [86a](#), [91b](#), [93a](#), [94b](#), [105c](#), [106a](#), [130b](#), [131a](#), [133b](#), [139ab](#), [140ac](#), [142ab](#), [143b](#), [147a](#), [148c](#), [151ac](#), [152de](#), [154c](#), [155ac](#), [160b](#), [170](#), [172](#), [173a](#), [176b](#), [177ab](#), [178d](#), [179d](#), [180](#), [181](#), [190ab](#), [192d](#), [193e](#), [194a](#), [195a](#)

We save the invocation name of the command in a global variable `command_name` for use in error messages.

⟨ Global variable declarations 29c ⟩ + ≡

```
extern char *command_name;
```

□

29c · defined at [27](#), [29ac](#), [45c](#), [50a](#), [66b](#), [129b](#), [137e](#), [158b](#), [197a](#) · used in [21a](#) · defines `command_name` [29de](#), [31](#), [34](#), [37b](#), [39b](#), [44d](#), [47ab](#), [56a](#), [61](#), [67c](#), [73a](#), [75c](#), [77](#), [84d](#), [90e](#), [94d](#), [95b](#), [104](#), [116ab](#), [127a](#), [128b](#), [131ab](#), [132](#), [133b](#), [137d](#), [139ab](#), [140c](#), [147a](#), [148b](#), [152e](#), [160b](#), [170](#), [172](#), [173ab](#), [174a](#), [176b](#), [177a](#), [178cd](#), [179d](#), [180](#), [190b](#)

⟨ Global variable definitions 29d ⟩ + ≡

```
char *command_name = NULL;
```

□

29d · defined at [28](#), [29bd](#), [46a](#), [50b](#), [66c](#), [129c](#), [137f](#), [158c](#) · used in [25e](#) · uses `command_name` [29c](#)

The invocation name is conventionally passed in `argv[0]`.

⟨ Interpret command-line arguments 29e ⟩ ≡

```
command_name = argv[0];
```

□

29e · defined at [29ef](#) · used in [25f](#) · uses `command_name` [29c](#)

We need to examine the remaining entries in `argv`, looking for command-line arguments.

⟨ Interpret command-line arguments 29f ⟩ + ≡

```
while (arg < argc) {
    char *s = argv[arg];
    if (*s++ == '-') {
        ⟨ Check for long options 30 ⟩
        ⟨ Interpret the argument string s 34 ⟩
        arg++;
        ⟨ Perhaps get the prepend path 35a ⟩
        ⟨ Perhaps get the version info string 36a ⟩
        ⟨ Perhaps get the hyperref options 36b ⟩
        ⟨ Perhaps add an include path 35b ⟩
    }
}
```

```

    < Perhaps get the extract name 36c >
    < Perhaps get the reverse map argument 36d >
}
else break;
}
□

```

29f · defined at 29ef · used in 25f

2.2.2 Help Message

When the user passes `--help`, we display a comprehensive reference that serves both human users and AI assistants. The help text is structured in sections—usage, options grouped by purpose, examples, and an AI workflow summary—so that a language model can parse it and use `weft` correctly without additional context.

We intercept long options before the character-by-character flag parsing. Currently `--help` and `--version` are the only long options.

```

< Check for long options 30 > ≡
if (*s == '-') {
    if (strcmp(s + 1, "help") == 0) {
        < Print help message 31 >
        exit(0);
    }
    if (strcmp(s + 1, "version") == 0) {
        printf("weft %s\n", WEFT_VERSION);
        exit(0);
    }
    if (strcmp(s + 1, "bodies") == 0) {
        bodies_flag = TRUE;
        arg++;
        continue;
    }
    if (strcmp(s + 1, "prose") == 0) {
        prose_flag = TRUE;
        arg++;
        continue;
    }
    if (strcmp(s + 1, "diff") == 0) {
        diff_flag = TRUE;
        arg++;
        continue;
    }
    if (strcmp(s + 1, "callers") == 0) {
        callers_flag = TRUE;
        arg++;
        continue;
    }
    if (strcmp(s + 1, "lint") == 0) {
        lint_flag = TRUE;
        arg++;
        continue;
    }
    if (strcmp(s + 1, "errors") == 0) {
        errors_flag = TRUE;
        arg++;
        continue;
    }
    if (strcmp(s + 1, "mkdirs") == 0) {

```

```

    mkdirs_flag = TRUE;
    arg++;
    continue;
}
}
□

```

30 · used in 29f · uses [bodies_flag 27](#), [callers_flag 27](#), [diff_flag 27](#), [errors_flag 27](#), [exit 21b](#), [lint_flag 27](#), [mkdirs_flag 27](#), [prose 134a](#), [prose_flag 27](#), [TRUE 23a](#), [WEFT_VERSION 23b](#)

The help message itself. We write to `stdout` (not `stderr`) because `--help` is a deliberate request, not an error condition. The output is structured so that both humans and AI assistants can use `weft` effectively from `--help` alone: options grouped by function, per-file output flags, section marker format, a complete directives reference, and a concise AI workflow section.

⟨ Print help message 31 ⟩ ≡

```

printf("weft %s -- A Literate Programming Tool\n\n", WEFT_VERSION);
printf("USAGE\n");
printf(" %s [options] file...\n\n", command_name);
printf("WHAT IS WEFT?\n");
printf(" weft is a literate programming tool: write .weft files where\n");
printf(" documentation and code live as one narrative. weft TANGLES them\n");
printf(" into executable output files and optionally WEAVES formatted\n");
printf(" documentation (LaTeX or Markdown). .weft files are the ONLY\n");
printf(" source of truth; tangled outputs are GENERATED -- never edit them.\n\n");
printf("HOW IT WORKS\n");
printf(" A .weft file contains prose with embedded code chunks:\n");
printf("  @o file.c @{ ... @}      Output file: tangle writes this to file.c\n");
printf("  @d Name   @{ ... @}      Fragment: reusable code block, invoked by name\n");
printf("  @<Name>   Invocation: inserts the named fragment here\n");
printf("  @@       Literal @: escapes the meta-character\n");
printf("  @i other.weft Include: pulls in another .weft file\n\n");
printf(" Fragments compose hierarchically. An @o chunk may invoke @d fragments,\n");
printf(" which invoke other fragments, forming a tree that weft flattens into\n");
printf(" the final output file.\n\n");
printf("OPTIONS\n");
printf(" Tangle control:\n");
printf("  -o          Suppress output file generation (parse only)\n");
printf("  -c          Overwrite files without comparing first\n");
printf("  -p path     Prepend path to all output file names\n");
printf("  --mkdirs    Create output directories as needed (mkdir -p)\n");
printf("  -I path     Add directory to include search path (repeatable)\n");
printf("  -x          Include cross-reference numbers in section markers\n\n");
printf(" Weave control:\n");
printf("  -w [format] Weave documentation (format: md, tex, or html)\n");
printf("              Without format, uses @W directive from source\n");
printf("  -t          Suppress weave output (even if -w given)\n");
printf("  -n          Sequential scrap numbering (auto for Markdown)\n");
printf("  -s          Omit scrap composition lists\n\n");
printf(" Analysis (AI-friendly tools):\n");
printf("  -m          Emit JSON structure map to stdout (all fragments,\n");
printf("              files, definitions, and dependency graph)\n");
printf("  -e name     Extract named fragment and all its transitive\n");
printf("              dependencies as self-contained Markdown to stdout\n");
printf("  -R file[:line] Reverse map: given a tangled output file (and\n");
printf("              optional line number), prints the .weft source\n");
printf("              location that generated it. Needs NO .weft files\n");
printf("  --bodies    -- reads section markers embedded in the output\n");
printf("              Include raw scrap body text in -m JSON output\n");

```

```

printf("    --prose          Include preceding documentation in -m JSON output\n");
printf("    --diff           Show unified diff of what tangle would change\n");
printf("                    (no files written)\n");
printf("    --lint           Static analysis: unused fragments, undefined refs,\n");
printf("                    and fuzzy-match suggestions for likely typos\n");
printf("    --callers        With -e, show call chain from %co roots to fragment\n",
    nw_char);
printf("    --errors         Annotate compiler errors from stdin with .weft\n");
printf("                    source locations (reads section markers, no\n");
printf("                    .weft files needed)\n");
printf("                    Example: make 2>&1 | weft --errors\n\n");
printf("    LaTeX:\n");
printf("    -r              Enable hyperref cross-reference links\n");
printf("    -h options      Pass options to hyperref package\n");
printf("    -l             Deprecated no-op (listings is always used)\n");
printf("    -d             List dangling identifier references\n");
printf("    General:\n");
printf("    -V string       Set version string (expands @v in source)\n");
printf("    -v             Verbose progress output\n");
printf("    --help         Show this help message\n");
printf("    --version      Show version number\n");
printf("PER-FILE FLAGS (on @o)\n");
printf("    @o file [-d] [-i] [-t] [-s] [-cc|-c+|-cp] @{ ... @} \n");
printf("    -d             Emit #line N \"file\" directives (C/C++ debuggers)\n");
printf("    -i             Suppress auto-indentation of expanded fragments\n");
printf("    -t             Suppress tab expansion (write tabs literally)\n");
printf("    -cc           C block-comment markers: /* {N: file:line} */\n");
printf("    -c+          C++ line-comment markers: // {N: file:line}\n");
printf("    -cp          Hash-comment markers: # {N: file:line}\n");
printf("    -s           Suppress all section markers in output\n");
printf("    Without explicit flags, weft auto-detects from file extension\n");
printf("    (250+ extensions recognized). Use @l tag before @o to override.\n\n");
printf("SECTION MARKERS\n");
printf("    When comment style is known, tangled output includes markers:\n");
printf("    /* {5: source.weft:42} */ opening -- scrap 5, from line 42\n");
printf("    /* {5} */ closing\n");
printf("    Comment adapts to language: // for C++/Go, # for Python/shell, etc.\n");
printf("    Use weft -R to parse these markers and reverse-map to source.\n\n");
printf("DIRECTIVES REFERENCE\n");
printf("    Core structure:\n");
printf("    @o file [flags] @{ code @} Output file (tangled to disk)\n");
printf("    @d Name @{ code @} Define reusable named fragment\n");
printf("    @d+ Name @{ code @} Global fragment (visible across sectors)\n");
printf("    @q Name @{ code @} Quoted fragment (no expansion on include)\n");
printf("    @Q / @D / @Q Uppercase = big-format scrap in LaTeX\n");
printf("    @<Name@> Invoke (expand) a named fragment\n");
printf("    @<+Name@> Invoke the global version of a fragment\n");
printf("    @i file Include another .weft file\n");
printf("    @@ Literal @ character\n");
printf("    Fragment parameters:\n");
printf("    @'text@' Pass literal text argument to a fragment\n");
printf("    @1 .. @9 Substitute nth argument inside fragment\n");
printf("    @{ code @} as argument Pass anonymous inline scrap as argument\n\n");
printf("    Inside scraps:\n");
printf("    @f Current output file name\n");
printf("    @t Current fragment title (as invoked)\n");
printf("    @v Version string (defaults to weft's version; -V\n    overrides)\n");
printf("    @c Expand block comment at current indent\n");

```

```

printf("    @#                Suppress auto-indent for this line\n");
printf("    @%%              Comment to end of line (omitted)\n");
printf("    @| name            Mark identifier(s) as defined here\n");
printf("    @+ @- @*           Global-export / import / used markers\n");
printf("    @_ text @_         Bold (LaTeX weave); skipped in tangle\n");
printf("    @x label @x        Cross-reference label (anchor position)\n\n");
printf("Scrap delimiters (LaTeX weave):\n");
printf("    @{ ... @}          Verbatim code (default)\n");
printf("    @[ ... @]          Paragraph mode\n");
printf("    @( ... @)          Math mode\n\n");
printf("Sections and structure:\n");
printf("    @s                Start a new sector (section boundary)\n");
printf("    @S                Close current sector (return to base)\n");
printf("    @W fmt            Declare weave format: md, tex, or html\n\n");
printf("Language and formatting:\n");
printf("    @l tag             Set language for next @o or @d\n");
printf("    @L name style [+d] Define custom language (// # -- /* <!--)\n");
printf("    @r x               Change escape character from @ to x\n");
printf("Indexes (weave output only):\n");
printf("    @f @m @u          File / fragment / identifier indexes\n");
printf("    @m+ @u+           Global fragment / identifier indexes\n\n");
printf("EXAMPLES\n");
printf("    %s project.weft      Tangle (default)\n", command_name);
printf("    %s -w md project.weft Weave to Markdown\n", command_name);
printf("    %s -w tex project.weft Weave to LaTeX\n", command_name);
printf("    %s -w html project.weft Weave to HTML\n", command_name);
printf("    %s -m project.weft | jq '.files' JSON structure map\n", command_name);
printf("    %s -e \"\nValidate\" project.weft Extract fragment\n", command_name);
printf("    %s -R server.js:47 Reverse map line\n", command_name);
printf("    %s -R server.js All regions in file\n", command_name);
printf("    %s -v -p build/ project.weft Verbose + path prefix\n", command_name);
printf("    %s -o project.weft Parse only, no output\n", command_name);
);
printf("AI ASSISTANT WORKFLOW\n");
printf("GOLDEN RULE: .weft files are the ONLY source of truth.\n");
printf("Tangled outputs (*.c, *.go, etc.) are GENERATED -- never edit them.\n");
printf("If you edit a tangled file, your changes WILL be lost on next tangle.\n\n");
printf("Step 1 -- Understand the project structure:\n");
printf("    %s -m project.weft > map.json\n", command_name);
printf("    The JSON contains: files (all @o output files), fragments (all @d\n");
printf("    named blocks), and a dependency graph (which fragments invoke which).\n");
printf("    Useful jq queries:\n");
printf("        jq '.files'                List output files\n");
printf("        jq '.fragments'            List all fragments\n");
printf("        jq '.fragments[] | select(.name | test(\"Auth\"))' Search\n");
printf("Step 2 -- Read specific code without opening .weft files:\n");
printf("    %s -e \"\nfragment name\" project.weft\n", command_name);
printf("    Prints the fragment AND all its transitive dependencies as\n");
printf("    self-contained Markdown to stdout. Ideal for understanding\n");
printf("    one piece of the system without reading the whole source.\n\n");
printf("Step 3 -- Edit the .weft source (NEVER tangled output):\n");
printf("    Locate the right .weft file (use -m to find it), then edit\n");
printf("    the @d or @o scrap that contains the code you need to change.\n");
printf("    Use standard text editing -- the prose around code is LaTeX or\n");
printf("    plain text, and code lives between @{ and @}.\n\n");
printf("Step 4 -- Regenerate and verify:\n");
printf("    %s project.weft      Tangle (regenerate all output files)\n",
    command_name);
printf("    Then build and test as usual.\n\n");

```

```

printf(" Step 5 -- Debug errors with reverse map:\n");
printf("    %s -R output.c:42          Find .weft source for line 42\n", command_name);
printf("    %s -R output.c              Show ALL .weft regions in file\n", command_name);
printf("    Reads section markers in the tangled output -- needs NO .weft files.\n");
printf("    Fix errors in the .weft source, then re-tangle (step 4).\n");

```

□

31 · used in 30 · uses [command_name 29c](#), [FILE 21b](#), [first 197d](#), [Name 158a](#), [next 133c](#), [nw_char 29ab](#), [prose 134a](#), [reverse 183a](#), [scraps 134b](#), [search 187c](#), [sector 134a](#), [uses 134a](#), [WEFT_VERSION 23b](#)

2.2.3 Short Flags

Several flags can be stacked behind a single minus sign; therefore, we've got to loop through the string, handling them all. If this flag requires an argument we skip to getting its argument straight away. This allows arguments to butt up to their flags and also avoids ambiguity about which value goes with which flag.

⟨ *Interpret the argument string s* ³⁴ ⟩ ≡

```

{
    char c = *s++;
    while (c) {
        switch (c) {
            case 'c': compare_flag = FALSE;
                       break;
            case 'd': dangling_flag = TRUE;
                       break;
            case 'h': hyperopt_flag = TRUE;
                       goto HasValue;
            case 'I': includepath_flag = TRUE;
                       goto HasValue;
            case 'l': listings_flag = TRUE;
                       break;
            case 'n': number_flag = TRUE;
                       break;
            case 'o': output_flag = FALSE;
                       break;
            case 'p': prepend_flag = TRUE;
                       goto HasValue;
            case 'r': hyperref_flag = TRUE;
                       break;
            case 's': scrap_flag = FALSE;
                       break;
            case 't': tex_flag = FALSE;
                       break;
            case 'e': extract_flag = TRUE;
                       goto HasValue;
            case 'R': reverse_map_flag = TRUE;
                       goto HasValue;
            case 'm': map_flag = TRUE;
                       break;
            case 'v': verbose_flag = TRUE;
                       break;
            case 'V': version_info_flag = TRUE;
                       goto HasValue;
            case 'w': goto HasWeaveFormat;
            case 'x': xref_flag = TRUE;
                       break;
            default: fprintf(stderr, "%s: unexpected argument ignored. "
                           "Try '%s --help' for usage information.\n",
                           command_name, command_name);

```

```

        break;
    }
    c = *s++;
}
goto HasValue;
HasWeaveFormat:
{
    char *fmt = NULL;
    weave_flag = TRUE;
    if (*s != '\0')
        fmt = s;
    else if (arg + 1 < argc && argv[arg + 1][0] != '-') {
        /* Peek: only consume next arg if it looks like a format token */
        char *candidate = argv[arg + 1];
        if (strcmp(candidate, "md") == 0 || strcmp(candidate, "tex") == 0
            || strcmp(candidate, "html") == 0) {
            fmt = candidate;
            ++arg;
        }
    }
    if (fmt != NULL) {
        if (strcmp(fmt, "md") == 0) {
            weave_format = 2;
            markdown_flag = TRUE;
            weave_format_from_cli = TRUE;
        } else if (strcmp(fmt, "tex") == 0) {
            weave_format = 1;
            weave_format_from_cli = TRUE;
        } else if (strcmp(fmt, "html") == 0) {
            weave_format = 3;
            weave_format_from_cli = TRUE;
        } else {
            fprintf(stderr,
                "%s: unknown weave format '%s'. Use 'md', 'tex', or 'html'.\\n",
                command_name, fmt);
        }
    }
    /* If fmt is NULL, -w alone: format will come from @W */
    s = "";
}
HasValue:;
}

```

□

34 · used in 29f · uses [command_name](#) 29c, [compare_flag](#) 27, [dangling_flag](#) 27, [extract_flag](#) 27, [FALSE](#) 23a, [fprintf](#) 21b, [hyperopt_flag](#) 27, [hyperref_flag](#) 27, [includepath_flag](#) 27, [map_flag](#) 27, [markdown_flag](#) 27, [next](#) 133c, [number_flag](#) 27, [output_flag](#) 27, [reverse_map_flag](#) 27, [scrap_flag](#) 27, [stderr](#) 21b, [tex_flag](#) 27, [TRUE](#) 23a, [verbose_flag](#) 27, [version_info_flag](#) 27, [weave_flag](#) 27, [weave_format](#) 27, [weave_format_from_cli](#) 27, [xref_flag](#) 27

⟨ *Perhaps get the prepend path* 35a ⟩ ≡

```

if (prepend_flag)
{
    if (*s == '\0')
        s = argv[arg++];
    dirpath = s;
    prepend_flag = FALSE;
}

```

□

35a · used in 29f · uses [FALSE](#) 23a

⟨ *Perhaps add an include path* 35b ⟩ ≡

```
if (includepath_flag)
{
    struct incl * le
        = (struct incl *)arena_getmem(sizeof(struct incl));
    struct incl ** p = &include_list;

    if (*s == '\\0')
        s = argv[arg++];
    le->name = save_string(s);
    le->next = NULL;
    while (*p != NULL)
        p = &((*p)->next);
    *p = le;
    includepath_flag = FALSE;
}
```

□

35b · used in 29f · uses arena_getmem 197e, FALSE 23a, incl 27, includepath_flag 27, next 133c, save_string 159c

⟨ *Perhaps get the version info string* 36a ⟩ ≡

```
if (version_info_flag)
{
    if (*s == '\\0')
        s = argv[arg++];
    version_string = s;
    version_info_flag = FALSE;
}
```

□

36a · used in 29f · uses FALSE 23a, version_info_flag 27, version_string 27

⟨ *Perhaps get the hyperref options* 36b ⟩ ≡

```
if (hyperopt_flag)
{
    if (*s == '\\0')
        s = argv[arg++];
    hyperoptions = s;
    hyperopt_flag = FALSE;
    hyperref_flag = TRUE;
}
```

□

36b · used in 29f · uses FALSE 23a, hyperoptions 27, hyperopt_flag 27, hyperref_flag 27, TRUE 23a

⟨ *Perhaps get the extract name* 36c ⟩ ≡

```
if (extract_flag) {
    extract_name = (*s != '\\0') ? s : argv[arg++];
    extract_flag = FALSE;
}
```

□

36c · used in 29f · uses extract_flag 27, extract_name 27, FALSE 23a

⟨ *Perhaps get the reverse map argument* 36d ⟩ ≡

```
if (reverse_map_flag) {
    reverse_map_arg = (*s != '\\0') ? s : argv[arg++];
    reverse_map_flag = FALSE;
}
```

□

36d · used in 29f · uses FALSE 23a, reverse_map_arg 27, reverse_map_flag 27

If the user gave `-R file[:line]`, we handle it immediately and exit. No `.weft` source files are required—only the tangled output file.

```
< Handle reverse map mode 36e > ≡
if (errors_flag) {
    write_error_annotator();
    exit(0);
}
if (reverse_map_arg) {
    write_reverse_map(reverse_map_arg);
    exit(0);
}
```

□

36e · used in 25f · uses errors_flag 27, exit 21b, reverse_map_arg 27, write_error_annotator 124, write_reverse_map 121

In order to be able to process files in foreign languages we set the locale information. `isgraph()` and friends need this (see Section 3.3). Try to read `LC_CTYPE` from the environment. Use `LC_ALL` if that fails. Don't set the locale if reading `LC_ALL` fails, too. Print a warning if setting the program's locale fails.

```
< Set locale information 37a > ≡
{
    /* try to get locale information */
    char *s=getenv("LC_CTYPE");
    if (s==NULL) s=getenv("LC_ALL");

    /* set it */
    if (s!=NULL)
        if(setlocale(LC_CTYPE, s)==NULL)
            fprintf(stderr, "Setting locale failed\n");
}
```

□

37a · used in 25f · uses fprintf 21b, getenv 21b, setlocale 21b, stderr 21b

2.2.4 File Names

We expect at least one file name. While a missing file name might be ignored without causing any problems, we take the opportunity to report the usage convention.

```
< Process the remaining arguments (file names) 37b > ≡
{
    if (arg >= argc) {
        fprintf(stderr, "%s: expected a file name.\n"
            "Usage: %s [options] file...\n"
            "Try '%s --help' for more information.\n",
            command_name, command_name, command_name);
        exit(-1);
    }
    do {
        < Handle the file name in argv[arg] 38a >
        arg++;
    } while (arg < argc);
}
```

□

37b · used in 25f · uses `command_name` 29c, `exit` 21b, `fprintf` 21b, `stderr` 21b

The code to handle a particular file name is rather more tedious than the actual processing of the file. A file name may be an arbitrarily complicated path name, with an optional extension. If no extension is present, we add `.w` as a default. The extended path name will be kept in a local variable `source_name`. The resulting documentation file will be written in the current directory; its name will be kept in the variable `tex_name`.

```
< Handle the file name in argv[arg] 38a > ≡
{
    char source_name[FILENAME_MAX];
    char tex_name[FILENAME_MAX];
    char aux_name[FILENAME_MAX];
    char *trim;
    char *dot;
    < Build source_name and tex_name 38b >
    < Process a file 39b >
}
```

□

38a · used in 37b · uses source_name 129b

I bump the pointer `p` through all the characters in `argv[arg]`, copying all the characters into `source_name` (via the pointer `q`).

At each slash, I update `trim` to point just past the slash in `source_name`. The effect is that `trim` will point at the file name without any leading directory specifications.

The pointer `dot` is made to point at the file name extension, if present. If there is no extension, we add `.weft` to the source name. The `tex_name` (weave output name) is computed later in “Process a file” once the weave format is known.

```
< Build source_name and tex_name 38b > ≡
{
    char *p = argv[arg];
    char *q = source_name;
    char c;
    trim = q;
    dot = NULL;
    c = *p++;
    while (c) {
        *q++ = c;
        if (PATH_SEP(c)) {
            trim = q;
            dot = NULL;
        }
        else if (c == '.')
            dot = q - 1;
        c = *p++;
    }
    < Add the source path to the include path list 39a >
    *q = '\0';
    if (!dot) {
        *q++ = '.';
        *q++ = 'w';
        *q++ = 'e';
        *q++ = 'f';
        *q++ = 't';
        *q = '\0';
    }
    /* Find trim again for tex_name computation later */
    {
        char *t = source_name;
```

```

    trim = t;
    dot = NULL;
    while (*t) {
        if (PATH_SEP(*t)) { trim = t + 1; dot = NULL; }
        else if (*t == '.') dot = t;
        t++;
    }
}
}

```

□

38b · used in 38a · uses [source_name 129b](#)

If the source file has a directory part we add that directory to the end of the search path so that the path of last resort is the same as the source path, not, as one person put it, something irrelevant like the directory weft was invoked from.

⟨ *Add the source path to the include path list* 39a ⟩ ≡

```

if (trim != source_name) {
    struct incl * le
        = (struct incl *)arena_getmem(sizeof(struct incl));
    struct incl ** p = &include_list;
    char sv = *trim;

    *trim = '\0';
    le->name = save_string(source_name);
    le->next = NULL;
    while (*p != NULL)
        p = &((*p)->next);
    *p = le;
    *trim = sv;
}

```

□

39a · used in 38b · uses [arena_getmem 197e](#), [incl 27](#), [next 133c](#), [save_string 159c](#), [source_name 129b](#)

Now that we're finally ready to process a file, it's not really too complex. We bundle most of the work into three routines `pass1` (see Section 2.3), `write_tex` (see Section 2.4), and `write_files` (see Section 2.8). After we're finished with a particular file, we must remember to release its storage (see Section 3.6).

The default behavior is to tangle only. Weaving happens only when `-w` is given. The weave format is determined by (in priority order): a CLI argument (`-w md` or `-w tex`), or the `@W` directive in the source file. If `-w` is given without a format and no `@W` is present, weft reports an error and exits. The `-t` flag suppresses weaving even if `-w` was given.

⟨ *Process a file* 39b ⟩ ≡

```

{
    int do_weave = FALSE;
    int effective_format = 0;

    /* Reset weave_format for this file unless CLI locked it */
    if (!weave_format_from_cli)
        weave_format = 0;

    pass1(source_name);
    current_sector = 1;
    prev_sector = 1;

    if (lint_flag) {
        write_lint(file_names, macro_names);
    }
}

```

```

    /* skip weave and tangle */
} else if (map_flag) {
    write_map(file_names, macro_names);
    /* skip weave and tangle */
} else if (extract_name) {
    if (callers_flag)
        write_callers(extract_name, file_names, macro_names);
    else
        write_extract(extract_name, file_names, macro_names);
    /* skip weave and tangle */
} else {
    /* Determine whether to weave and with which format */
    if (weave_flag && !tex_flag) {
        /* -t suppresses weave: tex_flag==FALSE means user passed -t */
    } else if (weave_flag) {
        effective_format = weave_format;
        if (effective_format == 0) {
            fprintf(stderr, "%s: -w requires a weave format.\n"
                "   Use -w md, -w tex, or add %cW md/%cW tex to the source file.\n",
                command_name, nw_char, nw_char);
            exit(-1);
        }
        do_weave = TRUE;
    }

    if (do_weave) {
        /* Compute weave output name now that format is known */
        const char *weave_ext = (effective_format == 2) ? ".md"
            : (effective_format == 3) ? ".html" : ".tex";

        if (dot) {
            *dot = '\\0';
            sprintf(tex_name, "%s%s%s%s", dirpath, path_sep, trim, weave_ext);
            sprintf(aux_name, "%s%s%s.s.aux", dirpath, path_sep, trim);
            *dot = '.';
        } else {
            sprintf(tex_name, "%s%s%s%s", dirpath, path_sep, trim, weave_ext);
            sprintf(aux_name, "%s%s%s.s.aux", dirpath, path_sep, trim);
        }

        if (effective_format == 2) {
            markdown_flag = TRUE;
            number_flag = TRUE;
            collect_numbers(aux_name);
            write_md(source_name, tex_name, 0);
        } else if (effective_format == 3) {
            number_flag = TRUE;
            collect_numbers(aux_name);
            write_html(source_name, tex_name, 0);
        } else {
            collect_numbers(aux_name);
            write_tex(source_name, tex_name, 0);
        }
    }

    if (output_flag)
        write_files(file_names);
}
arena_free();
}

```

□

39b · used in 38a · uses arena_free 199, callers_flag 27, collect_numbers 156b, command_name 29c, current_sector 45c, exit 21b, extract_name 27, FALSE 23a, file_names 158b, fprintf 21b, lint_flag 27, macro_names 158b, map_flag 27, markdown_flag 27, number_flag 27, nw_char 29ab, output_flag 27, pass1 42c, prev_sector 45c, source_name 129b, stderr 21b, tex_flag 27, TRUE 23a, weave_flag 27, weave_format 27, weave_format_from_cli 27, write_callers 116a, write_extract 116b, write_files 125c, write_html 104, write_lint 118b, write_map 113a, write_md 84d, write_tex 56a

2.3 Pass One

During the first pass, we scan the file, recording the definitions of each fragment and file and accumulating all the scraps.

< Function prototypes 42a > ≡

```
extern void pass1(char *file_name);
```

□

42a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses file_name 134a, pass1 42c

The routine `pass1` takes a single argument, the name of the source file. It opens the file, then initializes the scrap structures (see Section 3.2) and the roots of the file-name tree, the fragment-name tree, and the tree of user-specified index entries (see Section 3.3). After completing all the necessary preparation, we make a pass over the file, filling in all our data structures. Next, we search all the scraps for references to the user-specified index entries. Finally, we must reverse all the cross-reference lists accumulated while scanning the scraps.

< pass1.c 42b > + ≡

```
static char prose_buf[4096];
static int prose_len = 0;
static char *pending_prose = NULL;
```

□

42b · defined at 24a, 42bc · defines pending_prose 42c, 43b, 46bc, prose_buf 43ab, prose_len 42c, 43ab

< pass1.c 42c > + ≡

```
void pass1(char *file_name)
{
    if (verbose_flag)
        fprintf(stderr, "reading %s\n", file_name);
    source_open(file_name);
    init_scraps();
    macro_names = NULL;
    file_names = NULL;
    user_names = NULL;
    prose_len = 0;
    pending_prose = NULL;
    < Scan the source file, looking for at-sequences 42d >
    if (tex_flag)
        search();
    < Reverse cross-reference lists 52a >
}
```

□

42c · defined at 24a, 42bc · defines pass1 39b, 42a, 73b, 93a · uses file_name 134a, file_names 158b, fprintf 21b, init_scraps 135b, macro_names 158b, pending_prose 42b, prose_len 42b, search 187c, source_open 133b, stderr 21b, tex_flag 27, user_names 158b, verbose_flag 27

The only thing we look for in the first pass are the command sequences. All ordinary text is skipped entirely.

< Scan the source file, looking for at-sequences 42d > ≡

```
{
    int c = source_get();
    while (c != EOF) {
        if (c == nw_char)
```

```

    < Scan at-sequence 44d >
    else if (prose_flag) {
        < Accumulate prose character 43a >
    }
    c = source_get();
}
}

```

□

42d · used in 42c · uses `nw_char 29ab`, `prose_flag 27`, `source_get 130b`

```

< Accumulate prose character 43a > ≡
if (prose_len < (int)sizeof(prose_buf) - 1)
    prose_buf[prose_len++] = c;

```

□

43a · used in 42d · uses `prose_buf 42b`, `prose_len 42b`

```

< Save pending prose 43b > ≡
if (prose_flag && prose_len > 0) {
    /* trim trailing whitespace */
    while (prose_len > 0 && (prose_buf[prose_len-1] == ' '
        || prose_buf[prose_len-1] == '\n'
        || prose_buf[prose_len-1] == '\t'
        || prose_buf[prose_len-1] == '\r'))
        prose_len--;
    prose_buf[prose_len] = '\0';
    pending_prose = save_string(prose_buf);
    prose_len = 0;
} else {
    pending_prose = NULL;
    prose_len = 0;
}

```

□

43b · used in 46bc · uses `pending_prose 42b`, `prose_buf 42b`, `prose_flag 27`, `prose_len 42b`, `save_string 159c`

The `@l` command (lowercase) sets the language for the next `@o` or `@d` directive by storing the tag in `pending_lang`. It replaces the earlier mechanism of detecting Markdown fences (‘‘`lang`’) and L^AT_EX markers (‘‘`%lang`’) during pass1. A typical usage is:

```

@l python
@d Validate input
@{import re
...
@}

```

```

< Collect language apply in pass1 43c > ≡
{
    char lang[64];
    char *lp = lang;
    int lc = source_get();
    /* skip optional whitespace */
    while (lc == ' ' || lc == '\t')
        lc = source_get();
    /* read language tag */
    while (lc != EOF && lc != '\n' && lc != ' ' && lc != '\t'
        && (lp - lang) < 62) {
        *lp++ = lc;
    }
}

```

```

    lc = source_get();
}
*lp = '\0';
if (lang[0] != '\0') {
    strncpy(pending_lang, lang, 63);
    pending_lang[63] = '\0';
}
/* consume rest of line */
while (lc != EOF && lc != '\n')
    lc = source_get();
}

```

□

43c · used in 44d · uses `pending_lang` 158b, `source_get` 130b

The `@L` command (Section 3.3.2) lets users register custom language mappings. During pass1, the actual parsing and registration is handled by `collect_lang_def()` in `names.c`. During the weave passes (L^AT_EX and Markdown), we simply consume the rest of the line so that the arguments do not leak into the typeset output.

⟨ *Collect user language definition in pass1* 44a ⟩ ≡
`collect_lang_def();`

□

44a · used in 44d · uses `collect_lang_def` 170

⟨ *Collect weave format declaration in pass1* 44b ⟩ ≡
`collect_weave_format();`

□

44b · used in 44d · uses `collect_weave_format` 172

⟨ *Skip at-L language definition line* 44c ⟩ ≡

```

{ int lc = source_get();
  while (lc != '\n' && lc != EOF)
    lc = source_get();
}
c = source_get();

```

□

44c · used in 59c, 86a, 106a · uses `source_get` 130b

Six at-sequences are interesting during the first pass: `@o` and `@0` (file definitions), `@d`, `@D`, and `@q` (fragment definitions), `@c` (block comments), `@s`/`@S` (sectors), `@l` (language apply), and `@L` (language definitions). We skip past all others immediately, warning if unexpected sequences are discovered.

⟨ *Scan at-sequence* 44d ⟩ ≡

```

{
    char quoted = 0;

    c = source_get();
    switch (c) {
        case 'r':
            c = source_get();
            nw_char = c;
            update_delimit_scrap();
            break;
        case '0':
        case 'o': ⟨ Build output file definition 46b ⟩
    }
}

```

```

        break;
    case 'Q':
    case 'q': quoted = 1;
        FALLTHROUGH;
    case 'D':
    case 'd': < Build fragment definition 46c >
        break;
    case 's':
        < Step to next sector 45a >
        break;
    case 'S':
        < Close the current sector 45b >
        break;
    case '<':
    case '(':
    case '[':
    case '{': < Skip over an in-text scrap 47a >
        break;
    case 'c': < Collect a block comment 48a >
        break;
    case 'l': < Collect language apply in pass1 43c >
        break;
    case 'L': < Collect user language definition in pass1 44a >
        break;
    case 'W': < Collect weave format declaration in pass1 44b >
        break;
    case 'x':
    case 'v':
    case 'u':
    case 'm':
    case 'f': /* ignore during this pass */
        break;
    default: if (c==nw_char) /* ignore during this pass */
        break;
        fprintf(stderr,
            "%s: unexpected %c sequence ignored (%s, line %d)\n",
            command_name, nw_char, source_name, source_line);
        break;
}
}

```

□

44d · used in 42d · uses `command_name` 29c, `FALLTHROUGH` 23a, `fprintf` 21b, `nw_char` 29ab, `source_get` 130b, `source_line` 129b, `source_name` 129b, `stderr` 21b, `update_delimit_scrap` 72a

< Step to next sector 45a > ≡

```

prev_sector += 1;
current_sector = prev_sector;
c = source_get();

```

□

45a · used in 44d, 59c, 86a, 106a · uses `current_sector` 45c, `prev_sector` 45c, `source_get` 130b

< Close the current sector 45b > ≡

```

current_sector = 1;
c = source_get();

```

□

45b · used in 44d, 59c, 86a, 106a · uses `current_sector` 45c, `source_get` 130b

⟨ Global variable declarations 45c ⟩ + ≡

```
extern unsigned char current_sector;  
extern unsigned char prev_sector;
```

□

45c · defined at 27, 29ac, 45c, 50a, 66b, 129b, 137e, 158b, 197a · used in 21a · defines `current_sector` 39b, 45ab, 46a, 79a, 82a, 97d, 99c, 109b, 110a, 138d, 140c, 176b, 179d, `prev_sector` 39b, 45a, 46a

⟨ Global variable definitions 46a ⟩ + ≡

```
unsigned char current_sector = 1;  
unsigned char prev_sector = 1;
```

□

46a · defined at 28, 29bd, 46a, 50b, 66c, 129c, 137f, 158c · used in 25e · uses `current_sector` 45c, `prev_sector` 45c

2.3.1 Accumulating Definitions

There are three steps required to handle a definition:

1. Build an entry for the name so we can look it up later.
2. Collect the scrap and save it in the table of scraps.
3. Attach the scrap to the name.

We go through the same steps for both file names and fragment names.

⟨ Build output file definition 46b ⟩ ≡

```
{  
    Name *name;  
    int scrap;  
    ⟨ Save pending prose 43b ⟩  
    name = collect_file_name(); /* returns a pointer to the name entry */  
    scrap = collect_scrap(); /* returns an index to the scrap */  
    if (pending_prose) set_scrap_prose(scrap, pending_prose);  
    pending_prose = NULL;  
    ⟨ Add scrap to name's definition list 46d ⟩  
}
```

□

46b · used in 44d · uses `collect_file_name` 173a, `collect_scrap` 138c, `Name` 158a, `pending_prose` 42b, `set_scrap_prose` 145a

⟨ Build fragment definition 46c ⟩ ≡

```
{  
    Name *name;  
    int scrap;  
    ⟨ Save pending prose 43b ⟩  
    name = collect_macro_name();  
    scrap = collect_scrap();  
    if (pending_prose) set_scrap_prose(scrap, pending_prose);  
    pending_prose = NULL;  
    ⟨ Add scrap to name's definition list 46d ⟩  
}
```

□

46c · used in 44d · uses `collect_macro_name` 176b, `collect_scrap` 138c, `Name` 158a, `pending_prose` 42b, `set_scrap_prose` 145a

Since a file or fragment may be defined by many scraps, we maintain them in a simple linked list. The list is actually built in reverse order, with each new definition being added to the head of the list.

⟨ Add scrap to name's definition list 46d ⟩ ≡

```

{
    Scrap_Node *def = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    def->scrap = scrap;
    def->quoted = quoted;
    def->next = name->defs;
    name->defs = def;
    set_scrap_owner(scrap, name);
}

```

□

46d · used in 46bc · uses arena_getmem 197e, defs 134a, next 133c, Scrap_Node 157e, set_scrap_owner 145b

⟨ *Skip over an in-text scrap* 47a ⟩ ≡

```

{
    int c;
    int depth = 1;
    while ((c = source_get()) != EOF) {
        if (c == nw_char)
            ⟨ Skip over at-sign or go to skipped 47b ⟩
    }
    fprintf(stderr, "%s: unexpected EOF in text at (%s, %d)\n",
               command_name, source_name, source_line);
    exit(-1);
}

```

skipped: ;

}

□

47a · used in 44d · uses command_name 29c, exit 21b, fprintf 21b, nw_char 29ab, source_get 130b, source_line 129b, source_name 129b, stderr 21b

⟨ *Skip over at-sign or go to skipped* 47b ⟩ ≡

```

{
    c = source_get();
    switch (c) {
        case '{': case '[': case '(': case '<':
            depth += 1;
            break;
        case '}': case ']': case ')': case '>':
            if (--depth == 0)
                goto skipped;
        case 'x': case '|': case ',':
        case '%': case '1': case '2':
        case '3': case '4': case '5': case '6':
        case '7': case '8': case '9': case '_':
        case 'f': case '#': case '+': case '-':
        case 'v': case '*': case 'c': case '\\':
        case 's':
            break;
        default:
            if (c != nw_char) {
                fprintf(stderr, "%s: unexpected %c%c in text at (%s, %d)\n",
                           command_name, nw_char, c, source_name, source_line);
                exit(-1);
            }
            break;
    }
}

```

```
}
```

```
□
```

47b · used in 47a · uses `command_name` 29c, `exit` 21b, `fprintf` 21b, `nw_char` 29ab, `source_get` 130b, `source_line` 129b, `source_name` 129b, `stderr` 21b

2.3.2 Block Comments

⟨ *Collect a block comment* 48a ⟩ ≡

```
{
    char * p = blockBuff;
    char * e = blockBuff + (sizeof(blockBuff)/sizeof(blockBuff[0])) - 1;

    ⟨ Skip whitespace 48b ⟩
    while (p < e)
    {
        ⟨ Add one char to the block buffer 48c ⟩
    }
    if (p == e)
    {
        ⟨ Skip to the next nw-char 49c ⟩
    }
    *p = '\000';
}
```

```
□
```

48a · used in 44d · uses `blockBuff` 50a

⟨ *Skip whitespace* 48b ⟩ ≡

```
while (source_peek == ' '
      || source_peek == '\t'
      || source_peek == '\n')
    (void)source_get();
```

```
□
```

48b · used in 48a · uses `source_get` 130b, `source_peek` 130b

⟨ *Add one char to the block buffer* 48c ⟩ ≡

```
int c = source_get();

if (c == nw_char)
{
    ⟨ Add an at character to the block or break 48d ⟩
}
else if (c == EOF)
{
    source_ungetc(&c);
    break;
}
else
{
    ⟨ Add any other character to the block 49a ⟩
}
```

```
□
```

48c · used in 48a · uses `nw_char` 29ab, `source_get` 130b, `source_ungetc` 130d

⟨ *Add an at character to the block or break* 48d ⟩ ≡

```

int cc = source_peek;

if (cc == 'c')
{
    do
        c = source_get();
        while (c <= ' ');

    break;
}
else if (cc == 'd'
        || cc == 'D'
        || cc == 'q'
        || cc == 'Q'
        || cc == 'o'
        || cc == 'O'
        || cc == EOF)
{
    source_ungetc(&c);
    break;
}
else
{
    *p++ = c;
    *p++ = source_get();
}

```

□

48d · used in 48c · uses [source_get 130b](#), [source_peek 130b](#), [source_ungetc 130d](#)

⟨ *Add any other character to the block* 49a ⟩ ≡

⟨ *Perhaps skip white-space* 49b ⟩
 *p++ = c;

□

49a · used in 48c

⟨ *Perhaps skip white-space* 49b ⟩ ≡

```

if (c == ' ')
{
    while (source_peek == ' ')
        c = source_get();
}
if (c == '\n')
{
    if (source_peek == '\n')
    {
        do
            c = source_get();
        while (source_peek == '\n');
    }
    else
        c = ' ';
}

```

□

49b · used in 49a · uses [source_get 130b](#), [source_peek 130b](#)

⟨ *Skip to the next nw-char* 49c ⟩ ≡

```
int c;
```

```
while ((c = source_get()), c != nw_char && c != EOF)/* Skip */
source_ungetc(&c);
```

□

49c · used in 48a · uses `nw_char` 29ab, `source_get` 130b, `source_ungetc` 130d

⟨ *Global variable declarations* 50a ⟩ + ≡

```
extern char blockBuff[6400];
```

□

50a · defined at 27, 29ac, 45c, 50a, 66b, 129b, 137e, 158b, 197a · used in 21a · defines `blockBuff` 48a, 50bd

⟨ *Global variable definitions* 50b ⟩ + ≡

```
char blockBuff[6400];
```

□

50b · defined at 28, 29bd, 46a, 50b, 66c, 129c, 137f, 158c · used in 25e · uses `blockBuff` 50a

We don't show block comments inside scraps in the T_EX file, but we do show where they go.

⟨ *Show presence of a block comment* 50c ⟩ ≡

```
{
  begin_tex_escape(file);
  fprintf(file, "\\hbox{\\sffamily\\slshape (Comment)}");
  end_tex_escape(file);
}
```

□

50c · used in 73b · uses `begin_tex_escape` 71a, `end_tex_escape` 71a, `fprintf` 21b

The block comment is presently in the block buffer. Here we copy it into the scrap whence we shall copy it into the output file.

⟨ *Include block comment in a scrap* 50d ⟩ ≡

```
{
  char * p = blockBuff;

  push(nw_char, &writer);
  do
  {
    push(c, &writer);
    c = *p++;
  } while (c != '\\0');
}
```

□

50d · used in 139b · uses `blockBuff` 50a, `nw_char` 29ab, `push` 138a

⟨ *Copy block comment from scrap* 50e ⟩ ≡

```
{
  int bgn = indent + global_indent;
  int posn = bgn + strlen(comment_begin[comment_flag]);
  int i;
```

⟨ *Perhaps put a delayed indent* 154a ⟩

```
c = pop(&reader);
fputs(comment_begin[comment_flag], file);
while (c != '\\0')
```

```

{
  < Move a word to the file 51a >
  < If we break the line at this word 51b >
  {
    putc('\n', file);
    for (i = 0; i < bgn ; i++)
      putc(' ', file);
    c = pop(&reader);
    if (c != '\0')
    {
      posn = bgn + strlen(comment_mid[comment_flag]);
      fputs(comment_mid[comment_flag], file);
    }
  }
}
fputs(comment_end[comment_flag], file);
}

```

□

50e · used in 151c · uses comment_begin 176a, comment_end 176a, comment_mid 176a, fputs 21b, pop 143a, putc 21b, strlen 21b

< Move a word to the file 51a > ≡

```

do
{
  putc(c, file);
  posn += 1;
  c = pop(&reader);
} while (c > ' ');

```

□

51a · used in 50e · uses pop 143a, putc 21b

< If we break the line at this word 51b > ≡

```

if (c == '\n' || (c == ' ' && posn > 60))

```

□

51b · used in 50e

< Skip over a block comment 51c > ≡

```

if (last == nw_char && c == 'c')
  while ((c = pop(reader.m)) != '\0')
    /* Skip */;

```

□

51c · used in 189b · uses nw_char 29ab, pop 143a

< Begin or end a block comment 51d > ≡

```

if (inBlock)
{
  < End block 65a >
}
else
{
  < Start block 64b >
}

```

□

51d · used in 59c

2.3.3 Fixing the Cross References

Since the definition and reference lists for each name are accumulated in reverse order, we take the time at the end of `pass1` to reverse them all so they'll be simpler to print out prettily. The code for `reverse_lists` appears in Section 3.3.

```
< Reverse cross-reference lists 52a > ≡  
{  
  reverse_lists(file_names);  
  reverse_lists(macro_names);  
  reverse_lists(user_names);  
}  
□
```

52a · used in 42c · uses `file_names` 158b, `macro_names` 158b, `reverse_lists` 182f, `user_names` 158b

2.3.4 Dealing with fragment parameters

Fragment parameters were added on later in nuweb's development. There still is not, for example, an index of fragment parameters. We need a data type to keep track of fragment parameters.

```
< Type declarations 52b > + ≡  
typedef int *Parameters;  
□
```

52b · defined at 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e · used in 21a · defines `Parameters` 54a, 87b, 135a, 147ac, 148c

When we are copying a scrap to the output, we check for an embedded parameter. If we find it we copy it. Otherwise it's an old-style parameter and we can then pull the *n*th string from the `Parameters` list when we see an @1 @2, etc.

```
< Handle macro parameter substitution 52c > ≡  
  
case '1': case '2': case '3':  
case '4': case '5': case '6':  
case '7': case '8': case '9':  
{  
  Arglist * args;  
  Name * name;  
  
  lookup(c - '1', inArgs, inParams, &name, &args);  
  
  if (name == (Name *)1) {  
    Embed_Node * q = (Embed_Node *)args;  
    indent = write_scraps(file, spelling, q->defs,  
                          global_indent + indent,  
                          indent_chars, debug_flag,  
                          tab_flag, indent_flag,  
                          0, 0,  
                          q->args, inParams,  
                          local_parameters, "");  
  }  
  else if (name != NULL) {  
    int i, narg;  
    char * p = name->spelling;  
    Arglist *q = args;  
  
    < Perhaps comment this macro 153 >
```

```

        indent = write_scraps(file, spelling, name->defs,
                               global_indent + indent,
                               indent_chars, debug_flag,
                               tab_flag, indent_flag,
                               comment_flag, location_flag,
                               args, name->arg,
                               local_parameters, p);
    }
    else if (args != NULL) {
        if (delayed_indent) {
            < Put out the indent 150e >
        }
        fputs((char *)args, file);
    }
    else if ( parameters && parameters[c - '1'] ) {
        Scrap_Node param_defs;
        param_defs.scrap = parameters[c - '1'];
        param_defs.next = 0;
        write_scraps(file, spelling, &param_defs,
                     global_indent + indent,
                     indent_chars, debug_flag,
                     tab_flag, indent_flag,
                     comment_flag, location_flag,
                     NULL, NULL, 0, "");
    } else if (delayed_indent) {
        < Put out the indent 150e >
    }
}

```

□

52c · used in 151c · uses Arglist 179b, defs 134a, delayed_indent 148a, Embed_Node 182c, fputs 21b, lookup 145d, Name 158a, next 133c, Scrap_Node 157e, write_scraps 147c

Now onto actually parsing fragment parameters from a call. We start off checking for fragment parameters, an @ (sequence followed by parameters separated by @, sequences, and terminated by a @) sequence.

We collect separate scraps for each parameter, and write the scrap numbers down in the text. For example, if the file has:

```
@<foo @( param1 @, param2 @)@>
```

we actually make new scraps, say 10 and 11, for param1 and param2, and write in the collected scrap:

```
@<foo @(10@,11@)@>
```

< Save macro parameters 53 > ≡

```

{
    int param_scrap;
    char param_buf[10];

    push(nw_char, &writer);
    push('(', &writer);
    do {
        param_scrap = collect_scrap();
        sprintf(param_buf, "%d", param_scrap);
        pushs(param_buf, &writer);
        push(nw_char, &writer);
        push(scrap_ended_with, &writer);
        add_to_use(name, current_scrap);
    } while( scrap_ended_with == ',' );
    do

```

```

    c = source_get();
while( ' ' == c );
if (c == nw_char) {
    c = source_get();
}
if (c != '>') {
    /* ZZZ print error */;
}
}

```

□

53 · used in 142a · uses [add_to_use 142c](#), [collect_scrap 138c](#), [nw_char 29ab](#), [push 138a](#), [pushs 138b](#), [source_get 130b](#)

If we get inside, we have at least one parameter, which will be at the beginning of the parms buffer, and we prime the pump with the first character.

⟨ *Check for macro parameters* 54a ⟩ ≡

```

if (c == '(') {
    Parameters res = arena_getmem(10 * sizeof(int));
    int *p2 = res;
    int count = 0;
    int scrapnum;

    while( c && c != ')' ) {
        scrapnum = 0;
        c = pop(manager);
        while( '0' <= c && c <= '9' ) {
            scrapnum = scrapnum * 10 + c - '0';
            c = pop(manager);
        }
        if ( c == nw_char ) {
            c = pop(manager);
        }
        *p2++ = scrapnum;
    }
    while (count < 10) {
        *p2++ = 0;
        count++;
    }
    while( c && c != nw_char ) {
        c = pop(manager);
    }
    if ( c == nw_char ) {
        c = pop(manager);
    }
    *parameters = res;
}

```

□

54a · used in 147b · uses [arena_getmem 197e](#), [nw_char 29ab](#), [Parameters 52b](#), [pop 143a](#)

These are used in `write_tex` to output the argument list for a fragment.

⟨ *Format macro parameters* 54b ⟩ ≡

```

char sep;

sep = '(';

```

```

do {
    fputc(sep,file);

    fputs("{\\footnotesize ", file);
    write_single_scrap_ref(file, scraps + 1);
    fprintf(file, "\\label{scrap%d}", scraps + 1);
    fputs(" }", file);

    source_last = '{';
    {
        /* The parameter is rendered inside a fragment reference, i.e.
           inside a listings escape, so it must be single-line LaTeX with
           the special characters neutralised: switch copy_scrap into its
           inline mode for the duration. */
        int saved_inline = code_inline;
        code_inline = TRUE;
        copy_scrap(file, TRUE, NULL);
        code_inline = saved_inline;
    }

    ++scraps;

    sep = ',';
} while ( source_last != ')' && source_last != EOF );
fputs(" ) ",file);
do
    c = source_get();
while(c != nw_char && c != EOF);
if (c == nw_char) {
    c = source_get();
}

```

□

54b · used in 75c, 77 · uses `code_inline` 71a, `copy_scrap` 71b, `fprintf` 21b, `fputs` 21b, `nw_char` 29ab, `scraps` 134b, `source_get` 130b, `source_last` 130b, `TRUE` 23a, `write_single_scrap_ref` 135d

2.4 Writing the Latex File

The second pass (invoked via a call to `write_tex`) copies most of the text from the source file straight into a `.tex` file. Definitions are formatted slightly and cross-reference information is printed out.

Note that all the formatting is handled in this section. If you don't like the format of definitions or indices or whatever, it'll be in this section somewhere. Similarly, if someone wanted to modify `weft` to work with a different typesetting system, this would be the place to look.

⟨ *Function prototypes* 55a ⟩ + ≡

```
extern void write_tex(char *file_name, char *tex_name, unsigned char sector);
```

□

55a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `file_name` 134a, `sector` 134a, `write_tex` 56a

We need a few local function declarations before we get into the body of `write_tex`.

⟨ *latex.c* 55b ⟩ + ≡

```
static void copy_scrap(FILE *file, int prefix, Name *name);           /* formats the
    body of a scrap */
static void latex_code_putc(FILE *file, int c);                      /* escapes one
    code character for LaTeX */
```

```

static const char *latex_language(const char *lang);           /* maps a weft
    language tag to a listings name */
static void print_scrap_numbers(FILE *tex_file, Scrap_Node *scraps); /* formats a
    list of scrap numbers */
static void format_entry(Name *name, FILE *tex_file, unsigned char sector);
    /* formats an index entry */
static void format_file_entry(Name *name, FILE *tex_file);     /* formats a file index
    entry */
static void format_user_entry(Name *name, FILE *tex_file, unsigned char sector);
static void write_arg(FILE *tex_file, char *p);
static void write_literal(FILE *tex_file, char *p, int mode);
static void write_ArglistElement(FILE *file, Arglist *args, char **params);

```

□

55b · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · uses Arglist 179b, copy_scrap 71b, FILE 21b, format_entry 79c, format_file_entry 78b, format_user_entry 82b, latex_code_putc 69a, latex_language 69b, Name 158a, print_scrap_numbers 67d, scraps 134b, Scrap_Node 157e, sector 134a, write_arg 63c, write_ArglistElement 77, write_literal 70c

The routine `write_tex` takes two file names as parameters: the name of the web source file and the name of the `.tex` output file.

```

⟨ latex.c 56a ⟩ + ≡
void write_tex(char *file_name, char *tex_name, unsigned char sector)
{
    (void)sector;
    FILE *tex_file = fopen(tex_name, "w");
    if (tex_file) {
        if (verbose_flag)
            fprintf(stderr, "writing %s\n", tex_name);
        source_open(file_name);
        ⟨ Write LaTeX limbo definitions 56b ⟩
        ⟨ Copy source_file into tex_file 59b ⟩
        fclose(tex_file);
    }
    else
        fprintf(stderr, "%s: can't open %s\n", command_name, tex_name);
}

```

□

56a · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `write_tex` 39b, 55a · uses `command_name` 29c, `fclose` 21b, `FILE` 21b, `file_name` 134a, `fopen` 21b, `fprintf` 21b, `sector` 134a, `source_open` 133b, `stderr` 21b, `verbose_flag` 27

Weft's woven output relies on a family of `\WEFT...` macros. We write default definitions for all of them into the document's limbo so that source files need not define anything new. If a user wants to change any of the macros (to use `hyperref`, to write in some language other than english, or to tune the code layout) he or she can redefine the commands *after* the point where weft's definitions are read, or (better) redefine them in the preamble with `\renewcommand`.

Three groups of macros are written. The first group are the textual and cross-reference macros (`\WEFTtarget`, `\WEFTlink`, the `\WEFTtxt...` labels, and so on). The second group, the *modern decoration macros* of §2.4, give a scrap its visual identity (fragment name, scrap number, the `≡/+≡` marker, the one-line footer) and route all colour through the degradable `\WEFTtint` indirection. The third group, written by the scrap-formatting fragment below, are the layout macros that frame a scrap.

Code bodies are typeset with the `listings` package: every scrap becomes an `lstlisting` environment. This gives syntax highlighting (driven by the scrap's language, see `latex_language`), automatic line wrapping (`breaklines`), and page breaking for free. The document *must* therefore load `\usepackage{listings}`

in its preamble — `weft` cannot inject it, because the `limbo` is written before `\documentclass` (a colour package, `color` or `xcolor`, is likewise the user's to load, and enables the accent and syntax colours; without it the output still compiles in black and white). Fragment references that appear in the middle of code are typeset by briefly escaping out of the listing back to L^AT_EX; see §2.4 and the `escapeinside` option.

```
< Write LaTeX limbo definitions 56b > ≡
if (hyperref_flag) {
  fputs("\\newcommand{\\WEFTtarget}[2]{\\hypertarget{#1}{#2}}\\n", tex_file);
  fputs("\\newcommand{\\WEFTlink}[2]{\\hyperlink{#1}{#2}}\\n", tex_file);
} else {
  fputs("\\newcommand{\\WEFTtarget}[2]{#2}\\n", tex_file);
  fputs("\\newcommand{\\WEFTlink}[2]{#2}\\n", tex_file);
}
fputs("\\newcommand{\\WEFTtxtMacroDefBy}{defined at}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtMacroRefIn}{used in}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtMacroNoRef}{unreferenced}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtDefBy}{defined at}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtRefIn}{used in}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtNoRef}{unreferenced}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtFileDefBy}{defined at}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtIdsentsUsed}{uses}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtIdsentsNotUsed}{never used}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtIdsentsDefed}{defines}\\n", tex_file);
fputs("\\newcommand{\\WEFTsep}{\\par\\nobreak\\noindent{\\WEFTtint{weftcom}}
      \"\\fboxsep=0pt\\fboxrule=.3pt\\fbox{\\rule{0pt}{.85ex}}\\kern.85ex}}\"
      \"\\par}\\n", tex_file);
fputs("\\newcommand{\\WEFTnotglobal}{(not defined globally)}\\n", tex_file);
< Write the modern decoration macros 58b >
< Write the scrap formatting macros 57 >
fputs("\\newcommand{\\WEFTuseHyperlinks}{", tex_file);
if (hyperoptions[0] != '0')
{
  < Write the hyperlink usage macro 58a >
}
fputs("}\\n", tex_file);
< Write the AtBeginDocument setup 59a >
□
```

56b · used in 56a · uses `fputs` 21b, `hyperoptions` 27, `hyperref_flag` 27, `uses` 134a

The layout macros. `\\WEFTbegin`/`\\WEFTend` open and close the vertical block that holds a scrap's header line, its `lstlisting` body, and its one-line metadata footer. `\\WEFTbegin` sets the block ragged-right at the document's ordinary size (the old `\\small` is gone: the header now reads at full size, and the code carries its own, tighter, listings size). A fragment reference appearing in the middle of code is assembled inline and wrapped in an `\\hbox` so that `listings`' line-breaker never splits it while it sits inside an escape. (We use `\\hbox` rather than `\\mbox` because a fragment name may legitimately contain `\\verb`, which is illegal in a macro argument such as `\\mbox`'s but fine inside a primitive `\\hbox`.) `\\WEFTlstsetup` now carries `weft`'s *modern* `\\lstset` defaults: a compact monospace with tightened leading, coloured keywords/strings/comments (through `\\WEFTtint`, so colour is optional), line breaking, and *no* frame, background, or left indent. The user need not call it — the `\\AtBeginDocument` hook of §2.4 applies it automatically once `listings` is loaded — but an explicit call, or a `\\renewcommand`, still lets the user override the house style.

```
< Write the scrap formatting macros 57 > ≡
fputs("\\newcommand{\\WEFTbegin}{\\par\\addvspace{2.3ex plus .6ex}\"
      \"\\begingroup\\raggedright}\\n", tex_file);
fputs("\\newcommand{\\WEFTend}{\\par\\endgroup\"
      \"\\addvspace{2.3ex plus .6ex}}\\n", tex_file);
fputs("\\providecommand{\\WEFTlstsetup}{\\lstset{basicstyle=\\ttfamily"
```

```

"\fontsize{10pt}{10.8pt}\selectfont,"
"keywordstyle={\WEFTtint{weftkw}\bfseries},"
"commentstyle={\WEFTtint{weftcom}\itshape},"
"stringstyle={\WEFTtint{weftstr}},"
"breaklines=true,breakatwhitespace=false,"
"columns=fullflexible,keepspaces=true,showstringspaces=false,"
"tabsize=8,frame=none,xleftmargin=0pt,"
"aboveskip=0pt,belowskip=0pt}}\n", tex_file);

```

□

57 · used in 56b · uses fputs 21b

⟨ *Write the hyperlink usage macro* 58a ⟩ ≡

```
fprintf(tex_file, "\\usepackage[%s]{hyperref}", hyperoptions);
```

□

58a · used in 56b · uses fprintf 21b, hyperoptions 27

The modern decoration macros

The visual identity of a woven scrap — the way a fragment name reads, the little scrap number beside it, the definition/continuation marker, the one-line metadata footer — is factored into a handful of small macros written here. Emitting them once, as named commands, keeps the per-scrap output that follows compact and lets a reader (or the user) re-style the whole document by redefining a single command.

The design is *sober and self-contained*: a fragment name is set in the document's ordinary italic tinted with one accent colour, its scrap number in a small grey, the enclosing ⟨ ⟩ in the same grey, and the modern layout carries *no* rules, boxes, or shaded backgrounds — highlighting alone distinguishes the code.

Colour is applied through one level of indirection, `\WEFTtint`. By default it is a no-op: `\WEFTtint{weftaccent}` expands to nothing, so a document that loads *no* colour package still compiles, merely in black and white. When a colour package *is* present, the `\AtBeginDocument` hook below swaps in `\WEFTtintOn`, and the same call becomes `\color{weftaccent}`. This is why every colour site is written `{\WEFTtint{name}...}` rather than `\color` directly: it degrades gracefully.

`\WEFTlangle`/`\WEFTrangle` are the grey angle brackets; `\WEFTnumsep` is the space set between a fragment name and its scrap number (wide enough that `name 41a` never reads as one token); `\WEFTeq` and `\WEFTpluseq` are the definition ($\equiv$) and additive-continuation ($+ \equiv$) markers; `\WEFTdot` is the mid-dot that separates footer items; and `\WEFTmetabegin`/`\WEFTmetaend` open and close the small grey metadata line, which opens with the scrap's own number.

`\WEFTsep` is the end-of-scrap tombstone: a small *hollow* square, tinted like the metadata, on a line of its own flushed to the *left* margin directly below the code. It replaces the old floating diamond, whose math-axis glyph read as noise. The tombstone belongs to the chunk — it closes the code — so the metadata footer, which is information *about* the chunk rather than part of it, is set below the tombstone (the code emits `\WEFTsep` right after `\end{lstlisting}`, before `\WEFTmetabegin`). It is drawn as an empty `\fbox` (zero `\fboxsep`, a hairline `\fboxrule`) around an invisible strut, so it needs no maths or symbol package. Note that a stand-alone glyph adds vertical height, and because `weft` resolves a scrap's local/global sector number from the *page* its `\label` lands on, that height can shift a page break and renumber scraps in a multi-page, multi-sector document — a deliberate trade for the clearer layout.

⟨ *Write the modern decoration macros* 58b ⟩ ≡

```

fputs("\\newcommand{\\WEFTtint}[1]{\\n", tex_file);
fputs("\\newcommand{\\WEFTtintOn}[1]{\\color{#1}}\\n", tex_file);
fputs("\\newcommand{\\WEFTlangle}{\\WEFTtint{weftcom}$\\langle$}\\n",
tex_file);
fputs("\\newcommand{\\WEFTrangle}{\\WEFTtint{weftcom}$\\rangle$}\\n",
tex_file);
fputs("\\newcommand{\\WEFTeq}{\\WEFTtint{weftcom}$\\equiv$}\\n",
tex_file);
fputs("\\newcommand{\\WEFTpluseq}{\\WEFTtint{weftcom}$\\plus$}\\n",
tex_file);

```

```

    "$\\mathord{+}\\equiv$}\\n", tex_file);
fputs("\\newcommand{\\WEFTdot}{\\ {\\WEFTtint{weftcom}"
    "\\textperiodcentered}\\ }\\n", tex_file);
fputs("\\newcommand{\\WEFTnumsep}{\\ }\\n", tex_file);
fputs("\\newcommand{\\WEFTmetabegin}[1]{\\par\\vspace{3pt}\\begingroup"
    "\\footnotesize\\WEFTtint{weftcom}#1}\\n", tex_file);
fputs("\\newcommand{\\WEFTmetaend}{\\endgroup\\par}\\n", tex_file);

```

□

58b · used in 56b · uses fputs 21b

The colours themselves, and the automatic application of `\WEFTlstsetup`, are deferred to `\AtBeginDocument`. This is essential: the limbo we are writing precedes `\documentclass`, where `\definecolor` and `\lstset` cannot run (no colour or listings package is loaded yet, and loading one there fails outright). The kernel hook `\AtBeginDocument` runs its body at `\begin{document}`, by which time the user's preamble has loaded its packages. We guard each half with ε -TeX's `\ifdefined` so the output still compiles when a colour package or `listings` is absent. The palette uses the `rgb` model, understood by both `color` and `xcolor`, so no specific colour package is mandated.

⟨ *Write the AtBeginDocument setup* 59a ⟩ ≡

```

fputs("\\AtBeginDocument{\\n", tex_file);
fputs("\\ifdefined\\definecolor\\n", tex_file);
fputs("\\definecolor{weftaccent}{rgb}{0.184,0.435,0.922}\\n", tex_file);
fputs("\\definecolor{weftkw}{rgb}{0.020,0.314,0.682}\\n", tex_file);
fputs("\\definecolor{weftstr}{rgb}{0.067,0.388,0.161}\\n", tex_file);
fputs("\\definecolor{weftcom}{rgb}{0.431,0.467,0.506}\\n", tex_file);
fputs("\\let\\WEFTtint\\WEFTtintOn\\n", tex_file);
fputs("\\fi\\n", tex_file);
fputs("\\ifdefined\\lstset\\WEFTlstsetup\\fi\\n", tex_file);
fputs("}\\n", tex_file);

```

□

59a · used in 56b · uses fputs 21b

We make our second (and final) pass through the source web, this time copying characters straight into the `.tex` file. However, we keep an eye peeled for `@` characters, which signal a command sequence.

⟨ *Copy source_file into tex_file* 59b ⟩ ≡

```

{
    int inBlock = FALSE;
    int c = source_get();
    while (c != EOF) {
        if (c == nw_char)
        {
            ⟨ Interpret at-sequence 59c ⟩
        }
        else {
            putc(c, tex_file);
            c = source_get();
        }
    }
}

```

□

59b · used in 56a · uses FALSE 23a, nw_char 29ab, putc 21b, source_get 130b

⟨ *Interpret at-sequence* 59c ⟩ ≡

```

{
    int big_definition = FALSE;

```

```

c = source_get();
switch (c) {
  case 'r':
    c = source_get();
    nw_char = c;
    update_delimit_scrap();
    break;
  case '0': big_definition = TRUE;
             FALLTHROUGH;
  case 'o': < Write output file definition 62 >
             break;
  case 'Q':
  case 'D': big_definition = TRUE;
             FALLTHROUGH;
  case 'q':
  case 'd': < Write macro definition 63a >
             break;
  case 's':
             < Step to next sector 45a >
             break;
  case 'S':
             < Close the current sector 45b >
             break;
  case '{':
  case '[':
  case '(': < Write in-text scrap 66d >
             break;
  case '<': < Expand macro into tex_file 61 >
             break;
  case 'x': < Copy label from source into(60a tex_file ) 74b >
             c = source_get();
             break;
  case 'c': < Begin or end a block comment 51d >
             c = source_get();
             break;
  case 'f': < Write index of file names 78a >
             break;
  case 'm': < Write index of macro names 79a >
             break;
  case 'u': < Write index of user-specified names 82a >
             break;
  case 'v': < Copy version info into tex file 60b >
             break;
  case 'l':
  case 'L': < Skip at-L language definition line 44c >
             break;
  case 'W': < Skip at-L language definition line 44c >
             break;
  default:
    if (c==nw_char)
      putc(c, tex_file);
    c = source_get();
    break;
}
}

```

□

60a · used in [59b](#)

⟨ *Copy version info into tex file* 60b ⟩ ≡

```
fputs(version_string, tex_file);  
c = source_get();
```

□

60b · used in 59c · uses [fputs](#) 21b, [source_get](#) 130b, [version_string](#) 27

A fragment reference `@<name@>` written in the *documentation* text (outside any scrap) is a *cross-reference*, not an inclusion. weft renders it exactly like a reference in the middle of code — the grey `< >` brackets, the accent-tinted name, and a link to the fragment’s first defining scrap — only without the `listings` escape, since prose is already ordinary L^AT_EX text. This is a deliberate change from the old nuweb behaviour, which *expanded* the fragment’s whole body inline into the prose: a chunk name mentioned in a sentence would dump code into the middle of it. Referencing, not expanding, is what a literate narrative wants (“as we saw in `@<Say hello@>`”). To show a fragment’s *code* in the prose, define and reference it as a scrap in the usual way.

⟨ *Expand macro into file* 61 ⟩ ≡

```
{  
    Arglist *a = collect_scrap_name(0);  
    Name *name = a->name;  
    char *p = name->spelling;  
    int i = 0;  
    fputs("\\WEFTtriangle{\\WEFTtint{weftaccent}\\itshape ", file);  
    while (*p != '\\000') {  
        if (*p == ARG_CHR) {  
            write_arg(file, name->arg[i++]);  
            p++;  
        }  
        else  
            fputc(*p++, file);  
    }  
    fputs("{}\\nobreak\\WEFTnumsep{\\footnotesize\\WEFTtint{weftcom}", file);  
    if (name->defs) {  
        Scrap_Node *q = name->defs;  
        fputs("\\WEFTlink{weft", file);  
        write_single_scrap_ref(file, q->scrap);  
        fputs("{", file);  
        write_single_scrap_ref(file, q->scrap);  
        fputs("}", file);  
        if (q->next)  
            fputs(", \\ldots\\ ", file);  
    }  
    else {  
        putc('?', file);  
        fprintf(stderr, "%s: never defined <%s>\\n",  
                command_name, name->spelling);  
    }  
    fputs("{}\\WEFTtriangle", file);  
    c = source_get();  
}
```

□

61 · used in 59c · uses [Arglist](#) 179b, [ARG_CHR](#) 178a, [collect_scrap_name](#) 179d, [command_name](#) 29c, [defs](#) 134a, [fprintf](#) 21b, [fputs](#) 21b, [Name](#) 158a, [next](#) 133c, [putc](#) 21b, [Scrap_Node](#) 157e, [source_get](#) 130b, [stderr](#) 21b, [write_arg](#) 63c, [write_single_scrap_ref](#) 135d

2.4.1 Formatting Definitions

We go through a fair amount of effort to format a fragment definition. The L^AT_EX for the previous fragment definition should look like this (perhaps modulo the scrap references). Note the shape of a modern header: the grey `\WEFTlangle/\WEFTrangle` brackets, the accent-tinted italic name, the small grey scrap number, and the `\WEFTeq` ($\equiv$) kind marker; then a hair of space before the listing; and, after it, the one-line `\WEFTmetabegin... \WEFTmetaend` footer instead of the old bulleted list.

```
\WEFTbegin
\label{scrap70}
\WEFTtarget{weft70}{\WEFTlangle{\WEFTtint{weftaccent}}\itshape %
  Interpret at-sequence}\nobreak\,{\footnotesize\WEFTtint{weftcom}70}%
  \WEFTrangle\WEFTeq\par\nobreak\vspace{0.6ex}\nobreak
\begin{lstlisting}[escapeinside={(*<}{>*)},language=C]
{
  int big_definition = FALSE;
  c = source_get();
  switch (c) {
    case 'o': (*<\hbox{\normalfont \WEFTlangle{\WEFTtint{weftaccent}}%
\itshape Write output file definition}\nobreak\,{\footnotesize%
\WEFTtint{weftcom}\WEFTlink{weft19a}{19a}}\WEFTrangle}>*)
  }
\end{lstlisting}
\WEFTmetabegin{70}\WEFTdot \WEFTtxtMacroRefIn\ \WEFTlink{weft17b}{17b}\WEFTmetaend
\WEFTend
```

The code body is now an `lstlisting`. Because `listings` reads the body verbatim, the special L^AT_EX characters need *no* escaping — they are printed as typed. The `escapeinside={(*<}{>*)}` option carves out the one place we *do* need L^AT_EX: a fragment reference in the middle of code, which is emitted between `(*<` and `>*)` and wrapped in an `\hbox` so `breaklines` cannot split it. The delimiter pair `(*<...>*)` is deliberately chosen to be a sequence that never occurs in real source (weft warns if a scrap body contains it). `listings` supplies page breaking, line wrapping, and — via the `language=` option, taken from the scrap's language tag — syntax highlighting.

Fragment and file definitions are formatted nearly identically. I've factored the common parts out into separate scraps.

Both headers now carry a *kind marker* after the closing angle bracket: `\WEFTeq` ($\equiv$) for a definition, or `\WEFTpluseq` ($+\equiv$) for an *additive continuation* — a scrap that appends to a fragment (or file) already begun elsewhere. We know which is which because `name->defs` is, after pass one's `reverse_lists`, in source order: its head is the *first* scrap that defines the name. So the current scrap is a continuation exactly when `name->defs->scrap` differs from the scrap we are emitting. The file header sets the file name in monospace (it *is* a filename); the fragment header sets its name in the document's italic. Both are tinted with the accent colour and followed, on the next line after a hair of vertical space, by the code.

```
< Write output file definition 62 >  $\equiv$ 
{
  Name *name = collect_file_name();
  int additive = (name->defs && name->defs->scrap != scraps);
  < Begin the scrap environment 64a >
  fputs("\WEFTtarget{weft", tex_file);
  write_single_scrap_ref(tex_file, scraps);
  fputs("}{", tex_file);
  fprintf(tex_file, "\\WEFTlangle{\\WEFTtint{weftaccent}}\\verb%c%s%c}\\nobreak"
    "\\WEFTnumsep{\\footnotesize\\WEFTtint{weftcom}", nw_char, name->spelling,
    nw_char);
  write_single_scrap_ref(tex_file, scraps);
  fputs("\\WEFTrangle", tex_file);
  fputs(additive ? "\\WEFTpluseq" : "\\WEFTeq", tex_file);
  fputs("\\par\\nobreak\\vspace{0.6ex}\\nobreak\\n", tex_file);
```

```

    < Fill in the middle of the scrap environment 65b >
    < Begin the cross-reference environment 66e >
    if ( scrap_flag ) {
        < Write file defs 67a >
    }
    format_defs_refs(tex_file, scraps);
    format_uses_refs(tex_file, scraps++);
    < Finish the cross-reference environment 66f >
    < Finish the scrap environment 66a >
}

```

62 · used in 59c · uses collect_file_name 173a, defs 134a, format_defs_refs 193c, format_uses_refs 192b, fprintf 21b, fputs 21b, Name 158a, nw_char 29ab, scraps 134b, scrap_flag 27, write_single_scrap_ref 135d

The fragment name's *content* is still emitted verbatim by the *Write the macro's name* fragment, so a programmer may put italics or bold face in the midst of the name; the surrounding `\itshape` and accent tint are only the default dress.

```

< Write macro definition 63a > ≡
{
    Name *name = collect_macro_name();
    int additive = (name->defs && name->defs->scrap != scraps);

    < Begin the scrap environment 64a >
    fputs("\\WEFTtarget{weft", tex_file);
    write_single_scrap_ref(tex_file, scraps);
    fputs("{}{}\\WEFTlangle{\\WEFTtint{weftaccent}\\itshape ", tex_file);
    < Write the macro's name 63b >
    fputs("\\nobreak\\WEFTnumsep{\\footnotesize\\WEFTtint{weftcom}", tex_file);
    write_single_scrap_ref(tex_file, scraps);
    fputs("\\WEFTrangle", tex_file);
    fputs(additive ? "\\WEFTpluseq" : "\\WEFTeq", tex_file);
    fputs("\\par\\nobreak\\vspace{0.6ex}\\nobreak\\n", tex_file);
    < Fill in the middle of the scrap environment 65b >
    < Begin the cross-reference environment 66e >
    < Write macro defs 67b >
    < Write macro refs 67c >
    format_defs_refs(tex_file, scraps);
    format_uses_refs(tex_file, scraps++);
    < Finish the cross-reference environment 66f >
    < Finish the scrap environment 66a >
}

```

63a · used in 59c · uses collect_macro_name 176b, defs 134a, format_defs_refs 193c, format_uses_refs 192b, fputs 21b, Name 158a, scraps 134b, write_single_scrap_ref 135d

```

< Write the macro's name 63b > ≡
{
    char * p = name->spelling;
    int i = 0;

    while (*p != '\\000') {
        if (*p == ARG_CHR) {
            write_arg(tex_file, name->arg[i++]);
            p++;
        }
        else

```

```

        fputc(*p++, tex_file);
    }
}
□
63b · used in 63a, 80c · uses ARG_CHR 178a, write_arg 63c

⟨ latex.c 63c ⟩ + ≡
static void write_arg(FILE * tex_file, char * p)
{
    fputs("\\hbox{\\slshape\\sffamily ", tex_file);
    while (*p)
    {
        switch (*p)
        {
            case '$':
            case '_':
            case '^':
            case '#':
                fputc('\\', tex_file);
                break;
            default:
                break;
        }
        fputc(*p, tex_file);
        p++;
    }

    fputs("\\\\/", tex_file);
}

```

□

63c · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines write_arg 55b, 61, 63b, 73b, 77 · uses FILE 21b, fputs 21b

A scrap is opened with `\WEFTbegin` and closed with `\WEFTend` (both defined among the scrap-formatting macros above). Both big (`@0`, `@D`) and small (`@o`, `@d`) definitions use the same breakable block; the `inBlock` bookkeeping lets a run of small definitions share one open group, separated by a little vertical space.

⟨ *Begin the scrap environment* 64a ⟩ ≡

```

{
    if (big_definition)
    {
        if (inBlock)
        {
            ⟨ End block 65a ⟩
        }
        fputs("\\WEFTbegin", tex_file);
    }
    else
    {
        if (inBlock)
        {
            ⟨ Switch block 64c ⟩
        }
        else
        {
            ⟨ Start block 64b ⟩
        }
    }
}

```

```

    }
    fprintf(tex_file, "\\label{scrap%d}\\n", scraps);
}

```

□

64a · used in 62, 63a · uses `fprintf 21b`, `fputs 21b`, `scraps 134b`

⟨ *Start block* 64b ⟩ ≡

```

fputs("\\WEFTbegin\\n", tex_file);
inBlock = TRUE;

```

□

64b · used in 51d, 64a · uses `fputs 21b`, `TRUE 23a`

⟨ *Switch block* 64c ⟩ ≡

```

fputs("\\par\\vspace{\\baselineskip}\\n", tex_file);

```

□

64c · used in 64a · uses `fputs 21b`

⟨ *End block* 65a ⟩ ≡

```

fputs("\\WEFTend\\n", tex_file);
inBlock = FALSE;

```

□

65a · used in 51d, 64a, 66a · uses `FALSE 23a`, `fputs 21b`

The body of a code scrap (`@{...@}`) is emitted as an `lstlisting`. We tag it with our fixed `escapeinside` pair and, when the scrap has a language listings understands, with a `language=` option so the code is highlighted.

Both ends of the listing must sit on their own lines. We emit a newline right after the closing `]` of the options, so the first line of code starts fresh; without it a scrap beginning with `#` (a C `#ifndef`, say) would land immediately after `]`, and `listings` reads that `#` as a macro parameter and aborts with *Illegal parameter number*. Symmetrically, `\end{lstlisting}` must start its own line, so we emit a newline before it even when the scrap body did not end in one. Immediately after the listing we emit the end-of-scrap tombstone `\WEFTsep` (see above) on its own line, closing the code; the metadata footer then follows below it. A non-code block scrap (the unusual `@d name @[...@]` or `@(...@)` forms) is emitted as before, without a listing and without a tombstone.

⟨ *Fill in the middle of the scrap environment* 65b ⟩ ≡

```

{
    int is_code = (source_last == '{');
    if (is_code) {
        const char *lstlang = latex_language(name->lang);
        /* the delimiter halves are split so this source line does not itself
           contain the escape sequence when weft typesets its own source */
        fputs("\\begin{lstlisting}[escapeinside={(*" "<{}>" "*)}]", tex_file);
        if (lstlang)
            fprintf(tex_file, ",language=%s", lstlang);
        /* newline after ']' so the first code line starts fresh; a scrap
           beginning with '#' would otherwise break listings (see prose above) */
        fputs("]\\n", tex_file);
    }
    extra_scraps = 0;
    copy_scrap(tex_file, TRUE, name);
    if (is_code) {
        /* the closing tag is split across two literals so that this very
           line does not end a listing when weft typesets its own source
           (the same reason this comment says "the closing tag" rather than

```

```

        spelling it out); the leading newline keeps it on its own line */
fputs("\n\\end{ls} " "tlisting}\n", tex_file);
/* the end-of-scrap tombstone (\WEFTsep): a left-flushed square on its
   own line, closing the code above the metadata footer that follows */
fputs("\\WEFTsep\n", tex_file);
}
}
□
65b · used in 62, 63a · uses copy_scrap 71b, extra_scraps 66b, first 197d, fprintf 21b, fputs 21b, latex_language 69b,
prose 134a, source_last 130b, TRUE 23a

```

We've got one last spacing command, controlling the amount of white space after a scrap.

Note also the whitespace eater. I use it to remove any blank lines that appear after a scrap in the source file. This way, text following a scrap will not be indented. Again, this is a matter of personal taste.

```
< Finish the scrap environment 66a > ≡
{
    scraps += extra_scraps;
    if (big_definition)
        fputs("\\WEFTend\\n", tex_file);
    else
    {
        < End block 65a >
    }
    do
        c = source_get();
    while (isspace(c));
}
```

66a · used in 62, 63a · uses `extra_scraps` 66b, `fputs` 21b, `isspace` 21b, `scraps` 134b, `source_get` 130b

```
< Global variable declarations 66b > + ≡
extern int extra_scraps;
```

66b · defined at 27, 29ac, 45c, 50a, 66b, 129b, 137e, 158b, 197a · used in 21a · defines `extra_scraps` 65b, 66ac, 76b, 89ab, 95b, 107c, 108a

```
< Global variable definitions 66c > + ≡
int extra_scraps = 0;
```

66c · defined at 28, 29bd, 46a, 50b, 66c, 129c, 137f, 158c · used in 25e · uses `extra_scraps` 66b

```
< Write in-text scrap 66d > ≡
copy_scrap(tex_file, FALSE, NULL);
c = source_get();
```

66d · used in 59c · uses `copy_scrap` 71b, `FALSE` 23a, `source_get` 130b

Formatting Cross References

```
< Begin the cross-reference environment 66e > ≡
{
    fputs("\\WEFTmetabegin{", tex_file);
    write_single_scrap_ref(tex_file, scraps);
    fputs("}", tex_file);}

```

66e · used in 62, 63a · uses `fputs` 21b, `scraps` 134b, `write_single_scrap_ref` 135d

The footer is one horizontal line, opened above with the scrap's own number. Every subsequent item is introduced by `\WEFTdot`, the grey separator, so there is never a leading or trailing dot to special-case; an item that has nothing to say simply emits nothing.

```
< Finish the cross-reference environment 66f > ≡
```

```
{
  fputs("\\WEFTmetaend\\n", tex_file);
}
```

□

66f · used in 62, 63a · uses fputs 21b

⟨ *Write file defs* 67a ⟩ ≡

```
{
  if (name->defs) {
    if (name->defs->next) {
      fputs("\\WEFTdot \\WEFTtxtFileDefBy\\ ", tex_file);
      print_scrap_numbers(tex_file, name->defs);
    }
  } else {
    fprintf(stderr,
      "would have crashed in 'Write file defs' for '%s'\\n",
      name->spelling);
  }
}
```

□

67a · used in 62 · uses defs 134a, fprintf 21b, fputs 21b, next 133c, print_scrap_numbers 67d, stderr 21b

⟨ *Write macro defs* 67b ⟩ ≡

```
{
  if (name->defs->next) {
    fputs("\\WEFTdot \\WEFTtxtMacroDefBy\\ ", tex_file);
    print_scrap_numbers(tex_file, name->defs);
  }
}
```

□

67b · used in 63a · uses defs 134a, fputs 21b, next 133c, print_scrap_numbers 67d

⟨ *Write macro refs* 67c ⟩ ≡

```
{
  if (name->uses) {
    if (name->uses->next) {
      fputs("\\WEFTdot \\WEFTtxtMacroRefIn\\ ", tex_file);
      print_scrap_numbers(tex_file, name->uses);
    }
    else {
      fputs("\\WEFTdot \\WEFTtxtMacroRefIn\\ ", tex_file);
      fputs("\\WEFTlink{weft", tex_file);
      write_single_scrap_ref(tex_file, name->uses->scrap);
      fputs("}{", tex_file);
      write_single_scrap_ref(tex_file, name->uses->scrap);
      fputs("}", tex_file);
    }
  }
  else {
    fputs("\\WEFTdot {\\WEFTtxtMacroNoRef}", tex_file);
    fprintf(stderr, "%s: <%s> never referenced.\\n",
      command_name, name->spelling);
  }
}
```

□

67c · used in 63a · uses `command_name` 29c, `fprintf` 21b, `fputs` 21b, `next` 133c, `print_scrap_numbers` 67d, `stderr` 21b, `uses` 134a, `write_single_scrap_ref` 135d

```
<latex.c 67d> +≡
static void print_scrap_numbers(FILE *tex_file, Scrap_Node *scraps)
{
    int page;
    fputs("\\WEFTlink{weft", tex_file);
    write_scrap_ref(tex_file, scraps->scrap, -1, &page);
    fputs("{}{", tex_file);
    write_scrap_ref(tex_file, scraps->scrap, TRUE, &page);
    fputs("}", tex_file);
    scraps = scraps->next;
    while (scraps) {
        fputs("\\WEFTlink{weft", tex_file);
        write_scrap_ref(tex_file, scraps->scrap, -1, &page);
        fputs("{}{", tex_file);
        write_scrap_ref(tex_file, scraps->scrap, FALSE, &page);
        scraps = scraps->next;
        fputs("}", tex_file);
    }
}
```

□

67d · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b ·

defines `print_scrap_numbers` 55b, 67abc, 78d, 81a · uses `FALSE` 23a, `FILE` 21b, `fputs` 21b, `next` 133c, `page` 134a, `scraps` 134b, `Scrap_Node` 157e, `TRUE` 23a, `write_scrap_ref` 135c

Formatting a Scrap

This is the only place we really care whether a scrap is delimited with `@{...@}`, `@[...@]`, or `@(...@)`, and we base our output sequences on that. The code (`@{...@}`) mode is now handled by `listings`, so its per-line strings are all empty: `copy_scrap` just copies the body out verbatim, one raw line per source line, and the surrounding `lstlisting` does the formatting. The `@[...@]` (verbatim `TEX`) and `@(...@)` (math) modes keep their old inline treatment.

We still keep the array `delimit_scrap`: it carries the small *begin/end* strings for the math mode (a pair of \$) and, for every mode, the single character that stands in for the weft character. Upon initialisation we copy the strings from `orig_delimit_scrap` into a writeable array; we might change the *insert nw_char* entry later, when the weft character changes.

```
<latex.c 68a> +≡
static char *orig_delimit_scrap[3][5] = {
    /* {} mode (code): begin, end, insert nw_char, prefix, suffix.
       All empty: listings copies the body verbatim, one raw line per
       source line, so there is no per-line wrapper. */
    { "", "", "@", "", "" },
    /* [] mode: begin, end, insert nw_char, prefix, suffix */
    { "", "", "@", "", "" },
    /* () mode: begin, end, insert nw_char, prefix, suffix */
    { "$", "$", "@", "", "" },
};
```

```
static char *delimit_scrap[3][5];
```

□

68a · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · uses `nw_char` 29ab

The function `initialise_delimit_scrap_array` just copies the originals into a writeable array.

< latex.c 68b > + ≡

```
void initialise_delimit_scrap_array(void) {
    int i,j;
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 5; j++) {
            if((delimit_scrap[i][j] = strdup(orig_delimit_scrap[i][j])) == NULL) {
                fprintf(stderr, "Not enough memory for string allocation\n");
                exit(EXIT_FAILURE);
            }
        }
    }
}
```

□

68b · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · uses `exit` 21b, `fprintf` 21b, `stderr` 21b

Copying one code character to the `.tex` file. Because the code is no longer wrapped in `\verb`, every character that is special to $\text{\LaTeX}$ has to be turned into something harmless in `\ttfamily` text. A literal space becomes `\WEFTsp` (a breakable, fixed-width the code body itself needs none of this — `listings` copies it verbatim. `latex_code_putc` is used only for the little bits of text that are emitted *inside* an escape (a fragment reference's name and its substituted arguments), which land in ordinary $\text{\LaTeX}$ text mode and so must have the $\text{\LaTeX}$ -special characters neutralised.

< latex.c 69a > + ≡

```
static void latex_code_putc(FILE *file, int c)
{
    switch (c) {
        case '\\': fputs("\\textbackslash{}", file); break;
        case '{': fputs("\\{", file); break;
        case '}': fputs("\\}", file); break;
        case '$': fputs("\\$", file); break;
        case '&': fputs("\\&", file); break;
        case '#': fputs("\\#", file); break;
        case '%': fputs("\\%", file); break;
        case '_': fputs("\\_", file); break;
        case '^': fputs("\\textasciicircum{}", file); break;
        case '~': fputs("\\textasciitilde{}", file); break;
        default: putc(c, file); break;
    }
}
```

□

69a · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `latex_code_putc` 55b, 70c, 71b, 76b · uses `FILE` 21b, `fputs` 21b, `putc` 21b

The language mapper turns a scrap's weft language tag (`"c"`, `"python"`, ...) into the exact name `listings` expects, or `NULL` when we have no safe mapping. Returning `NULL` (rather than guessing) matters: `listings` aborts the run on an unknown `language=`, so we only ever name languages it is known to ship.

< latex.c 69b > + ≡

```
static const char *latex_language(const char *lang)
{
    static const struct { const char *tag, *lst; } map[] = {
        {"c", "C"}, {"h", "C"},
        {"cpp", "C++"}, {"c++", "C++"}, {"cc", "C++"}, {"cxx", "C++"},
        {"hpp", "C++"}, {"hxx", "C++"},
        {"java", "Java"},
    }
```

```

{"python","Python"}, {"py","Python"},
{"sql","SQL"},
{"html","HTML"}, {"htm","HTML"}, {"xml","XML"},
{"php","PHP"},
{"perl","Perl"}, {"pl","Perl"},
{"ruby","Ruby"}, {"rb","Ruby"},
{"pascal","Pascal"}, {"pas","Pascal"},
{"lisp","Lisp"}, {"el","Lisp"}, {"scheme","Lisp"},
{"matlab","Matlab"},
{"tex","[LaTeX]TeX"}, {"latex","[LaTeX]TeX"},
{"make","make"}, {"makefile","make"},
{"awk","Awk"},
{"fortran","Fortran"}, {"f","Fortran"}, {"f90","Fortran"},
{"r","R"}, {"lua","Lua"}, {"ada","Ada"},
{"vhdl","VHDL"}, {"verilog","Verilog"},
{"erlang","erlang"}, {"cobol","COBOL"}, {"eiffel","Eiffel"},
{"ocaml","Caml"}, {"caml","Caml"},
{NULL,NULL}
};
int i;
if (lang == NULL || lang[0] == '\0')
    return NULL;
for (i = 0; map[i].tag; i++) {
    const char *a = lang, *b = map[i].tag;
    while (*a && *b && tolower((unsigned char)*a) == (unsigned char)*b) { a++; b++; }
    if (*a == '\0' && *b == '\0')
        return map[i].lst;
}
return NULL;
}

```

□

69b · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `latex_language` 55b, 65b

⟨ *Function prototypes* 70a ⟩ + ≡

```
void initialise_delimit_scrap_array(void);
```

□

70a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a

⟨ `latex.c` 70b ⟩ + ≡

```
int scrap_type = 0;
```

□

70b · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `scrap_type` 71ab, 73ab, 75c

⟨ `latex.c` 70c ⟩ + ≡

```
static void write_literal(FILE * tex_file, char * p, int mode)
{
    fputs(delimit_scrap[mode][0], tex_file);
    while (*p != '\000') {
        if (*p == nw_char)
            fputs(delimit_scrap[mode][2], tex_file);
        else if (mode == 0)
            latex_code_putc(tex_file, (unsigned char)*p);
        else
            fputc(*p, tex_file);
        p++;
    }
}
```

```

    }
    fputs(delimit_scrap[mode][1], tex_file);
}

```

70c · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `write_literal` 55b, 75c, 77 · uses `FILE` 21b, `fputs` 21b, `latex_code_putc` 69a, `nw_char` 29ab

Two little helpers bracket every place where we must step out of the verbatim body to run L^AT_EX (a fragment reference, a bold keyword, an italic title, a substituted parameter). In code mode this opens and closes a `listings` escape; in math mode it closes and reopens the math run; in the verbatim-T_EX (`@[...@]`) mode both are empty.

The escape delimiters are written with the two halves as separate string literals ("`*`" "`<`"). The C compiler concatenates them, so the emitted text is exactly `(*<`; but the *source* never contains the delimiter as one contiguous token, which matters because weft typesets its own source — otherwise these very lines would trip the `listings` tokeniser when they appear in a scrap.

When `code_inline` is set we are already rendering inside an escape (a parameter of a fragment reference), so we must *not* open a nested escape — both helpers do nothing in code mode.

`<latex.c 71a> + ≡`

```
int code_inline = FALSE;
```

```
static void begin_tex_escape(FILE *file) /* leave the verbatim body */
{
    if (scrap_type == 0) { if (!code_inline) fputs("(*" "<", file); }
    else fputs(delimit_scrap[scrap_type][1], file);
}
static void end_tex_escape(FILE *file) /* re-enter the verbatim body */
{
    if (scrap_type == 0) { if (!code_inline) fputs(">" "*)", file); }
    else fputs(delimit_scrap[scrap_type][0], file);
}

```

71a · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `begin_tex_escape` 50c, 73b, 75abc, `code_inline` 54b, 71b, `end_tex_escape` 50c, 73b, 75abc · uses `FALSE` 23a, `FILE` 21b, `fputs` 21b, `scrap_type` 70b, 71b

`<latex.c 71b> + ≡`

```
static void copy_scrap(FILE *file, int prefix, Name *name)
{
    int indent = 0;
    int c;
    int coll_h1 = 0, coll_h2 = 0, coll_warned = FALSE;
    char ** params = name->arg;
    if (source_last == '{') scrap_type = 0;
    if (source_last == '[') scrap_type = 1;
    if (source_last == '(') scrap_type = 2;
    c = source_get();
    if (prefix) fputs(delimit_scrap[scrap_type][3], file);
    fputs(delimit_scrap[scrap_type][0], file);
    while (1) {
        switch (c) {
            case '\n': if (code_inline) {
                putc(' ', file); /* an inline parameter stays on one line */
                indent = 0;
                break;
            }
        }
    }
}

```

```

        fputs(delimit_scrap[scrap_type][1], file);
        if (prefix) fputs(delimit_scrap[scrap_type][4], file);
        fputs("\n", file);
        if (prefix) fputs(delimit_scrap[scrap_type][3], file);
        fputs(delimit_scrap[scrap_type][0], file);
        indent = 0;
        break;
    case '\t': < Expand tab into spaces 72c >
        break;
    default:
        if (c==nw_char)
        {
            < Check at-sequence for end-of-scrap 73b >
            break;
        }
        if (code_inline)
            latex_code_putc(file, c);    /* inside an escape: neutralise LaTeX */
        else {
            < Note a delimiter collision 73a >
            putc(c, file);
        }

        indent++;
        break;
    }
    c = source_get();
}
}

```

□

71b · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `copy_scrap` 54b, 55b, 65b, 66d, `scrap_type` 70b, 71a, 73ab, 75c · uses `code_inline` 71a, `FALSE` 23a, `FILE` 21b, `fputs` 21b, `latex_code_putc` 69a, `Name` 158a, `nw_char` 29ab, `putc` 21b, `source_get` 130b, `source_last` 130b

When we encounter the command to change the ‘weft character’ we call this function. It updates the scrap formatting directives accordingly.

```

< latex.c 72a > + ≡
void update_delimit_scrap(void)
{
    /* Every mode's "insert nw_char" string is the single character that
       stands in for the weft character inside a scrap; keep them in sync
       with the (possibly changed) weft character. */
    delimit_scrap[0][2][0] = nw_char;
    delimit_scrap[1][2][0] = nw_char;
    delimit_scrap[2][2][0] = nw_char;
}

```

□

72a · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `update_delimit_scrap` 44d, 59c, 72b, 86a, 106a · uses `nw_char` 29ab

< Function prototypes 72b > + ≡

```
void update_delimit_scrap(void);
```

□

72b · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `update_delimit_scrap` 72a

< Expand tab into spaces 72c > ≡

```

{
    int delta = 8 - (indent % 8);
    indent += delta;
    while (delta > 0) {
        putc(' ', file);
        delta--;
    }
}

```

□

72c · used in 71b, 151b · uses `putc` 21b

Since the code body is copied verbatim, a scrap that literally contains our escape delimiter ((*< or >*)) would desynchronise listings' tokeniser. This never happens in ordinary source, but we watch a two-character rolling window and warn once per scrap if it does, so a corrupted listing is never silent.

⟨ *Note a delimiter collision* 73a ⟩ ≡

```

if (scrap_type == 0) {
    if ((coll_h2 == '(' && coll_h1 == '*' && c == '<') ||
        (coll_h2 == '>' && coll_h1 == '*' && c == '>')) {
        if (!coll_warned) {
            fprintf(stderr,
                "%s: warning: a scrap body contains the listings escape "
                "delimiter; its typeset code may be corrupted.\n", command_name);
            coll_warned = TRUE;
        }
    }
    coll_h2 = coll_h1;
    coll_h1 = c;
}

```

□

73a · used in 71b · uses `command_name` 29c, `fprintf` 21b, `scrap_type` 70b, 71b, `stderr` 21b, `TRUE` 23a

⟨ *Check at-sequence for end-of-scrap* 73b ⟩ ≡

```

{
    c = source_get();
    switch (c) {
        case 'c': ⟨ Show presence of a block comment 50c ⟩
            break;
        case 'x': ⟨ Copy label from source into(73c file) 74b ⟩
            break;
        case 'v': ⟨ Copy version info into file 74a ⟩
        case 's':
            break;
        case '+':
        case '-':
        case '*':
        case '|': ⟨ Skip over index entries 74d ⟩
            FALLTHROUGH;
        case ',':
        case ')':
        case ']':
        case '}': fputs(delimit_scrap[scrap_type][1], file);
            return;
        case '<': ⟨ Format macro name 75c ⟩
            break;
        case '%': ⟨ Skip commented-out code 74e ⟩
            break;
    }
}

```

```

case '_': < Bold Keyword 75a >
    break;
case 't': < Italic "fragment title" 75b >
    break;
case 'f': < Italic "file name" 75b >
    break;
case '1': case '2': case '3':
case '4': case '5': case '6':
case '7': case '8': case '9':
    if (name == NULL
        || name->arg[c - '1'] == NULL) {
        fputs(delimit_scrap[scrap_type][2], file);
        fputc(c, file);
    }
    else {
        begin_tex_escape(file);
        write_arg(file, name->arg[c - '1']);
        end_tex_escape(file);
    }
    break;
default:
    if (c==nw_char)
    {
        fputs(delimit_scrap[scrap_type][2], file);
        break;
    }
    /* ignore these since pass1 will have warned about them */
    break;
}
}

```

□

73c · used in 71b

< *Copy version info into file* 74a > ≡
fputs(version_string, file);

□

74a · used in 73b, 93a, 151c · uses fputs 21b, version_string 27

< *Copy label from source into* 74b > ≡
{
 < *Get label from*(74c source.get()) 195a >
 write_label(label_name, @1);
}

□

74c · used in 59c, 73b, 86a, 93a, 106a · uses source_get 130b

There's no need to check for errors here, since we will have already pointed out any during the first pass.

< *Skip over index entries* 74d > ≡
{
 do {
 do
 c = source_get();
 while (c != nw_char);
 c = source_get();
 } while (c != '}' && c != ']' && c != ')');
 }
}

□

74d · used in 73b, 93a · uses `nw_char` 29ab, `source_get` 130b

⟨ *Skip commented-out code* 74e ⟩ ≡

```
{
    do
        c = source_get();
        while (c != '\n');
}
```

□

74e · used in 73b, 93a, 139b · uses `source_get` 130b

This scrap helps deal with bold keywords:

⟨ *Bold Keyword* 75a ⟩ ≡

```
{
    begin_tex_escape(file);
    fprintf(file, "\\hbox{\\sffamily\\bfseries ");
    c = source_get();
    do {
        fputc(c, file);
        c = source_get();
    } while (c != nw_char);
    c = source_get();
    fprintf(file, "}");
    end_tex_escape(file);
}
```

□

75a · used in 73b · uses `begin_tex_escape` 71a, `end_tex_escape` 71a, `fprintf` 21b, `nw_char` 29ab, `source_get` 130b

⟨ *Italic "whatever"* 75b ⟩ ≡

```
{
    begin_tex_escape(file);
    fprintf(file, "\\hbox{\\sffamily\\slshape whatever}");
    end_tex_escape(file);
}
```

□

75b · used in 73b · uses `begin_tex_escape` 71a, `end_tex_escape` 71a, `fprintf` 21b

⟨ *Format macro name* 75c ⟩ ≡

```
{
    Arglist *args = collect_scrap_name(-1);
    Name *name = args->name;
    char *p = name->spelling;
    Arglist *q = args->args;
    int nargs = 0;

    begin_tex_escape(file);
    if (prefix)
        /* \normalfont resets the listing's \ttfamily basicstyle, so the
           fragment name is set in the ordinary document font (roman italic)
           rather than as italic typewriter. */
        fputs("\\hbox{\\normalfont ", file);
    fputs("\\WEFTlangle{\\WEFTtint{weftaccent}\\itshape ", file);
    while (*p != '\\000') {
        if (*p == ARG_CHR) {
```

```

    if (q == NULL) {
        write_literal(file, name->arg[narg], scrap_type);
    }
    else {
        write_ArglistElement(file, q, params);
        q = q->next;
    }
    p++;
    narg++;
}
else
    fputc(*p++, file);
}
fputs("{}\\nobreak", file);
if (scrap_name_has_parameters) {
    < Format macro parameters 54b >
}
fprintf(file, "\\WEFTnumsep{\\footnotesize\\WEFTtint{weftcom}");
if (name->defs)
    < Write abbreviated definition list 76a >
else {
    putc('?', file);
    fprintf(stderr, "%s: never defined <%s>\\n",
        command_name, name->spelling);
}
fputs("{}\\WEFTtriangle", file);
if (prefix)
    fputs("}", file);
end_tex_escape(file);
}

```

□

75c · used in 73b · uses Arglist 179b, ARG_CHR 178a, begin_tex_escape 71a, collect_scrap_name 179d, command_name 29c, defs 134a, end_tex_escape 71a, fprintf 21b, fputs 21b, Name 158a, next 133c, putc 21b, scrap_type 70b, 71b, stderr 21b, write_ArglistElement 77, write_literal 70c

< *Write abbreviated definition list* 76a > ≡

```

{
    Scrap_Node *p = name->defs;
    fputs("\\WEFTlink{weft", file);
    write_single_scrap_ref(file, p->scrap);
    fputs("{}{", file);
    write_single_scrap_ref(file, p->scrap);
    fputs("}", file);
    p = p->next;
    if (p)
        fputs(", \\ldots\\ ", file);
}

```

□

76a · used in 75c, 77 · uses defs 134a, fputs 21b, next 133c, Scrap_Node 157e, write_single_scrap_ref 135d

A quoted scrap (`@{...@}`) can appear as the argument of a parameterised fragment reference, e.g. `@<Name @{code@} eman@>`. In the old `\verb`-based output that code was simply set in a verbatim run in the middle of the reference. Under `listings` we cannot leave the escape here — the reference is being built inside an `\hbox`, and closing the escape mid-box would leave the braces straddling the escape boundary (a ! Extra } error). Instead we capture the scrap's expansion into a memory buffer and re-emit it *inside* the escape, as L^AT_EX-escaped typewriter text on a single line, so the box stays balanced.

< Emit a quoted scrap argument inline 76b > ≡

```
{
  Scrap_Node * qq = (Scrap_Node *)q;
  char *buf = NULL;
  size_t sz = 0;
  FILE *mem = open_memstream(&buf, &sz);
  qq->quoted = TRUE;
  if (mem) {
    char *s;
    write_scraps(mem, "", qq, -1, "", 0, 0, 0, 0, 0, NULL, params, 0, "");
    fclose(mem);
    fputs("\\hbox{\\ttfamily ", file);
    for (s = buf; *s != '\\0'; s++) {
      if (*s == '\\n')
        putc(' ', file);
      else
        latex_code_putc(file, (unsigned char)*s);
    }
    fputs("}", file);
    free(buf);
  }
  extra_scraps++;
  qq->quoted = FALSE;
}
```

□

76b · used in 77 · uses [extra_scraps 66b](#), [FALSE 23a](#), [fclose 21b](#), [FILE 21b](#), [fputs 21b](#), [latex_code_putc 69a](#), [putc 21b](#), [Scrap_Node 157e](#), [size_t 21b](#), [TRUE 23a](#), [write_scraps 147c](#)

< latex.c 77 > + ≡

```
static void
write_ArglistElement(FILE * file, Arglist * args, char ** params)
{
  Name *name = args->name;
  Arglist *q = args->args;

  if (name == NULL) {
    char * p = (char*)q;

    if (p[0] == ARG_CHR) {
      write_arg(file, params[p[1] - '1']);
    } else {
      write_literal(file, (char *)q, 0);
    }
  } else if (name == (Name *)1) {
    < Emit a quoted scrap argument inline 76b >
  } else {
    char * p = name->spelling;
    fputs("$\\langle\\,$\\itshape ", file);
    while (*p != '\\000') {
      if (*p == ARG_CHR) {
        write_ArglistElement(file, q, params);
        q = q->next;
        p++;
      }
      else
        fputc(*p++, file);
    }
    fputs("\\nobreak\\ ", file);
  }
}
```

```

if (scrap_name_has_parameters) {
    int c;

    < Format macro parameters 54b >
}
fprintf(file, "{\\footnotesize ");
if (name->defs)
    < Write abbreviated definition list 76a >
else {
    putc('?', file);
    fprintf(stderr, "%s: never defined <%s>\n",
            command_name, name->spelling);
}
fputs("}$\\,\\rangle$", file);
}
}

```

□

77 · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b ·

defines `write_ArglistElement` 55b, 75c · uses `Arglist` 179b, `ARG_CHR` 178a, `command_name` 29c, `defs` 134a, `FILE` 21b, `fprintf` 21b, `fputs` 21b, `Name` 158a, `next` 133c, `putc` 21b, `stderr` 21b, `write_arg` 63c, `write_literal` 70c

2.4.2 Generating the Indices

The three back-of-book indices — of file names, fragment names, and user-specified identifiers — are each a compact vertical list, one entry per line. Their entries share the scrap presentation’s visual language: a fragment name is set in the accent-tinted italic inside the grey `\WEFTlangle/\WEFTrangle` brackets, a file name and an identifier in accent and keyword-coloured monospace respectively, and each entry’s `\WEFTtxtDefBy/\WEFTtxtRefIn` metadata in the small grey used everywhere else. Colour flows through the same `\WEFTtint` indirection, so the indices too degrade to black and white when no colour package is loaded.

```

< Write index of file names 78a > ≡
{
    if (file_names) {
        fputs("\n{\\small\\begin{list}{}{\\setlength{\\itemsep}{-\\parsep}",
              tex_file);
        fputs("\\setlength{\\itemindent}{-\\leftmargin}}\\n", tex_file);
        format_file_entry(file_names, tex_file);
        fputs("\\end{list}}", tex_file);
    }
    c = source_get();
}

```

□

78a · used in 59c · uses `file_names` 158b, `format_file_entry` 78b, `fputs` 21b, `source_get` 130b

```

< latex.c 78b > + ≡
static void format_file_entry(Name *name, FILE *tex_file)
{
    while (name) {
        format_file_entry(name->llink, tex_file);
        < Format a file index entry 78c >
        name = name->rlink;
    }
}

```

□

78b · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `format_file_entry` 55b, 78a · uses `FILE` 21b, `Name` 158a

< Format a file index entry 78c > ≡

```
fputs("\\item ", tex_file);
fprintf(tex_file, "{\\WEFTtint{weftaccent}\\verb%c%s%c}\\ ",
        nw_char, name->spelling, nw_char);
< Write file's defining scrap numbers 78d >
putc('\\n', tex_file);
```

□

78c · used in 78b · uses [fprintf 21b](#), [fputs 21b](#), [nw_char 29ab](#), [putc 21b](#)

< Write file's defining scrap numbers 78d > ≡

```
{
    Scrap_Node *p = name->defs;
    fputs("{\\footnotesize\\WEFTtint{weftcom} {\\WEFTtxtDefBy}", tex_file);
    putc(' ', tex_file);
    if (p->next) {
        print_scrap_numbers(tex_file, p);
    }
    else {
        fputs("\\WEFTlink{weft", tex_file);
        write_single_scrap_ref(tex_file, p->scrap);
        fputs("{}{", tex_file);
        write_single_scrap_ref(tex_file, p->scrap);
        fputs("}", tex_file);
    }
    putc('}', tex_file);
}
```

□

78d · used in 78c · uses [defs 134a](#), [fputs 21b](#), [next 133c](#), [print_scrap_numbers 67d](#), [putc 21b](#), [Scrap_Node 157e](#), [write_single_scrap_ref 135d](#)

< Write index of macro names 79a > ≡

```
{
    unsigned char sector = current_sector;
    int c = source_get();
    if (c == '+')
        sector = 0;
    else
        source_ungetc(&c);
    if (has_sector(macro_names, sector)) {
        fputs("\\n{\\small\\begin{list}{}{\\setlength{\\itemsep}{-\\parsep}",
              tex_file);
        fputs("\\setlength{\\itemindent}{-\\leftmargin}}\\n", tex_file);
        format_entry(macro_names, tex_file, sector);
        fputs("\\end{list}}", tex_file);
    } else {
        fputs("None.\\n", tex_file);
    }
}
c = source_get();
```

□

79a · used in 59c · uses [current_sector 45c](#), [format_entry 79c](#), [fputs 21b](#), [has_sector 81c](#), [macro_names 158b](#), [sector 134a](#), [source_get 130b](#), [source_ungetc 130d](#)

< latex.c 79b > + ≡

```
static int load_entry(Name * name, Name ** nms, int n)
{
```

```

    while (name) {
        n = load_entry(name->llink, nms, n);
        nms[n++] = name;
        name = name->rlink;
    }
    return n;
}

```

□

79b · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `load_entry` 79c · uses `Name` 158a

⟨`latex.c` 79c⟩ + ≡

```

static void format_entry(Name *name, FILE *tex_file, unsigned char sector)
{
    Name ** nms = malloc(num_scraps()*sizeof(Name *));
    int n = load_entry(name, nms, 0);
    int i;

    ⟨Sort nms of size n for ⟨Rob's ordering 80a⟩ 80b⟩
    for (i = 0; i < n; i++)
    {
        Name * name = nms[i];

        ⟨Format an index entry 80c⟩
    }
}

```

□

79c · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `format_entry` 55b, 79a · uses `FILE` 21b, `load_entry` 79b, `malloc` 21b, `Name` 158a, `num_scraps` 134b, `sector` 134a

⟨*Rob's ordering* 80a⟩ ≡

```

robs_strcmp(ki->spelling, kj->spelling) < 0

```

□

80a · used in 79c, 98b · uses `robs_strcmp` 161c

⟨*Sort key of size n for ordering* 80b⟩ ≡

```

int j;
for (j = 1; j < n; j++)
{
    int i = j - 1;
    Name * kj = key[j];

    do
    {
        Name * ki = key[i];

        if (ordering)
            break;
        key[i + 1] = ki;
        i -= 1;
    } while (i >= 0);
    key[i + 1] = kj;
}

```

□

80b · used in 79c, 98b · uses `Name` 158a

```

< Format an index entry 80c > ≡
if (name->sector == sector){
    fputs("\\item ", tex_file);
    fputs("\\WEFTlangle{\\WEFTtint{weftaccent}\\itshape ", tex_file);
    < Write the macro's name 63b >
    fputs("\\nobreak\\WEFTnumsep{\\footnotesize\\WEFTtint{weftcom}", tex_file);
    < Write defining scrap numbers 80d >
    fputs("\\WEFTrangle\\ ", tex_file);
    < Write referencing scrap numbers 81a >
    putc('\\n', tex_file);
}

```

□

80c · used in 79c · uses fputs 21b, putc 21b, sector 134a

```

< Write defining scrap numbers 80d > ≡
{
    Scrap_Node *p = name->defs;
    if (p) {
        int page;
        fputs("\\WEFTlink{weft", tex_file);
        write_scrap_ref(tex_file, p->scrap, -1, &page);
        fputs("{", tex_file);
        write_scrap_ref(tex_file, p->scrap, TRUE, &page);
        fputs("}", tex_file);
        p = p->next;
        while (p) {
            fputs("\\WEFTlink{weft", tex_file);
            write_scrap_ref(tex_file, p->scrap, -1, &page);
            fputs("{", tex_file);
            write_scrap_ref(tex_file, p->scrap, FALSE, &page);
            fputs("}", tex_file);
            p = p->next;
        }
    }
    else
        putc('?', tex_file);
}

```

□

80d · used in 80c · uses defs 134a, FALSE 23a, fputs 21b, next 133c, page 134a, putc 21b, Scrap_Node 157e, TRUE 23a, write_scrap_ref 135c

```

< Write referencing scrap numbers 81a > ≡
{
    Scrap_Node *p = name->uses;
    fputs("{\\footnotesize\\WEFTtint{weftcom} ", tex_file);
    if (p) {
        fputs("{\\WEFTtxtRefIn}", tex_file);
        putc(' ', tex_file);
        if (p->next) {
            print_scrap_numbers(tex_file, p);
        }
        else {
            fputs("\\WEFTlink{weft", tex_file);
            write_single_scrap_ref(tex_file, p->scrap);
            fputs("{", tex_file);
            write_single_scrap_ref(tex_file, p->scrap);
            fputs("}", tex_file);
        }
    }
}

```

```

    }
}
else
    fputs("\\WEFTtxtNoRef", tex_file);
putc('}', tex_file);
}

```

□

81a · used in 80c · uses `fputs` 21b, `next` 133c, `print_scrap_numbers` 67d, `putc` 21b, `Scrap_Node` 157e, uses 134a, `write_single_scrap_ref` 135d

⟨ *Function prototypes* 81b ⟩ + ≡

```
extern int has_sector(Name *, unsigned char);
```

□

81b · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `has_sector` 81c, `Name` 158a

⟨ *latex.c* 81c ⟩ + ≡

```
int has_sector(Name * name, unsigned char sector)
```

```

{
    while(name) {
        if (name->sector == sector)
            return TRUE;
        if (has_sector(name->llink, sector))
            return TRUE;
        name = name->rlink;
    }
    return FALSE;
}

```

□

81c · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines `has_sector` 79a, 81b, 82a, 97d, 99c, 109b, 110a · uses `FALSE` 23a, `Name` 158a, `sector` 134a, `TRUE` 23a

⟨ *Write index of user-specified names* 82a ⟩ ≡

```

{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    if (has_sector(user_names, sector)) {
        fputs("\n{\\small\\begin{list}}{\\setlength{\\itemsep}{-\\parsep}",
            tex_file);
        fputs("\\setlength{\\itemindent}{-\\leftmargin}}\\n", tex_file);
        format_user_entry(user_names, tex_file, sector);
        fputs("\\end{list}}", tex_file);
    }
}

```

□

82a · used in 59c · uses `current_sector` 45c, `format_user_entry` 82b, `fputs` 21b, `has_sector` 81c, `sector` 134a, `source_get` 130b, `user_names` 158b

⟨ *latex.c* 82b ⟩ + ≡

```

static void format_user_entry(Name *name, FILE *tex_file, unsigned char sector)
{
    while (name) {

```

```

    format_user_entry(name->llink, tex_file, sector);
    < Format a user index entry 82c >
    name = name->rlink;
}
}
□
82b · defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b · defines format_user_entry 55b,
82a · uses FILE 21b, Name 158a, sector 134a

< Format a user index entry 82c > ≡
if (name->sector == sector){
    Scrap_Node *uses = name->uses;
    if ( uses || dangling_flag ) {
        int page;
        Scrap_Node *defs = name->defs;
        fprintf(tex_file, "\\item {\\WEFTtint{weftkw}\\verb%c%s%c}: ",
            nw_char,name->spelling,nw_char);
        if (!uses) {
            fputs("\\underline{", tex_file);
            fputs("\\WEFTlink{weft", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("}{", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("})", tex_file);
            page = -2;
            defs = defs->next;
        }
        else
            if (!defs || uses->scrap < defs->scrap) {
                fputs("\\WEFTlink{weft", tex_file);
                write_scrap_ref(tex_file, uses->scrap, -1, &page);
                fputs("}{", tex_file);
                write_scrap_ref(tex_file, uses->scrap, TRUE, &page);
                fputs("}", tex_file);
                uses = uses->next;
            }
        else {
            if (defs->scrap == uses->scrap)
                uses = uses->next;
            fputs("\\underline{", tex_file);

            fputs("\\WEFTlink{weft", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("}{", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("})", tex_file);
            page = -2;
            defs = defs->next;
        }
    }
    while (uses || defs) {
        if (uses && (!defs || uses->scrap < defs->scrap)) {
            fputs("\\WEFTlink{weft", tex_file);
            write_scrap_ref(tex_file, uses->scrap, -1, &page);
            fputs("}{", tex_file);
            write_scrap_ref(tex_file, uses->scrap, FALSE, &page);
            fputs("}", tex_file);
            uses = uses->next;
        }
    }
}

```

```

    else {
        if (uses && defs->scrap == uses->scrap)
            uses = uses->next;
        fputs(" \\underline{", tex_file);

        fputs("\\WEFTlink{weft", tex_file);
        write_single_scrap_ref(tex_file, defs->scrap);
        fputs("{", tex_file);
        write_single_scrap_ref(tex_file, defs->scrap);
        fputs("}", tex_file);

        putc('}', tex_file);
        page = -2;
        defs = defs->next;
    }
}
fputs(".\\n", tex_file);
}

```

□

82c · used in 82b · uses [dangling_flag](#) 27, [defs](#) 134a, [FALSE](#) 23a, [fprintf](#) 21b, [fputs](#) 21b, [next](#) 133c, [nw_char](#) 29ab, [page](#) 134a, [putc](#) 21b, [Scrap_Node](#) 157e, [sector](#) 134a, [TRUE](#) 23a, [uses](#) 134a, [write_scrap_ref](#) 135c, [write_single_scrap_ref](#) 135d

2.5 Writing the Markdown File

This section implements the Markdown weave output, activated by the `-w md` command-line flag. It is structurally parallel to the L^AT_EX output in Section 2.4, but emits Markdown with embedded HTML for scraps (using `<pre><code>` blocks so that fragment references inside code can be clickable links).

⟨ *Function prototypes* 84a ⟩ + ≡

```
extern void write_md(char *file_name, char *md_name, unsigned char sector);
```

□

84a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses [file_name](#) 134a, [sector](#) 134a, [write_md](#) 84d

The HTML weaver (Section 2.6) reuses two format-agnostic helpers from here, so they are declared `extern` rather than `static`: `md_copy_scrap` copies a scrap body emitting HTML-escaped code and clickable inline fragment references, and `md_print_scrap_numbers` emits a comma-separated list of `<a href="#weftN">` cross-reference links. Both are pure HTML and identical to what the HTML output needs.

⟨ *Function prototypes* 84b ⟩ + ≡

```
extern void md_copy_scrap(FILE *file, int prefix, Name *name);
extern void md_print_scrap_numbers(FILE *md_file, Scrap_Node *scraps);
```

□

84b · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses [FILE](#) 21b, [md_copy_scrap](#) 91b, [md_print_scrap_numbers](#) 91a, [Name](#) 158a, [scraps](#) 134b, [Scrap_Node](#) 157e

We need a few local function declarations before we get into the body of `write_md`.

⟨ *markdown.c* 84c ⟩ + ≡

```
static void md_format_entry(Name *name, FILE *md_file, unsigned char sector);
static void md_format_file_entry(Name *name, FILE *md_file);
static void md_format_user_entry(Name *name, FILE *md_file, unsigned char sector);
static void md_write_arg(FILE *md_file, char *p);

```

```
static void md_write_literal(FILE *md_file, char *p);
static void md_write_ArglistElement(FILE *file, Arglist *args, char **params);
static void md_format_uses_refs(FILE *md_file, int scrap);
static void md_format_defs_refs(FILE *md_file, int scrap);
```

□

84c · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · uses Arglist 179b, FILE 21b, md_format_defs_refs 102c, md_format_entry 98b, md_format_file_entry 97a, md_format_user_entry 100a, md_format_uses_refs 101a, md_write_arg 88c, md_write_ArglistElement 95b, md_write_literal 95a, Name 158a, sector 134a

The routine `write_md` takes two file names as parameters: the name of the web source file and the name of the .md output file.

⟨markdown.c 84d⟩ + ≡

```
void write_md(char *file_name, char *md_name, unsigned char sector)
{
    (void)sector;
    FILE *md_file = fopen(md_name, "w");
    if (md_file) {
        if (verbose_flag)
            fprintf(stderr, "writing %s\n", md_name);
        source_open(file_name);
        ⟨ Copy source_file into md_file 85a ⟩
        fclose(md_file);
    }
    else
        fprintf(stderr, "%s: can't open %s\n", command_name, md_name);
}
```

□

84d · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines write_md 39b, 84a · uses command_name 29c, fclose 21b, FILE 21b, file_name 134a, fopen 21b, fprintf 21b, sector 134a, source_open 133b, stderr 21b, verbose_flag 27

We make our second pass through the source web, copying characters straight into the .md file. We watch for @ characters which signal a command sequence. We also detect and discard cosmetic fence lines (lines that are exactly “” optionally followed by a language tag).

⟨ Copy source_file into md_file 85a ⟩ ≡

```
{
    int c = source_get();
    int at_line_start = TRUE;
    while (c != EOF) {
        if (c == nw_char)
        {
            ⟨ Interpret md at-sequence 86a ⟩
        }
        else if (c == ' ' && at_line_start) {
            ⟨ Check for cosmetic fence line 85b ⟩
        }
        else {
            if (c == '\n') at_line_start = TRUE;
            else at_line_start = FALSE;
            putc(c, md_file);
            c = source_get();
        }
    }
}
```

□

85a · used in 84d · uses FALSE 23a, nw_char 29ab, putc 21b, source_get 130b, TRUE 23a

Cosmetic fence lines are lines that start with ‘‘‘. We peek ahead to see if the line matches; if so, we skip to end of line. Otherwise we output what we already consumed and continue.

```
⟨ Check for cosmetic fence line 85b ⟩ ≡
{
  int c2 = source_get();
  int c3 = (c2 == ' ') ? source_get() : 0;
  if (c2 == ' ' && c3 == ' ') {
    /* Skip rest of fence line */
    int fc = source_get();
    while (fc != '\n' && fc != EOF)
      fc = source_get();
    c = source_get();
    at_line_start = TRUE;
  } else {
    /* Not a fence, output what we consumed */
    putc(' ', md_file);
    if (c2 == ' ') {
      putc(' ', md_file);
      c = c3;
    } else {
      c = c2;
    }
    at_line_start = FALSE;
  }
}
□
```

85b · used in 85a · uses FALSE 23a, putc 21b, source_get 130b, TRUE 23a

```
⟨ Interpret md at-sequence 86a ⟩ ≡
{
  int big_definition = FALSE;
  c = source_get();
  switch (c) {
    case 'r':
      c = source_get();
      nw_char = c;
      update_delimit_scrap();
      break;
    case '0': big_definition = TRUE;
              FALLTHROUGH;
    case 'o': ⟨ Write md output file definition 87c ⟩
              break;
    case 'Q':
    case 'D': big_definition = TRUE;
              FALLTHROUGH;
    case 'q':
    case 'd': ⟨ Write md macro definition 88a ⟩
              break;
    case 's':
              ⟨ Step to next sector 45a ⟩
              break;
    case 'S':
              ⟨ Close the current sector 45b ⟩
              break;
    case '{':
```

```

case '[':
case '(': < Write md in-text scrap 89c >
    break;
case '<': < Expand md macro into md_file 87b >
    break;
case 'x': < Copy label from source into(86b md_file ) 74b >
    c = source_get();
    break;
case 'c': /* Block comments: no special handling in Markdown */
    c = source_get();
    break;
case 'f': < Write md index of file names 96 >
    break;
case 'm': < Write md index of macro names 97d >
    break;
case 'u': < Write md index of user-specified names 99c >
    break;
case 'v': < Copy version info into md file 87a >
    break;
case 'l':
case 'L': < Skip at-L language definition line 44c >
    break;
case 'W': < Skip at-L language definition line 44c >
    break;
default:
    if (c==nw_char)
        putc(c, md_file);
    c = source_get();
    break;
}
}

```

□
86b · used in 85a

< Copy version info into md file 87a > ≡
fputs(version_string, md_file);
c = source_get();

□

87a · used in 86a · uses fputs 21b, source_get 130b, version_string 27

< Expand md macro into file 87b > ≡

```

{
    Parameters local_parameters = 0;
    int changed;
    char indent_chars[MAX_INDENT];
    Arglist *a;
    Name *name;
    Arglist *args;
    a = collect_scrap_name(0);
    name = a->name;
    args = instance(a->args, NULL, NULL, &changed);
    name->mark = TRUE;
    write_scrap(file, md_name, name->defs, 0, indent_chars, 0, 0, 1, 0, 0,
        args, name->arg, local_parameters, md_name);
    name->mark = FALSE;
    c = source_get();
}

```

□

87b · used in 86a · uses [Arglist 179b](#), [collect_scrap_name 179d](#), [defs 134a](#), [FALSE 23a](#), [instance 145e](#), [MAX_INDENT 125d](#), [Name 158a](#), [Parameters 52b](#), [source_get 130b](#), [TRUE 23a](#), [write_scraps 147c](#)

2.5.1 Formatting Definitions in Markdown

File and fragment definitions use HTML anchors and formatting. A file definition header looks like:

```
<a id="weft1"></a>
**"filename" ** *1** =
<pre><code>...code...</code></pre>
<div align="left">&#9633;</div>
```

A fragment definition header looks like:

```
<a id="weft1"></a>
**&lt;*Fragment Name* 1&gt;* ** =
<pre><code>...code...</code></pre>
<div align="left">&#9633;</div>
```

The left-flushed □ square is the end-of-scrap tombstone, closing the code before the cross-reference blockquotes — the Markdown twin of the L^AT_EX \WEFTsep and the HTML .weft-end.

```
< Write md output file definition 87c > ≡
{
  Name *name = collect_file_name();
  (void)big_definition;
  fputs("\n<a id=\"weft\", md_file);
  write_single_scrap_ref(md_file, scraps);
  fputs("\n"></a>\n", md_file);
  fprintf(md_file, "**\"%s\"**", name->spelling);
  fputs(" **", md_file);
  write_single_scrap_ref(md_file, scraps);
  fputs("** =\n", md_file);
  < Fill in the md scrap 89a >
  < Write md file cross-refs 90a >
  < Finish md scrap 89b >
}
```

□

87c · used in 86a · uses [collect_file_name 173a](#), [fprintf 21b](#), [fputs 21b](#), [Name 158a](#), [scraps 134b](#), [write_single_scrap_ref 135d](#)

```
< Write md macro definition 88a > ≡
{
  Name *name = collect_macro_name();
  (void)big_definition;
  fputs("\n<a id=\"weft\", md_file);
  write_single_scrap_ref(md_file, scraps);
  fputs("\n"></a>\n", md_file);
  fputs("**&lt;* ", md_file);
  < Write the md macro's name 88b >
  fputs("* ", md_file);
  write_single_scrap_ref(md_file, scraps);
  fputs("&gt;* =\n", md_file);
  < Fill in the md scrap 89a >
  < Write md macro cross-refs 90c >
  < Finish md scrap 89b >
}
```

□

88a · used in 86a · uses `collect_macro_name` 176b, `fputs` 21b, `Name` 158a, `scraps` 134b, `write_single_scrap_ref` 135d

⟨ *Write the md macro's name* 88b ⟩ ≡

```
{
    char * p = name->spelling;
    int i = 0;

    while (*p != '\000') {
        if (*p == ARG_CHR) {
            md_write_arg(md_file, name->arg[i++]);
            p++;
        }
        else
            fputc(*p++, md_file);
    }
}
```

□

88b · used in 88a, 98c · uses `ARG_CHR` 178a, `md_write_arg` 88c

⟨ *markdown.c* 88c ⟩ + ≡

```
static void md_write_arg(FILE * md_file, char * p)
{
    fputs("<i>", md_file);
    while (*p)
    {
        switch (*p)
        {
            case '<':
                fputs("&lt;", md_file);
                break;
            case '>':
                fputs("&gt;", md_file);
                break;
            case '&':
                fputs("&amp;", md_file);
                break;
            default:
                fputc(*p, md_file);
                break;
        }
        p++;
    }
    fputs("</i>", md_file);
}
```

□

88c · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines `md_write_arg` 84c, 88b, 93a, 95b · uses `FILE` 21b, `fputs` 21b

⟨ *Fill in the md scrap* 89a ⟩ ≡

```
{
    if (name->lang[0] != '\0')
        fprintf(md_file, "<pre><code class=\"%s\">", name->lang);
    else
        fputs("<pre><code>", md_file);
    extra_scraps = 0;
    md_copy_scrap(md_file, TRUE, name);
}
```

```

fputs("</code></pre>\n", md_file);
/* end-of-scrap tombstone: a left-aligned square closing the code, the
   Markdown twin of the LaTeX \WEFTsep and the HTML .weft-end. We use the
   GitHub-safe <div align> since a Markdown file carries no stylesheet */
fputs("<div align=\"left\">&#9633;</div>\n", md_file);
}

```

□

89a · used in 87c, 88a · uses [extra_scrap](#) 66b, [fprintf](#) 21b, [fputs](#) 21b, [md_copy_scrap](#) 91b, [TRUE](#) 23a

⟨ *Finish md scrap* 89b ⟩ ≡

```

{
    scraps += extra_scrap;
    do
        c = source_get();
        while (isspace(c));
        at_line_start = TRUE;
    }
}

```

□

89b · used in 87c, 88a · uses [extra_scrap](#) 66b, [isspace](#) 21b, [scraps](#) 134b, [source_get](#) 130b, [TRUE](#) 23a

⟨ *Write md in-text scrap* 89c ⟩ ≡

```

md_copy_scrap(md_file, FALSE, NULL);
c = source_get();

```

□

89c · used in 86a · uses [FALSE](#) 23a, [md_copy_scrap](#) 91b, [source_get](#) 130b

Cross References in Markdown

Cross-references appear as blockquotes after each scrap.

⟨ *Write md file cross-refs* 90a ⟩ ≡

```

{
    if ( scrap_flag ) {
        ⟨ Write md file defs 90b ⟩
    }
    md_format_defs_refs(md_file, scraps);
    md_format_uses_refs(md_file, scraps++);
}

```

□

90a · used in 87c · uses [md_format_defs_refs](#) 102c, [md_format_uses_refs](#) 101a, [scraps](#) 134b, [scrap_flag](#) 27

⟨ *Write md file defs* 90b ⟩ ≡

```

{
    if (name->defs) {
        if (name->defs->next) {
            fputs("> *File defined by* ", md_file);
            md_print_scrap_numbers(md_file, name->defs);
        }
    } else {
        fprintf(stderr,
            "would have crashed in 'Write md file defs' for '%s'\n",
            name->spelling);
    }
}

```

□

90b · used in 90a · uses defs 134a, fprintf 21b, fputs 21b, md_print_scrap_numbers 91a, next 133c, stderr 21b

```
< Write md macro cross-refs 90c > ≡
{
  < Write md macro defs 90d >
  < Write md macro refs 90e >
  md_format_defs_refs(md_file, scraps);
  md_format_uses_refs(md_file, scraps++);
}
□
```

90c · used in 88a · uses md_format_defs_refs 102c, md_format_uses_refs 101a, scraps 134b

```
< Write md macro defs 90d > ≡
{
  if (name->defs->next) {
    fputs("> *Fragment defined by* ", md_file);
    md_print_scrap_numbers(md_file, name->defs);
  }
}
□
```

90d · used in 90c · uses defs 134a, fputs 21b, md_print_scrap_numbers 91a, next 133c

```
< Write md macro refs 90e > ≡
{
  if (name->uses) {
    if (name->uses->next) {
      fputs("> *Fragment referenced in* ", md_file);
      md_print_scrap_numbers(md_file, name->uses);
    }
    else {
      fputs("> *Fragment referenced in* ", md_file);
      fputs("<a href=\"#weft\"", md_file);
      write_single_scrap_ref(md_file, name->uses->scrap);
      fputs("\>", md_file);
      write_single_scrap_ref(md_file, name->uses->scrap);
      fputs("</a>", md_file);
      fputs(".\n", md_file);
    }
  }
  else {
    fputs("> *Fragment never referenced*.\\n", md_file);
    fprintf(stderr, "%s: <%s> never referenced.\\n",
            command_name, name->spelling);
  }
}
□
```

90e · used in 90c · uses command_name 29c, fprintf 21b, fputs 21b, md_print_scrap_numbers 91a, next 133c, stderr 21b, uses 134a, write_single_scrap_ref 135d

```
< markdown.c 91a > + ≡
void md_print_scrap_numbers(FILE *md_file, Scrap_Node *scraps)
{
  int page;
  fputs("<a href=\"#weft\"", md_file);
  write_scrap_ref(md_file, scraps->scrap, -1, &page);
  fputs("\>", md_file);
}
```

```

write_scrap_ref(md_file, scraps->scrap, TRUE, &page);
fputs("</a>", md_file);
scraps = scraps->next;
while (scraps) {
    fputs("<a href=\"#weft\"", md_file);
    write_scrap_ref(md_file, scraps->scrap, -1, &page);
    fputs("\">", md_file);
    write_scrap_ref(md_file, scraps->scrap, FALSE, &page);
    fputs("</a>", md_file);
    scraps = scraps->next;
}
fputs(".\n", md_file);
}

```

□

91a · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines `md_print_scrap_numbers` 84b, 90bde, 97c, 99b, 108bc, 109ac, 110b · uses `FALSE` 23a, `FILE` 21b, `fputs` 21b, `next` 133c, `page` 134a, `scraps` 134b, `Scrap_Node` 157e, `TRUE` 23a, `write_scrap_ref` 135c

Formatting a Scrap in Markdown

Scraps are output inside `<pre><code>` blocks. Characters `<`, `>`, and `&` are HTML-escaped. At-sequences within scraps (macro references, keywords, parameters, etc.) emit HTML markup directly.

```

<markdown.c 91b> + ≡
static int md_scrap_type = 0;

void md_copy_scrap(FILE *file, int prefix, Name *name)
{
    int indent = 0;
    int c;
    char **params = name ? name->arg : NULL;
    if (source_last == '{') md_scrap_type = 0;
    if (source_last == '[') md_scrap_type = 1;
    if (source_last == '(') md_scrap_type = 2;
    c = source_get();
    while (1) {
        switch (c) {
            case '\n': fputs("\n", file);
                        indent = 0;
                        break;
            case '\t': <Expand md tab into spaces 92b>
                        break;
            default:
                if (c==nw_char)
                {
                    <Check md at-sequence for end-of-scrap 93a>
                    break;
                }
                if (md_scrap_type == 0 && prefix) {
                    <HTML-escape character c 92a>
                } else {
                    putc(c, file);
                }
                indent++;
                break;
        }
        c = source_get();
    }
}

```

}

□

91b · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines `md_copy_scrap` 84b, 89ac, 106a, 107c, `md_scrap_type` 93a · uses `FILE` 21b, `fputs` 21b, `Name` 158a, `nw_char` 29ab, `putc` 21b, `source_get` 130b, `source_last` 130b

⟨ *HTML-escape character c* 92a ⟩ ≡

```
switch (c) {
    case '<': fputs("<", file); break;
    case '>': fputs(">", file); break;
    case '&': fputs("&", file); break;
    default: putc(c, file); break;
}
```

□

92a · used in 91b · uses `fputs` 21b, `putc` 21b

⟨ *Expand md tab into spaces* 92b ⟩ ≡

```
{
    int delta = 8 - (indent % 8);
    indent += delta;
    while (delta > 0) {
        putc(' ', file);
        delta--;
    }
}
```

□

92b · used in 91b · uses `putc` 21b

⟨ *Check md at-sequence for end-of-scrap* 93a ⟩ ≡

```
{
    c = source_get();
    switch (c) {
        case 'c': ⟨ Show md presence of a block comment 94a ⟩
            break;
        case 'x': ⟨ Copy label from source into(93b file)  74b ⟩
            break;
        case 'v': ⟨ Copy version info into file 74a ⟩
        case 's':
            break;
        case '+':
        case '-':
        case '*':
        case '|': ⟨ Skip over index entries 74d ⟩
        case ',':
        case ')':
        case ']':
        case '}' : return;
        case '<': ⟨ Format md macro name 94d ⟩
            break;
        case '%': ⟨ Skip commented-out code 74e ⟩
            break;
        case '_': ⟨ Bold md Keyword 94b ⟩
            break;
        case 't': ⟨ Italic md "fragment title" 94c ⟩
            break;
        case 'f': ⟨ Italic md "file name" 94c ⟩
            break;
        case '1': case '2': case '3':
```

```

case '4': case '5': case '6':
case '7': case '8': case '9':
    if (name == NULL
        || name->arg[c - '1'] == NULL) {
        if (md_scrap_type == 0 && prefix)
            fputs("&", file);
        else
            putc(nw_char, file);
        fputc(c, file);
    }
    else {
        md_write_arg(file, name->arg[c - '1']);
    }
    break;
default:
    if (c==nw_char)
    {
        if (md_scrap_type == 0 && prefix)
            fputs("&", file);
        else
            putc(nw_char, file);
        break;
    }
    /* ignore these since pass1 will have warned about them */
    break;
}
}

```

□

93b · used in 91b

⟨ *Show md presence of a block comment* 94a ⟩ ≡

```

{
    fputs("<i>(Comment)</i>", file);
}

```

□

94a · used in 93a · uses `fputs` 21b

⟨ *Bold md Keyword* 94b ⟩ ≡

```

{
    fprintf(file, "<b>");
    c = source_get();
    do {
        if (c == '<') fputs("<lt;", file);
        else if (c == '>') fputs("<gt;", file);
        else if (c == '&') fputs("<amp;", file);
        else fputc(c, file);
        c = source_get();
    } while (c != nw_char);
    c = source_get();
    fprintf(file, "</b>");
}

```

□

94b · used in 93a · uses `fprintf` 21b, `fputs` 21b, `nw_char` 29ab, `source_get` 130b

⟨ *Italic md "whatever"* 94c ⟩ ≡

```

{
    fprintf(file, "<i>whatever</i>");
}

```

```
}  
□
```

94c · used in 93a · uses [fprintf 21b](#)

⟨ *Format md macro name* 94d ⟩ ≡

```
{  
  Arglist *args = collect_scrap_name(-1);  
  Name *name = args->name;  
  char *p = name->spelling;  
  Arglist *q = args->args;  
  int narg = 0;  
  
  fputs("&lt;<a href=\"#weft\", file);  
  if (name->defs) {  
    write_single_scrap_ref(file, name->defs->scrap);  
  } else {  
    putc('?', file);  
  }  
  fputs("&gt;<i>", file);  
  while (*p != '\\000') {  
    if (*p == ARG_CHR) {  
      if (q == NULL) {  
        md_write_literal(file, name->arg[narg]);  
      }  
      else {  
        md_write_ArglistElement(file, q, params);  
        q = q->next;  
      }  
      p++;  
      narg++;  
    }  
    else {  
      if (*p == '<') fputs("&lt;", file);  
      else if (*p == '>') fputs("&gt;", file);  
      else if (*p == '&') fputs("&amp;", file);  
      else fputc(*p, file);  
      p++;  
    }  
  }  
  fputs("&lt;/i> ", file);  
  if (name->defs) {  
    Scrap_Node *dp = name->defs;  
    write_single_scrap_ref(file, dp->scrap);  
    dp = dp->next;  
    if (dp)  
      fputs(", ...", file);  
  }  
  else {  
    putc('?', file);  
    fprintf(stderr, "%s: never defined <%s>\n",  
            command_name, name->spelling);  
  }  
  fputs("&lt;/a>&gt;", file);  
}
```

□

94d · used in 93a · uses [Arglist 179b](#), [ARG_CHR 178a](#), [collect_scrap_name 179d](#), [command_name 29c](#), [defs 134a](#), [fprintf 21b](#), [fputs 21b](#), [md_write_ArglistElement 95b](#), [md_write_literal 95a](#), [Name 158a](#), [next 133c](#), [putc 21b](#), [Scrap_Node 157e](#), [stderr 21b](#), [write_single_scrap_ref 135d](#)

<markdown.c 95a> + ≡

```
static void md_write_literal(FILE * md_file, char * p)
{
    while (*p!= '\000') {
        if (*p == '<') fputs("<lt;\"", md_file);
        else if (*p == '>') fputs("<gt;\"", md_file);
        else if (*p == '&') fputs("<amp;\"", md_file);
        else fputc(*p, md_file);
        p++;
    }
}
```

□

95a · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines md_write_literal 84c, 94d, 95b ·
uses FILE 21b, fputs 21b

<markdown.c 95b> + ≡

```
static void
md_write_ArglistElement(FILE * file, Arglist * args, char ** params)
{
    Name *name = args->name;
    Arglist *q = args->args;

    if (name == NULL) {
        char * p = (char*)q;

        if (p[0] == ARG_CHR) {
            md_write_arg(file, params[p[1] - '1']);
        } else {
            md_write_literal(file, (char *)q);
        }
    } else if (name == (Name *)1) {
        Scrap_Node * qq = (Scrap_Node *)q;
        qq->quoted = TRUE;
        write_scraps(file, "", qq,
                     -1, "", 0, 0, 0, 0, 0,
                     NULL, params, 0, "");
        extra_scraps++;
        qq->quoted = FALSE;
    } else {
        char * p = name->spelling;
        fputs("<lt;<i>", file);
        while (*p != '\000') {
            if (*p == ARG_CHR) {
                md_write_ArglistElement(file, q, params);
                q = q->next;
                p++;
            }
            else {
                if (*p == '<') fputs("<lt;\"", file);
                else if (*p == '>') fputs("<gt;\"", file);
                else if (*p == '&') fputs("<amp;\"", file);
                else fputc(*p, file);
                p++;
            }
        }
        fputs("<lt;/i> ", file);
        if (name->defs) {
            Scrap_Node *dp = name->defs;
```

```

        write_single_scrap_ref(file, dp->scrap);
        dp = dp->next;
        if (dp)
            fputs(", ...", file);
    }
    else {
        putc('?', file);
        fprintf(stderr, "%s: never defined <%s>\n",
                command_name, name->spelling);
    }
    fputs("&gt;", file);
}
}

```

□

95b · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines `md_write_ArglistElement` 84c, 94d · uses `Arglist` 179b, `ARG_CHR` 178a, `command_name` 29c, `defs` 134a, `extra_scraps` 66b, `FALSE` 23a, `FILE` 21b, `fprintf` 21b, `fputs` 21b, `md_write_arg` 88c, `md_write_literal` 95a, `Name` 158a, `next` 133c, `putc` 21b, `Scrap_Node` 157e, `stderr` 21b, `TRUE` 23a, `write_scraps` 147c, `write_single_scrap_ref` 135d

2.5.2 Generating the Indices in Markdown

⟨ *Write md index of file names* 96 ⟩ ≡

```

{
    if (file_names) {
        fputs("\n", md_file);
        md_format_file_entry(file_names, md_file);
        fputs("\n", md_file);
    }
    c = source_get();
}

```

□

96 · used in 86a · uses `file_names` 158b, `fputs` 21b, `md_format_file_entry` 97a, `source_get` 130b

⟨ *markdown.c* 97a ⟩ + ≡

```

static void md_format_file_entry(Name *name, FILE *md_file)
{
    while (name) {
        md_format_file_entry(name->llink, md_file);
        ⟨ Format a md file index entry 97b ⟩
        name = name->rlink;
    }
}

```

□

97a · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines `md_format_file_entry` 84c, 96 · uses `FILE` 21b, `Name` 158a

⟨ *Format a md file index entry* 97b ⟩ ≡

```

fputs("- ", md_file);
fprintf(md_file, "'\'%s\'" " ", name->spelling);
⟨ Write md file's defining scrap numbers 97c ⟩
putc('\n', md_file);

```

□

97b · used in 97a · uses `fprintf` 21b, `fputs` 21b, `putc` 21b

⟨ *Write md file's defining scrap numbers* 97c ⟩ ≡

```

{
    Scrap_Node *p = name->defs;
    fputs("*Defined by* ", md_file);
    if (p->next) {
        md_print_scrap_numbers(md_file, p);
    }
    else {
        fputs("<a href=\"#weft\", md_file);
        write_single_scrap_ref(md_file, p->scrap);
        fputs(">", md_file);
        write_single_scrap_ref(md_file, p->scrap);
        fputs("</a>.", md_file);
    }
}

```

□

97c · used in 97b · uses [defs 134a](#), [fputs 21b](#), [md_print_scrap_numbers 91a](#), [next 133c](#), [Scrap_Node 157e](#), [write_single_scrap_ref 135d](#)

⟨ *Write md index of macro names* 97d ⟩ ≡

```

{
    unsigned char sector = current_sector;
    int c = source_get();
    if (c == '+')
        sector = 0;
    else
        source_ungetc(&c);
    if (has_sector(macro_names, sector)) {
        fputs("\n", md_file);
        md_format_entry(macro_names, md_file, sector);
        fputs("\n", md_file);
    } else {
        fputs("None.\n", md_file);
    }
}
c = source_get();

```

□

97d · used in 86a · uses [current_sector 45c](#), [fputs 21b](#), [has_sector 81c](#), [macro_names 158b](#), [md_format_entry 98b](#), [sector 134a](#), [source_get 130b](#), [source_ungetc 130d](#)

⟨ *markdown.c* 98a ⟩ + ≡

```

static int md_load_entry(Name * name, Name ** nms, int n)
{
    while (name) {
        n = md_load_entry(name->llink, nms, n);
        nms[n++] = name;
        name = name->rlink;
    }
    return n;
}

```

□

98a · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines [md_load_entry 98b](#) · uses [Name 158a](#)

⟨ *markdown.c* 98b ⟩ + ≡

```

static void md_format_entry(Name *name, FILE *md_file, unsigned char sector)
{
    Name ** nms = malloc(num_scrap()*sizeof(Name *));
    int n = md_load_entry(name, nms, 0);

```

```

int i;

< Sort nms of size n for < Rob's ordering 80a > 80b >
for (i = 0; i < n; i++)
{
    Name * name = nms[i];

    < Format a md index entry 98c >
}
free(nms);
}
□

98b · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines md_format_entry 84c, 97d · uses FILE 21b,
malloc 21b, md_load_entry 98a, Name 158a, num_scraps 134b, sector 134a

< Format a md index entry 98c > ≡
if (name->sector == sector){
    fputs("-", md_file);
    fputs("<*", md_file);
    < Write the md macro's name 88b >
    fputs("* ", md_file);
    < Write md defining scrap numbers 99a >
    fputs("> ", md_file);
    < Write md referencing scrap numbers 99b >
    putc('\n', md_file);
}
□

98c · used in 98b · uses fputs 21b, putc 21b, sector 134a

< Write md defining scrap numbers 99a > ≡
{
    Scrap_Node *p = name->defs;
    if (p) {
        int page;
        fputs("<a href=\"#weft", md_file);
        write_scrap_ref(md_file, p->scrap, -1, &page);
        fputs(">", md_file);
        write_scrap_ref(md_file, p->scrap, TRUE, &page);
        fputs("</a>", md_file);
        p = p->next;
        while (p) {
            fputs("<a href=\"#weft", md_file);
            write_scrap_ref(md_file, p->scrap, -1, &page);
            fputs(">", md_file);
            write_scrap_ref(md_file, p->scrap, FALSE, &page);
            fputs("</a>", md_file);
            p = p->next;
        }
    }
    else
        putc('?', md_file);
}
□

99a · used in 98c · uses defs 134a, FALSE 23a, fputs 21b, next 133c, page 134a, putc 21b, Scrap_Node 157e, TRUE 23a,
write_scrap_ref 135c

```

⟨ *Write md referencing scrap numbers 99b* ⟩ ≡

```
{
    Scrap_Node *p = name->uses;
    if (p) {
        fputs("*Referenced in* ", md_file);
        if (p->next) {
            md_print_scrap_numbers(md_file, p);
        }
        else {
            fputs("<a href=\"#weft\", md_file);
            write_single_scrap_ref(md_file, p->scrap);
            fputs(\">\", md_file);
            write_single_scrap_ref(md_file, p->scrap);
            fputs("</a>.", md_file);
        }
    }
    else
        fputs("*Not referenced*.", md_file);
}
```

□

99b · used in 98c · uses fputs 21b, md_print_scrap_numbers 91a, next 133c, Scrap_Node 157e, uses 134a, write_single_scrap_ref 135d

⟨ *Write md index of user-specified names 99c* ⟩ ≡

```
{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    if (has_sector(user_names, sector)) {
        fputs("\n", md_file);
        md_format_user_entry(user_names, md_file, sector);
        fputs("\n", md_file);
    }
}
```

□

99c · used in 86a · uses current_sector 45c, fputs 21b, has_sector 81c, md_format_user_entry 100a, sector 134a, source_get 130b, user_names 158b

⟨ *markdown.c 100a* ⟩ + ≡

```
static void md_format_user_entry(Name *name, FILE *md_file, unsigned char sector)
{
    while (name) {
        md_format_user_entry(name->llink, md_file, sector);
        ⟨ Format a md user index entry 100b ⟩
        name = name->rlink;
    }
}
```

□

100a · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines md_format_user_entry 84c, 99c · uses FILE 21b, Name 158a, sector 134a

⟨ *Format a md user index entry 100b* ⟩ ≡

```
if (name->sector == sector){
```

```

Scrap_Node *uses = name->uses;
if ( uses || dangling_flag ) {
    int page;
    Scrap_Node *defs = name->defs;
    fprintf(md_file, "- '%s': ", name->spelling);
    if (!uses) {
        fputs("**", md_file);
        fputs("<a href=\"#weft\"", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs(">", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs("</a>**", md_file);
        page = -2;
        defs = defs->next;
    }
    else
        if (!defs || uses->scrap < defs->scrap) {
            fputs("<a href=\"#weft\"", md_file);
            write_scrap_ref(md_file, uses->scrap, -1, &page);
            fputs(">", md_file);
            write_scrap_ref(md_file, uses->scrap, TRUE, &page);
            fputs("</a>", md_file);
            uses = uses->next;
        }
    else {
        if (defs->scrap == uses->scrap)
            uses = uses->next;
        fputs("**", md_file);

        fputs("<a href=\"#weft\"", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs(">", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs("</a>**", md_file);
        page = -2;
        defs = defs->next;
    }
}
while (uses || defs) {
    if (uses && (!defs || uses->scrap < defs->scrap)) {
        fputs("<a href=\"#weft\"", md_file);
        write_scrap_ref(md_file, uses->scrap, -1, &page);
        fputs(">", md_file);
        write_scrap_ref(md_file, uses->scrap, FALSE, &page);
        fputs("</a>", md_file);
        uses = uses->next;
    }
    else {
        if (uses && defs->scrap == uses->scrap)
            uses = uses->next;
        fputs(", **", md_file);

        fputs("<a href=\"#weft\"", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs(">", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs("</a>**", md_file);
        page = -2;
        defs = defs->next;
    }
}

```

```

    }
    fputs(".\n", md_file);
}
}

```

100b · used in 100a · uses [dangling_flag 27](#), [defs 134a](#), [FALSE 23a](#), [fprintf 21b](#), [fputs 21b](#), [next 133c](#), [page 134a](#), [Scrap_Node 157e](#), [sector 134a](#), [TRUE 23a](#), [uses 134a](#), [write_scrap_ref 135c](#), [write_single_scrap_ref 135d](#)

2.5.3 Identifier Cross-References in Markdown

These functions are the Markdown equivalents of `format_uses_refs` and `format_defs_refs` from `scraps.c`, but emit Markdown blockquote format.

[\(markdown.c 101a\)](#) + ≡

```

static void
md_format_uses_refs(FILE * md_file, int scrap)
{
    Uses * p = get_scrap_uses(scrap);
    if (p != NULL)
        \( Write md uses references 101b \)
}

```

101a · defined at [24c](#), [84cd](#), [88c](#), [91ab](#), [95ab](#), [97a](#), [98ab](#), [100a](#), [101a](#), [102c](#) · defines [md_format_uses_refs 84c](#), [90ac](#) · uses [FILE 21b](#), [get_scrap_uses 136a](#), [Uses 191d](#)

[\(Write md uses references 101b \)](#) ≡

```

{
    char join = ' ';
    fputs("> *Uses:*", md_file);
    do {
        \( Write one md use reference 102a \)
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".\n", md_file);
}

```

101b · used in 101a · uses [fputs 21b](#), [next 133c](#), [Uses 191d](#)

[\(Write one md use reference 102a \)](#) ≡

```

Name * name = p->defn;
Scrap_Node *defs = name->defs;
int first = TRUE, page = -1;
fprintf(md_file,
        "%c '%s' ",
        join, name->spelling);
if (defs)
{
    do {
        \( Write one md referenced scrap 102b \)
        first = FALSE;
        defs = defs->next;
    }while (defs!= NULL);
}
else

```

```
{
    fputs("(not defined globally)", md_file);
}
```

□

102a · used in 101b · uses [defs 134a](#), [FALSE 23a](#), [first 197d](#), [fprintf 21b](#), [fputs 21b](#), [Name 158a](#), [next 133c](#), [page 134a](#), [Scrap_Node 157e](#), [TRUE 23a](#)

⟨ *Write one md referenced scrap* 102b ⟩ ≡

```
fputs("<a href=\"#weft\", md_file);
write_scrap_ref(md_file, defs->scrap, -1, &page);
fputs(\">\", md_file);
write_scrap_ref(md_file, defs->scrap, first, &page);
fputs("</a>\", md_file);
```

□

102b · used in 102a, 103a · uses [defs 134a](#), [first 197d](#), [fputs 21b](#), [page 134a](#), [write_scrap_ref 135c](#)

⟨ *markdown.c* 102c ⟩ + ≡

```
static void
md_format_defs_refs(FILE * md_file, int scrap)
{
    Uses * p = get_scrap_defs(scrap);
    if (p != NULL)
        ⟨ Write md defs references 102d ⟩
}
```

□

102c · defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c · defines [md_format_defs_refs 84c](#), 90ac · uses [FILE 21b](#), [get_scrap_defs 136b](#), [Uses 191d](#)

⟨ *Write md defs references* 102d ⟩ ≡

```
{
    char join = ' ';
    fputs("> *Defines:*", md_file);
    do {
        ⟨ Write one md def reference 103a ⟩
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".\n", md_file);
}
```

□

102d · used in 102c · uses [fputs 21b](#), [next 133c](#)

⟨ *Write one md def reference* 103a ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->uses;
int first = TRUE, page = -1;
fprintf(md_file,
        "%c '%s' ",
        join, name->spelling);
if (defs == NULL
    || (defs->scrap == scrap && defs->next == NULL)) {
    fputs("*Never used*", md_file);
}
else {
    do {
```

```

    if (defs->scrap != scrap) {
        < Write one md referenced scrap 102b >
        first = FALSE;
    }
    defs = defs->next;
}while (defs!= NULL);
}

```

□

103a · used in 102d · uses [defs 134a](#), [FALSE 23a](#), [first 197d](#), [fprintf 21b](#), [fputs 21b](#), [Name 158a](#), [next 133c](#), [page 134a](#), [Scrap_Node 157e](#), [TRUE 23a](#), uses 134a

2.6 Writing the HTML File

This section implements the native HTML weave, activated by `-w html` (or `@W html`). It produces a complete, self-contained `.html` document — doctype, an embedded style sheet, and body — so a literate web becomes a browsable page with clickable cross-references and no external tooling.

It is a close sibling of the Markdown weave (Section 2.5): the two share the parts that are already HTML — scrap bodies, inline fragment references, and the `<a href="#weftN">` cross-reference links — through the reused `md_copy_scrap` and `md_print_scrap_numbers`. What this file adds is the document wrapper, the HTML scrap headers (with the `≡/+≡` markers), the one-line metadata footers, and the indices. The prose is copied verbatim: an HTML weave expects HTML prose, just as the $\text{\LaTeX}$ weave expects $\text{\LaTeX}$ and the Markdown weave expects Markdown.

< Function prototypes 103b > + ≡

```
extern void write_html(char *file_name, char *html_name, unsigned char sector);
```

□

103b · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses [file_name 134a](#), [sector 134a](#), [write_html 104](#)

2.6.1 HTML escaping helpers

A handful of small helpers escape the three HTML-special characters and render a fragment name (expanding embedded parameters to their default arguments) and a linked fragment reference.

< html.c 103c > + ≡

```
static void html_esc(FILE *f, int c)
{
    if (c == '<') fputs("&lt;", f);
    else if (c == '>') fputs("&gt;", f);
    else if (c == '&') fputs("&amp;", f);
    else putc(c, f);
}

```

```
static void html_puts(FILE *f, const char *s)
{
    while (s && *s)
        html_esc(f, (unsigned char)*s++);
}

```

```
static void html_emit_name(FILE *f, Name *name)
{
    char *p = name->spelling;
    int i = 0;
    while (*p != '\0') {
        if (*p == ARG_CHR) { html_puts(f, name->arg[i++]); p++; }
    }
}

```

```

    else html_esc(f, (unsigned char)*p++);
}
}

static void html_ref(FILE *f, Name *name)
{
    fputs("<<a href=\"#weft\", f);
    if (name->defs) write_single_scrap_ref(f, name->defs->scrap);
    else putc('?', f);
    fputs("><i>", f);
    html_emit_name(f, name);
    fputs("</i> ", f);
    if (name->defs) {
        write_single_scrap_ref(f, name->defs->scrap);
        if (name->defs->next) fputs(", ...", f);
    } else putc('?', f);
    fputs("</a>&gt;", f);
}

```

□

103c · defined at 24d, 103c, 104, 109ac, 110b · defines `html_emit_name` 107b, 109c, `html_esc` never used, `html_puts` 107a, 109a, 110b, `html_ref` 106a · uses `ARG_CHR` 178a, `defs` 134a, `FILE` 21b, `fputs` 21b, `Name` 158a, `next` 133c, `putc` 21b, `write_single_scrap_ref` 135d

2.6.2 The driver

```

<html.c 104> + ≡
static void html_format_file_entry(Name *name, FILE *f);
static void html_format_entry(Name *name, FILE *f, unsigned char sector);
static void html_format_user_entry(Name *name, FILE *f, unsigned char sector);

void write_html(char *file_name, char *html_name, unsigned char sector)
{
    (void)sector;
    FILE *html_file = fopen(html_name, "w");
    if (html_file) {
        if (verbose_flag)
            fprintf(stderr, "writing %s\n", html_name);
        < Write the HTML document head 105a >
        source_open(file_name);
        < Copy source into html_file 105c >
        < Write the HTML document tail 105b >
        fclose(html_file);
    }
    else
        fprintf(stderr, "%s: can't open %s\n", command_name, html_name);
}

```

□

104 · defined at 24d, 103c, 104, 109ac, 110b · defines `write_html` 39b, 103b · uses `command_name` 29c, `fclose` 21b, `FILE` 21b, `file_name` 134a, `fopen` 21b, `fprintf` 21b, `html_format_entry` 109c, `html_format_file_entry` 109a, `html_format_user_entry` 110b, `Name` 158a, `sector` 134a, `source_open` 133b, `stderr` 21b, `verbose_flag` 27

The document skeleton carries an embedded style sheet so the page needs no external files. The look mirrors the modern L^AT_EX weave: grey angle-bracketed fragment names in an accent colour, soft code blocks, accent links, and a small grey metadata line — and it adapts to the reader’s light or dark theme.

< *Write the HTML document head* 105a > ≡

```
fputs(
"<!doctype html>\n<html lang=\"en\">\n<head>\n<meta charset=\"utf-8\">\n"
"<meta name=\"viewport\" content=\"width=device-width,initial-scale=1\">\n"
"<title>weft</title>\n<style>\n"
":root{--acc:#2f6feb;--kw:#0550ae;--com:#6e7781;--bg:#f6f8fa;--rule:#d0d7de}\n"
"body{max-width:820px;margin:2rem auto;padding:0 1.2rem;color:#1f2328;\n"
" font:16px/1.6 -apple-system,Segoe UI,Roboto,Helvetica,Arial,sans-serif}\n"
"h1,h2,h3{font-weight:700;line-height:1.25;margin:1.8rem 0 .6rem}\n"
"a{color:var(--acc);text-decoration:none} a:hover{text-decoration:underline}\n"
".weft-name{margin:1.4rem 0 .1rem;font-size:1.02rem}\n"
".weft-name .b{color:var(--com)} .weft-name i{color:var(--acc);font-style:italic}\n"
".weft-name .f{color:var(--acc);font-family:ui-monospace,Menlo,Consolas,monospace}\n"
".weft-name .n{color:var(--com);font-size:.85em} .weft-name .eq{color:var(--com)}\n"


```
{background:var(--bg);border:1px solid var(--rule);border-radius:8px;\n"
padding:.8rem 1rem;overflow:auto;margin:.2rem 0}\n"
pre code{font:0.86em/1.45 ui-monospace,SFMono-Regular,Menlo,Consolas,monospace}\n"
pre code i{color:var(--acc);font-style:italic} pre code a{color:var(--acc)}\n"
.weft-meta{color:var(--com);font-size:.82rem;margin:.15rem 0 .6rem}\n"
.weft-meta a{color:var(--acc)} .weft-meta code{color:var(--kw)}\n"
.weft-end{color:var(--com);text-align:left;line-height:1;\n"
font-size:.75rem;margin:-.05rem 0 .1rem}\n"
code{background:var(--bg);padding:.1em .35em;border-radius:5px;\n"
font:0.9em ui-monospace,Menlo,Consolas,monospace}\n"
@media(prefers-color-scheme:dark){body{background:#0d1117;color:#e6edf3}\n"
:root{--bg:#161b22;--rule:#30363d;--com:#8b949e}}\n"
</style>\n</head>\n<body>\n", html_file);
```


```

□

105a · used in 104 · uses [fputs 21b](#), [root 184a](#)

⟨ *Write the HTML document tail* 105b ⟩ ≡

```
fputs("</body>\n</html>\n", html_file);
```

□

105b · used in 104 · uses [fputs 21b](#)

We make our second pass over the source, copying prose straight through (it is already HTML) and acting on at-sequences.

⟨ *Copy source into html_file* 105c ⟩ ≡

```
{
  int c = source_get();
  while (c != EOF) {
    if (c == nw_char)
    {
      ⟨ Interpret html at-sequence 106a ⟩
    }
    else {
      putc(c, html_file);
      c = source_get();
    }
  }
}
```

□

105c · used in 104 · uses [nw_char 29ab](#), [putc 21b](#), [source_get 130b](#)

⟨ *Interpret html at-sequence* 106a ⟩ ≡

```
{
  int big_definition = FALSE;
```

```

c = source_get();
switch (c) {
  case 'r': c = source_get(); nw_char = c; update_delimit_scrap(); break;
  case '0': big_definition = TRUE; FALLTHROUGH;
  case 'o': < Write html output file definition 107a > break;
  case 'Q':
  case 'D': big_definition = TRUE; FALLTHROUGH;
  case 'q':
  case 'd': < Write html macro definition 107b > break;
  case 's': < Step to next sector 45a > break;
  case 'S': < Close the current sector 45b > break;
  case '{':
  case '[':
  case '(': md_copy_scrap(html_file, FALSE, NULL); c = source_get(); break;
  case '<': { Arglist *a = collect_scrap_name(0);
             html_ref(html_file, a->name); c = source_get(); } break;
  case 'x': < Copy label from source into(106b html file ) 74b > c = source_get(); break;
  case 'c': c = source_get(); break;
  case 'f': < Write html index of file names 108d > break;
  case 'm': < Write html index of macro names 109b > break;
  case 'u': < Write html index of user names 110a > break;
  case 'v': fputs(version_string, html_file); c = source_get(); break;
  case 'l':
  case 'L': < Skip at-L language definition line 44c > break;
  case 'W': < Skip at-L language definition line 44c > break;
  default:
    if (c==nw_char) putc(c, html_file);
    c = source_get();
    break;
}
}

```

106b · used in 105c

2.6.3 HTML scrap definitions

A file or fragment definition writes an anchor, a header line naming the scrap (with the definition/continuation marker), the code in a `<pre><code>` block closed by a left-flushed end-of-scrap tombstone (`.weft-end`, the HTML twin of the L^AT_EX \WEFTsep square), and a one-line metadata footer.

```

< Write html output file definition 107a > ≡
{
  Name *name = collect_file_name();
  int additive = (name->defs && name->defs->scrap != scraps);
  (void)big_definition;
  fputs("<div class=\"weft-name\"><a id=\"weft\", html_file);
  write_single_scrap_ref(html_file, scraps);
  fputs("\"></a><span class=\"b\">&lt;</span> <span class=\"f\">", html_file);
  html_puts(html_file, name->spelling);
  fputs("</span> <span class=\"n\">", html_file);
  write_single_scrap_ref(html_file, scraps);
  fputs("</span> <span class=\"b\">&gt;</span> ", html_file);
  fputs(additive ? "<span class=\"eq\">+&equiv;</span>"
             : "<span class=\"eq\">&equiv;</span>", html_file);
  fputs("</div>\n", html_file);
  < Fill in the html scrap 107c >
  < Write html file footer 108b >
  < Finish html scrap 108a >

```

}

□

107a · used in 106a · uses [collect_file_name 173a](#), [defs 134a](#), [fputs 21b](#), [html_puts 103c](#), [Name 158a](#), [scraps 134b](#), [write_single_scrap_ref 135d](#)

⟨ *Write html macro definition* 107b ⟩ ≡

```
{
    Name *name = collect_macro_name();
    int additive = (name->defs && name->defs->scrap != scraps);
    (void)big_definition;
    fputs("<div class=\"weft-name\"><a id=\"weft\", html_file);
    write_single_scrap_ref(html_file, scraps);
    fputs("</a><span class=\"b\">&lt;</span> <i>", html_file);
    html_emit_name(html_file, name);
    fputs("</i> <span class=\"n\">", html_file);
    write_single_scrap_ref(html_file, scraps);
    fputs("</span> <span class=\"b\">&gt;</span> ", html_file);
    fputs(additive ? "<span class=\"eq\">+&equiv;</span>"
                  : "<span class=\"eq\">&equiv;</span>", html_file);
    fputs("</div>\n", html_file);
    ⟨ Fill in the html scrap 107c ⟩
    ⟨ Write html macro footer 108c ⟩
    ⟨ Finish html scrap 108a ⟩
}
```

□

107b · used in 106a · uses [collect_macro_name 176b](#), [defs 134a](#), [fputs 21b](#), [html_emit_name 103c](#), [Name 158a](#), [scraps 134b](#), [write_single_scrap_ref 135d](#)

⟨ *Fill in the html scrap* 107c ⟩ ≡

```
{
    if (name->lang[0] != '\0')
        fprintf(html_file, "<pre><code class=\"language-%s\">", name->lang);
    else
        fputs("<pre><code>", html_file);
    extra_scraps = 0;
    md_copy_scrap(html_file, TRUE, name);
    fputs("</code></pre>\n", html_file);
    /* end-of-scrap tombstone: a small grey square flushed left, the HTML
       twin of the LaTeX \WEFTsep, closing the code before the metadata line */
    fputs("<div class=\"weft-end\">&#9633;</div>\n", html_file);
}
```

□

107c · used in 107ab · uses [extra_scraps 66b](#), [fprintf 21b](#), [fputs 21b](#), [md_copy_scrap 91b](#), [TRUE 23a](#)

⟨ *Finish html scrap* 108a ⟩ ≡

```
{
    scraps += extra_scraps;
    do c = source_get(); while (isspace(c));
}
```

□

108a · used in 107ab · uses [extra_scraps 66b](#), [isspace 21b](#), [scraps 134b](#), [source_get 130b](#)

The footer is a single grey line: the scrap number, then where else the fragment is defined (for a multi-part definition) and where it is used, each a clickable link.

⟨ *Write html file footer* 108b ⟩ ≡

```
{
    fputs("<div class=\"weft-meta\\>", html_file);
    write_single_scrap_ref(html_file, scraps);
    if (scrap_flag && name->defs && name->defs->next) {
        fputs(" &mdot; also at ", html_file);
        md_print_scrap_numbers(html_file, name->defs);
    }
    fputs("</div>\n", html_file);
    scraps++;
}
```

□

108b · used in 107a · uses [defs 134a](#), [fputs 21b](#), [md_print_scrap_numbers 91a](#), [next 133c](#), [scraps 134b](#), [scrap_flag 27](#), [write_single_scrap_ref 135d](#)

⟨ *Write html macro footer* 108c ⟩ ≡

```
{
    fputs("<div class=\"weft-meta\\>", html_file);
    write_single_scrap_ref(html_file, scraps);
    if (name->defs && name->defs->next) {
        fputs(" &mdot; also at ", html_file);
        md_print_scrap_numbers(html_file, name->defs);
    }
    if (name->uses) {
        fputs(" &mdot; used in ", html_file);
        md_print_scrap_numbers(html_file, name->uses);
    } else {
        fputs(" &mdot; unreferenced", html_file);
    }
    fputs("</div>\n", html_file);
    scraps++;
}
```

□

108c · used in 107b · uses [defs 134a](#), [fputs 21b](#), [md_print_scrap_numbers 91a](#), [next 133c](#), [scraps 134b](#), [uses 134a](#), [write_single_scrap_ref 135d](#)

2.6.4 HTML indices

The @f, @m, and @u indices become HTML lists with links to the defining and referencing scraps.

⟨ *Write html index of file names* 108d ⟩ ≡

```
{
    if (file_names) {
        fputs("<ul class=\"weft-index\\>\n", html_file);
        html_format_file_entry(file_names, html_file);
        fputs("</ul>\n", html_file);
    }
    c = source_get();
}
```

□

108d · used in 106a · uses [file_names 158b](#), [fputs 21b](#), [html_format_file_entry 109a](#), [source_get 130b](#)

⟨ *html.c* 109a ⟩ + ≡

```
static void html_format_file_entry(Name *name, FILE *f)
{
    while (name) {
```

```

    html_format_file_entry(name->llink, f);
    fputs("<li><span class=\"f\">", f);
    html_puts(f, name->spelling);
    fputs("</span> ", f);
    if (name->defs) md_print_scrap_numbers(f, name->defs);
    fputs("</li>\n", f);
    name = name->rlink;
}
}

```

□

109a · defined at 24d, 103c, 104, 109ac, 110b · defines `html_format_file_entry` 104, 108d · uses `defs` 134a, `FILE` 21b, `fputs` 21b, `html_puts` 103c, `md_print_scrap_numbers` 91a, `Name` 158a

⟨ *Write html index of macro names* 109b ⟩ ≡

```

{
    unsigned char sector = current_sector;
    int cc = source_get();
    if (cc == '+') sector = 0; else source_ungetc(&cc);
    if (has_sector(macro_names, sector)) {
        fputs("<ul class=\"weft-index\">\n", html_file);
        html_format_entry(macro_names, html_file, sector);
        fputs("</ul>\n", html_file);
    }
    c = source_get();
}

```

□

109b · used in 106a · uses `current_sector` 45c, `fputs` 21b, `has_sector` 81c, `html_format_entry` 109c, `macro_names` 158b, `sector` 134a, `source_get` 130b, `source_ungetc` 130d

⟨ *html.c* 109c ⟩ + ≡

```

static void html_format_entry(Name *name, FILE *f, unsigned char sector)
{
    while (name) {
        html_format_entry(name->llink, f, sector);
        if (name->sector == sector) {
            fputs("<li><span class=\"b\">&lt;</span> <i>", f);
            html_emit_name(f, name);
            fputs("</i> <span class=\"b\">&gt;</span> ", f);
            if (name->defs) md_print_scrap_numbers(f, name->defs);
            else putc('?', f);
            fputs("</li>\n", f);
        }
        name = name->rlink;
    }
}

```

□

109c · defined at 24d, 103c, 104, 109ac, 110b · defines `html_format_entry` 104, 109b · uses `defs` 134a, `FILE` 21b, `fputs` 21b, `html_emit_name` 103c, `md_print_scrap_numbers` 91a, `Name` 158a, `putc` 21b, `sector` 134a

⟨ *Write html index of user names* 110a ⟩ ≡

```

{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') { sector = 0; c = source_get(); }
    if (has_sector(user_names, sector)) {
        fputs("<ul class=\"weft-index\">\n", html_file);

```

```

    html_format_user_entry(user_names, html_file, sector);
    fputs("</ul>\n", html_file);
}
}
□
110a · used in 106a · uses current_sector 45c, fputs 21b, has_sector 81c, html_format_user_entry 110b, sector 134a,
source_get 130b, user_names 158b

<html.c 110b> + ≡
static void html_format_user_entry(Name *name, FILE *f, unsigned char sector)
{
    while (name) {
        html_format_user_entry(name->llink, f, sector);
        if (name->sector == sector && (name->uses || dangling_flag)) {
            fputs("<li><code>", f);
            html_puts(f, name->spelling);
            fputs("</code>: ", f);
            if (name->uses) md_print_scrap_numbers(f, name->uses);
            fputs("</li>\n", f);
        }
        name = name->rlink;
    }
}
}
□
110b · defined at 24d, 103c, 104, 109ac, 110b · defines html_format_user_entry 104, 110a · uses dangling_flag 27, FILE 21b,
fputs 21b, html_puts 103c, md_print_scrap_numbers 91a, Name 158a, sector 134a, uses 134a

```

2.7 Map and Extract Output

Two modes support AI-assisted navigation of weft projects:

- m (map) emits a JSON object to stdout describing every output file and every fragment, their definition locations, usage relationships, and transitive dependencies.
- e *name* (extract) emits the scrap code and dependency information for a single fragment (matched by prefix) as Markdown to stdout.

Both modes run after `pass1` has built the name trees and do *not* invoke the weave or tangle phases.

2.7.1 JSON Map Output

The `write_map` function performs an in-order traversal of the `file_names` and `macro_names` binary search trees, emitting a JSON object with two arrays: "files" and "fragments".

For each `Name` node, it records:

- The spelling (name of the file or fragment).
- The sector number.
- The language tag (if any).
- Each definition scrap with its source file and line.
- Each usage scrap (scraps that reference this name).
- For definition scraps, the list of fragments referenced inside (via `get_scrap_uses`).

A helper function escapes strings for JSON output.

```
<output.c 111a> +≡
static void json_puts(FILE *f, const char *s)
{
    fputc('"', f);
    while (*s) {
        switch (*s) {
            case '\\': fputs("\\\\", f); break;
            case '\n': fputs("\\n", f); break;
            case '\r': fputs("\\r", f); break;
            case '\t': fputs("\\t", f); break;
            default:
                if ((unsigned char)*s < 0x20)
                    fprintf(f, "\\u%04x", (unsigned char)*s);
                else
                    fputc(*s, f);
        }
        s++;
    }
    fputc('"', f);
}
```

□

111a · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
 defines json_puts 111b, 121 · uses FILE 21b, fprintf 21b, fputs 21b

The `emit_name_json` helper emits a single name entry as a JSON object. The `is_file` flag distinguishes file names (which use "uses_fragments" for fragments referenced inside their definition scraps) from macro fragments (which use "used_by" for the scraps that invoke them, and "uses" for fragments they invoke).

```
<output.c 111b> +≡
static void emit_name_json(FILE *f, Name *name, int is_file, int *first)
{
    Scrap_Node *sn;
    int first_inner;

    if (!name) return;
    emit_name_json(f, name->llink, is_file, first);

    if (!*first) fputs(",\n", f);
    *first = 0;

    fputs("    {\n", f);
    fputs("        \"name\": ", f); json_puts(f, name->spelling);
    if (name->lang[0]) {
        fputs(",\n        \"lang\": ", f); json_puts(f, name->lang);
    }
    fprintf(f, ",\n        \"sector\": %d", name->sector);

    /* defs array */
    fputs(",\n        \"defs\": [", f);
    first_inner = 1;
    for (sn = name->defs; sn; sn = sn->next) {
        if (!first_inner) fputs(", ", f);
        first_inner = 0;
        fprintf(f, "{\"scrap\": %d, \"file\": ", sn->scrap);
        json_puts(f, scrap_file_name(sn->scrap));
    }
}
```

```

fprintf(f, ", \"line\": %d, \"end_line\": %d",
        scrap_file_line(sn->scrap), scrap_end_line(sn->scrap));
if (bodies_flag) {
    fputs(", \"body\": ", f);
    dump_scrap_json(f, sn->scrap);
}
if (prose_flag && scrap_prose(sn->scrap)) {
    fputs(", \"prose\": ", f);
    json_puts(f, scrap_prose(sn->scrap));
}
fputc('}', f);
fputs("]", f);

if (is_file) {
    /* uses_fragments: fragments referenced inside this file's scraps */
    fputs(",\n    \"uses_fragments\": [", f);
    first_inner = 1;
    for (sn = name->defs; sn; sn = sn->next) {
        Uses *u;
        for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
            if (!first_inner) fputs(", ", f);
            first_inner = 0;
            json_puts(f, u->defn->spelling);
        }
    }
    fputs("]", f);
} else {
    /* used_by: scraps that invoke this fragment */
    fputs(",\n    \"used_by\": [", f);
    first_inner = 1;
    for (sn = name->uses; sn; sn = sn->next) {
        if (!first_inner) fputs(", ", f);
        first_inner = 0;
        fprintf(f, "{\"scrap\": %d, \"file\": ", sn->scrap);
        json_puts(f, scrap_file_name(sn->scrap));
        fprintf(f, ", \"line\": %d}", scrap_file_line(sn->scrap));
    }
    fputs("]", f);

    /* uses: fragments referenced inside this fragment's def scraps */
    fputs(",\n    \"uses\": [", f);
    first_inner = 1;
    for (sn = name->defs; sn; sn = sn->next) {
        Uses *u;
        for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
            if (!first_inner) fputs(", ", f);
            first_inner = 0;
            json_puts(f, u->defn->spelling);
        }
    }
    fputs("]", f);
}

fputs("\n    }", f);

emit_name_json(f, name->rlink, is_file, first);
}

```

□

111b · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
 defines `emit_name_json` 113a · uses `bodies_flag` 27, `defs` 134a, `dump_scrap_json` 144, `end_line` 134a, `FILE` 21b, `first` 197d,
`fprintf` 21b, `fputs` 21b, `get_scrap_uses` 136a, `json_puts` 111a, `Name` 158a, `next` 133c, `prose` 134a, `prose_flag` 27,
`scraps` 134b, `scrap_end_line` 137a, `scrap_file_line` 136e, `scrap_file_name` 136d, `Scrap_Node` 157e, `scrap_prose` 137b,
`sector` 134a, `Uses` 191d, `uses` 134a

```

<output.c 113a> + ≡
void write_map(Name *files, Name *macros)
{
    int first;

    fputs("{\n  \"files\": [\n", stdout);
    first = 1;
    emit_name_json(stdout, files, 1, &first);
    fputs("\n  ],\n  \"fragments\": [\n", stdout);
    first = 1;
    emit_name_json(stdout, macros, 0, &first);
    fputs("\n  ]\n}\n", stdout);
}
□

```

113a · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
 defines `write_map` 39b, 125b · uses `emit_name_json` 111b, `first` 197d, `fputs` 21b, `Name` 158a

2.7.2 Fragment Extraction

The `write_extract` function finds a fragment by name (using prefix matching, like `@<name...@>` in the source), then emits the scrap code and dependency information as Markdown. It recurses into referenced fragments to provide the full transitive closure of the requested fragment.

To find a name in the BST without creating a new entry, we use a read-only search function:

```

<output.c 113b> + ≡
static Name *find_name(Name *node, const char *spelling)
{
    Name *found;
    if (!node) return NULL;
    if (robs_strcmp(node->spelling, (char *)spelling) == 0)
        return node;
    found = find_name(node->llink, spelling);
    if (found) return found;
    return find_name(node->rlink, spelling);
}
□

```

113b · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
 defines `find_name` 116ab · uses `Name` 158a, `robs_strcmp` 161c

A recursive helper emits one fragment and all its transitive dependencies. A visited array (indexed by Name pointer cast to a simple mark) prevents infinite recursion on cyclic references.

```

<output.c 114> + ≡
static void extract_fragment(FILE *f, Name *name, Name *files, Name *macros)
{
    Scrap_Node *sn;

    if (!name || name->mark) return;
    name->mark = TRUE;

```

```

for (sn = name->defs; sn; sn = sn->next) {
    fprintf(f, "\n## \"\%s\" --- %s:%d\n\n",
        name->spelling,
        scrap_file_name(sn->scrap),
        scrap_file_line(sn->scrap));
    if (name->lang[0])
        fprintf(f, "\"'\%s\n", name->lang);
    else
        fputs("\"'\n", f);
    dump_scrap_text(f, sn->scrap);
    fputs("\n'\n", f);

    /* Show dependencies */
    {
        Uses *u = get_scrap_uses(sn->scrap);
        if (u) {
            fputs("\n### Uses\n", f);
            for (; u; u = u->next)
                fprintf(f, "- %s\n", u->defn->spelling);
        }
    }

    /* Show used_by */
    if (name->uses) {
        Scrap_Node *usn;
        fputs("\n### Used by\n", f);
        for (usn = name->uses; usn; usn = usn->next) {
            fprintf(f, "- scrap %d (%s:%d)\n",
                usn->scrap,
                scrap_file_name(usn->scrap),
                scrap_file_line(usn->scrap));
        }
    }

    /* Recurse into dependencies */
    for (sn = name->defs; sn; sn = sn->next) {
        Uses *u;
        for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
            extract_fragment(f, u->defn, files, macros);
        }
    }
}

```

□

114 · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `extract_fragment` 116b · uses `defs` 134a, `dump_scrap_text` 143b, `FILE` 21b, `fprintf` 21b, `fputs` 21b,
`get_scrap_uses` 136a, `Name` 158a, `next` 133c, `scrap_file_line` 136e, `scrap_file_name` 136d, `Scrap_Node` 157e, `TRUE` 23a,
`Uses` 191d, `uses` 134a

Clear marks after extraction so the name trees remain usable:

```

<output.c 115a> + ≡
static void clear_marks(Name *node)
{
    if (!node) return;
    clear_marks(node->llink);
    node->mark = FALSE;
    clear_marks(node->rlink);
}

```

```
}
□
```

115a · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `clear_marks` 116ab · uses `FALSE` 23a, `Name` 158a

2.7.3 Callers (Upward Call Chain)

The `--callers` feature, used with `-e`, traces the upward call chain from a given fragment back to its @o file roots. Each scrap records its owning `Name` (set in `pass1` when the scrap is attached to a definition list), so we can walk upward through the `uses` list to find all callers.

```
<output.c 115b> + ≡
static int is_file_name(Name *root, Name *target)
{
    if (!root) return 0;
    if (root == target) return 1;
    if (is_file_name(root->llink, target)) return 1;
    return is_file_name(root->rlink, target);
}
□
```

115b · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `is_file_name` 115c · uses `Name` 158a, `root` 184a

```
<output.c 115c> + ≡
static void print_callers(Name *name, Name *file_names, int depth)
{
    Scrap_Node *usn;
    int i;

    if (!name || name->mark) return;
    name->mark = TRUE;

    for (i = 0; i < depth; i++) fputs("  ", stdout);
    if (is_file_name(file_names, name))
        printf("[file] %s\n", name->spelling);
    else
        printf("%s\n", name->spelling);

    for (usn = name->uses; usn; usn = usn->next) {
        Name *owner = scrap_owner(usn->scrap);
        if (owner && !owner->mark)
            print_callers(owner, file_names, depth + 1);
    }
}
□
```

115c · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `print_callers` 116a · uses `file_names` 158b, `fputs` 21b, `is_file_name` 115b, `Name` 158a, `next` 133c, `owner` 134a, `Scrap_Node` 157e, `scrap_owner` 137c, `TRUE` 23a, `uses` 134a

```
<output.c 116a> + ≡
void write_callers(char *name, Name *files, Name *macros)
{
    Name *found = find_name(macros, name);
    if (!found)
        found = find_name(files, name);
    if (!found) {
```

```

    fprintf(stderr, "%s: fragment \"%s\" not found\n", command_name, name);
    exit(1);
}
printf("Callers of \"%s\":\n\n", found->spelling);
printf("%s\n", found->spelling);
{
    Scrap_Node *usn;
    for (usn = found->uses; usn; usn = usn->next) {
        Name *owner = scrap_owner(usn->scrap);
        if (owner)
            print_callers(owner, files, 1);
    }
}
clear_marks(files);
clear_marks(macros);
}

```

□

116a · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `write_callers` 39b, 125b · uses `clear_marks` 115a, `command_name` 29c, `exit` 21b, `find_name` 113b, `fprintf` 21b,
`Name` 158a, `next` 133c, `owner` 134a, `print_callers` 115c, `Scrap_Node` 157e, `scrap_owner` 137c, `stderr` 21b, `uses` 134a

⟨`output.c` 116b⟩ + ≡

```

void write_extract(char *name, Name *files, Name *macros)
{
    Name *found = NULL;

    /* Search in macro_names first, then file_names */
    found = find_name(macros, name);
    if (!found)
        found = find_name(files, name);

    if (!found) {
        fprintf(stderr, "%s: fragment \"%s\" not found\n", command_name, name);
        exit(1);
    }

    fprintf(stdout, "# Extract: \"%s\"\n", found->spelling);
    extract_fragment(stdout, found, files, macros);

    /* Clean up marks */
    clear_marks(files);
    clear_marks(macros);
}

```

□

116b · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `write_extract` 39b, 125b · uses `clear_marks` 115a, `command_name` 29c, `exit` 21b, `extract_fragment` 114,
`file_names` 158b, `find_name` 113b, `first` 197d, `fprintf` 21b, `macro_names` 158b, `Name` 158a, `stderr` 21b

2.7.4 Lint (Static Analysis)

The `--lint` flag performs a post-pass1 static analysis of the fragment namespace. It reports three kinds of issues:

1. **Undefined references:** fragments that are used (`@<Name@>`) but never defined (`@d Name`).
2. **Unused fragments:** fragments defined but never referenced by any other scrap or output file.

3. **Fuzzy-match suggestions:** for undefined references, the closest defined name within Levenshtein distance 3.

⟨output.c 117a⟩ + ≡

```
static int levenshtein(const char *a, const char *b)
{
    int la = (int)strlen(a), lb = (int)strlen(b);
    int prev[256], curr[256];
    int i, j;
    if (la > 255 || lb > 255) return 999;
    for (j = 0; j <= lb; j++) prev[j] = j;
    for (i = 1; i <= la; i++) {
        curr[0] = i;
        for (j = 1; j <= lb; j++) {
            int cost = (a[i-1] != b[j-1]) ? 1 : 0;
            int del = prev[j] + 1;
            int ins = curr[j-1] + 1;
            int sub = prev[j-1] + cost;
            curr[j] = del < ins ? (del < sub ? del : sub) : (ins < sub ? ins : sub);
        }
        for (j = 0; j <= lb; j++) prev[j] = curr[j];
    }
    return prev[lb];
}
```

□

117a · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines levenshtein 117b · uses strlen 21b

⟨output.c 117b⟩ + ≡

```
#define MAX_LINT_NAMES 4096
static Name *lint_names[MAX_LINT_NAMES];
static int lint_name_count;

static void collect_lint_names(Name *node)
{
    if (!node) return;
    collect_lint_names(node->llink);
    if (node->defs && lint_name_count < MAX_LINT_NAMES)
        lint_names[lint_name_count++] = node;
    collect_lint_names(node->rlink);
}

static const char *find_closest(const char *target)
{
    int i, best_dist = 4;
    const char *best = NULL;
    for (i = 0; i < lint_name_count; i++) {
        int d = levenshtein(target, lint_names[i]->spelling);
        if (d < best_dist) {
            best_dist = d;
            best = lint_names[i]->spelling;
        }
    }
    return best;
}
```

□

117b · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
 defines `collect_lint_names` 118b, `find_closest` 118a, `lint_names` never used, `lint_name_count` 118b, `MAX_LINT_NAMES` never
 used · uses `defs` 134a, `levenshtein` 117a, `Name` 158a

```
<output.c 118a> + ≡
static int lint_name(Name *node, int check_unused, int *issues)
{
    if (!node) return 0;
    lint_name(node->llink, check_unused, issues);

    if (node->defs == NULL && node->uses != NULL) {
        /* undefined reference */
        const char *suggestion = find_closest(node->spelling);
        fprintf(stderr, "lint: undefined fragment \"%s\"", node->spelling);
        if (node->uses) {
            fprintf(stderr, " (used at %s:%d)",
                    scrap_file_name(node->uses->scrap),
                    scrap_file_line(node->uses->scrap));
        }
        if (suggestion)
            fprintf(stderr, " -- did you mean \"%s\"?", suggestion);
        fputc('\n', stderr);
        (*issues)++;
    }
    else if (node->defs != NULL && node->uses == NULL && check_unused) {
        fprintf(stderr, "lint: unused fragment \"%s\" (defined at %s:%d)\n",
                node->spelling,
                scrap_file_name(node->defs->scrap),
                scrap_file_line(node->defs->scrap));
        (*issues)++;
    }

    lint_name(node->rlink, check_unused, issues);
    return 0;
}
□
```

118a · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
 defines `lint_name` 118b · uses `defs` 134a, `find_closest` 117b, `fprintf` 21b, `Name` 158a, `scrap_file_line` 136e,
`scrap_file_name` 136d, `stderr` 21b, uses 134a

```
<output.c 118b> + ≡
void write_lint(Name *file_names, Name *macro_names)
{
    int issues = 0;
    lint_name_count = 0;
    collect_lint_names(file_names);
    collect_lint_names(macro_names);

    /* check files for undefined refs (not unused -- files are roots) */
    lint_name(file_names, 0, &issues);
    /* check macros for both undefined and unused */
    lint_name(macro_names, 1, &issues);

    if (issues)
        fprintf(stderr, "lint: %d issue(s) found\n", issues);
    else
        fprintf(stderr, "lint: no issues found\n");
}
```

□

118b · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `write_lint` 39b, 125b · uses `collect_lint_names` 117b, `file_names` 158b, `fprintf` 21b, `lint_name` 118a,
`lint_name_count` 117b, `macro_names` 158b, `Name` 158a, `stderr` 21b

2.7.5 Reverse Map Output

The reverse map feature provides programmatic navigation from tangled output back to the `.weft` source. Given a tangled output file (and optionally a line number), it parses the section markers embedded in that file and emits JSON mapping each region to its originating scrap, source file, and source line.

This operates entirely on the tangled output—no `.weft` files are needed and `pass1` is never invoked. The section markers have the form:

```
COMMENT_BEGIN{N: file:line}COMMENT_END      (opening)
COMMENT_BEGIN{:N}COMMENT_END                (closing)
```

where `COMMENT_BEGIN` and `COMMENT_END` vary by language (`/* ... */`, `// ...`, `# ...`, etc.).

A `MarkerFrame` records the data extracted from one opening marker. We maintain a stack of these to handle nested fragment expansions.

```
<output.c 119a> + ≡
typedef struct {
    int scrap;
    char file[FILENAME_MAX];
    int line;
    int start_line; /* line in the tangled file where this marker opened */
} MarkerFrame;
```

□

119a · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `MarkerFrame` 119b, 121, 123, 124

The `parse_open_marker` function scans a line for the pattern `{N: file:line}`. It returns 1 on success, filling in the supplied `MarkerFrame`. The scan is intentionally loose about what precedes the opening brace so that any comment style works.

```
<output.c 119b> + ≡
static int parse_open_marker(const char *line, MarkerFrame *mf)
{
    const char *p = line;
    while (*p) {
        if (*p == '{') {
            const char *start = p + 1;
            /* scrap number */
            if (*start < '0' || *start > '9') { p++; continue; }
            int scrap = 0;
            while (*start >= '0' && *start <= '9')
                scrap = scrap * 10 + (*start++ - '0');
            if (*start != ':' || *(start+1) != ' ') { p++; continue; }
            start += 2; /* skip ": " */
            /* file:line -- find the closing brace, then work backwards for :line */
            const char *end = strchr(start, '}');
            if (!end) { p++; continue; }
            /* find last ':' between start and end */
            const char *colon = NULL;
            for (const char *q = end - 1; q >= start; q--) {
                if (*q == ':') { colon = q; break; }
            }
        }
    }
}
```

```

    }
    if (!colon) { p++; continue; }
    /* digits after colon? */
    int src_line = 0;
    const char *d = colon + 1;
    if (d == end) { p++; continue; }
    while (d < end) {
        if (*d < '0' || *d > '9') break;
        src_line = src_line * 10 + (*d++ - '0');
    }
    if (d != end) { p++; continue; }
    /* success */
    mf->scrap = scrap;
    mf->line = src_line;
    {
        size_t len = (size_t)(colon - start);
        if (len >= FILENAME_MAX) len = FILENAME_MAX - 1;
        memcpy(mf->file, start, len);
        mf->file[len] = '\0';
    }
    return 1;
}
p++;
}
return 0;
}

```

□

119b · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines [parse_open_marker](#) 121, 123 · uses [MarkerFrame](#) 119a, [size_t](#) 21b

The `parse_close_marker` function scans for `{:N}` and returns the scrap number, or `-1` on failure.

⟨[output.c](#) 120⟩ + ≡

```

static int parse_close_marker(const char *line)
{
    const char *p = line;
    while (*p) {
        if (*p == '{' && *(p+1) == ':') {
            const char *start = p + 2;
            if (*start < '0' || *start > '9') { p++; continue; }
            int scrap = 0;
            while (*start >= '0' && *start <= '9')
                scrap = scrap * 10 + (*start++ - '0');
            if (*start == '}')
                return scrap;
        }
        p++;
    }
    return -1;
}

```

□

120 · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines [parse_close_marker](#) 121, 123

The main function `write_reverse_map` parses its argument to split an optional line number from the filename, opens the tangled file, and reads it line by line. A stack of `MarkerFrame` entries tracks nested scrap regions.

In *line mode* (a specific line was requested), as soon as `current_line` reaches the target the innermost stack frame is emitted and the function returns.

In *full mode* (no line number), every time a close marker is encountered the region from the matching open to the current line is emitted.

```
<output.c 121> +≡
void write_reverse_map(const char *arg)
{
    char filename[FILENAME_MAX];
    int target_line = -1;
    FILE *fp;
    char buf[8192];
    MarkerFrame stack[256];
    int depth = 0;
    int current_line = 0;
    int first_region = 1;

    /* Parse arg: find last ':' where suffix is all digits */
    {
        const char *colon = NULL;
        const char *p;
        for (p = arg + strlen(arg) - 1; p > arg; p--) {
            if (*p == ':') { colon = p; break; }
        }
        if (colon) {
            const char *d = colon + 1;
            int all_digits = (*d != '\0');
            while (*d) {
                if (*d < '0' || *d > '9') { all_digits = 0; break; }
                d++;
            }
            if (all_digits) {
                target_line = atoi(colon + 1);
                snprintf(filename, FILENAME_MAX, "%.s", (int)(colon - arg), arg);
            } else {
                snprintf(filename, FILENAME_MAX, "%s", arg);
            }
        } else {
            snprintf(filename, FILENAME_MAX, "%s", arg);
        }
    }

    fp = fopen(filename, "r");
    if (!fp) {
        fprintf(stderr, "weft: cannot open %s: ", filename);
        perror(NULL);
        exit(1);
    }

    if (target_line < 0) {
        /* Full mode header */
        fputs("{\"source\": ", stdout);
        json_puts(stdout, filename);
        fputs(", \"regions\": [\n", stdout);
    }

    while (fgets(buf, sizeof(buf), fp)) {
        MarkerFrame mf;
        int close_scrap;
```

```

current_line++;

if (parse_open_marker(buf, &mf)) {
    mf.start_line = current_line;
    if (depth < 256)
        stack[depth++] = mf;
    continue;
}

close_scrap = parse_close_marker(buf);
if (close_scrap >= 0) {
    /* Find matching frame on stack */
    int i;
    for (i = depth - 1; i >= 0; i--) {
        if (stack[i].scrap == close_scrap) {
            if (target_line < 0) {
                /* Full mode: emit region */
                if (!first_region) fputs("\n", stdout);
                first_region = 0;
                fprintf(stdout,
                    "  {\"start\": %d, \"end\": %d, \"scrap\": %d, \"file\": ",
                    stack[i].start_line + 1, current_line - 1, stack[i].scrap);
                json_puts(stdout, stack[i].file);
                fprintf(stdout, ", \"line\": %d}\n", stack[i].line);
            }
            /* Remove frame by shifting */
            depth--;
            for (; i < depth; i++)
                stack[i] = stack[i + 1];
            break;
        }
    }
    continue;
}

/* Regular line -- check for line mode match */
if (target_line > 0 && current_line == target_line) {
    if (depth > 0) {
        MarkerFrame *inner = &stack[depth - 1];
        fprintf(stdout, "{\"scrap\": %d, \"file\": ", inner->scrap);
        json_puts(stdout, inner->file);
        fprintf(stdout, ", \"line\": %d}\n", inner->line);
    } else {
        fputs("{}\n", stdout);
    }
    fclose(fp);
    return;
}

fclose(fp);

if (target_line > 0) {
    /* Line was beyond end of file or not inside a scrap */
    fputs("{}\n", stdout);
} else {
    fputs("\n]\n", stdout);
}
}

```

□

121 · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `write_reverse_map` 36e, 125b · uses `exit` 21b, `fclose` 21b, `FILE` 21b, `fopen` 21b, `fprintf` 21b, `fputs` 21b,
`json_puts` 111a, `MarkerFrame` 119a, `parse_close_marker` 120, `parse_open_marker` 119b, `stack` 130a, `stderr` 21b, `strlen` 21b

2.7.6 Error Annotator

The `--errors` flag reads compiler output from stdin and annotates lines matching `file:line:` patterns with the corresponding `.weft` source location. It works by opening each referenced file, parsing its section markers, and finding which scrap region contains the error line. Lines that don't match the pattern are passed through unchanged.

```
<output.c 123> + ≡
static int reverse_lookup_in_file(const char *filename, int target_line,
                                MarkerFrame *result)
{
    FILE *fp = fopen(filename, "r");
    char buf[8192];
    MarkerFrame stack[256];
    int depth = 0, current_line = 0;

    if (!fp) return 0;

    while (fgets(buf, sizeof(buf), fp)) {
        MarkerFrame mf;
        int close_scrap;
        current_line++;

        if (parse_open_marker(buf, &mf)) {
            mf.start_line = current_line;
            if (depth < 256)
                stack[depth++] = mf;
            continue;
        }

        close_scrap = parse_close_marker(buf);
        if (close_scrap >= 0) {
            int i;
            for (i = depth - 1; i >= 0; i--) {
                if (stack[i].scrap == close_scrap) {
                    depth--;
                    for (; i < depth; i++)
                        stack[i] = stack[i + 1];
                    break;
                }
            }
            continue;
        }

        if (current_line == target_line && depth > 0) {
            *result = stack[depth - 1];
            fclose(fp);
            return 1;
        }
    }
    fclose(fp);
    return 0;
}
```

□

123 · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `reverse_lookup_in_file` 124 · uses `fclose` 21b, `FILE` 21b, `fopen` 21b, `MarkerFrame` 119a, `parse_close_marker` 120,
`parse_open_marker` 119b, `stack` 130a

⟨`output.c` 124⟩ + ≡

```
void write_error_annotator(void)
{
    char line[8192];

    while (fgets(line, sizeof(line), stdin)) {
        const char *p = line;
        char err_file[FILENAME_MAX];
        int err_line = -1;
        int parsed = 0;

        /* Try to parse file:digits: pattern */
        /* Skip optional drive letter on Windows (e.g. C:) */
        if (p[0] && p[1] == ':' && ((p[0] >= 'A' && p[0] <= 'Z') ||
            (p[0] >= 'a' && p[0] <= 'z'))) {
            p += 2;
        }
        {
            const char *colon1 = strchr(p, ':');
            if (colon1 && colon1 > p) {
                const char *d = colon1 + 1;
                int num = 0;
                int has_digits = 0;
                while (*d >= '0' && *d <= '9') {
                    num = num * 10 + (*d - '0');
                    has_digits = 1;
                    d++;
                }
                if (has_digits && *d == ':') {
                    size_t flen = (size_t)(colon1 - line);
                    if (flen < FILENAME_MAX) {
                        memcpy(err_file, line, flen);
                        err_file[flen] = '\0';
                        err_line = num;
                        parsed = 1;
                    }
                }
            }
        }

        fputs(line, stdout);

        if (parsed) {
            MarkerFrame mf;
            if (reverse_lookup_in_file(err_file, err_line, &mf)) {
                int offset = err_line - mf.start_line;
                printf(" [weft: %s:%d, scrap %d",
                    mf.file, mf.line + offset, mf.scrap);
                printf("]\n");
            }
        }
    }
}
```

□

124 · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a · defines `write_error_annotator` 36e, 125b · uses `fputs` 21b, `letter` 134a, `MarkerFrame` 119a, `reverse_lookup_in_file` 123, `size_t` 21b

2.8 Writing the Output Files

⟨ *Function prototypes* 125a ⟩ + ≡

```
extern void write_files(Name *files);
```

□

125a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `Name` 158a, `write_files` 125c

⟨ *Function prototypes* 125b ⟩ + ≡

```
extern void write_map(Name *file_names, Name *macro_names);
extern void write_extract(char *name, Name *file_names, Name *macro_names);
extern void write_reverse_map(const char *arg);
extern void write_callers(char *name, Name *file_names, Name *macro_names);
extern void write_lint(Name *file_names, Name *macro_names);
extern void write_error_annotator(void);
```

□

125b · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `file_names` 158b, `macro_names` 158b, `Name` 158a, `write_callers` 116a, `write_error_annotator` 124, `write_extract` 116b, `write_lint` 118b, `write_map` 113a, `write_reverse_map` 121

⟨ *output.c* 125c ⟩ + ≡

```
static void make_parent_dirs(const char *path);    /* defined below */
```

```
void write_files(Name *files)
{
    while (files) {
        write_files(files->llink);
        ⟨ Write out files->spelling 126b ⟩
        files = files->rlink;
    }
}
```

□

125c · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a · defines `write_files` 39b, 125a · uses `make_parent_dirs` 126a, `Name` 158a

`MAX_INDENT` defines the maximum number of characters tracked per line for indentation purposes. When a fragment is expanded mid-line, the characters preceding it are recorded so that subsequent lines of the fragment can reproduce the same indentation. Languages like HTML, XML, and SVG routinely contain lines longer than 500 characters, so we set this to 8192. If a line exceeds this limit, indentation beyond the recorded portion falls back to spaces instead of causing a fatal error.

⟨ *Type declarations* 125d ⟩ + ≡

```
#define MAX_INDENT 8192
```

□

125d · defined at 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e · used in 21a · defines `MAX_INDENT` 87b, 126b, 149, 150e

When `--mkdirs` is given, `weft` creates the output directories itself before writing — both the `-p` prefix path and any subdirectories embedded in an `@o` name (`@o src/gen/foo.c`). Without the flag, the directories must already exist (the strict, typo-catching default). The helper is a portable recursive `mkdir -p`: it walks the path and creates each parent component, ignoring errors such as `EEXIST` (a real problem, like a permission error, surfaces later at the temporary-file or rename step). The final component — the file itself — is never created.

`<output.c 126a> + ≡`

```
#include <sys/stat.h>
#ifdef _WIN32
#include <direct.h>
#define WEFT_MKDIR(p) _mkdir(p)
#else
#define WEFT_MKDIR(p) mkdir((p), 0777)
#endif

static void make_parent_dirs(const char *path)
{
    char buf[FILENAME_MAX];
    char *p;
    if (strlen(path) >= sizeof(buf))
        return;
    strcpy(buf, path);
    for (p = buf + 1; *p != '\0'; p++) {
        if (*p == path_sep[0]) {
            *p = '\0';
            if (buf[0] != '\0')
                (void) WEFT_MKDIR(buf);
            *p = path_sep[0];
        }
    }
}
```

□

126a · defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a ·
defines `make_parent_dirs` 125c, 126b · uses `strlen` 21b

`<Write out files->spelling 126b> ≡`

```
{
    static char temp_name[FILENAME_MAX];
    static char real_name[FILENAME_MAX];
    static int temp_name_count = 0;
    char indent_chars[MAX_INDENT];
    FILE *temp_file;

    sprintf(real_name, "%s%s%s", dirpath, path_sep, files->spelling);
    if (mkdirs_flag)
        make_parent_dirs(real_name);

    <Find a free temporary file 127a>

    if (verbose_flag)
        fprintf(stderr, "writing %s [%s]\n", files->spelling, temp_name);
    write_scraps(temp_file, files->spelling, files->defs, 0, indent_chars,
        files->debug_flag, files->tab_flag, files->indent_flag,
        files->comment_flag, files->location_flag,
        NULL, NULL, 0, files->spelling);
    fclose(temp_file);
}
```

```

    < Move the temporary file to the target, if required 127b >
}
□
126b · used in 125c · uses defs 134a, fclose 21b, FILE 21b, fprintf 21b, make_parent_dirs 126a, MAX_INDENT 125d,
mkdirs_flag 27, stderr 21b, verbose_flag 27, write_scraps 147c

```

```

< Find a free temporary file 127a > ≡
for( temp_name_count = 0; temp_name_count < 10000; temp_name_count++) {
    sprintf(temp_name, "%s%snw%06d", dirpath, path_sep, temp_name_count);
#ifdef O_EXCL
    if (-1 != (temp_file_fd = open(temp_name, O_CREAT|O_WRONLY|O_EXCL))) {
        temp_file = fdopen(temp_file_fd, "w");
        break;
    }
#else
    if (0 != (temp_file = fopen(temp_name, "a"))) {
        if ( 0L == ftell(temp_file)) {
            break;
        } else {
            fclose(temp_file);
            temp_file = 0;
        }
    }
#endif
}
if (!temp_file) {
    fprintf(stderr, "%s: can't create %s for a temporary file\n",
        command_name, temp_name);
    exit(-1);
}
}
□

```

127a · used in 126b · uses command_name 29c, exit 21b, fclose 21b, fopen 21b, fprintf 21b, stderr 21b

Note the call to **remove** before **rename** – the ANSI/ISO C standard does *not* guarantee that renaming a file to an existing filename will overwrite the file.

```

< Move the temporary file to the target, if required 127b > ≡
if (diff_flag) {
    < Show diff of temp file vs real file 127c >
} else if (compare_flag)
    < Compare the temp file and the old file 128a >
else {
    remove(real_name);
    < Rename the temporary file to the target 128b >
}
}
□

```

127b · used in 126b · uses compare_flag 27, diff_flag 27, remove 21b

```

< Show diff of temp file vs real file 127c > ≡
{
    FILE *old_file = fopen(real_name, "r");
    if (!old_file) {
        /* New file: show all lines with + prefix */
        FILE *tf = fopen(temp_name, "r");
        if (tf) {
            char dbuf[8192];

```

```

        printf("new file: %s\n", real_name);
        while (fgets(dbuf, sizeof(dbuf), tf))
            printf("+%s", dbuf);
        fclose(tf);
    }
} else {
    char cmd[FILENAME_MAX * 2 + 32];
    FILE *dp;
    fclose(old_file);
    sprintf(cmd, "diff -u \"%s\" \"%s\"", real_name, temp_name);
    dp = popen(cmd, "r");
    if (dp) {
        char dbuf[8192];
        while (fgets(dbuf, sizeof(dbuf), dp))
            fputs(dbuf, stdout);
        pclose(dp);
    }
}
remove(temp_name);
}

```

127c · used in 127b · uses `fclose` 21b, `FILE` 21b, `fopen` 21b, `fputs` 21b, `remove` 21b

Again, we use a call to `remove` before `rename`.

```

< Compare the temp file and the old file 128a > ≡
{
    FILE *old_file = fopen(real_name, "r");
    if (old_file) {
        int x, y;
        temp_file = fopen(temp_name, "r");
        do {
            x = getc(old_file);
            y = getc(temp_file);
        } while (x == y && x != EOF);
        fclose(old_file);
        fclose(temp_file);
        if (x == y)
            remove(temp_name);
        else {
            remove(real_name);
            < Rename the temporary file to the target 128b >
        }
    }
    else
        < Rename the temporary file to the target 128b >
}

```

128a · used in 127b · uses `fclose` 21b, `FILE` 21b, `fopen` 21b, `getc` 21b, `remove` 21b

```

< Rename the temporary file to the target 128b > ≡
if (0 != rename(temp_name, real_name)) {
    fprintf(stderr, "%s: can't rename output file to %s\n",
            command_name, real_name);
    if (mkdirs_flag == FALSE)
        fprintf(stderr, " (does its directory exist? try --mkdirs)\n");
    remove(temp_name);
    exit(1);
}

```

}

□

128b · used in 127b, 128a · uses `command_name` 29c, `exit` 21b, `FALSE` 23a, `fprintf` 21b, `makedirs_flag` 27, `remove` 21b, `stderr` 21b

Chapter 3

The Support Routines

3.1 Source Files

3.1.1 Global Declarations

We need two routines to handle reading the source files.

⟨ *Function prototypes* 129a ⟩ + ≡

```
extern void source_open(char *name); /* pass in the name of the source file */
extern int source_get(void); /* no args; returns the next char or EOF */
extern int source_last; /* what last source_get() returned. */
extern int source_peek; /* The next character to get */
```

□

129a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses [next](#) 133c, [source_get](#) 130b, [source_last](#) 130b, [source_open](#) 133b, [source_peek](#) 130b

There are also two global variables maintained for use in error messages and such.

⟨ *Global variable declarations* 129b ⟩ + ≡

```
extern char *source_name; /* name of the current file */
extern int source_line; /* current line in the source file */
```

□

129b · defined at 27, 29ac, 45c, 50a, 66b, 129b, 137e, 158b, 197a · used in 21a · defines [source_line](#) 44d, 47ab, 129c, 130bd, 131ab, 132, 133ab, 138d, 139b, 140c, 160b, 170, 172, 173ab, 174a, 176b, 178c, 179d, 180, [source_name](#) 38ab, 39ab, 44d, 47ab, 129c, 131ab, 132, 133ab, 138d, 139b, 140c, 160b, 170, 172, 173ab, 174a, 176b, 177a, 178cd, 179d, 180

⟨ *Global variable definitions* 129c ⟩ + ≡

```
char *source_name = NULL;
int source_line = 0;
```

□

129c · defined at 28, 29bd, 46a, 50b, 66c, 129c, 137f, 158c · used in 25e · uses [source_line](#) 129b, [source_name](#) 129b

3.1.2 Local Declarations

⟨ *input.c* 129d ⟩ + ≡

```
static FILE *source_file; /* the current input file */
static int double_at;
static int include_depth;
```

□

129d · defined at 25a, 129d, 130abd, 133b · defines `double_at` 131a, 133b, `include_depth` 131b, 133ab, `source_file` 130bd, 131ab, 132, 133ab · uses `FILE` 21b

```
<input.c 130a> + ≡
static struct {
    FILE *file;
    char *name;
    int line;
} stack[10];
```

□

130a · defined at 25a, 129d, 130abd, 133b · defines `stack` 121, 123, 131b, 133a · uses `FILE` 21b

3.1.3 Reading a File

The routine `source_get` returns the next character from the current source file. It notices newlines and keeps the line counter `source_line` up to date. It also catches EOF and watches for @ characters. All other characters are immediately returned. We define `source_last` to let us tell which type of scrap we are defining.

<input.c 130b> + ≡

```
int source_peek;
int source_last;
int source_get(void)
{
    int c;
    source_last = c = source_peek;
    switch (c) {
        case EOF: <Handle EOF 133a>
            return c;
        case '\n': source_line++;
            FALLTHROUGH;
        default:
            if (c==nw_char)
            {
                <Handle an "at" character 131a>
                return c;
            }
            source_peek = getc(source_file);
            return c;
    }
}
```

□

130b · defined at 25a, 129d, 130abd, 133b · defines `source_get` 42d, 43c, 44cd, 45ab, 47ab, 48bcd, 49bc, 53, 54b, 59bc, 60b, 61, 66ad, 71b, 73b, 74cde, 75a, 78a, 79a, 82a, 85ab, 86a, 87ab, 89bc, 91b, 93a, 94b, 96, 97d, 99c, 105c, 106a, 108ad, 109b, 110a, 129a, 131b, 133a, 139ab, 140bc, 142a, 170, 172, 173ab, 174a, 176b, 177ab, 178d, 179d, 180, 181, `source_last` 54b, 65b, 71b, 91b, 129a, `source_peek` 48bd, 49b, 129a, 130d, 131ab, 133ab · uses `FALLTHROUGH` 23a, `getc` 21b, `nw_char` 29ab, `source_file` 129d, `source_line` 129b

`source_ungetc` pushes a read character back to the `source_file`.

<Function prototypes 130c> + ≡

```
extern void source_ungetc(int*);
```

□

130c · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `source_ungetc` 130d

```
<input.c 130d> + ≡
void source_ungetc(int *c)
{
    ungetc(source_peek, source_file);
    if(*c == '\n')
        source_line--;
    source_peek=*c;
}
□
```

130d · defined at 25a, 129d, 130abd, 133b · defines `source_ungetc` 48cd, 49c, 79a, 97d, 109b, 130c · uses `source_file` 129d, `source_line` 129b, `source_peek` 130b

This whole @ character handling mess is pretty annoying. I want to recognize @i so I can handle include files correctly. At the same time, it makes sense to recognize illegal @ sequences and complain; this avoids ever having to check anywhere else. Unfortunately, I need to avoid tripping over the @@ sequence; hence this whole unsatisfactory `double_at` business.

```
<Handle an "at" character 131a> ≡
{
    c = getc(source_file);
    if (double_at) {
        source_peek = c;
        double_at = FALSE;
        c = nw_char;
    }
    else
        switch (c) {
            case 'i': <Open an include file 131b>
                break;
            case '#': case 'f': case 'm': case 'u': case 'v':
            case 'd': case 'o': case 'D': case 'O': case 's':
            case 'q': case 'Q': case 'S': case 'l': case 'L': case 't':
            case 'W':
            case '+':
            case '-':
            case '*':
            case '\\':
            case '{': case '}': case '<': case '>': case '|':
            case '(': case ')': case '[': case ']':
            case '%': case '_':
            case ':': case ',': case 'x': case 'c':
            case '1': case '2': case '3': case '4': case '5':
            case '6': case '7': case '8': case '9':
            case 'r':
                source_peek = c;
                c = nw_char;
                break;
            default:
                if (c==nw_char)
                {
                    source_peek = c;
                    double_at = TRUE;
                    break;
                }
                fprintf(stderr, "%s: bad %c sequence %c[%d] (%s, line %d)\n",
```

```

        command_name, nw_char, c, c, source_name, source_line);
    exit(-1);
}
}
□

```

131a · used in 130b · uses [command_name 29c](#), [double_at 129d](#), [exit 21b](#), [FALSE 23a](#), [fprintf 21b](#), [getc 21b](#), [nw_char 29ab](#), [source_file 129d](#), [source_line 129b](#), [source_name 129b](#), [source_peek 130b](#), [stderr 21b](#), [TRUE 23a](#)

⟨ *Open an include file* 131b ⟩ ≡

```

{
    char name[FILENAME_MAX];
    char fullname[FILENAME_MAX];
    struct incl * p = include_list;

    if (include_depth >= 10) {
        fprintf(stderr, "%s: include nesting too deep (%s, %d)\n",
            command_name, source_name, source_line);
        exit(-1);
    }
    ⟨ Collect include-file name 132 ⟩
    stack[include_depth].file = source_file;
    fullname[0] = '\0';
    for (;;) {
        strcat(fullname, name);
        source_file = fopen(fullname, "r");
        if (source_file || !p)
            break;
        strcpy(fullname, p->name);
        strcat(fullname, "/");
        p = p->next;
    }
    if (!source_file) {
        fprintf(stderr, "%s: can't open include file %s\n",
            command_name, name);
        source_file = stack[include_depth].file;
    }
    else
    {
        stack[include_depth].name = source_name;
        stack[include_depth].line = source_line + 1;
        include_depth++;
        source_line = 1;
        source_name = save_string(fullname);
    }
    source_peek = getc(source_file);
    c = source_get();
}

```

□

131b · used in 131a · uses [command_name 29c](#), [exit 21b](#), [fopen 21b](#), [fprintf 21b](#), [getc 21b](#), [incl 27](#), [include_depth 129d](#), [next 133c](#), [save_string 159c](#), [source_file 129d](#), [source_get 130b](#), [source_line 129b](#), [source_name 129b](#), [source_peek 130b](#), [stack 130a](#), [stderr 21b](#)

⟨ *Collect include-file name* 132 ⟩ ≡

```

{
    char *p = name;
    do
        c = getc(source_file);

```

```

    while (c == ' ' || c == '\t');
    while (isgraph(c)) {
        *p++ = c;
        c = getc(source_file);
    }
    *p = '\0';
    if (c != '\n') {
        fprintf(stderr, "%s: unexpected characters after file name (%s, %d)\n",
            command_name, source_name, source_line);
        exit(-1);
    }
}

```

□

132 · used in 131b · uses `command_name` 29c, `exit` 21b, `fprintf` 21b, `getc` 21b, `isgraph` 21b, `source_file` 129d, `source_line` 129b, `source_name` 129b, `stderr` 21b

If an EOF is discovered, the current file must be closed and input from the next stacked file must be resumed. If no more files are on the stack, the EOF is returned.

⟨ *Handle EOF* 133a ⟩ ≡

```

{
    fclose(source_file);
    if (include_depth) {
        include_depth--;
        source_file = stack[include_depth].file;
        source_line = stack[include_depth].line;
        source_name = stack[include_depth].name;
        source_peek = getc(source_file);
        c = source_get();
    }
}

```

□

133a · used in 130b · uses `fclose` 21b, `getc` 21b, `include_depth` 129d, `source_file` 129d, `source_get` 130b, `source_line` 129b, `source_name` 129b, `source_peek` 130b, `stack` 130a

3.1.4 Opening a File

The routine `source_open` takes a file name and tries to open the file. If unsuccessful, it complains and halts. Otherwise, it sets `source_name`, `source_line`, and `double_at`.

⟨ *input.c* 133b ⟩ + ≡

```

void source_open(char *name)
{
    source_file = fopen(name, "r");
    if (!source_file) {
        fprintf(stderr, "%s: couldn't open %s\n", command_name, name);
        exit(-1);
    }
    nw_char = '@';
    source_name = name;
    source_line = 1;
    source_peek = getc(source_file);
    double_at = FALSE;
    include_depth = 0;
}

```

□

133b · defined at 25a, 129d, 130abd, 133b · defines `source_open` 42c, 56a, 84d, 104, 129a · uses `command_name` 29c, `double_at` 129d, `exit` 21b, `FALSE` 23a, `fopen` 21b, `fprintf` 21b, `getc` 21b, `include_depth` 129d, `nw_char` 29ab, `source_file` 129d, `source_line` 129b, `source_name` 129b, `source_peek` 130b, `stderr` 21b

3.2 Scraps

⟨ `scraps.c` 133c ⟩ + ≡
`#define SLAB_SIZE 1024`

```
typedef struct slab {
    struct slab *next;
    char chars[SLAB_SIZE];
} Slab;
```

□

133c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `next` 27, 31, 34, 35b, 39a, 46d, 52c, 61, 67abcd, 75c, 76a, 77, 78d, 80d, 81a, 82c, 90bde, 91a, 94d, 95b, 97c, 99ab, 100b, 101b, 102ad, 103ac, 108bc, 111b, 114, 115c, 116a, 129a, 131b, 138a, 141ab, 142c, 143a, 145de, 146bc, 147c, 154c, 155c, 157e, 179bc, 182bde, 183abcd, 184b, 186a, 188, 189ab, 191bcd, 192cd, 193de, 197c, 198ab, 200, `Slab` 134a, 137g, 138ad, 143a, `SLAB_SIZE` 138a, 143a, 145c, 187a · uses `slab` 134a

⟨ `scraps.c` 134a ⟩ + ≡

```
typedef struct {
    char *file_name;
    Slab *slab;
    struct uses *uses;
    struct uses *defs;
    int file_line;
    int end_line;
    int page;
    char letter;
    unsigned char sector;
    char *prose;
    struct name *owner;
} ScrapEntry;
```

□

134a · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `defs` 46d, 52c, 61, 62, 63a, 67ab, 75c, 76a, 77, 78d, 80d, 82c, 87b, 90bd, 94d, 95b, 97c, 99a, 100b, 102ab, 103ac, 107ab, 108bc, 109ac, 111b, 114, 117b, 118a, 126b, 135a, 136b, 138d, 141b, 146b, 147c, 148c, 150abc, 152de, 153, 154bc, 155b, 158a, 163a, 182cf, 189b, 192d, 193ace, `end_line` 111b, 137a, 138d, 139b, `file_line` 136e, 138d, 139a, 148c, `file_name` 42ac, 55a, 56a, 84ad, 103b, 104, 136d, 138d, 139a, 150ab, `letter` 124, 135c, 138d, 157d, `owner` 115c, 116a, 137c, 138d, 145b, `page` 67d, 80d, 82c, 91a, 99a, 100b, 102ab, 103a, 135acd, 138d, 156bc, 157bd, 192d, 193ae, 200, `prose` 27, 30, 31, 65b, 111b, 137b, 138d, 145a, 200, `ScrapEntry` 134b, 135b, 138d, `sector` 31, 55ab, 56a, 79ac, 80c, 81c, 82abc, 84acd, 97d, 98bc, 99c, 100ab, 103b, 104, 109bc, 110ab, 111b, 138d, 140c, 141ab, 142b, 143b, 152e, 155c, 158a, 159a, 160ab, 161a, 162b, 163a, 176b, 178c, 179d, 189b, `slab` 133c, 138d, 143b, 148c, 189b, `uses` 31, 56b, 67c, 81a, 82c, 90e, 99b, 100b, 103a, 108c, 110b, 111b, 114, 115c, 116a, 118a, 136a, 138d, 141a, 142c, 158a, 163ab, 182f, 189b, 191d, 192b, 193e · uses `Slab` 133c

There's some issue with reading the auxiliary file if it has more than 255 lines, so (as a workaround only) increase the size of the array of `ScrapEntry`s held in `SCRAP[n]`.

⟨ `scraps.c` 134b ⟩ + ≡
`#define SCRAP_BITS 10`

```

#define SCRAP_SIZE (1<<SCRAP_BITS)
#define SCRAP_MASK (SCRAP_SIZE - 1)
#define SCRAP_SHIFT SCRAP_BITS
static ScrapEntry *SCRAP[SCRAP_SIZE];

#define scrap_array(i) SCRAP[(i) >> SCRAP_SHIFT][(i) & SCRAP_MASK]

static int scraps;
int num_scraps(void)
{
    return scraps;
}
< Forward declarations for scraps.c 148a, ... >
□

134b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd,
184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines num_scraps 79c, 98b, 135a, SCRAP 135b, 138d,
scraps 24bcd, 27, 31, 54b, 55b, 62, 63a, 64a, 66ae, 67d, 84b, 87c, 88a, 89b, 90ac, 91a, 107ab, 108abc, 111b, 135b, 138d, 156bc,
157bc, 189b, 200, scrap_array 135c, 136abde, 137abc, 138d, 139ab, 141ab, 143b, 145ab, 148c, 150ab, 156bc, 157bd, 189b,
192b, 193c, SCRAP_BITS never used, SCRAP_MASK 138d, SCRAP_SHIFT 138d, SCRAP_SIZE 135b, 138d · uses ScrapEntry 134a

< Function prototypes 135a > + ≡
extern void init_scraps(void);
extern int collect_scrap(void);
extern int write_scraps(FILE *file, char *spelling, Scrap_Node *defs, int global_indent,
    char *indent_chars, char debug_flag, char tab_flag, char indent_flag, unsigned char
    comment_flag, char location_flag, Arglist *inArgs, char *inParams[9], Parameters
    parameters, char *title);
extern void write_scrap_ref(FILE *file, int num, int first, int *page);
extern void write_single_scrap_ref(FILE *file, int num);
extern int num_scraps(void);
extern Uses * get_scrap_uses(int scrap);
extern Uses * get_scrap_defs(int scrap);
□

135a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b,
192a, 193b, 195c, 197b · used in 21a · uses Arglist 179b, collect_scrap 138c, defs 134a, FILE 21b, first 197d,
get_scrap_defs 136b, get_scrap_uses 136a, init_scraps 135b, num_scraps 134b, page 134a, Parameters 52b,
Scrap_Node 157e, Uses 191d, write_scraps 147c, write_scrap_ref 135c, write_single_scrap_ref 135d

< scraps.c 135b > + ≡
void init_scraps(void)
{
    scraps = 1;
    SCRAP[0] = (ScrapEntry *) arena_getmem(SCRAP_SIZE * sizeof(ScrapEntry));
}
□

135b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd,
184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines init_scraps 42c, 135a · uses arena_getmem 197e,
SCRAP 134b, ScrapEntry 134a, scraps 134b, SCRAP_SIZE 134b

< scraps.c 135c > + ≡
void write_scrap_ref(FILE *file, int num, int first, int *page)
{
    if (scrap_array(num).page >= 0) {
        if (first!=0)
            fprintf(file, "%d", scrap_array(num).page);
        else if (scrap_array(num).page != *page)

```

```

    fprintf(file, ", %d", scrap_array(num).page);
    if (scrap_array(num).letter > 0)
        fputc(scrap_array(num).letter, file);
}
else {
    if (first!=0)
        putc('?', file);
    else
        fputs(" ?", file);
    < Warn (only once) about needing to rerun after Latex 137d >
}
if (first>=0)
    *page = scrap_array(num).page;
}

```

□

135c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcb, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `write_scrap_ref` 67d, 80d, 82c, 91a, 99a, 100b, 102b, 135ad, 193a · uses `FILE` 21b, `first` 197d, `fprintf` 21b, `fputs` 21b, `letter` 134a, `page` 134a, `putc` 21b, `scrap_array` 134b

```

< scraps.c 135d > + ≡
void write_single_scrap_ref(FILE *file, int num)
{
    int page;
    write_scrap_ref(file, num, TRUE, &page);
}

```

□

135d · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcb, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `write_single_scrap_ref` 54b, 61, 62, 63a, 66e, 67c, 76a, 78d, 81a, 82c, 87c, 88a, 90e, 94d, 95b, 97c, 99b, 100b, 103c, 107ab, 108bc, 135a, 153, 154b, 195d · uses `FILE` 21b, `page` 134a, `TRUE` 23a, `write_scrap_ref` 135c

```

< scraps.c 136a > + ≡
Uses * get_scrap_uses(int scrap)
{
    return scrap_array(scrap).uses;
}

```

□

136a · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcb, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `get_scrap_uses` 101a, 111b, 114, 135a · uses `scrap_array` 134b, `Uses` 191d, `uses` 134a

```

< scraps.c 136b > + ≡
Uses * get_scrap_defs(int scrap)
{
    return scrap_array(scrap).defs;
}

```

□

136b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcb, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `get_scrap_defs` 102c, 135a · uses `defs` 134a, `scrap_array` 134b, `Uses` 191d

These accessors expose scrap metadata that is needed by the `--map` and `--extract` modes (Section 2.7) without leaking the internal `ScrapEntry` or `Slab` types.

```

< Function prototypes 136c > + ≡

```

```

extern const char *scrap_file_name(int i);
extern int scrap_file_line(int i);
extern int scrap_end_line(int i);
extern const char *scrap_prose(int i);
extern Name *scrap_owner(int i);
extern void dump_scrap_text(FILE *file, int scrap_idx);
extern void dump_scrap_json(FILE *f, int scrap_idx);
extern void set_scrap_prose(int i, char *p);
extern void set_scrap_owner(int i, Name *n);

```

□

136c · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `dump_scrap_json` 144, `dump_scrap_text` 143b, `FILE` 21b, `Name` 158a, `scrap_end_line` 137a, `scrap_file_line` 136e, `scrap_file_name` 136d, `scrap_owner` 137c, `scrap_prose` 137b, `set_scrap_owner` 145b, `set_scrap_prose` 145a

⟨ `scraps.c` 136d ⟩ + ≡

```
const char *scrap_file_name(int i) { return scrap_array(i).file_name; }
```

□

136d · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `scrap_file_name` 111b, 114, 118a, 136c · uses `file_name` 134a, `scrap_array` 134b

⟨ `scraps.c` 136e ⟩ + ≡

```
int scrap_file_line(int i) { return scrap_array(i).file_line; }
```

□

136e · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `scrap_file_line` 111b, 114, 118a, 136c · uses `file_line` 134a, `scrap_array` 134b

⟨ `scraps.c` 137a ⟩ + ≡

```
int scrap_end_line(int i) { return scrap_array(i).end_line; }
```

□

137a · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `scrap_end_line` 111b, 136c · uses `end_line` 134a, `scrap_array` 134b

⟨ `scraps.c` 137b ⟩ + ≡

```
const char *scrap_prose(int i) { return scrap_array(i).prose; }
```

□

137b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `scrap_prose` 111b, 136c · uses `prose` 134a, `scrap_array` 134b

⟨ `scraps.c` 137c ⟩ + ≡

```
Name *scrap_owner(int i) { return scrap_array(i).owner; }
```

□

137c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `scrap_owner` 115c, 116a, 136c · uses `Name` 158a, `owner` 134a, `scrap_array` 134b

⟨ *Warn (only once) about needing to rerun after Latex* 137d ⟩ ≡

```

{
    if (!already_warned) {
        fprintf(stderr, "%s: you'll need to rerun weft after running latex\n",
                command_name);
        already_warned = TRUE;
    }
}

```

□

137d · used in 135c, 156c · uses already_warned 137e, command_name 29c, fprintf 21b, stderr 21b, TRUE 23a

⟨ *Global variable declarations* 137e ⟩ + ≡

```
extern int already_warned;
```

□

137e · defined at 27, 29ac, 45c, 50a, 66b, 129b, 137e, 158b, 197a · used in 21a · defines already_warned 137df, 156b

⟨ *Global variable definitions* 137f ⟩ + ≡

```
int already_warned = 0;
```

□

137f · defined at 28, 29bd, 46a, 50b, 66c, 129c, 137f, 158c · used in 25e · uses already_warned 137e

⟨ *scraps.c* 137g ⟩ + ≡

```
typedef struct {
    Slab *scrap;
    Slab *prev;
    int index;
} Manager;
```

□

137g · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines Manager 138abc, 143ab, 145c, 147a, 148c, 184d, 186c, 189b · uses Slab 133c

⟨ *scraps.c* 138a ⟩ + ≡

```
static void push(char c, Manager *manager)
{
    Slab *scrap = manager->scrap;
    int index = manager->index;
    scrap->chars[index++] = c;
    if (index == SLAB_SIZE) {
        Slab *new_slab = (Slab *) arena_getmem(sizeof(Slab));
        scrap->next = new_slab;
        manager->scrap = new_slab;
        index = 0;
    }
    manager->index = index;
}

```

□

138a · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines push 50d, 53, 138b, 139ab, 140a, 142ab · uses arena_getmem 197e, Manager 137g, next 133c, Slab 133c, SLAB_SIZE 133c

⟨ *scraps.c* 138b ⟩ + ≡

```
static void pushes(char *s, Manager *manager)
{
    while (*s)
        push(*s++, manager);
}

```

}

□

138b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `pushs` 53, 140a, 142b · uses `Manager` 137g, `push` 138a

⟨ `scraps.c` 138c ⟩ + ≡

```
int collect_scrap(void)
{
    int current_scrap, lblseq = 0;
    int depth = 1;
    Manager writer;
    ⟨ Create new scrap, managed by writer 138d ⟩
    ⟨ Accumulate scrap and return scraps++ 139a ⟩
}
```

□

138c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `collect_scrap` 46bc, 53, 135a, 182b · uses `Manager` 137g

⟨ Create new scrap, managed by writer 138d ⟩ ≡

```
{
    Slab *scrap = (Slab *) arena_getmem(sizeof(Slab));
    if ((scraps & SCRAP_MASK) == 0)
        SCRAP[scraps >> SCRAP_SHIFT] = (ScrapEntry *) arena_getmem(SCRAP_SIZE * sizeof(
            ScrapEntry));
    scrap_array(scraps).slab = scrap;
    scrap_array(scraps).file_name = save_string(source_name);
    scrap_array(scraps).file_line = source_line;
    scrap_array(scraps).page = -1;
    scrap_array(scraps).letter = 0;
    scrap_array(scraps).uses = NULL;
    scrap_array(scraps).defs = NULL;
    scrap_array(scraps).sector = current_sector;
    scrap_array(scraps).end_line = source_line;
    scrap_array(scraps).prose = NULL;
    scrap_array(scraps).owner = NULL;
    writer.scrap = scrap;
    writer.index = 0;
    current_scrap = scraps++;
}
```

□

138d · used in 138c · uses `arena_getmem` 197e, `current_sector` 45c, `defs` 134a, `end_line` 134a, `file_line` 134a, `file_name` 134a, `letter` 134a, `owner` 134a, `page` 134a, `prose` 134a, `save_string` 159c, `SCRAP` 134b, `ScrapEntry` 134a, `scraps` 134b, `scrap_array` 134b, `SCRAP_MASK` 134b, `SCRAP_SHIFT` 134b, `SCRAP_SIZE` 134b, `sector` 134a, `Slab` 133c, `slab` 134a, `source_line` 129b, `source_name` 129b, uses 134a

⟨ Accumulate scrap and return scraps++ 139a ⟩ ≡

```
{
    int c = source_get();
    while (1) {
        switch (c) {
            case EOF: fprintf(stderr, "%s: unexpect EOF in (%s, %d)\n",
                                command_name, scrap_array(current_scrap).file_name,
                                scrap_array(current_scrap).file_line);
                        exit(-1);
            default:
                if (c==nw_char)
```

```

        {
            < Handle at-sign during scrap accumulation 139b >
            break;
        }
        push(c, &writer);
        c = source_get();
        break;
    }
}
}

```

139a · used in 138c · uses `command_name` 29c, `exit` 21b, `file_line` 134a, `file_name` 134a, `fprintf` 21b, `nw_char` 29ab, `push` 138a, `scrap_array` 134b, `source_get` 130b, `stderr` 21b

< Handle at-sign during scrap accumulation 139b > ≡

```

{
    c = source_get();
    switch (c) {
        case '(':
        case '[':
        case '{': depth++;
                    break;

        case '+':
        case '-':
        case '*':
        case '|': < Collect user-specified index entries 140c >
                    /* Fall through */

        case ')':
        case ']':
        case '}': if (--depth > 0)
                    break;
                    /* else fall through */

        case ',':
                    push('\0', &writer);
                    scrap_ended_with = c;
                    scrap_array(current_scrap).end_line = source_line;
                    return current_scrap;

        case '<': < Handle macro invocation in scrap 142a >
                    break;

        case '%': < Skip commented-out code 74e >
                    /* emit line break to the output file to keep #line in sync. */
                    push('\n', &writer);
                    c = source_get();
                    break;

        case 'x': < Get label while collecting scrap 140a >
                    break;

        case 'c': < Include block comment in a scrap 50d >
                    break;

        case '1': case '2': case '3':
        case '4': case '5': case '6':
        case '7': case '8': case '9':
        case 'f': case '#': case 'v':
        case 't': case 's':
                    push(nw_char, &writer);
                    break;

        case '_': c = source_get();
                    break;

        default :
    }
}

```

```

        if (c==nw_char)
        {
            push(nw_char, &writer);
            push(nw_char, &writer);
            c = source_get();
            break;
        }
        fprintf(stderr, "%s: unexpected %c%c in scrap (%s, %d)\n",
                command_name, nw_char, c, source_name, source_line);
        exit(-1);
    }
}
□

```

139b · used in 139a · uses [command_name 29c](#), [end_line 134a](#), [exit 21b](#), [fprintf 21b](#), [nw_char 29ab](#), [push 138a](#), [scrap_array 134b](#), [source_get 130b](#), [source_line 129b](#), [source_name 129b](#), [stderr 21b](#)

```

< Get label while collecting scrap 140a > ≡
{
    < Get label from(140b source_get() ) 195a >
    < Save label to label store 195f >
    push(nw_char, &writer);
    push('x', &writer);
    pushs(label_name, &writer);
    push(nw_char, &writer);
}
□

```

140b · used in 139b · uses [source_get 130b](#)

```

< Collect user-specified index entries 140c > ≡
{
    do {
        int type = c;
        do {
            char new_name[MAX_NAME_LEN];
            char *p = new_name;
            unsigned int sector = 0;
            do
                c = source_get();
            while (isspace(c));
            if (c != nw_char) {
                Name *name;
                do {
                    *p++ = c;
                    c = source_get();
                } while (c != nw_char && !isspace(c));
                *p = '\0';
                switch (type) {
                    case '*':
                        sector = current_sector;
                        < Add user identifier use 141a >
                        break;
                    case '-':
                        < Add user identifier use 141a >
                        /* Fall through */
                    case '|':
                        sector = current_sector;
                        /* Fall through */

```

```

        case '+':
            < Add user identifier definition 141b >
            break;
    }
}
} while (c != nw_char);
c = source_get();
}while (c == '|' || c == '*' || c == '-' || c == '+');
if (c != ']' && c != ']' && c != ')') {
    fprintf(stderr, "%s: unexpected %c%c in index entry (%s, %d)\n",
        command_name, nw_char, c, source_name, source_line);
    exit(-1);
}
}
}

```

□

140c · used in 139b · uses [command_name 29c](#), [current_sector 45c](#), [exit 21b](#), [fprintf 21b](#), [isspace 21b](#), [MAX_NAME_LEN 23b](#), [Name 158a](#), [nw_char 29ab](#), [sector 134a](#), [source_get 130b](#), [source_line 129b](#), [source_name 129b](#), [stderr 21b](#)

```

< Add user identifier use 141a > ≡
name = name_add(&user_names, new_name, sector);
if (!name->uses || name->uses->scrap != current_scrap) {
    Scrap_Node *use = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    use->scrap = current_scrap;
    use->next = name->uses;
    name->uses = use;
    add_uses(&(scrap_array(current_scrap).uses), name);
}

```

□

141a · used in 140c · uses [add_uses 191c](#), [arena_getmem 197e](#), [name_add 162b](#), [next 133c](#), [scrap_array 134b](#), [Scrap_Node 157e](#), [sector 134a](#), [user_names 158b](#), [uses 134a](#)

```

< Add user identifier definition 141b > ≡
name = name_add(&user_names, new_name, sector);
if (!name->defs || name->defs->scrap != current_scrap) {
    Scrap_Node *def = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    def->scrap = current_scrap;
    def->next = name->defs;
    name->defs = def;
    add_uses(&(scrap_array(current_scrap).defs), name);
}

```

□

141b · used in 140c · uses [add_uses 191c](#), [arena_getmem 197e](#), [defs 134a](#), [name_add 162b](#), [next 133c](#), [scrap_array 134b](#), [Scrap_Node 157e](#), [sector 134a](#), [user_names 158b](#)

```

< Handle macro invocation in scrap 142a > ≡
{
    Arglist * args = collect_scrap_name(current_scrap);
    Name *name = args->name;
    < Save macro name 142b >
    add_to_use(name, current_scrap);
    if (scrap_name_has_parameters) {
        < Save macro parameters 53 >
    }
    push(nw_char, &writer);
    push('>', &writer);
    c = source_get();
}

```

}

□

142a · used in 139b · uses [add_to_use 142c](#), [Arglist 179b](#), [collect_scrap_name 179d](#), [Name 158a](#), [nw_char 29ab](#), [push 138a](#), [source_get 130b](#)

⟨ *Save macro name* 142b ⟩ ≡

```
{
    char buff[24];

    push(nw_char, &writer);
    push('<', &writer);
    push(name->sector, &writer);
    sprintf(buff, "%d", arglist_register(args));
    pushs(buff, &writer);
}
```

□

142b · used in 142a · uses [arglist_register 148b](#), [nw_char 29ab](#), [push 138a](#), [pushs 138b](#), [sector 134a](#)

⟨ [scraps.c 142c](#) ⟩ + ≡

```
void
add_to_use(Name * name, int current_scrap)
{
    if (!name->uses || name->uses->scrap != current_scrap) {
        Scrap_Node *use = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
        use->scrap = current_scrap;
        use->next = name->uses;
        name->uses = use;
    }
}
```

□

142c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines [add_to_use 53](#), 142ad, 182d · uses [arena_getmem 197e](#), [Name 158a](#), [next 133c](#), [Scrap_Node 157e](#), uses 134a

⟨ *Function prototypes* 142d ⟩ + ≡

```
extern void add_to_use(Name * name, int current_scrap);
```

□

142d · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses [add_to_use 142c](#), [Name 158a](#)

⟨ [scraps.c 143a](#) ⟩ + ≡

```
static char pop(Manager *manager)
{
    Slab *scrap = manager->scrap;
    int index = manager->index;
    char c = scrap->chars[index++];
    if (index == SLAB_SIZE) {
        manager->prev = scrap;
        manager->scrap = scrap->next;
        index = 0;
    }
    manager->index = index;
    return c;
}
```

□

143a · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines pop 50e, 51ac, 54a, 143b, 147ab, 148c, 150d, 151c, 152c, 185b · uses Manager 137g, next 133c, Slab 133c, SLAB_SIZE 133c

The function `dump_scrap_text` walks the slab chain for a given scrap and writes every printable character (plus newlines and tabs) to the supplied file handle. The internal meta-character sequences that encode fragment references are decoded back to their `@<name@>` form; other control sequences are silently skipped.

(`scraps.c` 143b) + ≡

```
void dump_scrap_text(FILE *file, int scrap_idx)
{
    Manager reader;
    char c;
    reader.scrap = scrap_array(scrap_idx).slab;
    reader.prev = NULL;
    reader.index = 0;
    c = pop(&reader);
    while (c) {
        if (c == nw_char) {
            /* handle internal control sequences */
            c = pop(&reader);
            if (c == '<') {
                /* fragment reference encoded as: sector_byte + decimal_index
                   followed by nw_char + '>' */
                char idx_buf[MAX_NAME_LEN];
                char *p = idx_buf;
                int idx;
                (void)pop(&reader); /* sector byte, discard */
                c = pop(&reader);
                while (c != nw_char && c != '\0') {
                    *p++ = c;
                    c = pop(&reader);
                }
                *p = '\0';
                if (sscanf(idx_buf, "%d", &idx) == 1) {
                    Arglist *a = arglist_lookup(idx);
                    if (a && a->name) {
                        fprintf(file, "%c<%s%c>", nw_char, a->name->spelling, nw_char);
                    }
                }
                /* skip past closing '>' marker */
            }
            if (c == nw_char)
                c = pop(&reader);
            /* skip optional parameter block */
            if (c == '(') {
                while (c && c != nw_char)
                    c = pop(&reader);
                if (c == nw_char)
                    c = pop(&reader);
            }
        }
        else {
            /* other control seq: skip */
        }
        c = pop(&reader);
        continue;
    }
    fputc(c, file);
    c = pop(&reader);
}
```

```

    }
}
□

```

143b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `dump_scrap_text` 114, 136c, 144 · uses `Arglist` 179b, `arglist_lookup` 148b, `FILE` 21b, `fprintf` 21b, `Manager` 137g, `MAX_NAME_LEN` 23b, `nw_char` 29ab, `pop` 143a, `scrap_array` 134b, `sector` 134a, `slab` 134a

The function `dump_scrap_json` is similar to `dump_scrap_text` but wraps the output in a JSON string, escaping special characters inline. It is used by `--bodies` to embed scrap code in the `-m` JSON output.

```

<scraps.c 144> + ≡
void dump_scrap_json(FILE *f, int scrap_idx)
{
    FILE *tmp = tmpfile();
    int ch;
    if (!tmp) { fputs("\\""", f); return; }
    dump_scrap_text(tmp, scrap_idx);
    fflush(tmp);
    fseek(tmp, 0, SEEK_SET);
    fputc('"', f);
    while ((ch = fgetc(tmp)) != EOF) {
        switch (ch) {
            case '\\': fputs("\\\\", f); break;
            case '\n': fputs("\\n", f); break;
            case '\r': fputs("\\r", f); break;
            case '\t': fputs("\\t", f); break;
            default:
                if (ch < 0x20)
                    fprintf(f, "\\u%04x", ch);
                else
                    fputc(ch, f);
        }
    }
    fputc('"', f);
    fclose(tmp);
}

```

□

144 · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `dump_scrap_json` 111b, 136c · uses `dump_scrap_text` 143b, `fclose` 21b, `FILE` 21b, `fprintf` 21b, `fputs` 21b

```

<scraps.c 145a> + ≡
void set_scrap_prose(int i, char *p) { scrap_array(i).prose = p; }

```

□

145a · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `set_scrap_prose` 46bc, 136c · uses `prose` 134a, `scrap_array` 134b

```

<scraps.c 145b> + ≡
void set_scrap_owner(int i, Name *n) { scrap_array(i).owner = n; }

```

□

145b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `set_scrap_owner` 46d, 136c · uses `Name` 158a,

owner 134a, scrap_array 134b

```
< scraps.c 145c > + ≡
static void backup(int n, Manager *manager)
{
    int index = manager->index;
    if (n > index
        && manager->prev != NULL)
    {
        manager->scrap = manager->prev;
        manager->prev = NULL;
        index += SLAB_SIZE;
    }
    manager->index = (n <= index ? index - n : 0);
}
```

□

145c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines backup 150d · uses Manager 137g, SLAB_SIZE 133c

```
< scraps.c 145d > + ≡
void
lookup(int n, Arglist * par, char * arg[9], Name **name, Arglist ** args)
{
    int i;
    Arglist * p = par;

    for (i = 0; i < n && p != NULL; i++)
        p = p->next;
    if (p == NULL) {
        char * a = arg[n];

        *name = NULL;
        *args = (Arglist *)a;
    }
    else {
        *name = p->name;
        *args = p->args;
    }
}
```

□

145d · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines lookup 52c, 146b · uses Arglist 179b, Name 158a, next 133c

```
< scraps.c 145e > + ≡
Arglist * instance(Arglist * a, Arglist * par, char * arg[9], int * ch)
{
    if (a != NULL) {
        int changed = 0;
        Arglist *args, *next;
        Name* name;
        < Set up name, args and next 146b >
        if (changed){
            < Build a new arglist 146c >
            *ch = 1;
        }
    }
}
```

```

    return a;
}

```

□

145e · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines instance 87b, 146ab, 152e · uses Arglist 179b, Name 158a, next 133c

⟨ *Function prototypes* 146a ⟩ + ≡

```

Arglist *instance(Arglist *a, Arglist *par, char *arg[9], int *ch);

```

□

146a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses Arglist 179b, instance 145e

⟨ *Set up name, args and next* 146b ⟩ ≡

```

next = instance(a->next, par, arg, &changed);
name = a->name;
if (name == (Name *)1) {
    Embed_Node * q = (Embed_Node *)arena_getmem(sizeof(Embed_Node));
    q->defs = (Scrap_Node *)a->args;
    q->args = par;
    args = (Arglist *)q;
    changed = 1;
} else if (name != NULL)
    args = instance(a->args, par, arg, &changed);
else {
    char * p = (char *)a->args;
    if (p[0] == ARG_CHR) {
        lookup(p[1] - '1', par, arg, &name, &args);
        changed = 1;
    }
    else {
        args = a->args;
    }
}
}

```

□

146b · used in 145e · uses arena_getmem 197e, Arglist 179b, ARG_CHR 178a, defs 134a, Embed_Node 182c, instance 145e, lookup 145d, Name 158a, next 133c, Scrap_Node 157e

⟨ *Build a new arglist* 146c ⟩ ≡

```

a = (Arglist *)arena_getmem(sizeof(Arglist));
a->name = name;
a->args = args;
a->next = next;

```

□

146c · used in 145e · uses arena_getmem 197e, Arglist 179b, next 133c

⟨ *scraps.c* 147a ⟩ + ≡

```

static Arglist *pop_scrap_name(Manager *manager, Parameters *parameters)
{
    char name[MAX_NAME_LEN];
    char *p = name;
    Arglist * args;
    int c;
    int idx;

```

```

(void)pop(manager); /* not sure why we have to pop twice */
c = pop(manager);

while (c != nw_char) {
    *p++ = c;
    c = pop(manager);
}
*p = '\000';
if (sscanf(name, "%d", &idx) != 1)
{
    fprintf(stderr, "%s: found an internal problem (2)\n", command_name);
    exit(-1);
}
args = arglist_lookup(idx);
< Check for end of scrap name 147b >
return args;
}

```

□

147a · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `pop_scrap_name` 152e · uses `Arglist` 179b, `arglist_lookup` 148b, `command_name` 29c, `exit` 21b, `fprintf` 21b, `Manager` 137g, `MAX_NAME_LEN` 23b, `nw_char` 29ab, `Parameters` 52b, `pop` 143a, `stderr` 21b

```

< Check for end of scrap name 147b > ≡
{
    c = pop(manager);
    < Check for macro parameters 54a >
}

```

□

147b · used in 147a · uses `pop` 143a

< `scraps.c` 147c > + ≡

```

int write_scraps(FILE *file, char *spelling, Scrap_Node *defs,
                int global_indent, char *indent_chars,
                char debug_flag, char tab_flag, char indent_flag,
                unsigned char comment_flag, char location_flag,
                Arglist *inArgs,
                char *inParams[9], Parameters parameters, char *title)
{
    /* This is in file file name */
    int indent = 0;
    int newline = 1;
    while (defs) {
        < Copy defs->scrap to file 148c >
        defs = defs->next;
    }
    return indent + global_indent;
}

```

□

147c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `write_scraps` 52c, 76b, 87b, 95b, 126b, 135a, 152e, 155b · uses `Arglist` 179b, `defs` 134a, `FILE` 21b, `next` 133c, `Parameters` 52b, `Scrap_Node` 157e

```

< Forward declarations for scraps.c 148a > ≡
int delayed_indent = 0;

```

□

148a · defined at 148ab, 155a, 176a, 191a · used in 134b · defines `delayed_indent` 52c, 148c, 151a, 152e, 153, 154a

< Forward declarations for scraps.c 148b > + ≡

```
#define MAX_ARGLISTS 4096
static Arglist *arglist_table[MAX_ARGLISTS];
static int arglist_count = 0;

void arglist_reset(void)
{
    arglist_count = 0;
}

static int arglist_register(Arglist *a)
{
    if (arglist_count >= MAX_ARGLISTS) {
        fprintf(stderr, "%s: too many macro argument lists (max %d)\n",
            command_name, MAX_ARGLISTS);
        exit(EXIT_FAILURE);
    }
    arglist_table[arglist_count] = a;
    return arglist_count++;
}

static Arglist *arglist_lookup(int idx)
{
    if (idx < 0 || idx >= arglist_count) {
        fprintf(stderr, "%s: invalid arglist index %d\n", command_name, idx);
        exit(EXIT_FAILURE);
    }
    return arglist_table[idx];
}
□
```

148b · defined at 148ab, 155a, 176a, 191a · used in 134b · defines `arglist_count` never used, `arglist_lookup` 143b, 147a, 190b, `arglist_register` 142b, `arglist_reset` 197b, 199, `arglist_table` never used, `MAX_ARGLISTS` never used ·
uses `Arglist` 179b, `command_name` 29c, `exit` 21b, `fprintf` 21b, `stderr` 21b

< Copy defs->scrap to file 148c > ≡

```
{
    char c;
    Manager reader;
    Parameters local_parameters = 0;
    int line_number = scrap_array(defs->scrap).file_line;
    reader.scrap = scrap_array(defs->scrap).slab;
    reader.prev = NULL;
    reader.index = 0;
    < Insert debugging information if required 150a >
    < Insert section opening marker 150b >
    if (delayed_indent)
    {
        < Insert appropriate indentation 150d >
    }
    c = pop(&reader);
    while (c) {
        switch (c) {
            case '\n':
                if (global_indent >= 0) {
                    putc(c, file);
                }
            }
        }
    }
```

```

        line_number++;
        newline = 1;
        delayed_indent = 0;
        < Insert appropriate indentation 150d >
        break;
    } else {
        /* Don't show newlines in embedded fragments */
        fputs(". . .", file);
        return 0;
    }
case '\t': < Handle tab characters on output 151b >
    delayed_indent = 0;
    break;
default:
    if (c==nw_char)
    {
        < Check for macro invocation in scrap 151c >
        break;
    }
    putc(c, file);
    if (global_indent >= 0) {
        < Add more indentation ' ' 149 >
    }
    indent++;
    if (c > ' ') newline = 0;
    delayed_indent = 0;
    break;
}
c = pop(&reader);
}
< Insert section closing marker 150c >
}

```

□

148c · used in 147c · uses [defs 134a](#), [delayed_indent 148a](#), [file_line 134a](#), [fputs 21b](#), [Manager 137g](#), [nw_char 29ab](#), [Parameters 52b](#), [pop 143a](#), [putc 21b](#), [scrap_array 134b](#), [slab 134a](#)

We need to make sure that we don't overflow `indent_chars[]`. If the line is longer than `MAX_INDENT`, we simply stop recording; the Put out the indent scrap will use spaces for any positions beyond the recorded range.

```

< Add more indentation char 149 > ≡
{
    if (global_indent + indent < MAX_INDENT)
        indent_chars[global_indent + indent] = char;
}

```

□

149 · used in 148c, 151bc · uses [MAX_INDENT 125d](#)

```

< Insert debugging information if required 150a > ≡
if (debug_flag) {
    fprintf(file, "\n#line %d \"%s\"\n",
            line_number, scrap_array(defs->scrap).file_name);
    < Insert appropriate indentation 150d >
}

```

□

150a · used in 148c, 152a · uses [defs 134a](#), [file_name 134a](#), [fprintf 21b](#), [scrap_array 134b](#)

Section markers in the Knuth style delimit each scrap with opening and closing comments that include the scrap number and source location. For example, in C: `/* {5: weft.w:28} */` and `/* {:5} */`.

⟨ Insert section opening marker 150b ⟩ ≡

```
if (location_flag && comment_flag) {
    fprintf(file, "%s{%d: %s:%d}%s\n",
            comment_begin[comment_flag],
            defs->scrap,
            scrap_array(defs->scrap).file_name,
            line_number,
            comment_end[comment_flag]);
}
```

□

150b · used in 148c · uses `comment_begin 176a`, `comment_end 176a`, `defs 134a`, `file_name 134a`, `fprintf 21b`, `scrap_array 134b`

⟨ Insert section closing marker 150c ⟩ ≡

```
if (location_flag && comment_flag) {
    fprintf(file, "%s{: %d}%s\n",
            comment_begin[comment_flag],
            defs->scrap,
            comment_end[comment_flag]);
}
```

□

150c · used in 148c · uses `comment_begin 176a`, `comment_end 176a`, `defs 134a`, `fprintf 21b`

⟨ Insert appropriate indentation 150d ⟩ ≡

```
{
    char c1 = pop(&reader);
    char c2 = pop(&reader);

    if (indent_flag && !(⟨ Indent suppressed 151a ⟩)) {
        ⟨ Put out the indent 150e ⟩
    }
    indent = 0;
    backup(2, &reader);
}
```

□

150d · used in 148c, 150a · uses `backup 145c`, `pop 143a`

⟨ Put out the indent 150e ⟩ ≡

```
if (tab_flag)
    for (indent=0; indent<global_indent; indent++)
        putc(' ', file);
else
    for (indent=0; indent<global_indent; indent++)
        putc(indent < MAX_INDENT ? indent_chars[indent] : ' ', file);
```

□

150e · used in 52c, 150d · uses `MAX_INDENT 125d`, `putc 21b`

Indent will be suppressed if the next character is a newline or if the next two characters are `@#`. If the next two characters are `@<`, we suppress the indent for now but mark that it may be needed when the next fragment is started.

⟨ Indent suppressed 151a ⟩ ≡

```

c1 == '\n'
|| (c1 == nw_char && (c2 == '#' || (delayed_indent != (c2 == '<'))))

```

□

151a · used in 150d · uses [delayed_indent 148a](#), [nw_char 29ab](#)

⟨ *Handle tab characters on output* 151b ⟩ ≡

```

{
    if (tab_flag)
        ⟨ Expand tab into spaces 72c ⟩
    else {
        putc('\t', file);
        if (global_indent >= 0) {
            ⟨ Add more indentation '\t' 149 ⟩
        }
        indent++;
    }
}

```

□

151b · used in 148c · uses [putc 21b](#)

⟨ *Check for macro invocation in scrap* 151c ⟩ ≡

```

{
    int oldin = indent;
    char oldcf = comment_flag;
    c = pop(&reader);
    switch (c) {
        case 't': ⟨ Copy fragment title into file 154b ⟩
            break;
        case 'c': ⟨ Copy block comment from scrap 50e ⟩
            break;
        case 'f': ⟨ Copy file name into file 152d ⟩
            break;
        case 'x': ⟨ Copy label from scrap into file 152b ⟩
        case '_': break;
        case 'v': ⟨ Copy version info into file 74a ⟩
            break;
        case 's': indent = -global_indent;
            comment_flag = 0;
            break;
        case '<': ⟨ Copy macro into file 152e ⟩
            ⟨ Resync the line number after a fragment reference 152a ⟩
            indent = oldin;
            comment_flag = oldcf;
            break;
        ⟨ Handle macro parameter substitution 52c ⟩
            indent = oldin;
            break;
        default:
            if(c==nw_char)
            {
                putc(c, file);
                if (global_indent >= 0) {
                    ⟨ Add more indentation ' ' 149 ⟩
                }
                indent++;
                break;
            }
        /* ignore, since we should already have a warning */
    }
}

```

```

        break;
    }
}
□

```

151c · used in 148c · uses [nw_char 29ab](#), [pop 143a](#), [putc 21b](#)

The `#line` marker that keeps the C preprocessor's line count in step is emitted *after* a fragment reference expands, to resync to the parent scrap's current line. That is right when the reference sat on a line of its own, but for a *mid-line* reference — `int port = @<port@>;` or `#define V "@<version@>"` — an unconditional resync would inject a stray `\n#line\n` into the middle of the line, splitting the expression or string literal across two lines (and breaking the C it produces). So we resync only when the reference *began* its line (`newline` is still set): a mid-line reference then splices its fragment's value in place and the surrounding text stays on one line, exactly as `@<port@>` becoming 4000 should. The child expansion already suppresses its own markers mid-line (see `Copy macro into file`); this closes the remaining gap left by the trailing resync.

< Resync the line number after a fragment reference 152a > ≡

```

if (newline) {
    < Insert debugging information if required 150a >
}
□

```

152a · used in 151c

< Copy label from scrap into file 152b > ≡

```

{
    < Get label from(152c pop(&reader) ) 195a >
    write_label(label_name, file);
}
□

```

152c · used in 151c · uses [pop 143a](#)

< Copy file name into file 152d > ≡

```

if (defs->quoted)
    fprintf(file, "%cf", nw_char);
else
    fputs(spelling, file);
□

```

152d · used in 151c · uses [defs 134a](#), [fprintf 21b](#), [fputs 21b](#), [nw_char 29ab](#)

< Copy macro into file 152e > ≡

```

{
    Arglist *a = pop_scrap_name(&reader, &local_parameters);
    Name *name = a->name;
    int changed;
    Arglist * args = instance(a->args, inArgs, inParams, &changed);
    int i, nargs;
    char * p = name->spelling;
    char * * inParams = name->arg;
    Arglist *q = args;

    if (name->mark) {
        fprintf(stderr, "%s: recursive macro discovered involving <%s>\n",
            command_name, name->spelling);
        exit(-1);
    }
    if (name->defs && !defs->quoted) {

```

```

unsigned char saved_comment = comment_flag;
char saved_location = location_flag;
char saved_debug = debug_flag;
if (name->lang[0] != '\0') {
    comment_flag = name->comment_flag;
    location_flag = name->location_flag;
    debug_flag = name->debug_flag;
}
if (!newline) {
    comment_flag = 0;
    location_flag = FALSE;
    debug_flag = FALSE;
}
< Perhaps comment this macro 153 >
name->mark = TRUE;
indent = write_scraps(file, spelling, name->defs, global_indent + indent,
                      indent_chars, debug_flag, tab_flag, indent_flag,
                      comment_flag, location_flag, args, name->arg,
                      local_parameters, name->spelling);
comment_flag = saved_comment;
location_flag = saved_location;
debug_flag = saved_debug;
indent -= global_indent;
name->mark = FALSE;
}
else
{
    if (delayed_indent)
    {
        for (i = indent + global_indent; --i >= 0; )
            putc(' ', file);
    }

    fprintf(file, "%c<", nw_char);
    if (name->sector == 0)
        fputc('+', file);
    < Comment this macro use 154c >
    fprintf(file, "%c>", nw_char);
    if (!defs->quoted && !tex_flag)
        fprintf(stderr, "%s: macro never defined <%s>\n",
                command_name, name->spelling);
}
}

```

□

152e · used in 151c · uses Arglist 179b, command_name 29c, defs 134a, delayed_indent 148a, exit 21b, FALSE 23a, fprintf 21b, instance 145e, Name 158a, nw_char 29ab, pop_scrap_name 147a, putc 21b, sector 134a, stderr 21b, tex_flag 27, TRUE 23a, write_scraps 147c

```

< Perhaps comment this macro 153 > ≡
if (comment_flag && newline) {
    < Perhaps put a delayed indent 154a >
    fputs(comment_begin[comment_flag], file);
    < Comment this macro use 154c >
    if (xref_flag) {
        putc(' ', file);
        write_single_scrap_ref(file, name->defs->scrap);
    }
    fputs(comment_end[comment_flag], file);
}

```

```

    putc('\n', file);
    if (!delayed_indent)
        for (i = indent + global_indent; --i >= 0; )
            putc(' ', file);
}

```

□

153 · used in 52c, 152e · uses [comment_begin 176a](#), [comment_end 176a](#), [defs 134a](#), [delayed_indent 148a](#), [fputs 21b](#), [putc 21b](#), [write_single_scrap_ref 135d](#), [xref_flag 27](#)

⟨ *Perhaps put a delayed indent 154a* ⟩ ≡

```

if (delayed_indent)
    for (i = indent + global_indent; --i >= 0; )
        putc(' ', file);

```

□

154a · used in 50e, 153 · uses [delayed_indent 148a](#), [putc 21b](#)

⟨ *Copy fragment title into file 154b* ⟩ ≡

```

{
    char * p = title;
    Arglist *q = inArgs;
    int narg;

    ⟨ Comment this macro use 154c ⟩
    if (xref_flag) {
        putc(' ', file);
        write_single_scrap_ref(file, defs->scrap);
    }
}

```

□

154b · used in 151c · uses [Arglist 179b](#), [defs 134a](#), [putc 21b](#), [write_single_scrap_ref 135d](#), [xref_flag 27](#)

⟨ *Comment this macro use 154c* ⟩ ≡

```

narg = 0;
while (*p != '\000') {
    if (*p == ARG_CHR) {
        if (q == NULL) {
            if (defs->quoted)
                fprintf(file, "%c'%s%c'", nw_char, inParams[narg], nw_char);
            else
                fprintf(file, "'%s'", inParams[narg]);
        }
        else {
            comment_ArglistElement(file, q, defs->quoted);
            q = q->next;
        }
        p++;
        narg++;
    }
    else
        fputc(*p++, file);
}

```

□

154c · used in 152e, 153, 154b · uses [ARG_CHR 178a](#), [comment_ArglistElement 155a](#), [defs 134a](#), [fprintf 21b](#), [next 133c](#), [nw_char 29ab](#)

⟨ *Forward declarations for scraps.c 155a* ⟩ + ≡

```

static void
comment_ArglistElement(FILE * file, Arglist * args, int quote)
{
    Name *name = args->name;
    Arglist *q = args->args;

    if (name == NULL) {
        if (quote)
            fprintf(file, "%c'%s%c'", nw_char, (char *)q, nw_char);
        else
            fprintf(file, "'%s'", (char *)q);
    } else if (name == (Name *)1) {
        < Include an embedded scrap in comment 155b >
    } else {
        < Include a fragment use in comment 155c >
    }
}

```

□

155a · defined at 148ab, 155a, 176a, 191a · used in 134b · defines `comment_ArglistElement` 154c, 155c · uses `Arglist` 179b, `FILE` 21b, `fprintf` 21b, `Name` 158a, `nw_char` 29ab

< Include an embedded scrap in comment 155b > ≡

```

Embed_Node * e = (Embed_Node *)q;
fputc('{', file);
write_scraps(file, "", e->defs, -1, "", 0, 0, 0, 0, e->args, NULL, NULL, "");
fputc('}', file);

```

□

155b · used in 155a · uses `defs` 134a, `Embed_Node` 182c, `write_scraps` 147c

< Include a fragment use in comment 155c > ≡

```

char * p = name->spelling;
if (quote)
    fputc(nw_char, file);
fputc('<', file);
if (quote && name->sector == 0)
    fputc('+', file);
while (*p != '\000') {
    if (*p == ARG_CHR) {
        comment_ArglistElement(file, q, quote);
        q = q->next;
        p++;
    }
    else
        fputc(*p++, file);
}
if (quote)
    fputc(nw_char, file);
fputc('>', file);

```

□

155c · used in 155a · uses `ARG_CHR` 178a, `comment_ArglistElement` 155a, `next` 133c, `nw_char` 29ab, `sector` 134a

3.2.1 Collecting Page Numbers

< Function prototypes 156a > + ≡

```

extern void collect_numbers(char *aux_name);

```

□

156a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `collect_numbers` 156b

```
< scraps.c 156b > + ≡
void collect_numbers(char *aux_name)
{
    if (number_flag) {
        int i;
        for (i=1; i<scraps; i++)
            scrap_array(i).page = i;
    }
    else {
        FILE *aux_file = fopen(aux_name, "r");
        already_warned = FALSE;
        if (aux_file) {
            char aux_line[500];
            while (fgets(aux_line, 500, aux_file)) {
                < Read line in .aux file 156c >
            }
            fclose(aux_file);
            < Add letters to scraps with duplicate page numbers 157a >
        }
    }
}
```

□

156b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcb, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `collect_numbers` 39b, 156a · uses `already_warned` 137e, `FALSE` 23a, `fclose` 21b, `FILE` 21b, `fopen` 21b, `number_flag` 27, `page` 134a, `scraps` 134b, `scrap_array` 134b

The *memoir* document class creates references in the `.aux` file with nested braces, so after each open brace we have to wait for the matching closing brace.

```
< Read line in .aux file 156c > ≡
int scrap_number;
int page_number;
int i;
int dummy_idx;
int bracket_depth = 1;
if (1 == sscanf(aux_line,
                "\\newlabel{scrap%d}{%n",
                &scrap_number,
                &dummy_idx)) {
    for (i = dummy_idx; i < (int)strlen(aux_line) && bracket_depth > 0; i++) {
        if (aux_line[i] == '{') bracket_depth++;
        else if (aux_line[i] == '}') bracket_depth--;
    }
    if (i > dummy_idx
        && i < (int)strlen(aux_line)
        && 1 == sscanf(aux_line+i, "{%d}", &page_number)) {
        if (scrap_number < scraps)
            scrap_array(scrap_number).page = page_number;
        else
            < Warn (only once) about needing to rerun after Latex 137d >
    }
}
```

□

156c · used in 156b · uses `page` 134a, `scraps` 134b, `scrap_array` 134b, `strlen` 21b

```

< Add letters to scraps with duplicate page numbers 157a > ≡
{
    int i = 0;

    < Step i to the next valid scrap 157b >
    < For all remaining scraps 157c > {
        int j = i;
        < Step j to the next valid scrap 157b >
        < Perhaps add letters to the page numbers 157d >
        i = j;
    }
}

```

□

157a · used in 156b

```

< Step i to the next valid scrap 157b > ≡
do
    i++;
while (i < scraps && scrap_array(i).page == -1);

```

□

157b · used in 157a · uses page 134a, scraps 134b, scrap_array 134b

```

< For all remaining scraps 157c > ≡
while (i < scraps)

```

□

157c · used in 157a · uses scraps 134b

```

< Perhaps add letters to the page numbers 157d > ≡
if (scrap_array(i).page == scrap_array(j).page) {
    if (scrap_array(i).letter == 0)
        scrap_array(i).letter = 'a';
    scrap_array(j).letter = scrap_array(i).letter + 1;
}

```

□

157d · used in 157a · uses letter 134a, page 134a, scrap_array 134b

3.3 Names

Each scrap is linked through a simple node:

```

< Type declarations 157e > + ≡
typedef struct scrap_node {
    struct scrap_node *next;
    int scrap;
    char quoted;
} Scrap_Node;

```

□

157e · defined at 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e · used in 21a · defines Scrap_Node 46d, 52c, 55b, 61, 67d, 76ab, 78d, 80d, 81a, 82c, 84b, 91a, 94d, 95b, 97c, 99ab, 100b, 102a, 103a, 111b, 114, 115c, 116a, 135a, 141ab, 142c, 146b, 147c, 158a, 182bcf, 183a, 189b, 191ab, 192d, 193e · uses next 133c

A Name represents a file or fragment definition. Beyond the spelling and the binary tree links, several flags control how the tangled output is formatted:

- `debug_flag` — if TRUE, emit `#line` directives (set by `-d` or auto-detected for C/C++).
- `comment_flag` — selects the comment style for section markers and block comments: 0=none, 1=C, 2=C++, 3=shell, 4=Lua/SQL, 5=HTML/XML (see Section 3.3.5).
- `location_flag` — if TRUE, emit Knuth-style opening and closing section markers around each scrap in the tangled output. Activated automatically by `@l` auto-detection or the `@L` command.
- `lang` — the language tag (e.g. "c", "python") used for syntax highlighting in the Markdown weave output. Set from a preceding `@l` command or inferred from the file extension of an `@o` name. Empty if unknown.

⟨ *Type declarations* 158a ⟩ + ≡

```
typedef struct name {
    char *spelling;
    struct name *llink;
    struct name *rlink;
    Scrap_Node *defs;
    Scrap_Node *uses;
    char * arg[9];
    int mark;
    char tab_flag;
    char indent_flag;
    char debug_flag;
    char location_flag;
    char suppress_markers;
    unsigned char comment_flag;
    char lang[64];
    unsigned char sector;
} Name;
```

□

158a · defined at 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e · used in 21a · defines [Name](#) 31, 46bc, 52c, 55b, 61, 62, 63a, 71b, 75c, 77, 78b, 79bc, 80b, 81bc, 82b, 84bc, 87bc, 88a, 91b, 94d, 95b, 97a, 98ab, 100a, 102a, 103ac, 104, 107ab, 109ac, 110b, 111b, 113ab, 114, 115abc, 116ab, 117b, 118ab, 125abc, 136c, 137c, 140c, 142acd, 145bde, 146b, 152e, 155a, 158bc, 159a, 160a, 161a, 162b, 163a, 173a, 176b, 178e, 179abc, 180, 182bf, 183b, 186a, 187cd, 189b, 191acd, 192d, 193e, 194b · uses [defs](#) 134a, [Scrap_Node](#) 157e, [sector](#) 134a, [uses](#) 134a

The variable `pending_lang` acts as a one-shot channel between the `@l` command in `pass1` (which fills it with a language tag like "js" or "python") and `collect_file_name` or `collect_macro_name` (which consume it to set the flags and the `lang` field on the next `@o` or `@d` name).

⟨ *Global variable declarations* 158b ⟩ + ≡

```
extern Name *file_names;
extern Name *macro_names;
extern Name *user_names;
extern int scrap_name_has_parameters;
extern int scrap_ended_with;
extern char pending_lang[64];
```

□

158b · defined at 27, 29ac, 45c, 50a, 66b, 129b, 137e, 158b, 197a · used in 21a · defines [file_names](#) 39b, 42c, 52a, 78a, 96, 108d, 115c, 116b, 118b, 125b, 158c, 173a, [macro_names](#) 39b, 42c, 52a, 79a, 97d, 109b, 116b, 118b, 125b, 158c, 178c, [pending_lang](#) 43c, 158c, 174b, 175, [user_names](#) 42c, 52a, 82a, 99c, 110a, 141ab, 158c, 187c · uses [Name](#) 158a

⟨ *Global variable definitions* 158c ⟩ + ≡

```
Name *file_names = NULL;
Name *macro_names = NULL;
Name *user_names = NULL;
```

```
int scrap_name_has_parameters;
int scrap_ended_with;
char pending_lang[64] = "";
```

□

158c · defined at 28, 29bd, 46a, 50b, 66c, 129c, 137f, 158c · used in 25e · uses [file_names 158b](#), [macro_names 158b](#), [Name 158a](#), [pending_lang 158b](#), [user_names 158b](#)

⟨ *Function prototypes* 159a ⟩ + ≡

```
extern Name *collect_file_name(void);
extern Name *collect_macro_name(void);
extern Arglist *collect_scrap_name(int current_scrap);
extern Name *name_add(Name **rt, char *spelling, unsigned char sector);
extern Name *prefix_add(Name **rt, char *spelling, unsigned char sector);
extern char *save_string(char *s);
extern void reverse_lists(Name *names);
extern void collect_lang_def(void);
extern void collect_weave_format(void);
```

□

159a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses [Arglist 179b](#), [collect_file_name 173a](#), [collect_lang_def 170](#), [collect_macro_name 176b](#), [collect_scrap_name 179d](#), [collect_weave_format 172](#), [Name 158a](#), [name_add 162b](#), [prefix_add 160a](#), [reverse_lists 182f](#), [save_string 159c](#), [sector 134a](#)

⟨ *names.c* 159b ⟩ + ≡

```
enum { LESS, GREATER, EQUAL, PREFIX, EXTENSION };
```

```
static int compare(char *x, char *y)
{
    int len, result;
    int xl = strlen(x);
    int yl = strlen(y);
    int xp = x[xl - 1] == ' ';
    int yp = y[yl - 1] == ' ';
    if (xp) xl--;
    if (yp) yl--;
    len = xl < yl ? xl : yl;
    result = strncmp(x, y, len);
    if (result < 0) return GREATER;
    else if (result > 0) return LESS;
    else if (xl < yl) {
        if (xp) return EXTENSION;
        else return LESS;
    }
    else if (xl > yl) {
        if (yp) return PREFIX;
        else return GREATER;
    }
    else return EQUAL;
}
```

□

159b · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · defines [compare 160a](#), [161a](#), [EQUAL 160a](#), [161a](#), [EXTENSION 160a](#), [161a](#), [GREATER 160a](#), [161a](#), [LESS 160a](#), [161a](#), [PREFIX 160a](#), [161a](#) · uses [strlen 21b](#)

⟨ *names.c* 159c ⟩ + ≡

```
char *save_string(char *s)
{
```

```

char *new_str = (char *) arena_getmem((strlen(s) + 1) * sizeof(char));
strcpy(new_str, s);
return new_str;
}

```

□

159c · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · defines `save_string` 35b, 39a, 43b, 131b, 138d, 159a, 160a, 163a, 179a, 182e · uses `arena_getmem` 197e, `strlen` 21b

⟨names.c 160a⟩ + ≡

```

static int ambiguous_prefix(Name *node, char *spelling, unsigned char sector);

```

```

static char * found_name = NULL;

```

```

Name *prefix_add(Name **rt, char *spelling, unsigned char sector)
{

```

```

    Name *node = *rt;
    int cmp;

```

```

    while (node) {
        switch ((cmp = compare(node->spelling, spelling))) {
            case GREATER:    rt = &node->rlink;
                            break;
            case LESS:      rt = &node->llink;
                            break;
            case EQUAL:
                found_name = node->spelling;
                FALLTHROUGH;
            case EXTENSION: if (node->sector > sector) {
                            rt = &node->rlink;
                            break;
                        }
                        else if (node->sector < sector) {
                            rt = &node->llink;
                            break;
                        }
                        if (cmp == EXTENSION)
                            node->spelling = save_string(spelling);
                        return node;
            case PREFIX:    ⟨ Check for ambiguous prefix 160b ⟩
                            return node;
        }
        node = *rt;
    }

```

⟨ Create new name entry 163a ⟩

```

}

```

□

160a · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · defines `prefix_add` 159a, 178c · uses `compare` 159b, `EQUAL` 159b, `EXTENSION` 159b, `FALLTHROUGH` 23a, `GREATER` 159b, `LESS` 159b, `Name` 158a, `PREFIX` 159b, `save_string` 159c, `sector` 134a

Since a very short prefix might match more than one fragment name, I need to check for other matches to avoid mistakes. Basically, I simply continue the search down *both* branches of the tree.

⟨ Check for ambiguous prefix 160b ⟩ ≡

```

{
    if (ambiguous_prefix(node->llink, spelling, sector) ||
        ambiguous_prefix(node->rlink, spelling, sector))

```

```

    fprintf(stderr,
        "%s: ambiguous prefix %c<%s...%c> (%s, line %d)\n",
        command_name, nw_char, spelling, nw_char, source_name, source_line);
}
□
160b · used in 160a · uses command_name 29c, fprintf 21b, nw_char 29ab, sector 134a, source_line 129b, source_name 129b,
stderr 21b

⟨names.c 161a⟩ + ≡
static int ambiguous_prefix(Name *node, char *spelling, unsigned char sector)
{
    while (node) {
        switch (compare(node->spelling, spelling)) {
            case GREATER:    node = node->rlink;
                            break;
            case LESS:      node = node->llink;
                            break;
            case EXTENSION:
            case PREFIX:
            case EQUAL:     if (node->sector > sector) {
                            node = node->rlink;
                            break;
                        }
                            else if (node->sector < sector) {
                            node = node->llink;
                            break;
                        }
                            return TRUE;
        }
    }
    return FALSE;
}
□
161a · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · uses compare 159b,
EQUAL 159b, EXTENSION 159b, FALSE 23a, GREATER 159b, LESS 159b, Name 158a, PREFIX 159b, sector 134a, TRUE 23a

```

Rob Shillingsburg suggested that I organize the index of user-specified identifiers more traditionally; that is, not relying on strict ASCII comparisons via `strcmp`. Ideally, we'd like to see the index ordered like this:

```

aardvark
Adam
atom
Atomic
atoms

```

The function `robs_strcmp` implements the desired predicate. It returns -2 for alphabetically less-than, -1 for less-than but only differing in case, zero for equal, 1 for greater-than but only in case and 2 for alphabetically greater-than.

```

⟨Function prototypes 161b⟩ + ≡
extern int robs_strcmp(char*, char*);
□
161b · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b,
192a, 193b, 195c, 197b · used in 21a · uses robs_strcmp 161c

⟨names.c 161c⟩ + ≡

```

```

int robs_strcmp(char* x, char* y)
{
    int cmp = 0;

    for (; *x && *y; x++, y++)
    {
        < Skip invisibles on x 162a >
        < Skip invisibles on y 162a >
        if (*x == *y)
            continue;
        if (islower(*x) && toupper(*x) == *y)
        {
            if (!cmp) cmp = 1;
            continue;
        }
        if (islower(*y) && *x == toupper(*y))
        {
            if (!cmp) cmp = -1;
            continue;
        }
        return 2*(toupper(*x) - toupper(*y));
    }
    if (*x)
        return 2;
    if (*y)
        return -2;
    return cmp;
}

```

□

161c · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · defines `robs_strcmp` 80a, 113b, 161b, 162b, 163a, 191c · uses `islower` 21b, `toupper` 21b

Certain character sequences are invisible when printed. We don't want them to be considered for the alphabetical ordering.

< Skip invisibles on p 162a > ≡

```

if (*p == '|')
    p++;

```

□

162a · used in 161c

< names.c 162b > + ≡

Name *name_add(Name **rt, char *spelling, unsigned char sector)

```

{
    Name *node = *rt;
    while (node) {
        int result = robs_strcmp(node->spelling, spelling);
        if (result > 0)
            rt = &node->llink;
        else if (result < 0)
            rt = &node->rlink;
        else
        {
            found_name = node->spelling;
            if (node->sector > sector)
                rt = &node->llink;
            else if (node->sector < sector)

```

```

        rt = &node->rlink;
    else
        return node;
    }
    node = *rt;
}
⟨ Create new name entry 163a ⟩
}

```

□

162b · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · defines `name_add` 141ab, 159a, 173a · uses `Name` 158a, `robs_strcmp` 161c, `sector` 134a

```

⟨ Create new name entry 163a ⟩ ≡
{
    node = (Name *) arena_getmem(sizeof(Name));
    if (found_name && robs_strcmp(found_name, spelling) == 0)
        node->spelling = found_name;
    else
        node->spelling = save_string(spelling);
    node->mark = FALSE;
    node->llink = NULL;
    node->rlink = NULL;
    node->uses = NULL;
    node->defs = NULL;
    node->arg[0] =
    node->arg[1] =
    node->arg[2] =
    node->arg[3] =
    node->arg[4] =
    node->arg[5] =
    node->arg[6] =
    node->arg[7] =
    node->arg[8] = NULL;
    node->tab_flag = TRUE;
    node->indent_flag = TRUE;
    node->debug_flag = FALSE;
    node->location_flag = FALSE;
    node->suppress_markers = FALSE;
    node->comment_flag = 0;
    node->lang[0] = '\0';
    node->sector = sector;
    *rt = node;
    return node;
}

```

□

163a · used in 160a, 162b · uses `arena_getmem` 197e, `defs` 134a, `FALSE` 23a, `Name` 158a, `robs_strcmp` 161c, `save_string` 159c, `sector` 134a, `TRUE` 23a, uses 134a

3.3.1 Language Auto-Detection

When a Markdown fence (‘‘‘c, ‘‘‘python) or a L^AT_EX marker (%%c) precedes an @o directive, weft auto-detects the language’s comment style and, for C/C++, enables #line directives. This is driven by a data table that maps fence tags to one of the six comment styles defined in Section 3.3.5.

Each entry records:

- **name** — the fence tag (lowercase, e.g. "js", "python", "lua").
- **comment** — index into the `comment_begin/comment_end` arrays (1–5).

- `line_directive` — TRUE only for languages whose toolchains honour `#line` (C family).

The built-in table covers over 250 entries (languages, file extensions, and aliases) organised by comment style. For any language not listed, the `@L` command (Section 3.3.2) lets users add entries at the document level. User-defined entries are searched first, so they can also override built-in defaults. If a fence tag matches neither table, weft defaults to C++ line comments (`//`) with no `#line`.

```
<names.c 163b> + ≡
typedef struct {
    const char *name;
    unsigned char comment;
    char line_directive;
} LangEntry;

static const LangEntry lang_table[] = {
    /* ---- Style 1: C block comments WITH #line ---- */
    {"c", 1, TRUE},
    {"h", 1, TRUE},
    {"hpp", 2, TRUE},
    {"hxx", 2, TRUE},
    {"hh", 2, TRUE},
    {"cpp", 2, TRUE},
    {"c++", 2, TRUE},
    {"cxx", 2, TRUE},
    {"cc", 2, TRUE},
    {"objc", 1, TRUE},
    {"objcpp", 2, TRUE},
    {"m", 1, TRUE},
    {"mm", 2, TRUE},
    {"ino", 2, TRUE}, /* Arduino */
    {"pde", 2, TRUE}, /* Processing (C-like) */
    /* ---- Style 1: C block comments WITHOUT #line ---- */
    {"css", 1, FALSE},
    {"scss", 1, FALSE},
    {"sass", 1, FALSE},
    {"less", 1, FALSE},
    {"stylus", 1, FALSE},
    {"styl", 1, FALSE},
    /* ---- Style 2: C++ line comments (//) ---- */
    /* --- JavaScript / TypeScript --- */
    {"js", 2, FALSE},
    {"javascript", 2, FALSE},
    {"jsx", 2, FALSE},
    {"mjs", 2, FALSE},
    {"cjs", 2, FALSE},
    {"ts", 2, FALSE},
    {"typescript", 2, FALSE},
    {"tsx", 2, FALSE},
    {"mts", 2, FALSE},
    {"cts", 2, FALSE},
    {"svelte", 2, FALSE},
    {"vue", 2, FALSE},
    /* --- JVM languages --- */
    {"java", 2, FALSE},
    {"scala", 2, FALSE},
    {"sc", 2, FALSE},
    {"kotlin", 2, FALSE},
    {"kt", 2, FALSE},
    {"kts", 2, FALSE},
```

```

{"groovy",      2, FALSE},
{"gradle",     2, FALSE},
{"clj",        2, FALSE}, /* Clojure (;; but // is close enough) */
{"cljs",       2, FALSE},
{"cljs",       2, FALSE},
/* --- .NET --- */
{"cs",         2, FALSE},
{"csharp",     2, FALSE},
{"fs",         2, FALSE},
{"fsharp",     2, FALSE},
{"fsx",        2, FALSE},
{"vb",         2, FALSE}, /* VB.NET uses ' but // is a safe fallback */
/* --- Systems languages --- */
{"go",         2, FALSE},
{"rust",       2, FALSE},
{"rs",         2, FALSE},
{"zig",        2, FALSE},
{"d",          2, FALSE},
{"v",          2, FALSE},
{"vlang",      2, FALSE},
{"swift",      2, FALSE},
{"dart",       2, FALSE},
{"odin",       2, FALSE},
{"jai",        2, FALSE},
{"carbon",     2, FALSE},
{"mojo",       2, FALSE},
/* --- Web / general --- */
{"php",        2, FALSE},
{"hack",       2, FALSE},
{"jsonc",      2, FALSE},
{"json5",      2, FALSE},
{"graphql",    2, FALSE},
{"gql",        2, FALSE},
{"prisma",     2, FALSE},
{"wgsl",       2, FALSE}, /* WebGPU shading language */
/* --- GPU / shaders --- */
{"glsl",       2, FALSE},
{"hlsl",       2, FALSE},
{"vert",       2, FALSE},
{"frag",       2, FALSE},
{"comp",       2, FALSE},
{"geom",       2, FALSE},
{"tesc",       2, FALSE},
{"tese",       2, FALSE},
{"metal",      2, FALSE},
{"cu",         2, TRUE}, /* CUDA: supports #line */
{"cuh",        2, TRUE},
{"cl",         1, FALSE}, /* OpenCL: C-style block comments */
/* --- Protocol / IDL / blockchain --- */
{"proto",      2, FALSE},
{"protobuf",   2, FALSE},
{"thrift",     2, FALSE},
{"flatbuffers", 2, FALSE},
{"fbs",        2, FALSE},
{"capnp",      2, FALSE},
{"avdl",       2, FALSE}, /* Avro IDL */
{"solidity",   2, FALSE},
{"sol",        2, FALSE},
{"vyper",      3, FALSE},

```

```

{"vy",          3, FALSE},
{"move",        2, FALSE},
/* --- Functional (// style) --- */
{"reason",      2, FALSE},
{"re",          2, FALSE},
{"res",         2, FALSE}, /* ReScript */
{"ml",          2, FALSE}, /* OCaml also uses (**) but // in comments */
{"mli",         2, FALSE},
{"sml",         2, FALSE},
{"sig",         2, FALSE},
{"fut",         2, FALSE}, /* Futhark */
/* --- Other // languages --- */
{"actionscript", 2, FALSE},
{"as",          2, FALSE},
{"apex",        2, FALSE},
{"bicep",       2, FALSE},
{"ceu",         2, FALSE},
{"chapel",      2, FALSE},
{"chpl",        2, FALSE},
{"cilk",        2, TRUE},
{"jsonnet",     2, FALSE},
{"libsonnet",   2, FALSE},
{"p4",          2, FALSE},
{"qml",         2, FALSE},
{"slang",       2, FALSE},
/* ---- Style 3: Shell / hash comments (#) ---- */
/* --- Python --- */
{"py",          3, FALSE},
{"python",      3, FALSE},
{"pyx",         3, FALSE}, /* Cython */
{"pxd",         3, FALSE},
{"pyi",         3, FALSE}, /* type stubs */
{"sage",        3, FALSE}, /* SageMath */
/* --- Shell --- */
{"sh",          3, FALSE},
{"bash",        3, FALSE},
{"zsh",         3, FALSE},
{"fish",        3, FALSE},
{"ksh",         3, FALSE},
{"csh",         3, FALSE},
{"tcsh",        3, FALSE},
{"dash",        3, FALSE},
{"nu",          3, FALSE}, /* Nushell */
/* --- Classic scripting --- */
{"perl",        3, FALSE},
{"pl",          3, FALSE},
{"pm",          3, FALSE},
{"ruby",        3, FALSE},
{"rb",          3, FALSE},
{"rake",        3, FALSE},
{"gemspec",     3, FALSE},
{"r",           3, FALSE},
{"rmd",         3, FALSE},
{"tcl",         3, FALSE},
{"awk",         3, FALSE},
{"sed",         3, FALSE},
/* --- Build / make --- */
{"makefile",    3, FALSE},
{"make",        3, FALSE},

```

```

{"mk",          3, FALSE},
{"cmake",       3, FALSE},
{"just",        3, FALSE},
{"justfile",    3, FALSE},
{"meson",       3, FALSE},
{"scons",       3, FALSE},
{"bazel",       3, FALSE},
{"bzl",         3, FALSE},
{"buck",        3, FALSE},
{"pants",       3, FALSE},
{"gnumakefile", 3, FALSE},
/* --- Scientific / modern --- */
{"julia",      3, FALSE},
{"jl",         3, FALSE},
{"nim",        3, FALSE},
{"nimble",     3, FALSE},
{"crystal",    3, FALSE},
{"cr",         3, FALSE},
{"elixir",     3, FALSE},
{"ex",         3, FALSE},
{"exs",        3, FALSE},
{"coco",       3, FALSE}, /* Coconut (Python superset) */
{"coffee",    3, FALSE}, /* CoffeeScript */
{"litcoffee", 3, FALSE},
/* --- Config / DevOps --- */
{"yaml",       3, FALSE},
{"yml",        3, FALSE},
{"toml",       3, FALSE},
{"dockerfile", 3, FALSE},
{"docker",     3, FALSE},
{"containerfile", 3, FALSE},
{"terraform",  3, FALSE},
{"tf",         3, FALSE},
{"tfvars",     3, FALSE},
{"hcl",        3, FALSE},
{"nix",        3, FALSE},
{"conf",       3, FALSE},
{"ini",        3, FALSE},
{"properties", 3, FALSE},
{"env",        3, FALSE},
{"editorconfig", 3, FALSE},
{"gitignore",  3, FALSE},
{"dockerignore", 3, FALSE},
{"flake8",     3, FALSE},
{"pylint",     3, FALSE},
{"powershell", 3, FALSE},
{"ps1",        3, FALSE},
{"psm1",       3, FALSE},
{"psd1",       3, FALSE},
/* --- Hardware description (# style) --- */
{"tcl",        3, FALSE},
{"xdc",        3, FALSE}, /* Xilinx constraints */
{"sdc",        3, FALSE}, /* Synopsys constraints */
/* --- Data / query (# style) --- */
{"gn",         3, FALSE}, /* GN build */
{"cfg",        3, FALSE},
{"pip",        3, FALSE},
{"cask",       3, FALSE}, /* Homebrew */
{"gemfile",    3, FALSE},

```

```

{"vagrantfile", 3, FALSE},
{"procfile", 3, FALSE},
/* ---- Style 4: double-dash comments (--) ---- */
{"lua", 4, FALSE},
{"luau", 4, FALSE}, /* Roblox Lua */
{"sql", 4, FALSE},
{"psql", 4, FALSE},
{"plsql", 4, FALSE},
{"plpgsql", 4, FALSE},
{"mysql", 4, FALSE},
{"sqlite", 4, FALSE},
{"hive", 4, FALSE},
{"presto", 4, FALSE},
{"sparksql", 4, FALSE},
{"hs", 4, FALSE},
{"haskell", 4, FALSE},
{"lhs", 4, FALSE},
{"elm", 4, FALSE},
{"purescript", 4, FALSE},
{"purs", 4, FALSE},
{"ada", 4, FALSE},
{"adb", 4, FALSE},
{"ads", 4, FALSE},
{"vhdl", 4, FALSE},
{"vhd", 4, FALSE},
{"lean", 4, FALSE},
{"lean4", 4, FALSE},
{"agda", 4, FALSE},
{"idr", 4, FALSE}, /* Idris */
{"idris", 4, FALSE},
{"erl", 4, FALSE}, /* Erlang uses % but -- is common in embedded SQL */
{"erlang", 4, FALSE},
/* ---- Style 5: HTML comments (<!-- -->) ---- */
{"html", 5, FALSE},
{"htm", 5, FALSE},
{"xml", 5, FALSE},
{"svg", 5, FALSE},
{"xhtml", 5, FALSE},
{"xsl", 5, FALSE},
{"xslt", 5, FALSE},
{"xsd", 5, FALSE},
{"dtd", 5, FALSE},
{"rss", 5, FALSE},
{"atom", 5, FALSE},
{"mathml", 5, FALSE},
{"plist", 5, FALSE},
{"xaml", 5, FALSE},
{"csproj", 5, FALSE},
{"fsproj", 5, FALSE},
{"vbproj", 5, FALSE},
{"vcxproj", 5, FALSE},
{"sln", 5, FALSE},
{"props", 5, FALSE},
{"targets", 5, FALSE},
{"nuspec", 5, FALSE},
{"resx", 5, FALSE},
{"wxml", 5, FALSE},
{"axml", 5, FALSE},
{"jspx", 5, FALSE},

```

```

{"jsp",          5, FALSE},
{"asp",          5, FALSE},
{"ascx",         5, FALSE},
{"aspx",         5, FALSE},
{"cshtml",       5, FALSE}, /* Razor */
{"vbhtml",       5, FALSE},
{"erb",          5, FALSE},
{"ejs",          5, FALSE},
{"njk",          5, FALSE}, /* Nunjucks */
{"hbs",          5, FALSE}, /* Handlebars */
{"mustache",     5, FALSE},
{"liquid",       5, FALSE},
{"twig",         5, FALSE},
{"blade",        5, FALSE}, /* Laravel Blade */
{"pug",          5, FALSE}, /* Pug/Jade */
{"slim",         5, FALSE},
{"haml",         5, FALSE},
{"md",           5, FALSE}, /* Markdown: HTML comments are valid */
{"markdown",     5, FALSE},
{"mdx",          5, FALSE},
{"rst",          5, FALSE}, /* reStructuredText (.. is native but <!-- works) */
{"tex",          5, FALSE}, /* LaTeX: % is native but no style for it */
{"latex",        5, FALSE},
/* ---- Style 0: no comments (data formats) ---- */
{"json",         0, FALSE},
{"json5",        0, FALSE},
{"csv",          0, FALSE},
{"tsv",          0, FALSE},
{"txt",          0, FALSE},
{"text",         0, FALSE},
{"log",          0, FALSE},
{"diff",         0, FALSE},
{"patch",        0, FALSE},
/* Sentinel */
{NULL,          0, FALSE}
};

/* User-defined languages via @L command */
#define MAX_USER_LANGS 256
static LangEntry user_langs[MAX_USER_LANGS];
static char user_lang_names[MAX_USER_LANGS][64];
static int user_lang_count = 0;

static const LangEntry *find_lang(const char *name)
{
    int i;
    const LangEntry *p;
    /* User-defined languages override built-in */
    for (i = 0; i < user_lang_count; i++)
        if (strcmp(user_langs[i].name, name) == 0) return &user_langs[i];
    for (p = lang_table; p->name; p++)
        if (strcmp(p->name, name) == 0) return p;
    return NULL;
}

static void lang_to_flags(const char *lang,
                        unsigned char *comment_flag,
                        char *debug_flag,
                        char *location_flag)

```

```

{
    const LangEntry *e = find_lang(lang);
    if (e) {
        *comment_flag = e->comment;
        *debug_flag = e->line_directive;
        *location_flag = (e->comment != 0);
    } else {
        /* Unknown language: default to // comments, no #line */
        *comment_flag = 2;
        *debug_flag = FALSE;
        *location_flag = TRUE;
    }
}

```

□

163b · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · defines `find_lang` 174b, `LangEntry` 174b, `lang_table` never used, `lang_to_flags` 174b, 175, `user_langs` 170, `user_lang_count` 170, `user_lang_names` 170 · uses `FALSE` 23a, `TRUE` 23a, uses 134a

3.3.2 User-Defined Languages

The `@L` command lets users register new language mappings (or override built-in ones) directly in the document preamble. This is useful for languages that are not in the built-in table, or when the default comment style for a language is not what the user wants.

Syntax: `@L name style [+d]`

- *name* — the fence tag that will trigger this mapping (e.g. `nim`, `glsl`, `wgsl`).
- *style* — one of `//`, `#`, `--`, `/*`, or `<!--`, selecting the comment delimiters.
- `+d` — optional; if present, enables `#line` directive generation for this language.

Examples:

```

@L nim #           % Nim uses # comments
@L glsl // +d      % GLSL uses // and supports #line
@L c //           % override: use // instead of /* */ for C

```

Up to 32 user-defined languages may be registered. They are searched before the built-in table, so a definition like `@L c //` effectively overrides the built-in mapping for C.

The `@L` command is processed during `pass1` and silently skipped during weaving. It should appear in the document preamble (before any `@o` or `@d` directives that depend on it).

`<names.c 170> + ≡`

```

void collect_lang_def(void)
{
    char name[64];
    char *p = name;
    char style[16];
    char *sp;
    int c;
    unsigned char comment = 0;
    char line_dir = FALSE;

    /* Skip whitespace after @L */
    c = source_get();
    while (c == ' ' || c == '\t')
        c = source_get();
}

```

```

/* Read language name */
while (c != EOF && c != ' ' && c != '\t' && c != '\n'
      && (p - name) < 62) {
    *p++ = c;
    c = source_get();
}
*p = '\0';

if (name[0] == '\0') {
    fprintf(stderr, "%s: expected language name after %cL (%s, %d)\n",
            command_name, nw_char, source_name, source_line);
    while (c != EOF && c != '\n') c = source_get();
    return;
}

/* Skip whitespace */
while (c == ' ' || c == '\t')
    c = source_get();

/* Read comment style token */
sp = style;
while (c != EOF && c != ' ' && c != '\t' && c != '\n'
      && (sp - style) < 14) {
    *sp++ = c;
    c = source_get();
}
*sp = '\0';

/* Map style string to comment index */
if (strcmp(style, "//") == 0)        comment = 2;
else if (strcmp(style, "#") == 0)    comment = 3;
else if (strcmp(style, "--") == 0)   comment = 4;
else if (strcmp(style, "/*") == 0)   comment = 1;
else if (strcmp(style, "<!--") == 0)  comment = 5;
else {
    fprintf(stderr,
            "%s: unknown comment style '%s' for %cL (%s, %d)\n",
            command_name, style, nw_char, source_name, source_line);
    fprintf(stderr, " Valid styles: // # -- /* <!--\n");
    while (c != EOF && c != '\n') c = source_get();
    return;
}

/* Check for optional +d flag (#line directives) */
while (c == ' ' || c == '\t')
    c = source_get();
if (c == '+') {
    c = source_get();
    if (c == 'd') line_dir = TRUE;
}

/* Consume rest of line */
while (c != EOF && c != '\n') c = source_get();

/* Register the user language */
if (user_lang_count < MAX_USER_LANGS) {
    snprintf(user_lang_names[user_lang_count],
            sizeof(user_lang_names[user_lang_count]), "%s", name);
}

```

```

    user_langs[user_lang_count].name = user_lang_names[user_lang_count];
    user_langs[user_lang_count].comment = comment;
    user_langs[user_lang_count].line_directive = line_dir;
    user_lang_count++;
} else {
    fprintf(stderr, "%s: too many %cL definitions (max %d) (%s, %d)\n",
            command_name, nw_char, MAX_USER_LANGS,
            source_name, source_line);
    exit(1);
}
}

```

□

170 · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · defines `collect_lang_def` 44a, 159a · uses `command_name` 29c, `exit` 21b, `FALSE` 23a, `fprintf` 21b, `nw_char` 29ab, `source_get` 130b, `source_line` 129b, `source_name` 129b, `stderr` 21b, `TRUE` 23a, `user_langs` 163b, `user_lang_count` 163b, `user_lang_names` 163b

3.3.3 Weave Format Declaration

The `@W` command declares the weave output format in the source file. Valid tokens are `md` (Markdown) and `tex` (L^AT_EX). If the user has already specified a format on the command line (`-w md` or `-w tex`), the declaration is silently ignored.

```

<names.c 172> + ≡
void collect_weave_format(void)
{
    char token[64];
    char *p = token;
    int c;

    /* Skip whitespace after @W */
    c = source_get();
    while (c == ' ' || c == '\t')
        c = source_get();

    /* Read format token */
    while (c != EOF && c != ' ' && c != '\t' && c != '\n'
           && (p - token) < 62) {
        *p++ = c;
        c = source_get();
    }
    *p = '\0';

    /* Consume rest of line */
    while (c != EOF && c != '\n') c = source_get();

    if (token[0] == '\0') {
        fprintf(stderr, "%s: expected format after %cW (md or tex) (%s, %d)\n",
                command_name, nw_char, source_name, source_line);
        return;
    }

    /* If CLI already set the format, skip silently */
    if (weave_format_from_cli)
        return;

    if (strcmp(token, "md") == 0)
        weave_format = 2;
    else if (strcmp(token, "tex") == 0)

```

```

    weave_format = 1;
else if (strcmp(token, "html") == 0)
    weave_format = 3;
else {
    fprintf(stderr, "%s: unknown weave format '%s' for %cW (%s, %d)\n",
        command_name, token, nw_char, source_name, source_line);
    fprintf(stderr, "    Valid formats: md tex html\n");
}
}

```

□

172 · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a ·
 defines collect_weave_format 44b, 159a · uses command_name 29c, fprintf 21b, nw_char 29ab, source_get 130b,
 source_line 129b, source_name 129b, stderr 21b, weave_format 27, weave_format_from_cli 27

3.3.4 Collecting File and Fragment Names

An output file name (@o) is terminated by whitespace. After reading the name we check for optional per-file flags (-d, -t, -i, -c) and then, if no explicit comment or debug flags were given, apply any pending language defaults from a preceding fence, %%% marker, or @L definition.

<names.c 173a> + ≡

```

Name *collect_file_name(void)
{
    Name *new_name;
    char name[MAX_NAME_LEN];
    char *p = name;
    int start_line = source_line;
    int c = source_get(), c2;
    while (isspace(c))
        c = source_get();
    while (isgraph(c)) {
        *p++ = c;
        c = source_get();
    }
    if (p == name) {
        fprintf(stderr, "%s: expected file name (%s, %d)\n",
            command_name, source_name, start_line);
        exit(-1);
    }
    *p = '\0';
    /* File names are always global. */
    new_name = name_add(&file_names, name, 0);
    <Handle optional per-file flags 173b>
    <Apply pending language if no explicit flags 174b>
    c2 = source_get();
    if (c != nw_char || (c2 != '{' && c2 != '(' && c2 != '[')) {
        fprintf(stderr, "%s: expected %c{, %c[, or %c( after file name (%s, %d)\n",
            command_name, nw_char, nw_char, nw_char, source_name, start_line);
        exit(-1);
    }
    return new_name;
}

```

□

173a · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a ·
 defines collect_file_name 46b, 62, 87c, 107a, 159a · uses command_name 29c, exit 21b, file_names 158b, fprintf 21b,
 isgraph 21b, isspace 21b, MAX_NAME_LEN 23b, Name 158a, name_add 162b, nw_char 29ab, source_get 130b, source_line 129b,
 source_name 129b, stderr 21b

⟨ *Handle optional per-file flags* 173b ⟩ ≡

```
{
  while (1) {
    while (isspace(c))
      c = source_get();
    if (c == '-') {
      c = source_get();
      do {
        switch (c) {
          case 't': new_name->tab_flag = FALSE;
                     break;
          case 'd': new_name->debug_flag = TRUE;
                     break;
          case 'i': new_name->indent_flag = FALSE;
                     break;
          case 's': new_name->suppress_markers = TRUE;
                     break;
          case 'c': ⟨ Get comment delimiters 174a ⟩
                     break;
          default : fprintf(stderr, "%s: unexpected per-file flag (%s, %d)\n",
                           command_name, source_name, source_line);
                     break;
        }
        c = source_get();
      } while (!isspace(c));
    }
    else break;
  }
}
```

□

173b · used in 173a · uses `command_name` 29c, `FALSE` 23a, `fprintf` 21b, `isspace` 21b, `source_get` 130b, `source_line` 129b, `source_name` 129b, `stderr` 21b, `TRUE` 23a

The explicit per-file comment flags (`-cc`, `-c+`, `-cp`) select among the first three comment styles. These flags predate the auto-detection system; they still work and take precedence over any fence-based or `@L`-based setting.

The `-s` flag suppresses `#line` directives and section markers in the tangled output for that file, while still allowing language auto-detection to set the comment style and the `lang` tag (used by the Markdown weaver for syntax highlighting classes). This is useful for bootstrap source files or any output where tangle markers are unwanted.

⟨ *Get comment delimiters* 174a ⟩ ≡

```
c = source_get();
if (c == 'c')
  new_name->comment_flag = 1;
else if (c == '+')
  new_name->comment_flag = 2;
else if (c == 'p')
  new_name->comment_flag = 3;
else
  fprintf(stderr, "%s: Unrecognised comment flag (%s, %d)\n",
          command_name, source_name, source_line);
```

□

174a · used in 173b · uses `command_name` 29c, `fprintf` 21b, `source_get` 130b, `source_line` 129b, `source_name` 129b, `stderr` 21b

If there is a pending language tag from a fence or `%%%` marker, and the user has not specified explicit comment or debug flags, we apply the language defaults automatically.

```

< Apply pending language if no explicit flags 174b > ≡
if (pending_lang[0] != '\0') {
    strncpy(new_name->lang, pending_lang, 63);
    new_name->lang[63] = '\0';
    if (new_name->comment_flag == 0 && !new_name->debug_flag) {
        lang_to_flags(pending_lang,
                      &new_name->comment_flag,
                      &new_name->debug_flag,
                      &new_name->location_flag);
    }
    pending_lang[0] = '\0';
} else if (new_name->lang[0] == '\0') {
    const char *dot = strrchr(new_name->spelling, '.');
    if (dot && dot[1]) {
        const LangEntry *e = find_lang(dot + 1);
        if (e) {
            strncpy(new_name->lang, dot + 1, 63);
            new_name->lang[63] = '\0';
            if (new_name->comment_flag == 0 && !new_name->debug_flag) {
                lang_to_flags(new_name->lang,
                              &new_name->comment_flag,
                              &new_name->debug_flag,
                              &new_name->location_flag);
            }
        }
    }
}
if (new_name->suppress_markers) {
    new_name->debug_flag = FALSE;
    new_name->location_flag = FALSE;
}

```

174b · used in 173a · uses FALSE 23a, find_lang 163b, LangEntry 163b, lang_to_flags 163b, pending_lang 158b

For @d (macro) definitions, the language tag is stored and flags are inferred from @l, just as for output files. At expansion time, if a fragment carries an explicit @l tag its own flags override the parent's—so @l txt before @d suppresses markers and newlines, enabling true inline expansion.

```

< Apply pending language to macro 175 > ≡
if (pending_lang[0] != '\0') {
    strncpy(node->lang, pending_lang, 63);
    node->lang[63] = '\0';
    if (node->comment_flag == 0 && !node->debug_flag) {
        lang_to_flags(pending_lang,
                      &node->comment_flag,
                      &node->debug_flag,
                      &node->location_flag);
    }
    pending_lang[0] = '\0';
}

```

175 · used in 177a, 178d · uses lang_to_flags 163b, pending_lang 158b

3.3.5 Comment Styles

weft supports six comment styles, indexed 0–5. The arrays below provide the opening, continuation, and closing delimiters for each. Style 0 means “no comments”; the remaining five cover the vast majority of

programming languages:

Index	Style	Begin	Mid	End
0	none	(empty)	(empty)	(empty)
1	C	/*	*	*/
2	C++	//	//	(empty)
3	Shell	#	#	(empty)
4	Lua/SQL	--	--	(empty)
5	HTML	<!--	(spaces)	-->

These are used both for block comments (@c) and for the Knuth-style section markers generated by the language auto-detection system.

⟨ *Forward declarations for scraps.c* 176a ⟩ + ≡

```
char * comment_begin[6] = { "", "/* ", "// ", "# ", "-- ", "<!-- "};
char * comment_mid[6] = { "", " * ", "// ", "# ", "-- ", " "};
char * comment_end[6] = { "", " */", "", "", "", " -->"};
```

□

176a · defined at 148ab, 155a, 176a, 191a · used in 134b · defines comment_begin 50e, 150bc, 153, comment_end 50e, 150bc, 153, comment_mid 50e

Name terminated by \n or @{; but keep skipping until @{

⟨ *names.c* 176b ⟩ + ≡

```
Name *collect_macro_name(void)
```

```
{
    char name[MAX_NAME_LEN];
    char args[1000];
    char * arg[9];
    char * argp = args;
    int argc = 0;
    char *p = name;
    int start_line = source_line;
    int c = source_get(), c2;
    unsigned char sector = current_sector;

    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    while (isspace(c))
        c = source_get();
    while (c != EOF) {
        Name * node;
        switch (c) {
            case '\t':
            case ' ': *p++ = ' ';
                    do
                        c = source_get();
                    while (c == ' ' || c == '\t');
                    break;
            case '\n': ⟨ Skip until scrap begins, then return name 178d ⟩
            default:
                if (c==nw_char)
                {
                    ⟨ Check for terminating at-sequence and return name 177a ⟩
                    break;
                }
        }
    }
}
```

```

    }
    *p++ = c;
    c = source_get();
    break;
}
}
fprintf(stderr, "%s: expected fragment name (%s, %d)\n",
        command_name, source_name, start_line);
exit(-1);
return NULL; /* unreachable return to avoid warnings on some compilers */
}

```

□

176b · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a ·

defines `collect_macro_name` 46c, 63a, 88a, 107b, 159a · uses `command_name` 29c, `current_sector` 45c, `exit` 21b, `fprintf` 21b, `isspace` 21b, `MAX_NAME_LEN` 23b, `Name` 158a, `nw_char` 29ab, `sector` 134a, `source_get` 130b, `source_line` 129b, `source_name` 129b, `stderr` 21b

⟨ *Check for terminating at-sequence and return name* 177a ⟩ ≡

```

{
    c = source_get();
    switch (c) {
        case '(':
        case '[':
        case '{': ⟨ Cleanup and install name 178c ⟩
                ⟨ Apply pending language to macro 175 ⟩
                return install_args(node, argc, arg);
        case '\\': ⟨ Enter the next argument 177b ⟩
                break;
        default:
            if (c==nw_char)
            {
                *p++ = c;
                break;
            }
            fprintf(stderr,
                "%s: unexpected %c%c in fragment definition name (%s, %d)\n",
                command_name, nw_char, c, source_name, start_line);
            exit(-1);
    }
}

```

□

177a · used in 176b · uses `command_name` 29c, `exit` 21b, `fprintf` 21b, `nw_char` 29ab, `source_get` 130b, `source_name` 129b, `stderr` 21b

⟨ *Enter the next argument* 177b ⟩ ≡

```

arg[argc] = argp;
while ((c = source_get()) != EOF) {
    if (c==nw_char) {
        c2 = source_get();
        if (c2=='\\') {
            ⟨ Make this argument 178b ⟩
            c = source_get();
            break;
        }
    }
    else
        *argp++ = c2;
}

```

```

    else
        *argp++ = c;
}
*p++ = ARG_CHR;

```

177b · used in 177a · uses ARG_CHR 178a, nw_char 29ab, source_get 130b

⟨ *Type declarations* 178a ⟩ + ≡

```
#define ARG_CHR '\001'
```

□

178a · defined at 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e · used in 21a · defines ARG_CHR 61, 63b, 75c, 77, 88b, 94d, 95b, 103c, 146b, 154c, 155c, 177b, 180, 182a

⟨ *Make this argument* 178b ⟩ ≡

```

if (argc < 9) {
    *argp++ = '\000';
    argc += 1;
}

```

□

178b · used in 177b

⟨ *Cleanup and install name* 178c ⟩ ≡

```

{
    if (p > name && p[-1] == ' ')
        p--;
    if (p - name > 3 && p[-1] == '.' && p[-2] == '.' && p[-3] == '.') {
        p[-3] = ' ';
        p -= 2;
    }
    if (p == name || name[0] == ' ') {
        fprintf(stderr, "%s: empty name (%s, %d)\n",
            command_name, source_name, source_line);
        exit(-1);
    }
    *p = '\0';
    node = prefix_add(&macro_names, name, sector);
}

```

□

178c · used in 177a, 178d, 180 · uses command_name 29c, exit 21b, fprintf 21b, macro_names 158b, prefix_add 160a, sector 134a, source_line 129b, source_name 129b, stderr 21b

⟨ *Skip until scrap begins, then return name* 178d ⟩ ≡

```

{
    do
        c = source_get();
        while (isspace(c));
        c2 = source_get();
        if (c != nw_char || (c2 != '{' && c2 != '(' && c2 != '[')) {
            fprintf(stderr, "%s: expected %c{ after fragment name (%s, %d)\n",
                command_name, nw_char, source_name, start_line);
            exit(-1);
        }
    }
    ⟨ Cleanup and install name 178c ⟩
    ⟨ Apply pending language to macro 175 ⟩
    return install_args(node, argc, arg);
}

```

□

178d · used in 176b · uses [command_name](#) 29c, [exit](#) 21b, [fprintf](#) 21b, [isspace](#) 21b, [nw_char](#) 29ab, [source_get](#) 130b, [source_name](#) 129b, [stderr](#) 21b

⟨ *Function prototypes* 178e ⟩ + ≡

```
extern Name *install_args(Name *name, int argc, char *arg[9]);
```

□

178e · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses [Name](#) 158a

⟨ *names.c* 179a ⟩ + ≡

```
Name *install_args(Name * name, int argc, char *arg[9])
```

```
{
    int i;

    for (i = 0; i < argc; i++) {
        if (name->arg[i] == NULL)
            name->arg[i] = save_string(arg[i]);
    }
    return name;
}
```

□

179a · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · uses [Name](#) 158a, [save_string](#) 159c

⟨ *Type declarations* 179b ⟩ + ≡

```
typedef struct arglist
{Name * name;
 struct arglist * args;
 struct arglist * next;
} Arglist;
```

□

179b · defined at 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e · used in 21a · defines [Arglist](#) 52c, 55b, 61, 75c, 77, 84c, 87b, 94d, 95b, 106a, 135a, 142a, 143b, 145de, 146abc, 147ac, 148b, 152e, 154b, 155a, 159a, 179cd, 182bce, 184c, 185a, 186a, 190b · uses [Name](#) 158a, [next](#) 133c

⟨ *names.c* 179c ⟩ + ≡

```
Arglist * buildArglist(Name * name, Arglist * a)
```

```
{
    Arglist * args = (Arglist *)arena_getmem(sizeof(Arglist));

    args->args = a;
    args->next = NULL;
    args->name = name;
    return args;
}
```

□

179c · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · defines [buildArglist](#) 180, 182be · uses [arena_getmem](#) 197e, [Arglist](#) 179b, [Name](#) 158a, [next](#) 133c

Terminated by @>

⟨ *names.c* 179d ⟩ + ≡

```

Arglist * collect_scrap_name(int current_scrap)
{
    char name[MAX_NAME_LEN];
    char *p = name;
    int c = source_get();
    unsigned char sector = current_sector;
    Arglist * head = NULL;
    Arglist ** tail = &head;

    if (c == '+')
    {
        sector = 0;
        c = source_get();
    }
    while (c == ' ' || c == '\t')
        c = source_get();
    while (c != EOF) {
        switch (c) {
            case '\t':
            case ' ': *p++ = ' ';
                        do
                            c = source_get();
                        while (c == ' ' || c == '\t');
                        break;
            default:
                if (c==nw_char)
                {
                    < Look for end of scrap name and return 180 >
                    break;
                }
                if (!isgraph(c)) {
                    fprintf(stderr,
                        "%s: unexpected character in fragment name (%s, %d)\n",
                        command_name, source_name, source_line);
                    exit(-1);
                }
                *p++ = c;
                c = source_get();
                break;
        }
    }
    fprintf(stderr, "%s: unexpected end of file (%s, %d)\n",
        command_name, source_name, source_line);
    exit(-1);
    return NULL; /* unreachable return to avoid warnings on some compilers */
}

```

□

179d · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a ·

defines collect_scrap_name 61, 75c, 87b, 94d, 106a, 142a, 159a, 182d · uses Arglist 179b, command_name 29c,

current_sector 45c, exit 21b, fprintf 21b, isgraph 21b, MAX_NAME_LEN 23b, nw_char 29ab, sector 134a, source_get 130b,

source_line 129b, source_name 129b, stderr 21b

< Look for end of scrap name and return 180 > ≡

```

{
    Name * node;

    c = source_get();
    switch (c) {

```

```

case '\\': {
    < Add plain string argument 181 >
}
*p++ = ARG_CHR;
c = source_get();
break;
case '1': case '2': case '3':
case '4': case '5': case '6':
case '7': case '8': case '9': {
    < Add a propagated argument 182a >
}
*p++ = ARG_CHR;
c = source_get();
break;
case '{': {
    < Add an inline scrap argument 182b >
}
*p++ = ARG_CHR;
c = source_get();
break;
case '<':
    < Add macro call argument 182d >
    *p++ = ARG_CHR;
    c = source_get();
    break;
case '(':
    scrap_name_has_parameters = 1;
    < Cleanup and install name 178c >
    return buildArglist(node, head);
case '>':
    scrap_name_has_parameters = 0;
    < Cleanup and install name 178c >
    return buildArglist(node, head);

default:
    if (c==nw_char)
    {
        *p++ = c;
        c = source_get();
        break;
    }
    fprintf(stderr,
        "%s: unexpected %c%c in fragment invocation name (%s, %d)\n",
        command_name, nw_char, c, source_name, source_line);
    exit(-1);
}
}

```

□

180 · used in 179d · uses ARG_CHR 178a, buildArglist 179c, command_name 29c, exit 21b, fprintf 21b, Name 158a, nw_char 29ab, source_get 130b, source_line 129b, source_name 129b, stderr 21b

A plain string argument has no name and a string as a value.

< Add plain string argument 181 > ≡

```

char buff[MAX_NAME_LEN];
char * s = buff;
int c, c2;

```

```

while ((c = source_get()) != EOF) {
    if (c==nw_char) {
        c2 = source_get();
        if (c2=='\''')
            break;
        *s++ = c2;
    }
    else
        *s++ = c;
}
*s = '\000';
< Add buff to current arg list 182e >

```

□

181 · used in 180 · uses MAX_NAME_LEN 23b, nw_char 29ab, source_get 130b

A parameter argument propagated into an interior fragment has no name and a string of ARG_CHAR followed by a digit as value.

```

< Add a propagated argument 182a > ≡
char buff[3];
buff[0] = ARG_CHAR;
buff[1] = c;
buff[2] = '\000';
< Add buff to current arg list 182e >

```

□

182a · used in 180 · uses ARG_CHAR 178a

```

< Add an inline scrap argument 182b > ≡
int s = collect_scrap();
Scrap_Node * d = (Scrap_Node *)arena_getmem(sizeof(Scrap_Node));
d->scrap = s;
d->quoted = 0;
d->next = NULL;
*tail = buildArglist((Name *)1, (Arglist *)d);
tail = &(*tail)->next;

```

□

182b · used in 180 · uses arena_getmem 197e, Arglist 179b, buildArglist 179c, collect_scrap 138c, Name 158a, next 133c, Scrap_Node 157e

```

< Type declarations 182c > + ≡
typedef struct embed {
    Scrap_Node * defs;
    Arglist * args;
} Embed_Node;

```

□

182c · defined at 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e · used in 21a · defines Embed_Node 52c, 146b, 155b · uses Arglist 179b, defs 134a, Scrap_Node 157e

```

< Add macro call argument 182d > ≡
*tail = collect_scrap_name(current_scrap);
if (current_scrap >= 0)
    add_to_use((*tail)->name, current_scrap);
tail = &(*tail)->next;

```

□

182d · used in 180 · uses `add_to_use 142c`, `collect_scrap_name 179d`, `next 133c`

⟨ *Add buff to current arg list* 182e ⟩ ≡

```
*tail = buildArglist(NULL, (Arglist *)save_string(buff));
tail = &(*tail)->next;
```

□

182e · used in 181, 182a · uses `Arglist 179b`, `buildArglist 179c`, `next 133c`, `save_string 159c`

⟨ `names.c` 182f ⟩ + ≡

```
static Scrap_Node *reverse(Scrap_Node *a); /* a forward declaration */
```

```
void reverse_lists(Name *names)
```

```
{
    while (names) {
        reverse_lists(names->llink);
        names->defs = reverse(names->defs);
        names->uses = reverse(names->uses);
        names = names->rlink;
    }
}
```

□

182f · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · defines `reverse_lists 52a`, 159a · uses `defs 134a`, `Name 158a`, `reverse 183a`, `Scrap_Node 157e`, uses 134a

Just for fun, here's a non-recursive version of the traditional list reversal code. Note that it reverses the list in place; that is, it does no new allocations.

⟨ `names.c` 183a ⟩ + ≡

```
static Scrap_Node *reverse(Scrap_Node *a)
```

```
{
    if (a) {
        Scrap_Node *b = a->next;
        a->next = NULL;
        while (b) {
            Scrap_Node *c = b->next;
            b->next = a;
            a = b;
            b = c;
        }
    }
    return a;
}
```

□

183a · defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a · defines `reverse 27`, 31, 182f · uses `next 133c`, `Scrap_Node 157e`

3.4 Searching for Index Entries

Given the array of scraps and a set of index entries, we need to search all the scraps for occurrences of each entry. The obvious approach to this problem would be quite expensive for large documents; however, there is an interesting paper describing an efficient solution [1].

⟨ `scraps.c` 183b ⟩ + ≡

```
typedef struct name_node {
    struct name_node *next;
    Name *name;
} Name_Node;
```

□

183b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines **Name_Node** 183c, 188, 189ab · uses **Name** 158a, **next** 133c

⟨scraps.c 183c⟩ + ≡

```
typedef struct goto_node {
    Name_Node *output;           /* list of words ending in this state */
    struct move_node *moves;     /* list of possible moves */
    struct goto_node *fail;      /* and where to go when no move fits */
    struct goto_node *next;      /* next goto node with same depth */
} Goto_Node;
```

□

183c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines **Goto_Node** 183d, 184ab, 187c, 188, 189ab · uses **Name_Node** 183b, **next** 133c

⟨scraps.c 183d⟩ + ≡

```
typedef struct move_node {
    struct move_node *next;
    Goto_Node *state;
    char c;
} Move_Node;
```

□

183d · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines **Move_Node** 184b, 188, 189a · uses **Goto_Node** 183c, **next** 133c

⟨scraps.c 184a⟩ + ≡

```
static Goto_Node *root[256];
static int max_depth;
static Goto_Node **depths;
```

□

184a · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines **depths** 187c, 188, 189a, **max_depth** 187c, 188, 189a, **root** 105a, 115b, 187c, 188, 189ab, 191ac · uses **Goto_Node** 183c

⟨scraps.c 184b⟩ + ≡

```
static Goto_Node *goto_lookup(char c, Goto_Node *g)
{
    Move_Node *m = g->moves;
    while (m && m->c != c)
        m = m->next;
    if (m)
        return m->state;
    else
        return NULL;
}
```

□

184b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines **goto_lookup** 188, 189ab · uses **Goto_Node** 183c, **Move_Node** 183d, **next** 133c

3.4.1 Retrieving scrap uses

`<scraps.c 184c> + ≡`

```
typedef struct ArgMgr_s
{
    char * pv;
    char * bgn;
    Arglist * arg;
    struct ArgMgr_s * old;
} ArgMgr;
```

□

184c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines [ArgMgr 184d](#), 185ab, 186c · uses [Arglist 179b](#)

`<scraps.c 184d> + ≡`

```
typedef struct ArgManager_s
{
    Manager * m;
    ArgMgr * a;
} ArgManager;
```

□

184d · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines [ArgManager 185ab](#), 186c, 187c, 189b, 194b · uses [ArgMgr 184c](#), [Manager 137g](#)

`<scraps.c 185a> + ≡`

```
static void
pushArglist(ArgManager * mgr, Arglist * a)
{
    ArgMgr * b = malloc(sizeof(ArgMgr));

    if (b == NULL)
    {
        fprintf(stderr, "Can't allocate space for an argument manager\n");
        exit(EXIT_FAILURE);
    }
    b->pv = b->bgn = NULL;
    b->arg = a;
    b->old = mgr->a;
    mgr->a = b;
}
```

□

185a · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines [pushArglist 186a](#), 190b · uses [Arglist 179b](#), [ArgManager 184d](#), [ArgMgr 184c](#), [exit 21b](#), [fprintf 21b](#), [malloc 21b](#), [stderr 21b](#)

`<scraps.c 185b> + ≡`

```
static char argpop(ArgManager * mgr)
{
    while (mgr->a != NULL)
    {
        ArgMgr * a = mgr->a;

        < Perhaps —return— a character from the current arg 185c >
        < Perhaps start a new arg 186a >
        < Otherwise pop the current arg 186b >
    }
}
```

```

    }

    return (pop(mgr->m));
}

```

□

185b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `argpop` 189b, 190ab · uses `ArgManager` 184d, `ArgMgr` 184c, `pop` 143a

We separate individual arguments using spaces.

⟨ *Perhaps —return— a character from the current arg* 185c ⟩ ≡

```

if (a->pv != NULL)
{
    char c = *a->pv++;

    if (c != '\0')
        return c;
    a->pv = NULL;
    return ' ';
}

```

□

185c · used in 185b

I'm pretty sure that only the first case is ever used. I don't think the others can occur in this context, which makes the whole —ArgManager— thing doubtful.

⟨ *Perhaps start a new arg* 186a ⟩ ≡

```

if (a->arg) {
    Arglist * b = a->arg;

    a->arg = b->next;
    if (b->name == NULL) {
        a->bgn = a->pv = (char *)b->args;
    } else if (b->name == (Name *)1) {
        a->bgn = a->pv = "{Embedded Scrap}";
    } else {
        pushArglist(mgr, b->args);
    }
}

```

□

186a · used in 185b · uses `Arglist` 179b, `Name` 158a, `next` 133c, `pushArglist` 185a

⟨ *Otherwise pop the current arg* 186b ⟩ ≡

```

} else {
    mgr->a = a->old;
    free(a);
}

```

□

186b · used in 185b

⟨ *scraps.c* 186c ⟩ + ≡

```

static char
prev_char(ArgManager * mgr, int n)
{
    char c = '\0';
    ArgMgr * a = mgr->a;

```

```

Manager * m = mgr->m;

if (a != NULL) {
    < Get the nth previous character from an argument 186d >
} else {
    < Get the nth previous character from a scrap 187a >
}

return c;
}

```

□

186c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `prev_char` 194b · uses `ArgManager` 184d, `ArgMgr` 184c, `Manager` 137g

```

< Get the nth previous character from an argument 186d > ≡
if (a->pv && a->pv - n >= a->bgn)
    c = *a->pv;
else if (a->bgn) {
    int j = strlen(a->bgn) + 1;

    if (n >= j)
        c = a->bgn[j - n];
    else
        c = ' ';
}

```

□

186d · used in 186c · uses `strlen` 21b

When the offset `k` falls within the current slab we index it directly. When `k` is negative we must reach into the previous slab: since `k` is already negative, `SLAB_SIZE + k` yields the correct index (e.g. `k = -1` gives `chars[1023]`, the last element).

```

< Get the nth previous character from a scrap 187a > ≡
int k = m->index - n - 2;

if (k >= 0)
    c = m->scrap->chars[k];
else if (m->prev)
    c = m->prev->chars[SLAB_SIZE + k];

```

□

187a · used in 186c · uses `SLAB_SIZE` 133c

3.4.2 Building the Automata

```

< Function prototypes 187b > + ≡
extern void search(void);

```

□

187b · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `search` 187c

```

< scraps.c 187c > + ≡
static void build_gotos(Name *tree);
static int reject_match(Name *name, char post, ArgManager *reader);

```

```

void search(void)
{
    int i;
    for (i=0; i<256; i++)
        root[i] = NULL;
    max_depth = 10;
    depths = (Goto_Node **) arena_getmem(max_depth * sizeof(Goto_Node *));
    for (i=0; i<max_depth; i++)
        depths[i] = NULL;
    build_gotos(user_names);
    < Build failure functions 189a >
    < Search scraps 189b >
}

```

□

187c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `search` 31, 42c, 187b, 200 · uses `arena_getmem` 197e, `ArgManager` 184d, `build_gotos` 187d, `depths` 184a, `Goto_Node` 183c, `max_depth` 184a, `Name` 158a, `reject_match` 194b, `root` 184a, `user_names` 158b

< scraps.c 187d > + ≡

```

static void build_gotos(Name *tree)
{
    while (tree) {
        < Extend goto graph with tree->spelling 188 >
        build_gotos(tree->rlink);
        tree = tree->llink;
    }
}

```

□

187d · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `build_gotos` 187c · uses `Name` 158a

< Extend goto graph with tree->spelling 188 > ≡

```

{
    int depth = 2;
    char *p = tree->spelling;
    char c = *p++;
    Goto_Node *q = root[(unsigned char)c];
    Name_Node * last;
    if (!q) {
        q = (Goto_Node *) arena_getmem(sizeof(Goto_Node));
        root[(unsigned char)c] = q;
        q->moves = NULL;
        q->fail = NULL;
        q->output = NULL;
        q->next = depths[1];
        depths[1] = q;
    }
    while ((c = *p++)) {
        Goto_Node *new_node = goto_lookup(c, q);
        if (!new_node) {
            Move_Node *new_move = (Move_Node *) arena_getmem(sizeof(Move_Node));
            new_node = (Goto_Node *) arena_getmem(sizeof(Goto_Node));
            new_node->moves = NULL;
            new_node->fail = NULL;
            new_node->output = NULL;
            new_move->state = new_node;

```

```

new_move->c = c;
new_move->next = q->moves;
q->moves = new_move;
if (depth == max_depth) {
    int i;
    Goto_Node **new_depths =
        (Goto_Node **) arena_getmem(2*depth*sizeof(Goto_Node *));
    max_depth = 2 * depth;
    for (i=0; i<depth; i++)
        new_depths[i] = depths[i];
    depths = new_depths;
    for (i=depth; i<max_depth; i++)
        depths[i] = NULL;
}
new_node->next = depths[depth];
depths[depth] = new_node;
}
q = new_node;
depth++;
}
last = q->output;
q->output = (Name_Node *) arena_getmem(sizeof(Name_Node));
q->output->next = last;
q->output->name = tree;
}

```

□

188 · used in 187d · uses arena_getmem 197e, depths 184a, goto_lookup 184b, Goto_Node 183c, max_depth 184a, Move_Node 183d, Name_Node 183b, next 133c, root 184a

⟨ Build failure functions 189a ⟩ ≡

```

{
    int depth;
    for (depth=1; depth<max_depth; depth++) {
        Goto_Node *r = depths[depth];
        while (r) {
            Move_Node *m = r->moves;
            while (m) {
                char a = m->c;
                Goto_Node *s = m->state;
                Goto_Node *state = r->fail;
                while (state && !goto_lookup(a, state))
                    state = state->fail;
                if (state)
                    s->fail = goto_lookup(a, state);
                else
                    s->fail = root[(unsigned char)a];
                if (s->fail) {
                    Name_Node *p = s->fail->output;
                    while (p) {
                        Name_Node *q = (Name_Node *) arena_getmem(sizeof(Name_Node));
                        q->name = p->name;
                        q->next = s->output;
                        s->output = q;
                        p = p->next;
                    }
                }
                m = m->next;
            }
        }
    }
}

```

```

    r = r->next;
  }
}

```

□

189a · used in 187c · uses arena_getmem 197e, depths 184a, goto_lookup 184b, Goto_Node 183c, max_depth 184a, Move_Node 183d, Name_Node 183b, next 133c, root 184a

3.4.3 Searching the Scraps

⟨ Search scraps 189b ⟩ ≡

```

{
  for (i=1; i<scraps; i++) {
    char c, last = '\0';
    Manager rd;
    ArgManager reader;
    Goto_Node *state = NULL;
    rd.prev = NULL;
    rd.scrap = scrap_array(i).slab;
    rd.index = 0;
    reader.m = &rd;
    reader.a = NULL;
    c = argpop(&reader);
    while (c) {
      while (state && !goto_lookup(c, state))
        state = state->fail;
      if (state)
        state = goto_lookup(c, state);
      else
        state = root[(unsigned char)c];
      ⟨ Skip over at at 190a ⟩
      ⟨ Skip over a scrap use 190b ⟩
      ⟨ Skip over a block comment 51c ⟩
      last = c;
      c = argpop(&reader);
      if (state && state->output) {
        Name_Node *p = state->output;
        do {
          Name *name = p->name;
          if (!reject_match(name, c, &reader) &&
              scrap_array(i).sector == name->sector &&
              (!name->uses || name->uses->scrap != i)) {
            Scrap_Node *new_use =
              (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
            new_use->scrap = i;
            new_use->next = name->uses;
            name->uses = new_use;
            if (!scrap_is_in(name->defs, i))
              add_uses(&(scrap_array(i).uses), name);
          }
          p = p->next;
        } while (p);
      }
    }
  }
}

```

□

189b · used in 187c · uses [add_uses 191c](#), [arena_getmem 197e](#), [ArgManager 184d](#), [argpop 185b](#), [defs 134a](#), [goto_lookup 184b](#), [Goto_Node 183c](#), [Manager 137g](#), [Name 158a](#), [Name_Node 183b](#), [next 133c](#), [reject_match 194b](#), [root 184a](#), [scraps 134b](#), [scrap_array 134b](#), [scrap_is_in 191b](#), [Scrap_Node 157e](#), [sector 134a](#), [slab 134a](#), [uses 134a](#)

```

< Skip over at at 190a > ≡
if (last == nw_char && c == nw_char)
{
    last = '\0';
    c = argpop(&reader);
}
□

```

190a · used in 189b · uses [argpop 185b](#), [nw_char 29ab](#)

```

< Skip over a scrap use 190b > ≡
if (last == nw_char && c == '<')
{
    char buf[MAX_NAME_LEN];
    char * p = buf;
    Arglist * args;
    int idx;

    c = argpop(&reader);
    while ((c = argpop(&reader)) != nw_char)
        *p++ = c;
    c = argpop(&reader);
    *p = '\0';
    if (sscanf(buf, "%d", &idx) != 1) {
        fprintf(stderr, "%s: found an internal problem (3)\n", command_name);
        exit(-1);
    }
    args = arglist_lookup(idx);
    pushArglist(&reader, args);
}
□

```

190b · used in 189b · uses [Arglist 179b](#), [arglist_lookup 148b](#), [argpop 185b](#), [command_name 29c](#), [exit 21b](#), [fprintf 21b](#), [MAX_NAME_LEN 23b](#), [nw_char 29ab](#), [pushArglist 185a](#), [stderr 21b](#)

< Forward declarations for scraps.c 191a > + ≡

```

static void add_uses(Uses **root, Name *name);
static int scrap_is_in(Scrap_Node *list, int i);
□

```

191a · defined at [148ab](#), [155a](#), [176a](#), [191a](#) · used in [134b](#) · uses [add_uses 191c](#), [Name 158a](#), [root 184a](#), [scrap_is_in 191b](#), [Scrap_Node 157e](#), [Uses 191d](#)

< scraps.c 191b > + ≡

```

static int scrap_is_in(Scrap_Node * list, int i)
{
    while (list != NULL) {
        if (list->scrap == i)
            return TRUE;
        list = list->next;
    }
    return FALSE;
}

```

□

191b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `scrap_is_in` 189b, 191a · uses `FALSE` 23a, `next` 133c, `Scrap_Node` 157e, `TRUE` 23a

⟨ `scraps.c` 191c ⟩ + ≡

```
static void add_uses(Uses * * root, Name *name)
{
    int cmp;
    Uses *p, **q = root;

    while ((p = *q, p != NULL)
           && (cmp = robs_strcmp(p->defn->spelling, name->spelling)) < 0)
        q = &(p->next);
    if (p == NULL || cmp > 0)
    {
        Uses *new_use = arena_getmem(sizeof(Uses));
        new_use->next = p;
        new_use->defn = name;
        *q = new_use;
    }
}
```

□

191c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `add_uses` 141ab, 189b, 191a · uses `arena_getmem` 197e, `Name` 158a, `next` 133c, `robs_strcmp` 161c, `root` 184a, `Uses` 191d

⟨ *Type declarations* 191d ⟩ + ≡

```
typedef struct uses {
    struct uses *next;
    Name *defn;
} Uses;
```

□

191d · defined at 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e · used in 21a · defines `Uses` 101ab, 102c, 111b, 114, 135a, 136ab, 191ac, 192b, 193c · uses `Name` 158a, `next` 133c, `uses` 134a

⟨ *Function prototypes* 192a ⟩ + ≡

```
extern void format_uses_refs(FILE *, int);
```

□

192a · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `FILE` 21b, `format_uses_refs` 192b

⟨ `scraps.c` 192b ⟩ + ≡

```
void
format_uses_refs(FILE * tex_file, int scrap)
{
    Uses * p = scrap_array(scrap).uses;
    if (p != NULL)
        ⟨ Write uses references 192c ⟩
}
```

□

192b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `format_uses_refs` 62, 63a, 192a · uses `FILE` 21b,

scrap_array 134b, Uses 191d, uses 134a

⟨ *Write uses references* 192c ⟩ ≡

```
{
    char join = ' ';
    fputs("\\WEFTdot \\WEFTtxtIdentsUsed\\nobreak\\", tex_file);
    do {
        ⟨ Write one use reference 192d ⟩
        join = ',';
        p = p->next;
    }while (p != NULL);
}
```

□

192c · used in 192b · uses fputs 21b, next 133c

⟨ *Write one use reference* 192d ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->defs;
int first = TRUE, page = -1;
fprintf(tex_file,
        "%c {\\WEFTtint{weftkw}\\verb%c%s%c}\\nobreak\\ ",
        join, nw_char, name->spelling, nw_char);
if (defs)
{
    do {
        ⟨ Write one referenced scrap 193a ⟩
        first = FALSE;
        defs = defs->next;
    }while (defs!= NULL);
}
else
{
    fputs("\\WEFTnotglobal", tex_file);
}
```

□

192d · used in 192c · uses defs 134a, FALSE 23a, first 197d, fprintf 21b, fputs 21b, Name 158a, next 133c, nw_char 29ab, page 134a, Scrap_Node 157e, TRUE 23a

⟨ *Write one referenced scrap* 193a ⟩ ≡

```
fputs("\\WEFTlink{weft", tex_file);
write_scrap_ref(tex_file, defs->scrap, -1, &page);
fputs("{}{", tex_file);
write_scrap_ref(tex_file, defs->scrap, first, &page);
fputs("}", tex_file);
```

□

193a · used in 192d, 193e · uses defs 134a, first 197d, fputs 21b, page 134a, write_scrap_ref 135c

⟨ *Function prototypes* 193b ⟩ + ≡

```
extern void format_defs_refs(FILE *, int);
```

□

193b · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses FILE 21b, format_defs_refs 193c

⟨ *scraps.c* 193c ⟩ + ≡

```

void
format_defs_refs(FILE * tex_file, int scrap)
{
    Uses * p = scrap_array(scrap).defs;
    if (p != NULL)
        < Write defs references 193d >
}
□

193c · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd,
184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines format_defs_refs 62, 63a, 193b · uses defs 134a,
FILE 21b, scrap_array 134b, Uses 191d

< Write defs references 193d > ≡
{
    char join = ' ';
    fputs("\\WEFTdot \\WEFTtxtIdentsDefed\\nobreak\\", tex_file);
    do {
        < Write one def reference 193e >
        join = ',';
        p = p->next;
    }while (p != NULL);
}
□

193d · used in 193c · uses fputs 21b, next 133c

< Write one def reference 193e > ≡
Name * name = p->defn;
Scrap_Node *defs = name->uses;
int first = TRUE, page = -1;
fprintf(tex_file,
        "%c {\\WEFTtint{weftkw}\\verb%c%s%c}\\nobreak\\ ",
        join, nw_char, name->spelling, nw_char);
if (defs == NULL
    || (defs->scrap == scrap && defs->next == NULL)) {
    fputs("\\WEFTtxtIdentsNotUsed", tex_file);
}
else {
    do {
        if (defs->scrap != scrap) {
            < Write one referenced scrap 193a >
            first = FALSE;
        }
        defs = defs->next;
    }while (defs!= NULL);
}
□

193e · used in 193d · uses defs 134a, FALSE 23a, first 197d, fprintf 21b, fputs 21b, Name 158a, next 133c, nw_char 29ab,
page 134a, Scrap_Node 157e, TRUE 23a, uses 134a

```

Rejecting Matches

A problem with simple substring matching is that the string “he” would match longer strings like “she” and “her.” Norman Ramsey suggested examining the characters occurring immediately before and after a match and rejecting the match if it appears to be part of a longer token. Of course, the concept of *token* is

language-dependent, so we may be occasionally mistaken. For the present, we'll consider the mechanism an experiment.

`< scraps.c 194a > + ≡`

```
#define sym_char(c) (isalnum(c) || (c) == '_')

static int op_char(char c)
{
    switch (c) {
        case '!':          case '#': case '%': case '$': case '^':
        case '&': case '*': case '-': case '+': case '=': case '/':
        case '|': case '~': case '<': case '>':
            return TRUE;
        default:
            return c==nw_char ? TRUE : FALSE;
    }
}
```

□

194a · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `op_char` 194b, `sym_char` 194b · uses `FALSE` 23a, `nw_char` 29ab, `TRUE` 23a

`< scraps.c 194b > + ≡`

```
static int reject_match(Name *name, char post, ArgManager *reader)
{
    int len = strlen(name->spelling);
    char first = name->spelling[0];
    char last = name->spelling[len - 1];
    char prev = prev_char(reader, len);
    if (sym_char(last) && sym_char(post)) return TRUE;
    if (sym_char(first) && sym_char(prev)) return TRUE;
    if (op_char(last) && op_char(post)) return TRUE;
    if (op_char(first) && op_char(prev)) return TRUE;
    return FALSE; /* Here is 194b-01 */
}
```

□

194b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines `reject_match` 187c, 189b · uses `ArgManager` 184d, `FALSE` 23a, `first` 197d, `Name` 158a, `op_char` 194a, `prev_char` 186c, `strlen` 21b, `sym_char` 194a, `TRUE` 23a

3.5 Labels

Refer to 195a-01. And another one 194b-01.

`< Get label from 195a > ≡`

```
char label_name[MAX_NAME_LEN];
char * p = label_name;
while (c = @1, c != nw_char) /* Here is 195a-01 */
    *p++ = c;
*p = '\0';
c = @1;
```

□

195a · used in 74b, 140a, 152b · uses `MAX_NAME_LEN` 23b, `nw_char` 29ab

`< scraps.c 195b > + ≡`

```

void
write_label(char label_name[], FILE * file)
< Search for label(< Write the label to file 195d>,< Complain about missing label 195e>) 196c >
□

195b · defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd,
184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b · defines write_label 74b, 152b, 195c · uses FILE 21b

< Function prototypes 195c > + ≡
void write_label(char label_name[], FILE * file);
□

195c · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b,
192a, 193b, 195c, 197b · used in 21a · uses FILE 21b, write_label 195b

< Write the label to file 195d > ≡
write_single_scrap_ref(file, lbl->scrap);
fprintf(file, "-%02d", lbl->seq);
□

195d · used in 195b · uses fprintf 21b, write_single_scrap_ref 135d

< Complain about missing label 195e > ≡
fprintf(stderr, "Can't find label %s.\n", label_name);
□

195e · used in 195b · uses fprintf 21b, stderr 21b

< Save label to label store 195f > ≡
if (label_name[0])
< Search for label(< Complain about duplicate labels 196a>,< Create a new label entry 195g>) 196c >
else
{
    < Complain about empty label 196b >
}
□

195f · used in 140a

< Create a new label entry 195g > ≡
lbl = (label_node *)arena_getmem(sizeof(label_node) + (p - label_name));
lbl->left = lbl->right = NULL;
strcpy(lbl->name, label_name);
lbl->scrap = current_scrap;
lbl->seq = ++lblseq;
*plbl = lbl;
□

195g · used in 195f · uses arena_getmem 197e, label_node 196e

< Complain about duplicate labels 196a > ≡
fprintf(stderr, "Duplicate label %s.\n", label_name);
□

196a · used in 195f · uses fprintf 21b, stderr 21b

< Complain about empty label 196b > ≡
fprintf(stderr, "Empty label.\n");
□

196b · used in 195f · uses fprintf 21b, stderr 21b

```

⟨ *Search for label(Found,Notfound)* 196c ⟩ ≡

```
{
    label_node * * plbl = &label_tab;
    for (;;)
    {
        label_node * lbl = *plbl;

        if (lbl)
        {
            int cmp = label_name[0] - lbl->name[0];

            if (cmp == 0)
                cmp = strcmp(label_name + 1, lbl->name + 1);
            if (cmp < 0)
                plbl = &lbl->left;
            else if (cmp > 0)
                plbl = &lbl->right;
            else
            {
                Found
                break;
            }
        }
        else
        {
            Notfound
            break;
        }
    }
}
```

□

196c · used in 195bf · uses [label_node](#) 196e, [label_tab](#) 196d

⟨ *global.c* 196d ⟩ + ≡

```
label_node * label_tab = NULL;
```

□

196d · defined at 25e, 196d · defines [label_tab](#) 196c, 197a · uses [label_node](#) 196e

⟨ *Type declarations* 196e ⟩ + ≡

```
typedef struct l_node
{
    struct l_node * left, * right;
    int scrap, seq;
    char name[1];
} label_node;
```

□

196e · defined at 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e · used in 21a · defines [label_node](#) 195g, 196cd, 197a

⟨ *Global variable declarations* 197a ⟩ + ≡

```
extern label_node * label_tab;
```

□

197a · defined at 27, 29ac, 45c, 50a, 66b, 129b, 137e, 158b, 197a · used in 21a · uses [label_node](#) 196e, [label_tab](#) 196d

3.6 Memory Management

I manage memory using a simple scheme inspired by Hanson's idea of *arenas* [2]. Basically, I allocate all the storage required when processing a source file (primarily for names and scraps) using calls to `arena_getmem(n)`, where `n` specifies the number of bytes to be allocated. When the storage is no longer required, the entire arena is freed with a single call to `arena_free()`. Both operations are quite fast.

< Function prototypes 197b > + ≡

```
extern void *arena_getmem(size_t n);
extern void arena_free(void);
extern void arglist_reset(void);
```

□

197b · defined at 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b · used in 21a · uses `arena_free` 199, `arena_getmem` 197e, `arglist_reset` 148b, `size_t` 21b

< arena.c 197c > + ≡

```
typedef struct chunk {
    struct chunk *next;
    char *limit;
    char *avail;
} Chunk;
```

□

197c · defined at 25d, 197cde, 199 · defines `Chunk` 197d, 198ab · uses `next` 133c

We define an empty chunk called `first`. The variable `arena` points at the current chunk of memory; it's initially pointed at `first`. As soon as some storage is required, a “real” chunk of memory will be allocated and attached to `first->next`; storage will be allocated from the new chunk (and later chunks if necessary).

< arena.c 197d > + ≡

```
static Chunk first = { NULL, NULL, NULL };
static Chunk *arena = &first;
```

□

197d · defined at 25d, 197cde, 199 · defines `arena` 197e, 198ab, 199, `first` 31, 65b, 102ab, 103a, 111b, 113a, 116b, 135ac, 192d, 193ae, 194b, 199, 200 · uses `Chunk` 197c

3.6.1 Allocating Memory

The routine `arena_getmem(n)` returns a pointer to (at least) `n` bytes of memory. Note that `n` is rounded up to ensure that returned pointers are always aligned. We align to the nearest 8-byte segment, since that'll satisfy the more common 2-byte and 4-byte alignment restrictions too.

< arena.c 197e > + ≡

```
void *arena_getmem(size_t n)
{
    char *q;
    char *p = arena->avail;
    n = (n + 7) & ~7;           /* ensuring alignment to 8 bytes */
    q = p + n;
    if (q <= arena->limit) {
        arena->avail = q;
        return p;
    }
    < Find a new chunk of memory 198a >
}
```

□

197e · defined at 25d, 197cde, 199 · defines `arena_getmem` 35b, 39a, 46d, 54a, 135b, 138ad, 141ab, 142c, 146bc, 159c, 163a, 179c, 182b, 187c, 188, 189ab, 191c, 195g, 197b · uses `arena` 197d, `size_t` 21b

If the current chunk doesn't have adequate space (at least `n` bytes) we examine the rest of the list of chunks (starting at `arena->next`) looking for a chunk with adequate space. If `n` is very large, we may not find it right away or we may not find a suitable chunk at all.

⟨ *Find a new chunk of memory* 198a ⟩ ≡

```
{
    Chunk *ap = arena;
    Chunk *np = ap->next;
    while (np) {
        char *v = sizeof(Chunk) + (char *) np;
        if (v + n <= np->limit) {
            np->avail = v + n;
            arena = np;
            return v;
        }
        ap = np;
        np = ap->next;
    }
}
⟨ Allocate a new chunk of memory 198b ⟩
```

□

198a · used in 197e · uses `arena` 197d, `Chunk` 197c, `next` 133c

If there isn't a suitable chunk of memory on the free list, then we need to allocate a new one.

⟨ *Allocate a new chunk of memory* 198b ⟩ ≡

```
{
    size_t m = n + 10000;
    np = (Chunk *) malloc(m);
    np->limit = m + (char *) np;
    np->avail = n + sizeof(Chunk) + (char *) np;
    np->next = NULL;
    ap->next = np;
    arena = np;
    return sizeof(Chunk) + (char *) np;
}
```

□

198b · used in 198a · uses `arena` 197d, `Chunk` 197c, `malloc` 21b, `next` 133c, `size_t` 21b

3.6.2 Freeing Memory

To free all the memory in the arena, we need only point `arena` back to the first empty chunk.

⟨ *arena.c* 199 ⟩ + ≡

```
void arena_free(void)
{
    arena = &first;
    arglist_reset();
}
```

□

199 · defined at 25d, 197cde, 199 · defines `arena_free` 39b, 197b · uses `arena` 197d, `arglist_reset` 148b, `first` 197d

Chapter 4

Man page

Here is the UNIX man page for weft:

```
<weft.1 200> ≡
.TH WEFT 1 "March 2026"
.SH NAME
weft, a literate programming tool
.SH SYNOPSIS
.B weft
.br
\fBweft\fP [options] [file] ...
.SH DESCRIPTION
.I weft
is a literate programming tool like Knuth's
.I WEB,
only simpler.
A
.I weft
file contains program source code interleaved with documentation.
When
.I weft
is given a
.I weft
file, it writes the program file(s).
By default only tangling is performed. Weaving (documentation
generation) is requested with \fB-w\fP.
.SH COMMAND LINE OPTIONS
.br
\fB-t\fP Suppresses weave output (even if \fB-w\fP is given).
.br
\fB-o\fP Suppresses generation of the output files.
.br
\fB-d\fP List dangling identifier references in indexes.
.br
\fB-c\fP Forces output files to overwrite old files of the same
name without comparing for equality first.
.br
\fB-v\fP The verbose flag. Forces output of progress reports.
.br
\fB-n\fP Forces sequential numbering of scraps (instead of page
numbers).
.br
\fB-s\fP Doesn't print list of scraps making up file at end of
```

each scrap.

`\fB-p path`\fP Prepend path to the filenames for all the output files.

`\fB-V string`\fP Provide the string for replacement of the @v operation. This is intended as a means for including version information in generated output.

`\fB-x`\fP Include cross-references in comments in output files.

`\fB-h options`\fP Turn on hyperlinks using the hyperref package of LaTeX and provide the options to the package.

`\fB-r`\fP Turn on hyperlinks using the hyperref package of LaTeX, with the package options being in the text.

`\fB-I path`\fP Provide a directory to search for included files. This may appear several times.

.br

`\fB-m`\fP Emit a JSON map of all output files, fragments, their definitions, and dependency relationships to stdout. No weave or tangle is performed. Useful for AI tooling and project navigation.

.br

`\fB-e name`\fP Extract the named fragment and all its transitive dependencies as Markdown to stdout. Fragment names support prefix matching. No weave or tangle is performed.

.br

`\fB-R file[:line]`\fP Read section markers from a tangled output file and emit JSON mapping to stdout. If a line number is given, emit the single innermost scrap covering that line. Without a line number, emit all scrap regions. No .weft source files are needed.

.br

`\fB-w [format]`\fP Request weave output. Without a format token, the format is taken from the `\fB@W`\fP directive in the source file. With `\fBmd`\fP, a `\fB.md`\fP file is generated; with `\fBtex`\fP, a `\fB.tex`\fP file is generated. A CLI format overrides `\fB@W`\fP. If `\fB-w`\fP is given without a format and no `\fB@W`\fP exists, weft exits with an error.

.br

`\fB--bodies`\fP Include raw scrap body text in `\fB-m`\fP JSON output. Each definition entry gets a `\fB"body"`\fP field with the JSON-escaped source code of the scrap.

.br

`\fB--prose`\fP Include the documentation paragraph preceding each `\fB@d`\fP/\fB@o\fP directive as a `\fB"prose"`\fP field in `\fB-m`\fP JSON output.

.br

`\fB--diff`\fP Dry-run tangle: show a unified diff of what would change in each output file without writing anything. If a file does not yet exist, all lines are shown with a `\fB+`\fP prefix.

.br

`\fB--lint`\fP Static analysis of the literate source. Reports unused fragments (defined but never referenced), undefined references (used but never defined), and fuzzy-match suggestions for likely typos (Levenshtein distance ≤ 3).

.br

`\fB--callers`\fP When combined with `\fB-e name`\fP, show the upward call chain from `\fB@o`\fP roots down to the named fragment as an indented tree.

.br

`\fB--errors`\fP Read compiler or linter output from stdin and annotate `\fBfile:line`\fP patterns with `\fB.weft`\fP source locations by parsing section markers embedded in the tangled output. No `\fB.weft`\fP source files are needed. Example: `\fBmake 2>&1 | weft --errors`\fP.

.br

`\fB--help`\fP Print a comprehensive help message including all options,

usage examples, and an AI-assisted workflow guide, then exit.

.SH FORMAT OF WEFT FILES

A

.I weft

file contains mostly ordinary

.I LaTeX.

The file is read and copied to output (.tex file) unless a

.I weft

command is encountered. All

.I weft

commands start with an ‘‘at-sign’’ (@).

Files and fragments are defined with the following commands:

.PP

@o \fIfile-name flags scrap\fP where scrap is smaller than one page.

.br

@O \fIfile-name flags scrap\fP where scrap is bigger than one page.

.br

@d \fIfragment-name scrap\fP. Where scrap is smaller than one page.

.br

@D \fIfragment-name scrap\fP. Where scrap is bigger than one page.

.PP

@q \fIfragment-name scrap\fP. Where scrap is smaller than one page.

The scrap is not expanded in the output, allowing you to construct output files which can, perhaps after further processing, be input to

.I weft.

.br

@Q \fIfragment-name scrap\fP. Where scrap is bigger than one page.

Likewise.

.PP

Scraps have specific begin and end

markers;

which begin and end marker you use determines how the scrap will be typeset in the .tex file:

.br

\fB@{\fP...\fB@}\fP for code, typeset with the LaTeX \fBlistings\fP package (load \fB\usepackage{listings}\fP in your preamble).

.br

\fB@[\fP...\fB@]\fP for LaTeX paragraph mode formatting, and

.br

\fB@(\fP...\fB@)\fP for LaTeX math mode formatting.

.br

Any amount of whitespace

(including carriage returns) may appear between a name and the beginning of a scrap.

.PP

Several code/file scraps may have the same name;

.I weft

concatenates their definitions to produce a single scrap.

Code scrap definitions are like macro definitions;

.I weft

extracts a program by expanding one scrap.

The definition of that scrap contains references to other scraps, which are themselves expanded, and so on.

\fIweft\fP’s output is readable; it preserves the indentation of expanded scraps with respect to the scraps in which they appear.

.PP

.SH PER FILE OPTIONS

When defining an output file, the programmer has the option of using flags

to control the output.

.PP

\fB-d\fR option,

.I weft

will emit line number indications at scrap boundaries.

.br

\fB-i\fR option,

.I weft

supresses the indentation of fragments (useful for \fBFortran\fR).

.br

\fB-t\fP option makes \fIweft\fP

copy tabs untouched from input to output.

.br

\fB-c\fIx\fP Include comments in the output file.

\fIx\fP may be \fBc\fP for C-style comments, \fB+\fP for C++ and

\fBp\fP for perl and similar.

.br

\fB-s\fP Suppress section markers and \fB#line\fP directives in the

tangled output for this file. Language auto-detection still sets

the comment style and the \fBlang\fP tag (used for Markdown syntax

highlighting), but no markers appear in the output.

.PP

.SH MINOR COMMANDS

.br

@@ Causes a single ‘‘at-sign’’ to be copied into the output.

.br

@_ Causes the text between it and the next {\tt @_} to be made bold

(for keywords, etc.) in the formatted document

.br

@% Comments out a line so that it doesn’t appear in the output.

.br

@i \fBfilename\fR causes the file named to be included.

.br

@f Creates an index of output files.

.br

@m Creates an index of fragments.

.br

@u Creates an index of user-specified identifiers.

.br

@W \fIformat\fP Declares the weave output format (\fBmd\fP or \fBtex\fP).

Used by \fB-w\fP when no explicit format is given on the command line.

.br

@l \fItag\fP Sets the language for the next @o or @d directive.

The tag is looked up in the language table to determine the comment

style for section markers and the \fBclass\fP attribute for Markdown

syntax highlighting. Place it on the line before the directive it

applies to.

.br

@L \fIname style\fP [+d] Defines a custom language mapping.

\fIname\fP is the tag (e.g. \fBnim\fP, \fBgls\fP),

\fIstyle\fP is one of \fB/\fP, \fB#\fP, \fB--\fP, \fB/*\fP, or \fB<!--\fP.

The optional \fB+d\fP flag enables \fB#line\fP directive generation.

User-defined languages override built-in entries. Must appear in the

document preamble before any @o or @d that depends on it.

.PP

.SH SECTION MARKERS

By default, when a language is detected (via @l, @L, or file extension

of @o), weft wraps each scrap in the tangled output with opening and

closing section markers using the appropriate comment syntax. For example,

C output includes:
.nf

```
/* {1: source.w:10} */  
int main() { return 0; }  
/* {1} */
```

.fi
The opening marker shows the scrap number and source location; the closing marker shows only the scrap number. Nested fragment expansions produce nested markers.

.PP
Languages with \fB#line\fP support (C, C++, Objective-C, etc.) also emit \fB#line\fP directives at scrap boundaries.

.PP
The \fB-s\fP per-file flag suppresses both section markers and \fB#line\fP directives.

.PP
.SH LANGUAGE INFERENCE
When no @l tag precedes an @o directive, weft infers the language from the output file's extension. For example, \fB@o app.js\fP automatically detects JavaScript (// comments, no #line). The inferred language is used for section markers in the tangle and for the \fBclass="language-X"\fP attribute in the Markdown weave.

.PP
Over 250 file extensions are recognised, covering C, C++, Python, JavaScript, Go, Rust, Ruby, Lua, SQL, HTML, and many more. Extensions not in the table produce no markers.

.PP
To mark an identifier for inclusion in the index, it must be mentioned at the end of the scrap it was defined in. The line starts with @| and ends with the \fBend of scrap\fP mark \fB@}\fP.

.PP
.SH ERROR MESSAGES

.PP
.SH BUGS

.PP
.SH AUTHOR

Preston Briggs.
Internet address \fBpreston@cs.rice.edu\fP.

.SH MAINTAINER
Simon Wright
Internet address \fBsimon@pushface.org\fP

.br
Keith Harwood
Internet address \fBKeith.Harwood@vitalmis.com\fP

□

200 · uses [exit 21b](#), [FILE 21b](#), [first 197d](#), [main 25f](#), [next 133c](#), [page 134a](#), [prose 134a](#), [scraps 134b](#), [search 187c](#)

Chapter 5

Indices

Three sets of indices can be created automatically: an index of file names, an index of fragment names, and an index of user-specified identifiers. An index entry includes the name of the entry, where it was defined, and where it was referenced.

5.1 Files

`arena.c` defined at 25d, 197cde, 199
`global.c` defined at 25e, 196d
`global.h` defined at 21a
`html.c` defined at 24d, 103c, 104, 109ac, 110b
`input.c` defined at 25a, 129d, 130abd, 133b
`latex.c` defined at 24b, 55b, 56a, 63c, 67d, 68ab, 69ab, 70bc, 71ab, 72a, 77, 78b, 79bc, 81c, 82b
`main.c` defined at 23c, 25f
`markdown.c` defined at 24c, 84cd, 88c, 91ab, 95ab, 97a, 98ab, 100a, 101a, 102c
`names.c` defined at 25c, 159bc, 160a, 161ac, 162b, 163b, 170, 172, 173a, 176b, 179acd, 182f, 183a
`output.c` defined at 24e, 111ab, 113ab, 114, 115abc, 116ab, 117ab, 118ab, 119ab, 120, 121, 123, 124, 125c, 126a
`pass1.c` defined at 24a, 42bc
`scraps.c` defined at 25b, 133c, 134ab, 135bcd, 136abde, 137abcg, 138abc, 142c, 143ab, 144, 145abcde, 147ac, 156b, 183bcd, 184abcd, 185ab, 186c, 187cd, 191bc, 192b, 193c, 194ab, 195b
`weft.1` defined at 200

5.2 Fragments

⟨ Accumulate prose character 43a ⟩ used in 42d
⟨ Accumulate scrap and return scraps++ 139a ⟩ used in 138c
⟨ Add a propagated argument 182a ⟩ used in 180
⟨ Add an at character to the block or break 48d ⟩ used in 48c
⟨ Add an inline scrap argument 182b ⟩ used in 180
⟨ Add any other character to the block 49a ⟩ used in 48c
⟨ Add buff to current arg list 182e ⟩ used in 181, 182a
⟨ Add letters to scraps with duplicate page numbers 157a ⟩ used in 156b
⟨ Add macro call argument 182d ⟩ used in 180
⟨ Add more indentation char 149 ⟩ used in 148c, 151bc
⟨ Add one char to the block buffer 48c ⟩ used in 48a
⟨ Add plain string argument 181 ⟩ used in 180
⟨ Add the source path to the include path list 39a ⟩ used in 38b
⟨ Add user identifier definition 141b ⟩ used in 140c
⟨ Add user identifier use 141a ⟩ used in 140c
⟨ Add scrap to name's definition list 46d ⟩ used in 46bc
⟨ Allocate a new chunk of memory 198b ⟩ used in 198a

< *Apply pending language if no explicit flags* 174b > used in 173a
 < *Apply pending language to macro* 175 > used in 177a, 178d
 < *Begin or end a block comment* 51d > used in 59c
 < *Begin the cross-reference environment* 66e > used in 62, 63a
 < *Begin the scrap environment* 64a > used in 62, 63a
 < *Bold Keyword* 75a > used in 73b
 < *Bold md Keyword* 94b > used in 93a
 < *Build a new arglist* 146c > used in 145e
 < *Build failure functions* 189a > used in 187c
 < *Build fragment definition* 46c > used in 44d
 < *Build output file definition* 46b > used in 44d
 < *Build source_name and tex_name* 38b > used in 38a
 < *Check at-sequence for end-of-scrap* 73b > used in 71b
 < *Check for ambiguous prefix* 160b > used in 160a
 < *Check for cosmetic fence line* 85b > used in 85a
 < *Check for end of scrap name* 147b > used in 147a
 < *Check for long options* 30 > used in 29f
 < *Check for macro invocation in scrap* 151c > used in 148c
 < *Check for macro parameters* 54a > used in 147b
 < *Check for terminating at-sequence and return name* 177a > used in 176b
 < *Check md at-sequence for end-of-scrap* 93a > used in 91b
 < *Cleanup and install name* 178c > used in 177a, 178d, 180
 < *Close the current sector* 45b > used in 44d, 59c, 86a, 106a
 < *Collect a block comment* 48a > used in 44d
 < *Collect include-file name* 132 > used in 131b
 < *Collect language apply in pass1* 43c > used in 44d
 < *Collect user language definition in pass1* 44a > used in 44d
 < *Collect user-specified index entries* 140c > used in 139b
 < *Collect weave format declaration in pass1* 44b > used in 44d
 < *Comment this macro use* 154c > used in 152e, 153, 154b
 < *Compare the temp file and the old file* 128a > used in 127b
 < *Complain about duplicate labels* 196a > used in 195f
 < *Complain about empty label* 196b > used in 195f
 < *Complain about missing label* 195e > used in 195b
 < *Copy block comment from scrap* 50e > used in 151c
 < *Copy file name into file* 152d > used in 151c
 < *Copy fragment title into file* 154b > used in 151c
 < *Copy label from scrap into file* 152b > used in 151c
 < *Copy label from source into* 74b > used in 59c, 73b, 86a, 93a, 106a
 < *Copy macro into file* 152e > used in 151c
 < *Copy source into html_file* 105c > used in 104
 < *Copy version info into file* 74a > used in 73b, 93a, 151c
 < *Copy version info into md file* 87a > used in 86a
 < *Copy version info into tex file* 60b > used in 59c
 < *Copy defs->scrap to file* 148c > used in 147c
 < *Copy source_file into md_file* 85a > used in 84d
 < *Copy source_file into tex_file* 59b > used in 56a
 < *Create a new label entry* 195g > used in 195f
 < *Create new name entry* 163a > used in 160a, 162b
 < *Create new scrap, managed by writer* 138d > used in 138c
 < *Emit a quoted scrap argument inline* 76b > used in 77
 < *End block* 65a > used in 51d, 64a, 66a
 < *Enter the next argument* 177b > used in 177a
 < *Expand macro into file* 61 > used in 59c
 < *Expand md macro into file* 87b > used in 86a
 < *Expand md tab into spaces* 92b > used in 91b
 < *Expand tab into spaces* 72c > used in 71b, 151b
 < *Extend goto graph with tree->spelling* 188 > used in 187d

< *Fill in the html scrap* 107c > used in 107ab
 < *Fill in the md scrap* 89a > used in 87c, 88a
 < *Fill in the middle of the scrap environment* 65b > used in 62, 63a
 < *Find a free temporary file* 127a > used in 126b
 < *Find a new chunk of memory* 198a > used in 197e
 < *Finish html scrap* 108a > used in 107ab
 < *Finish md scrap* 89b > used in 87c, 88a
 < *Finish the cross-reference environment* 66f > used in 62, 63a
 < *Finish the scrap environment* 66a > used in 62, 63a
 < *For all remaining scraps* 157c > used in 157a
 < *Format a file index entry* 78c > used in 78b
 < *Format a md file index entry* 97b > used in 97a
 < *Format a md index entry* 98c > used in 98b
 < *Format a md user index entry* 100b > used in 100a
 < *Format a user index entry* 82c > used in 82b
 < *Format an index entry* 80c > used in 79c
 < *Format macro name* 75c > used in 73b
 < *Format macro parameters* 54b > used in 75c, 77
 < *Format md macro name* 94d > used in 93a
 < *Forward declarations for scraps.c* 148ab, 155a, 176a, 191a > used in 134b
 < *Function prototypes* 42a, 55a, 70a, 72b, 81b, 84ab, 103b, 125ab, 129a, 130c, 135a, 136c, 142d, 146a, 156a, 159a, 161b, 178e, 187b, 192a, 193b, 195c, 197b > used in 21a
 < *Get comment delimiters* 174a > used in 173b
 < *Get label from* 195a > used in 74b, 140a, 152b
 < *Get label while collecting scrap* 140a > used in 139b
 < *Get the nth previous character from a scrap* 187a > used in 186c
 < *Get the nth previous character from an argument* 186d > used in 186c
 < *Global variable declarations* 27, 29ac, 45c, 50a, 66b, 129b, 137e, 158b, 197a > used in 21a
 < *Global variable definitions* 28, 29bd, 46a, 50b, 66c, 129c, 137f, 158c > used in 25e
 < *Handle an “at” character* 131a > used in 130b
 < *Handle at-sign during scrap accumulation* 139b > used in 139a
 < *Handle macro invocation in scrap* 142a > used in 139b
 < *Handle macro parameter substitution* 52c > used in 151c
 < *Handle optional per-file flags* 173b > used in 173a
 < *Handle reverse map mode* 36e > used in 25f
 < *Handle tab characters on output* 151b > used in 148c
 < *Handle the file name in argv[arg]* 38a > used in 37b
 < *Handle EOF* 133a > used in 130b
 < *HTML-escape character c* 92a > used in 91b
 < *If we break the line at this word* 51b > used in 50e
 < *Include a fragment use in comment* 155c > used in 155a
 < *Include an embedded scrap in comment* 155b > used in 155a
 < *Include block comment in a scrap* 50d > used in 139b
 < *Include files* 21b > used in 21a
 < *Indent suppressed* 151a > used in 150d
 < *Insert appropriate indentation* 150d > used in 148c, 150a
 < *Insert debugging information if required* 150a > used in 148c, 152a
 < *Insert section closing marker* 150c > used in 148c
 < *Insert section opening marker* 150b > used in 148c
 < *Interpret at-sequence* 59c > used in 59b
 < *Interpret command-line arguments* 29ef > used in 25f
 < *Interpret html at-sequence* 106a > used in 105c
 < *Interpret md at-sequence* 86a > used in 85a
 < *Interpret the argument string s* 34 > used in 29f
 < *Italic “whatever”* 75b > used in 73b
 < *Italic md “whatever”* 94c > used in 93a
 < *Limits* 23b > used in 21a
 < *Look for end of scrap name and return* 180 > used in 179d

< *Make this argument 178b* > used in 177b
 < *Move a word to the file 51a* > used in 50e
 < *Move the temporary file to the target, if required 127b* > used in 126b
 < *Note a delimiter collision 73a* > used in 71b
 < *Open an include file 131b* > used in 131a
 < *Operating System Dependencies 26* > used in 21a, 25e
 < *Otherwise pop the current arg 186b* > used in 185b
 < *Perhaps add an include path 35b* > used in 29f
 < *Perhaps add letters to the page numbers 157d* > used in 157a
 < *Perhaps comment this macro 153* > used in 52c, 152e
 < *Perhaps get the extract name 36c* > used in 29f
 < *Perhaps get the hyperref options 36b* > used in 29f
 < *Perhaps get the prepend path 35a* > used in 29f
 < *Perhaps get the reverse map argument 36d* > used in 29f
 < *Perhaps get the version info string 36a* > used in 29f
 < *Perhaps put a delayed indent 154a* > used in 50e, 153
 < *Perhaps —return— a character from the current arg 185c* > used in 185b
 < *Perhaps skip white-space 49b* > used in 49a
 < *Perhaps start a new arg 186a* > used in 185b
 < *Print help message 31* > used in 30
 < *Process a file 39b* > used in 38a
 < *Process the remaining arguments (file names) 37b* > used in 25f
 < *Put out the indent 150e* > used in 52c, 150d
 < *Read line in .aux file 156c* > used in 156b
 < *Rename the temporary file to the target 128b* > used in 127b, 128a
 < *Resync the line number after a fragment reference 152a* > used in 151c
 < *Reverse cross-reference lists 52a* > used in 42c
 < *Rob's ordering 80a* > used in 79c, 98b
 < *Save label to label store 195f* > used in 140a
 < *Save macro name 142b* > used in 142a
 < *Save macro parameters 53* > used in 142a
 < *Save pending prose 43b* > used in 46bc
 < *Scan at-sequence 44d* > used in 42d
 < *Scan the source file, looking for at-sequences 42d* > used in 42c
 < *Search for label(Found,Notfound) 196c* > used in 195bf
 < *Search scraps 189b* > used in 187c
 < *Set locale information 37a* > used in 25f
 < *Set up name, args and next 146b* > used in 145e
 < *Show diff of temp file vs real file 127c* > used in 127b
 < *Show md presence of a block comment 94a* > used in 93a
 < *Show presence of a block comment 50c* > used in 73b
 < *Skip at-L language definition line 44c* > used in 59c, 86a, 106a
 < *Skip commented-out code 74e* > used in 73b, 93a, 139b
 < *Skip invisibles on p 162a* > used in 161c
 < *Skip over a block comment 51c* > used in 189b
 < *Skip over a scrap use 190b* > used in 189b
 < *Skip over an in-text scrap 47a* > used in 44d
 < *Skip over at at 190a* > used in 189b
 < *Skip over at-sign or go to skipped 47b* > used in 47a
 < *Skip over index entries 74d* > used in 73b, 93a
 < *Skip to the next nw-char 49c* > used in 48a
 < *Skip until scrap begins, then return name 178d* > used in 176b
 < *Skip whitespace 48b* > used in 48a
 < *Sort key of size n for ordering 80b* > used in 79c, 98b
 < *Start block 64b* > used in 51d, 64a
 < *Step i to the next valid scrap 157b* > used in 157a
 < *Step to next sector 45a* > used in 44d, 59c, 86a, 106a
 < *Switch block 64c* > used in 64a

< *Type declarations* 23a, 52b, 125d, 157e, 158a, 178a, 179b, 182c, 191d, 196e > used in 21a
 < *version 2* > used in 23b
 < *Warn (only once) about needing to rerun after LaTeX* 137d > used in 135c, 156c
 < *Write abbreviated definition list* 76a > used in 75c, 77
 < *Write defining scrap numbers* 80d > used in 80c
 < *Write defs references* 193d > used in 193c
 < *Write file defs* 67a > used in 62
 < *Write file's defining scrap numbers* 78d > used in 78c
 < *Write html file footer* 108b > used in 107a
 < *Write html index of file names* 108d > used in 106a
 < *Write html index of macro names* 109b > used in 106a
 < *Write html index of user names* 110a > used in 106a
 < *Write html macro definition* 107b > used in 106a
 < *Write html macro footer* 108c > used in 107b
 < *Write html output file definition* 107a > used in 106a
 < *Write in-text scrap* 66d > used in 59c
 < *Write index of file names* 78a > used in 59c
 < *Write index of macro names* 79a > used in 59c
 < *Write index of user-specified names* 82a > used in 59c
 < *Write LaTeX limbo definitions* 56b > used in 56a
 < *Write macro definition* 63a > used in 59c
 < *Write macro defs* 67b > used in 63a
 < *Write macro refs* 67c > used in 63a
 < *Write md defining scrap numbers* 99a > used in 98c
 < *Write md defs references* 102d > used in 102c
 < *Write md file cross-refs* 90a > used in 87c
 < *Write md file defs* 90b > used in 90a
 < *Write md file's defining scrap numbers* 97c > used in 97b
 < *Write md in-text scrap* 89c > used in 86a
 < *Write md index of file names* 96 > used in 86a
 < *Write md index of macro names* 97d > used in 86a
 < *Write md index of user-specified names* 99c > used in 86a
 < *Write md macro cross-refs* 90c > used in 88a
 < *Write md macro definition* 88a > used in 86a
 < *Write md macro defs* 90d > used in 90c
 < *Write md macro refs* 90e > used in 90c
 < *Write md output file definition* 87c > used in 86a
 < *Write md referencing scrap numbers* 99b > used in 98c
 < *Write md uses references* 101b > used in 101a
 < *Write one def reference* 193e > used in 193d
 < *Write one md def reference* 103a > used in 102d
 < *Write one md referenced scrap* 102b > used in 102a, 103a
 < *Write one md use reference* 102a > used in 101b
 < *Write one referenced scrap* 193a > used in 192d, 193e
 < *Write one use reference* 192d > used in 192c
 < *Write out files->spelling* 126b > used in 125c
 < *Write output file definition* 62 > used in 59c
 < *Write referencing scrap numbers* 81a > used in 80c
 < *Write the AtBeginDocument setup* 59a > used in 56b
 < *Write the HTML document head* 105a > used in 104
 < *Write the HTML document tail* 105b > used in 104
 < *Write the hyperlink usage macro* 58a > used in 56b
 < *Write the label to file* 195d > used in 195b
 < *Write the macro's name* 63b > used in 63a, 80c
 < *Write the md macro's name* 88b > used in 88a, 98c
 < *Write the modern decoration macros* 58b > used in 56b
 < *Write the scrap formatting macros* 57 > used in 56b
 < *Write uses references* 192c > used in 192b

5.3 Identifiers

Knuth prints his index of identifiers in a two-column format. I could force this automatically by emitting the `\twocolumn` command; but this has the side effect of forcing a new page. Therefore, it seems better to leave it this up to the user.

`add_to_use`: 53, 142a, [142c](#), 142d, 182d.
`add_uses`: 141ab, 189b, 191a, [191c](#).
`already_warned`: 137d, [137e](#), 137f, 156b.
`arena`: [197d](#), 197e, 198ab, 199.
`arena_free`: 39b, 197b, [199](#).
`arena_getmem`: 35b, 39a, 46d, 54a, 135b, 138ad, 141ab, 142c, 146bc, 159c, 163a, 179c, 182b, 187c, 188, 189ab, 191c, 195g, 197b, [197e](#).
`Arglist`: 52c, 55b, 61, 75c, 77, 84c, 87b, 94d, 95b, 106a, 135a, 142a, 143b, 145de, 146abc, 147ac, 148b, 152e, 154b, 155a, 159a, [179b](#), 179cd, 182bce, 184c, 185a, 186a, 190b.
`arglist_count`: [148b](#).
`arglist_lookup`: 143b, 147a, [148b](#), 190b.
`arglist_register`: 142b, [148b](#).
`arglist_reset`: [148b](#), 197b, 199.
`arglist_table`: [148b](#).
`ArgManager`: [184d](#), 185ab, 186c, 187c, 189b, 194b.
`ArgMgr`: [184c](#), 184d, 185ab, 186c.
`argpop`: [185b](#), 189b, 190ab.
`ARG_CHR`: 61, 63b, 75c, 77, 88b, 94d, 95b, 103c, 146b, 154c, 155c, 177b, [178a](#), 180, 182a.
`backup`: [145c](#), 150d.
`begin_tex_escape`: 50c, [71a](#), 73b, 75abc.
`blockBuff`: 48a, [50a](#), 50bd.
`bodies_flag`: [27](#), 28, 30, 111b.
`buildArglist`: [179c](#), 180, 182be.
`build_gotos`: 187c, [187d](#).
`callers_flag`: [27](#), 28, 30, 39b.
`Chunk`: [197c](#), 197d, 198ab.
`clear_marks`: [115a](#), 116ab.
`code_inline`: 54b, [71a](#), 71b.
`collect_file_name`: 46b, 62, 87c, 107a, 159a, [173a](#).
`collect_lang_def`: 44a, 159a, [170](#).
`collect_lint_names`: [117b](#), 118b.
`collect_macro_name`: 46c, 63a, 88a, 107b, 159a, [176b](#).
`collect_numbers`: 39b, 156a, [156b](#).
`collect_scrap`: 46bc, 53, 135a, [138c](#), 182b.
`collect_scrap_name`: 61, 75c, 87b, 94d, 106a, 142a, 159a, [179d](#), 182d.
`collect_weave_format`: 44b, 159a, [172](#).
`command_name`: [29c](#), 29de, 31, 34, 37b, 39b, 44d, 47ab, 56a, 61, 67c, 73a, 75c, 77, 84d, 90e, 94d, 95b, 104, 116ab, 127a, 128b, 131ab, 132, 133b, 137d, 139ab, 140c, 147a, 148b, 152e, 160b, 170, 172, 173ab, 174a, 176b, 177a, 178cd, 179d, 180, 190b.
`comment_ArglistElement`: 154c, [155a](#), 155c.
`comment_begin`: 50e, 150bc, 153, [176a](#).
`comment_end`: 50e, 150bc, 153, [176a](#).
`comment_mid`: 50e, [176a](#).
`compare`: [159b](#), 160a, 161a.
`compare_flag`: [27](#), 28, 34, 127b.
`copy_scrap`: 54b, 55b, 65b, 66d, [71b](#).
`current_sector`: 39b, 45ab, [45c](#), 46a, 79a, 82a, 97d, 99c, 109b, 110a, 138d, 140c, 176b, 179d.
`dangling_flag`: [27](#), 28, 34, 82c, 100b, 110b.
`defs`: 46d, 52c, 61, 62, 63a, 67ab, 75c, 76a, 77, 78d, 80d, 82c, 87b, 90bd, 94d, 95b, 97c, 99a, 100b, 102ab, 103ac, 107ab, 108bc, 109ac, 111b, 114, 117b, 118a, 126b, [134a](#), 135a, 136b, 138d, 141b, 146b, 147c, 148c, 150abc, 152de, 153, 154bc, 155b, 158a, 163a, 182cf, 189b, 192d, 193ace.
`delayed_indent`: 52c, [148a](#), 148c, 151a, 152e, 153, 154a.
`depths`: [184a](#), 187c, 188, 189a.

diff_flag: [27](#), [28](#), [30](#), [127b](#).
double_at: [129d](#), [131a](#), [133b](#).
dump_scrap_json: [111b](#), [136c](#), [144](#).
dump_scrap_text: [114](#), [136c](#), [143b](#), [144](#).
Embed_Node: [52c](#), [146b](#), [155b](#), [182c](#).
emit_name_json: [111b](#), [113a](#).
end_line: [111b](#), [134a](#), [137a](#), [138d](#), [139b](#).
end_tex_escape: [50c](#), [71a](#), [73b](#), [75abc](#).
EQUAL: [159b](#), [160a](#), [161a](#).
errors_flag: [27](#), [28](#), [30](#), [36e](#).
exit: [21b](#), [25f](#), [30](#), [36e](#), [37b](#), [39b](#), [47ab](#), [68b](#), [116ab](#), [121](#), [127a](#), [128b](#), [131ab](#), [132](#), [133b](#), [139ab](#), [140c](#), [147a](#), [148b](#), [152e](#), [170](#), [173a](#), [176b](#), [177a](#), [178cd](#), [179d](#), [180](#), [185a](#), [190b](#), [200](#).
EXTENSION: [159b](#), [160a](#), [161a](#).
extract_flag: [27](#), [28](#), [34](#), [36c](#).
extract_fragment: [114](#), [116b](#).
extract_name: [27](#), [28](#), [36c](#), [39b](#).
extra_scraps: [65b](#), [66a](#), [66b](#), [66c](#), [76b](#), [89ab](#), [95b](#), [107c](#), [108a](#).
FALLTHROUGH: [23a](#), [44d](#), [59c](#), [73b](#), [86a](#), [106a](#), [130b](#), [160a](#).
FALSE: [23a](#), [27](#), [28](#), [34](#), [35ab](#), [36abcd](#), [39b](#), [59bc](#), [65a](#), [66d](#), [67d](#), [71ab](#), [76b](#), [80d](#), [81c](#), [82c](#), [85ab](#), [86a](#), [87b](#), [89c](#), [91a](#), [95b](#), [99a](#), [100b](#), [102a](#), [103a](#), [106a](#), [115a](#), [128b](#), [131a](#), [133b](#), [152e](#), [156b](#), [161a](#), [163ab](#), [170](#), [173b](#), [174b](#), [191b](#), [192d](#), [193e](#), [194ab](#).
fclose: [21b](#), [56a](#), [76b](#), [84d](#), [104](#), [121](#), [123](#), [126b](#), [127ac](#), [128a](#), [133a](#), [144](#), [156b](#).
FILE: [21b](#), [31](#), [55b](#), [56a](#), [63c](#), [67d](#), [69a](#), [70c](#), [71ab](#), [76b](#), [77](#), [78b](#), [79c](#), [82b](#), [84bcd](#), [88c](#), [91ab](#), [95ab](#), [97a](#), [98b](#), [100a](#), [101a](#), [102c](#), [103c](#), [104](#), [109ac](#), [110b](#), [111ab](#), [114](#), [121](#), [123](#), [126b](#), [127c](#), [128a](#), [129d](#), [130a](#), [135acd](#), [136c](#), [143b](#), [144](#), [147c](#), [155a](#), [156b](#), [192ab](#), [193bc](#), [195bc](#), [200](#).
file_line: [134a](#), [136e](#), [138d](#), [139a](#), [148c](#).
file_name: [42ac](#), [55a](#), [56a](#), [84ad](#), [103b](#), [104](#), [134a](#), [136d](#), [138d](#), [139a](#), [150ab](#).
file_names: [39b](#), [42c](#), [52a](#), [78a](#), [96](#), [108d](#), [115c](#), [116b](#), [118b](#), [125b](#), [158b](#), [158c](#), [173a](#).
find_closest: [117b](#), [118a](#).
find_lang: [163b](#), [174b](#).
find_name: [113b](#), [116ab](#).
first: [31](#), [65b](#), [102ab](#), [103a](#), [111b](#), [113a](#), [116b](#), [135ac](#), [192d](#), [193ae](#), [194b](#), [197d](#), [199](#), [200](#).
fopen: [21b](#), [56a](#), [84d](#), [104](#), [121](#), [123](#), [127ac](#), [128a](#), [131b](#), [133b](#), [156b](#).
format_defs_refs: [62](#), [63a](#), [193b](#), [193c](#).
format_entry: [55b](#), [79a](#), [79c](#).
format_file_entry: [55b](#), [78a](#), [78b](#).
format_user_entry: [55b](#), [82a](#), [82b](#).
format_uses_refs: [62](#), [63a](#), [192a](#), [192b](#).
fprintf: [21b](#), [34](#), [37ab](#), [39b](#), [42c](#), [44d](#), [47ab](#), [50c](#), [54b](#), [56a](#), [58a](#), [61](#), [62](#), [64a](#), [65b](#), [67ac](#), [68b](#), [73a](#), [75abc](#), [77](#), [78c](#), [82c](#), [84d](#), [87c](#), [89a](#), [90be](#), [94bcd](#), [95b](#), [97b](#), [100b](#), [102a](#), [103a](#), [104](#), [107c](#), [111ab](#), [114](#), [116ab](#), [118ab](#), [121](#), [126b](#), [127a](#), [128b](#), [131ab](#), [132](#), [133b](#), [135c](#), [137d](#), [139ab](#), [140c](#), [143b](#), [144](#), [147a](#), [148b](#), [150abc](#), [152de](#), [154c](#), [155a](#), [160b](#), [170](#), [172](#), [173ab](#), [174a](#), [176b](#), [177a](#), [178cd](#), [179d](#), [180](#), [185a](#), [190b](#), [192d](#), [193e](#), [195de](#), [196ab](#).
fputs: [21b](#), [50e](#), [52c](#), [54b](#), [56b](#), [57](#), [58b](#), [59a](#), [60b](#), [61](#), [62](#), [63ac](#), [64abc](#), [65ab](#), [66aef](#), [67abcd](#), [69a](#), [70c](#), [71ab](#), [73b](#), [74a](#), [75c](#), [76ab](#), [77](#), [78acd](#), [79a](#), [80cd](#), [81a](#), [82ac](#), [87ac](#), [88ac](#), [89a](#), [90bde](#), [91ab](#), [92a](#), [93a](#), [94abd](#), [95ab](#), [96](#), [97bcd](#), [98c](#), [99abc](#), [100b](#), [101b](#), [102abd](#), [103ac](#), [105ab](#), [106a](#), [107abc](#), [108bcd](#), [109abc](#), [110ab](#), [111ab](#), [113a](#), [114](#), [115c](#), [121](#), [124](#), [127c](#), [135c](#), [144](#), [148c](#), [152d](#), [153](#), [192cd](#), [193ade](#).
getc: [21b](#), [128a](#), [130b](#), [131ab](#), [132](#), [133ab](#).
getenv: [21b](#), [37a](#).
get_scrap_defs: [102c](#), [135a](#), [136b](#).
get_scrap_uses: [101a](#), [111b](#), [114](#), [135a](#), [136a](#).
goto_lookup: [184b](#), [188](#), [189ab](#).
Goto_Node: [183c](#), [183d](#), [184ab](#), [187c](#), [188](#), [189ab](#).
GREATER: [159b](#), [160a](#), [161a](#).
has_sector: [79a](#), [81b](#), [81c](#), [82a](#), [97d](#), [99c](#), [109b](#), [110a](#).
html_emit_name: [103c](#), [107b](#), [109c](#).
html_esc: [103c](#).
html_format_entry: [104](#), [109b](#), [109c](#).
html_format_file_entry: [104](#), [108d](#), [109a](#).

[html_format_user_entry](#): 104, 110a, [110b](#).
[html_puts](#): [103c](#), 107a, 109a, 110b.
[html_ref](#): [103c](#), 106a.
[hyperoptions](#): [27](#), 28, 36b, 56b, 58a.
[hyperopt_flag](#): [27](#), 28, 34, 36b.
[hyperref_flag](#): [27](#), 28, 34, 36b, 56b.
[incl](#): [27](#), 28, 35b, 39a, 131b.
[includepath_flag](#): [27](#), 28, 34, 35b.
[include_depth](#): [129d](#), 131b, 133ab.
[init_scraps](#): 42c, 135a, [135b](#).
[instance](#): 87b, [145e](#), 146ab, 152e.
[isgraph](#): [21b](#), 132, 173a, 179d.
[islower](#): [21b](#), 161c.
[isspace](#): [21b](#), 66a, 89b, 108a, 140c, 173ab, 176b, 178d.
[is_file_name](#): [115b](#), 115c.
[json_puts](#): [111a](#), 111b, 121.
[label_node](#): 195g, 196cd, [196e](#), 197a.
[label_tab](#): 196c, [196d](#), 197a.
[LangEntry](#): [163b](#), 174b.
[lang_table](#): [163b](#).
[lang_to_flags](#): [163b](#), 174b, 175.
[latex_code_putc](#): 55b, [69a](#), 70c, 71b, 76b.
[latex_language](#): 55b, 65b, [69b](#).
[LESS](#): [159b](#), 160a, 161a.
[letter](#): 124, [134a](#), 135c, 138d, 157d.
[levenshtein](#): [117a](#), 117b.
[lint_flag](#): [27](#), 28, 30, 39b.
[lint_name](#): [118a](#), 118b.
[lint_names](#): [117b](#).
[lint_name_count](#): [117b](#), 118b.
[load_entry](#): [79b](#), 79c.
[lookup](#): 52c, [145d](#), 146b.
[macro_names](#): 39b, 42c, 52a, 79a, 97d, 109b, 116b, 118b, 125b, [158b](#), 158c, 178c.
[main](#): [25f](#), 200.
[make_parent_dirs](#): 125c, [126a](#), 126b.
[malloc](#): [21b](#), 79c, 98b, 185a, 198b.
[Manager](#): [137g](#), 138abc, 143ab, 145c, 147a, 148c, 184d, 186c, 189b.
[map_flag](#): [27](#), 28, 34, 39b.
[markdown_flag](#): [27](#), 28, 34, 39b.
[MarkerFrame](#): [119a](#), 119b, 121, 123, 124.
[MAX_ARGLISTS](#): [148b](#).
[max_depth](#): [184a](#), 187c, 188, 189a.
[MAX_INDENT](#): 87b, [125d](#), 126b, 149, 150e.
[MAX_LINT_NAMES](#): [117b](#).
[MAX_NAME_LEN](#): 23b, 140c, 143b, 147a, 173a, 176b, 179d, 181, 190b, 195a.
[md_copy_scrap](#): 84b, 89ac, [91b](#), 106a, 107c.
[md_format_defs_refs](#): 84c, 90ac, [102c](#).
[md_format_entry](#): 84c, 97d, [98b](#).
[md_format_file_entry](#): 84c, 96, [97a](#).
[md_format_user_entry](#): 84c, 99c, [100a](#).
[md_format_uses_refs](#): 84c, 90ac, [101a](#).
[md_load_entry](#): [98a](#), 98b.
[md_print_scrap_numbers](#): 84b, 90bde, [91a](#), 97c, 99b, 108bc, 109ac, 110b.
[md_scrap_type](#): [91b](#), 93a.
[md_write_arg](#): 84c, 88b, [88c](#), 93a, 95b.
[md_write_ArglistElement](#): 84c, 94d, [95b](#).
[md_write_literal](#): 84c, 94d, [95a](#), 95b.
[mkdirs_flag](#): [27](#), 28, 30, 126b, 128b.

Move_Node: [183d](#), [184b](#), [188](#), [189a](#).
 Name: [31](#), [46bc](#), [52c](#), [55b](#), [61](#), [62](#), [63a](#), [71b](#), [75c](#), [77](#), [78b](#), [79bc](#), [80b](#), [81bc](#), [82b](#), [84bc](#), [87bc](#), [88a](#), [91b](#), [94d](#), [95b](#), [97a](#), [98ab](#), [100a](#), [102a](#), [103ac](#), [104](#), [107ab](#), [109ac](#), [110b](#), [111b](#), [113ab](#), [114](#), [115abc](#), [116ab](#), [117b](#), [118ab](#), [125abc](#), [136c](#), [137c](#), [140c](#), [142acd](#), [145bde](#), [146b](#), [152e](#), [155a](#), [158a](#), [158bc](#), [159a](#), [160a](#), [161a](#), [162b](#), [163a](#), [173a](#), [176b](#), [178e](#), [179abc](#), [180](#), [182bf](#), [183b](#), [186a](#), [187cd](#), [189b](#), [191acd](#), [192d](#), [193e](#), [194b](#).
 name_add: [141ab](#), [159a](#), [162b](#), [173a](#).
 Name_Node: [183b](#), [183c](#), [188](#), [189ab](#).
 next: [27](#), [31](#), [34](#), [35b](#), [39a](#), [46d](#), [52c](#), [61](#), [67abcd](#), [75c](#), [76a](#), [77](#), [78d](#), [80d](#), [81a](#), [82c](#), [90bde](#), [91a](#), [94d](#), [95b](#), [97c](#), [99ab](#), [100b](#), [101b](#), [102ad](#), [103ac](#), [108bc](#), [111b](#), [114](#), [115c](#), [116a](#), [129a](#), [131b](#), [133c](#), [138a](#), [141ab](#), [142c](#), [143a](#), [145de](#), [146bc](#), [147c](#), [154c](#), [155c](#), [157e](#), [179bc](#), [182bde](#), [183abcd](#), [184b](#), [186a](#), [188](#), [189ab](#), [191bcd](#), [192cd](#), [193de](#), [197c](#), [198ab](#), [200](#).
 number_flag: [27](#), [28](#), [34](#), [39b](#), [156b](#).
 num_scraps: [79c](#), [98b](#), [134b](#), [135a](#).
 nw_char: [29a](#), [29b](#), [31](#), [39b](#), [42d](#), [44d](#), [47ab](#), [48c](#), [49c](#), [50d](#), [51c](#), [53](#), [54ab](#), [59bc](#), [62](#), [68a](#), [70c](#), [71b](#), [72a](#), [73b](#), [74d](#), [75a](#), [78c](#), [82c](#), [85a](#), [86a](#), [91b](#), [93a](#), [94b](#), [105c](#), [106a](#), [130b](#), [131a](#), [133b](#), [139ab](#), [140ac](#), [142ab](#), [143b](#), [147a](#), [148c](#), [151ac](#), [152de](#), [154c](#), [155ac](#), [160b](#), [170](#), [172](#), [173a](#), [176b](#), [177ab](#), [178d](#), [179d](#), [180](#), [181](#), [190ab](#), [192d](#), [193e](#), [194a](#), [195a](#).
 op_char: [194a](#), [194b](#).
 output_flag: [27](#), [28](#), [34](#), [39b](#).
 owner: [115c](#), [116a](#), [134a](#), [137c](#), [138d](#), [145b](#).
 page: [67d](#), [80d](#), [82c](#), [91a](#), [99a](#), [100b](#), [102ab](#), [103a](#), [134a](#), [135acd](#), [138d](#), [156bc](#), [157bd](#), [192d](#), [193ae](#), [200](#).
 Parameters: [52b](#), [54a](#), [87b](#), [135a](#), [147ac](#), [148c](#).
 parse_close_marker: [120](#), [121](#), [123](#).
 parse_open_marker: [119b](#), [121](#), [123](#).
 pass1: [39b](#), [42a](#), [42c](#), [73b](#), [93a](#).
 pending_lang: [43c](#), [158b](#), [158c](#), [174b](#), [175](#).
 pending_prose: [42b](#), [42c](#), [43b](#), [46bc](#).
 pop: [50e](#), [51ac](#), [54a](#), [143a](#), [143b](#), [147ab](#), [148c](#), [150d](#), [151c](#), [152c](#), [185b](#).
 pop_scrap_name: [147a](#), [152e](#).
 PREFIX: [159b](#), [160a](#), [161a](#).
 prefix_add: [159a](#), [160a](#), [178c](#).
 prev_char: [186c](#), [194b](#).
 prev_sector: [39b](#), [45a](#), [45c](#), [46a](#).
 print_callers: [115c](#), [116a](#).
 print_scrap_numbers: [55b](#), [67abc](#), [67d](#), [78d](#), [81a](#).
 prose: [27](#), [30](#), [31](#), [65b](#), [111b](#), [134a](#), [137b](#), [138d](#), [145a](#), [200](#).
 prose_buf: [42b](#), [43ab](#).
 prose_flag: [27](#), [28](#), [30](#), [42d](#), [43b](#), [111b](#).
 prose_len: [42b](#), [42c](#), [43ab](#).
 push: [50d](#), [53](#), [138a](#), [138b](#), [139ab](#), [140a](#), [142ab](#).
 pushArglist: [185a](#), [186a](#), [190b](#).
 pushes: [53](#), [138b](#), [140a](#), [142b](#).
 putc: [21b](#), [50e](#), [51a](#), [59bc](#), [61](#), [69a](#), [71b](#), [72c](#), [75c](#), [76b](#), [77](#), [78cd](#), [80cd](#), [81a](#), [82c](#), [85ab](#), [86a](#), [91b](#), [92ab](#), [93a](#), [94d](#), [95b](#), [97b](#), [98c](#), [99a](#), [103c](#), [105c](#), [106a](#), [109c](#), [135c](#), [148c](#), [150e](#), [151bc](#), [152e](#), [153](#), [154ab](#).
 reject_match: [187c](#), [189b](#), [194b](#).
 remove: [21b](#), [127bc](#), [128ab](#).
 reverse: [27](#), [31](#), [182f](#), [183a](#).
 reverse_lists: [52a](#), [159a](#), [182f](#).
 reverse_lookup_in_file: [123](#), [124](#).
 reverse_map_arg: [27](#), [28](#), [36de](#).
 reverse_map_flag: [27](#), [28](#), [34](#), [36d](#).
 robs_strcmp: [80a](#), [113b](#), [161b](#), [161c](#), [162b](#), [163a](#), [191c](#).
 root: [105a](#), [115b](#), [184a](#), [187c](#), [188](#), [189ab](#), [191ac](#).
 save_string: [35b](#), [39a](#), [43b](#), [131b](#), [138d](#), [159a](#), [159c](#), [160a](#), [163a](#), [179a](#), [182e](#).
 SCRAP: [134b](#), [135b](#), [138d](#).
 ScrapEntry: [134a](#), [134b](#), [135b](#), [138d](#).
 scraps: [24bcd](#), [27](#), [31](#), [54b](#), [55b](#), [62](#), [63a](#), [64a](#), [66ae](#), [67d](#), [84b](#), [87c](#), [88a](#), [89b](#), [90ac](#), [91a](#), [107ab](#), [108abc](#), [111b](#), [134b](#), [135b](#), [138d](#), [156bc](#), [157bc](#), [189b](#), [200](#).

scrap_array: [134b](#), [135c](#), [136abde](#), [137abc](#), [138d](#), [139ab](#), [141ab](#), [143b](#), [145ab](#), [148c](#), [150ab](#), [156bc](#), [157bd](#), [189b](#), [192b](#), [193c](#).
 SCRAP_BITS: [134b](#).
 scrap_end_line: [111b](#), [136c](#), [137a](#).
 scrap_file_line: [111b](#), [114](#), [118a](#), [136c](#), [136e](#).
 scrap_file_name: [111b](#), [114](#), [118a](#), [136c](#), [136d](#).
 scrap_flag: [27](#), [28](#), [34](#), [62](#), [90a](#), [108b](#).
 scrap_is_in: [189b](#), [191a](#), [191b](#).
 SCRAP_MASK: [134b](#), [138d](#).
 Scrap_Node: [46d](#), [52c](#), [55b](#), [61](#), [67d](#), [76ab](#), [78d](#), [80d](#), [81a](#), [82c](#), [84b](#), [91a](#), [94d](#), [95b](#), [97c](#), [99ab](#), [100b](#), [102a](#), [103a](#), [111b](#), [114](#), [115c](#), [116a](#), [135a](#), [141ab](#), [142c](#), [146b](#), [147c](#), [157e](#), [158a](#), [182bcf](#), [183a](#), [189b](#), [191ab](#), [192d](#), [193e](#).
 scrap_owner: [115c](#), [116a](#), [136c](#), [137c](#).
 scrap_prose: [111b](#), [136c](#), [137b](#).
 SCRAP_SHIFT: [134b](#), [138d](#).
 SCRAP_SIZE: [134b](#), [135b](#), [138d](#).
 scrap_type: [70b](#), [71a](#), [71b](#), [73ab](#), [75c](#).
 search: [31](#), [42c](#), [187b](#), [187c](#), [200](#).
 sector: [31](#), [55ab](#), [56a](#), [79ac](#), [80c](#), [81c](#), [82abc](#), [84acd](#), [97d](#), [98bc](#), [99c](#), [100ab](#), [103b](#), [104](#), [109bc](#), [110ab](#), [111b](#), [134a](#), [138d](#), [140c](#), [141ab](#), [142b](#), [143b](#), [152e](#), [155c](#), [158a](#), [159a](#), [160ab](#), [161a](#), [162b](#), [163a](#), [176b](#), [178c](#), [179d](#), [189b](#).
 setlocale: [21b](#), [37a](#).
 set_scrap_owner: [46d](#), [136c](#), [145b](#).
 set_scrap_prose: [46bc](#), [136c](#), [145a](#).
 size_t: [21b](#), [76b](#), [119b](#), [124](#), [197be](#), [198b](#).
 Slab: [133c](#), [134a](#), [137g](#), [138ad](#), [143a](#).
 slab: [133c](#), [134a](#), [138d](#), [143b](#), [148c](#), [189b](#).
 SLAB_SIZE: [133c](#), [138a](#), [143a](#), [145c](#), [187a](#).
 source_file: [129d](#), [130bd](#), [131ab](#), [132](#), [133ab](#).
 source_get: [42d](#), [43c](#), [44cd](#), [45ab](#), [47ab](#), [48bcd](#), [49bc](#), [53](#), [54b](#), [59bc](#), [60b](#), [61](#), [66ad](#), [71b](#), [73b](#), [74cde](#), [75a](#), [78a](#), [79a](#), [82a](#), [85ab](#), [86a](#), [87ab](#), [89bc](#), [91b](#), [93a](#), [94b](#), [96](#), [97d](#), [99c](#), [105c](#), [106a](#), [108ad](#), [109b](#), [110a](#), [129a](#), [130b](#), [131b](#), [133a](#), [139ab](#), [140bc](#), [142a](#), [170](#), [172](#), [173ab](#), [174a](#), [176b](#), [177ab](#), [178d](#), [179d](#), [180](#), [181](#).
 source_last: [54b](#), [65b](#), [71b](#), [91b](#), [129a](#), [130b](#).
 source_line: [44d](#), [47ab](#), [129b](#), [129c](#), [130bd](#), [131ab](#), [132](#), [133ab](#), [138d](#), [139b](#), [140c](#), [160b](#), [170](#), [172](#), [173ab](#), [174a](#), [176b](#), [178c](#), [179d](#), [180](#).
 source_name: [38ab](#), [39ab](#), [44d](#), [47ab](#), [129b](#), [129c](#), [131ab](#), [132](#), [133ab](#), [138d](#), [139b](#), [140c](#), [160b](#), [170](#), [172](#), [173ab](#), [174a](#), [176b](#), [177a](#), [178cd](#), [179d](#), [180](#).
 source_open: [42c](#), [56a](#), [84d](#), [104](#), [129a](#), [133b](#).
 source_peek: [48bd](#), [49b](#), [129a](#), [130b](#), [130d](#), [131ab](#), [133ab](#).
 source_ungetc: [48cd](#), [49c](#), [79a](#), [97d](#), [109b](#), [130c](#), [130d](#).
 stack: [121](#), [123](#), [130a](#), [131b](#), [133a](#).
 stderr: [21b](#), [34](#), [37ab](#), [39b](#), [42c](#), [44d](#), [47ab](#), [56a](#), [61](#), [67ac](#), [68b](#), [73a](#), [75c](#), [77](#), [84d](#), [90be](#), [94d](#), [95b](#), [104](#), [116ab](#), [118ab](#), [121](#), [126b](#), [127a](#), [128b](#), [131ab](#), [132](#), [133b](#), [137d](#), [139ab](#), [140c](#), [147a](#), [148b](#), [152e](#), [160b](#), [170](#), [172](#), [173ab](#), [174a](#), [176b](#), [177a](#), [178cd](#), [179d](#), [180](#), [185a](#), [190b](#), [195e](#), [196ab](#).
 strlen: [21b](#), [50e](#), [117a](#), [121](#), [126a](#), [156c](#), [159bc](#), [186d](#), [194b](#).
 sym_char: [194a](#), [194b](#).
 tex_flag: [27](#), [28](#), [34](#), [39b](#), [42c](#), [152e](#).
 toupper: [21b](#), [161c](#).
 TRUE: [23a](#), [27](#), [28](#), [30](#), [34](#), [36b](#), [39b](#), [54b](#), [59c](#), [64b](#), [65b](#), [67d](#), [73a](#), [76b](#), [80d](#), [81c](#), [82c](#), [85ab](#), [86a](#), [87b](#), [89ab](#), [91a](#), [95b](#), [99a](#), [100b](#), [102a](#), [103a](#), [106a](#), [107c](#), [114](#), [115c](#), [131a](#), [135d](#), [137d](#), [152e](#), [161a](#), [163ab](#), [170](#), [173b](#), [191b](#), [192d](#), [193e](#), [194ab](#).
 update_delimit_scrap: [44d](#), [59c](#), [72a](#), [72b](#), [86a](#), [106a](#).
 user_langs: [163b](#), [170](#).
 user_lang_count: [163b](#), [170](#).
 user_lang_names: [163b](#), [170](#).
 user_names: [42c](#), [52a](#), [82a](#), [99c](#), [110a](#), [141ab](#), [158b](#), [158c](#), [187c](#).
 Uses: [101ab](#), [102c](#), [111b](#), [114](#), [135a](#), [136ab](#), [191ac](#), [191d](#), [192b](#), [193c](#).
 uses: [31](#), [56b](#), [67c](#), [81a](#), [82c](#), [90e](#), [99b](#), [100b](#), [103a](#), [108c](#), [110b](#), [111b](#), [114](#), [115c](#), [116a](#), [118a](#), [134a](#), [136a](#), [138d](#), [141a](#), [142c](#), [158a](#), [163ab](#), [182f](#), [189b](#), [191d](#), [192b](#), [193e](#).
 verbose_flag: [27](#), [28](#), [34](#), [42c](#), [56a](#), [84d](#), [104](#), [126b](#).

version_info_flag: [27](#), [28](#), [34](#), [36a](#).
version_string: [27](#), [28](#), [36a](#), [60b](#), [74a](#), [87a](#), [106a](#).
weave_flag: [27](#), [28](#), [34](#), [39b](#).
weave_format: [27](#), [28](#), [34](#), [39b](#), [172](#).
weave_format_from_cli: [27](#), [28](#), [34](#), [39b](#), [172](#).
WEFT_VERSION: [23b](#), [28](#), [30](#), [31](#).
write_arg: [55b](#), [61](#), [63b](#), [63c](#), [73b](#), [77](#).
write_ArglistElement: [55b](#), [75c](#), [77](#).
write_callers: [39b](#), [116a](#), [125b](#).
write_error_annotator: [36e](#), [124](#), [125b](#).
write_extract: [39b](#), [116b](#), [125b](#).
write_files: [39b](#), [125a](#), [125c](#).
write_html: [39b](#), [103b](#), [104](#).
write_label: [74b](#), [152b](#), [195b](#), [195c](#).
write_lint: [39b](#), [118b](#), [125b](#).
write_literal: [55b](#), [70c](#), [75c](#), [77](#).
write_map: [39b](#), [113a](#), [125b](#).
write_md: [39b](#), [84a](#), [84d](#).
write_reverse_map: [36e](#), [121](#), [125b](#).
write_scraps: [52c](#), [76b](#), [87b](#), [95b](#), [126b](#), [135a](#), [147c](#), [152e](#), [155b](#).
write_scrap_ref: [67d](#), [80d](#), [82c](#), [91a](#), [99a](#), [100b](#), [102b](#), [135a](#), [135c](#), [135d](#), [193a](#).
write_single_scrap_ref: [54b](#), [61](#), [62](#), [63a](#), [66e](#), [67c](#), [76a](#), [78d](#), [81a](#), [82c](#), [87c](#), [88a](#), [90e](#), [94d](#), [95b](#), [97c](#), [99b](#), [100b](#),
 [103c](#), [107ab](#), [108bc](#), [135a](#), [135d](#), [153](#), [154b](#), [195d](#).
write_tex: [39b](#), [55a](#), [56a](#).
xref_flag: [27](#), [28](#), [34](#), [153](#), [154b](#).

Bibliography

- [1] A[lfred] V. Aho and M[argaret] J. Corasick, *Efficient string matching: An aid to bibliographic search.*, CACM **18** (1975), no. 6, 333–340. [188](#)
- [2] C[hris] Hanson, *Efficient stack allocation for tail-recursive languages.*, (1990). [203](#)
- [3] Donald E. Knuth, *Literate programming*, The Computer Journal **27** (1984), no. 2, 97–111. [1](#)
- [4] ———, *The T_EXbook*, Computers and Typesetting, vol. 1986aA, Addison-Wesley, Reading, MA, USA, 1986. [1](#)
- [5] ———, *T_EX: The program*, Computers and Typesetting, vol. B, Addison-Wesley, Reading, MA, USA, 1986b1986. [1](#)
- [6] ———, *METAFont: THE PROGRAM*, Computers and Typesetting, vol. D, Addison-Wesley, Reading, MA, USA, 1986d1986. [1](#)
- [7] Leslie Lamport, *L^AT_EXa document preparation system user's guide and reference manual*, Addison-Wesley, Reading, MA, USA, 1985. [2](#)
- [8] Silvio Levy, *WEB adapted to C, another approach*, TUB **8** (1987), no. 1, 12–13. [1](#)
- [9] Norman Ramsey, *Literate-programming tools need not be complex*, Report at `ftp.cs.princeton.edu` in `/reports/1991/351.ps.Z`. Software at `ftp.cs.princeton.edu` in `/pub/noweb.shar.Z` and at `bellcore.com` in `/pub/norman/noweb.shar.Z`. CS-TR-351-91, Department of Computer Science, Princeton University, August 1992, Submitted to *IEEE Software*. [1](#)
- [10] Ross Williams, *Funnelweb user's manual*, `ftp.adelaide.edu.au` in `/pub/compression` and `/pub/funnelweb`, University of Adelaide, Adelaide, South Australia, Australia, 1992. [1](#)